Sheaf-Descent Semantic Typing for Native Python

Maximal Automatic Inference of Input Requirements, Output Guarantees, Local Site Types, and Runtime-Error Explanations in DepPy

Design and implementation monograph for the next DepPy architecture

March 11, 2026

Contents

0.1 Problem statement, target behavior, and acceptance criteria . . . 1
0.2 Design stance: native Python first, rich semantics second, silence never . . . 1
0.3 Running examples and the target shape of answers . . . 2
0.4 Semantic kernel: site categories, presheaves, boundaries, and errors . . . 3
0.5 The information order and the optimization objective . . . 4
0.6 From native Python source to site covers . . . 4
0.7 Local stalk theories and decidable kernels . . . 5
0.8 Bidirectional synthesis: safe input requirements, rich local sections, and maximal output guarantees . . . 6
0.9 Runtime errors as first-class semantic sites . . . 6
0.10 Interprocedural transport, wrappers, and local transparency . . . 7
0.11 Proof-oriented capabilities in native Python: an F*-like surface without leaving Python 8
0.12 Torch as the flagship library theory . . . 8
0.13 General Python, protocols, heaps, classes, and exceptions . . . 9
0.14 Trust boundary, solver boundary, and explicit residuals . . . 9
0.15 Precise implementation blueprint and migration from the current code base . . . 10
0.16 Package-by-package implementation blueprint . . . 10
0.16.1 Package `src/deppy/assist` . . . 10
0.16.2 Package `src/deppy/contracts` . . . 11
0.16.3 Package `src/deppy/core` . . . 11
0.16.4 Package `src/deppy/frontend` . . . 12
0.16.5 Package `src/deppy/generate` . . . 12
0.16.6 Package `src/deppy/harvest` . . . 13
0.16.7 Package `src/deppy/interprocedural` . . . 13
0.16.8 Package `src/deppy/kernel` . . . 13
0.16.9 Package `src/deppy/library_theories` . . . 14
0.16.10 Package `src/deppy/native_python` . . . 14
0.16.11 Package `src/deppy/normalize` . . . 15
0.16.12 Package `src/deppy/observe` . . . 15
0.16.13 Package `src/deppy/optimize` . . . 16
0.16.14 Package `src/deppy/predicates` . . . 16
0.16.15 Package `src/deppy/proof` . . . 16
0.16.16 Package `src/deppy/render` . . . 17
0.16.17 Package `src/deppy/solver` . . . 17
0.16.18 Package `src/deppy/static_analysis` . . . 18
0.16.19 Package `src/deppy/surface` . . . 18
0.16.20 Package `src/deppy/types` . . . 19
0.17 Site-family catalogue . . . 19

0.17.1 Argument-boundary sites . . . 19
0.17.2 Return and output-boundary sites . . . 20
0.17.3 SSA-value sites . . . 20
0.17.4 Branch-guard sites . . . 20
0.17.5 Call-result sites . . . 21
0.17.6 Tensor-shape sites . . . 21
0.17.7 Tensor-order sites . . . 22
0.17.8 Tensor-indexing sites . . . 22
0.17.9 Heap/protocol sites . . . 22
0.17.10 Proof sites . . . 23
0.17.11 Trace sites . . . 23
0.17.12 Error sites . . . 24
0.17.13 Loop-header and invariant sites . . . 24
0.17.14 Module-summary sites . . . 24
0.18 Theory-family catalogue . . . 25
0.18.1 Linear arithmetic and intervals . . . 25
0.18.2 Boolean guard theory . . . 25
0.18.3 Nullability and optionality . . . 26
0.18.4 Heap and protocol theory . . . 26
0.18.5 Tensor shape theory . . . 26
0.18.6 Tensor order theory . . . 27
0.18.7 Tensor indexing theory . . . 27
0.18.8 Device and dtype compatibility theory . . . 28
0.18.9 Sequence and collection theory . . . 28
0.18.10 Exception theory . . . 28
0.18.11 Witness and equality transport theory . . . 29
0.18.12 Loop invariant and decreases theory . . . 29
0.19 Algorithmic realization . . . 30
0.19.1 Cover synthesis . . . 30
0.19.2 Local solving . . . 30
0.19.3 Backward safe-input synthesis . . . 31
0.19.4 Forward output synthesis . . . 31
0.19.5 Obstruction extraction . . . 31
0.19.6 Interprocedural section transport . . . 32
0.20 Native-Python contract and proof surface . . . 32
0.21 Example corpus and benchmark expectations . . . 33
0.21.1 `examples/protocol_mismatch.py` . . . 33
0.21.2 `examples/heap_field_mismatch.py` . . . 33
0.21.3 `examples/safe_guarded_head.py` . . . 33
0.21.4 `examples/pytorch_shape_bug_from_tensorguard.py` . . . 33
0.21.5 `examples/pytorch_sort_bad_return.py` . . . 34
0.21.6 `examples/pytorch_semantic_showcase.py` . . . 34
0.21.7 `examples/pytorch_long_capability_audit.py` . . . 34
0.21.8 `examples/dependent_sigma_witness.py` . . . 34
0.21.9 `examples/constructor_usage_bug.py` . . . 34
0.21.10 `examples/heap_protocol_branch_bug.py` . . . 35
0.21.11 `examples/mixed_mode_sort_equivalence.py` . . . 35
0.21.12 `examples/theorem_carrying_benchmark.proof` . . . 35

0.22 Runtime-error catalogue and safety synthesis . . . 35
0.22.1 IndexError on standard sequences . . . 35
0.22.2 KeyError on mappings . . . 36
0.22.3 AttributeError . . . 36
0.22.4 TypeError from wrong callable or wrong operand . . . 36
0.22.5 ValueError from invalid reshape or view . . . 36
0.22.6 RuntimeError from broadcast mismatch . . . 37
0.22.7 Out-of-range gather/index-select . . . 37
0.22.8 Reduction on an empty dimension . . . 37
0.22.9 Device mismatch . . . 38
0.22.10 Unsupported dtype/operator combination . . . 38
0.22.11 ZeroDivisionError . . . 38
0.22.12 Assertion failure . . . 38
0.22.13 Constructor invariant failure . . . 39
0.22.14 Protocol misuse in method dispatch . . . 39
0.22.15 Proof transport failure . . . 39
0.22.16 Decreases or invariant failure . . . 40
0.23 Cross-component interaction profiles . . . 40
0.23.1 Interaction: Argument-boundary sites with Linear arithmetic and intervals . 40
0.23.2 Interaction: Argument-boundary sites with Boolean guard theory . . . 40
0.23.3 Interaction: Argument-boundary sites with Nullability and optionality . . . 41
0.23.4 Interaction: Argument-boundary sites with Heap and protocol theory . . . 41
0.23.5 Interaction: Argument-boundary sites with Tensor shape theory . . . 42
0.23.6 Interaction: Argument-boundary sites with Tensor order theory . . . 42
0.23.7 Interaction: Argument-boundary sites with Tensor indexing theory . . . 43
0.23.8 Interaction: Argument-boundary sites with Device and dtype compatibility theory . . . 43
0.23.9 Interaction: Argument-boundary sites with Sequence and collection theory . 44
0.23.10 Interaction: Argument-boundary sites with Exception theory . . . 44
0.23.11 Interaction: Argument-boundary sites with Witness and equality transport theory . . . 44
0.23.12 Interaction: Argument-boundary sites with Loop invariant and decreases theory 45
0.23.13 Interaction: Return and output-boundary sites with Linear arithmetic and intervals . . . 45
0.23.14 Interaction: Return and output-boundary sites with Boolean guard theory . . 46
0.23.15 Interaction: Return and output-boundary sites with Nullability and optionality 46
0.23.16 Interaction: Return and output-boundary sites with Heap and protocol theory 47
0.23.17 Interaction: Return and output-boundary sites with Tensor shape theory . . 47
0.23.18 Interaction: Return and output-boundary sites with Tensor order theory . . . 48
0.23.19 Interaction: Return and output-boundary sites with Tensor indexing theory . 48
0.23.20 Interaction: Return and output-boundary sites with Device and dtype compatibility theory . . . 49
0.23.21 Interaction: Return and output-boundary sites with Sequence and collection theory . . . 49
0.23.22 Interaction: Return and output-boundary sites with Exception theory . . . 50
0.23.23 Interaction: Return and output-boundary sites with Witness and equality transport theory . . . 50

0.23.24 Interaction: Return and output-boundary sites with Loop invariant and decreases theory . . . 51
0.23.25 Interaction: SSA-value sites with Linear arithmetic and intervals . . . 51
0.23.26 Interaction: SSA-value sites with Boolean guard theory . . . 52
0.23.27 Interaction: SSA-value sites with Nullability and optionality . . . 52
0.23.28 Interaction: SSA-value sites with Heap and protocol theory . . . 52
0.23.29 Interaction: SSA-value sites with Tensor shape theory . . . 53
0.23.30 Interaction: SSA-value sites with Tensor order theory . . . 53
0.23.31 Interaction: SSA-value sites with Tensor indexing theory . . . 54
0.23.32 Interaction: SSA-value sites with Device and dtype compatibility theory . . . 54
0.23.33 Interaction: SSA-value sites with Sequence and collection theory . . . 55
0.23.34 Interaction: SSA-value sites with Exception theory . . . 55
0.23.35 Interaction: SSA-value sites with Witness and equality transport theory . . . 55
0.23.36 Interaction: SSA-value sites with Loop invariant and decreases theory . . . 56
0.23.37 Interaction: Branch-guard sites with Linear arithmetic and intervals . . . 56
0.23.38 Interaction: Branch-guard sites with Boolean guard theory . . . 57
0.23.39 Interaction: Branch-guard sites with Nullability and optionality . . . 57
0.23.40 Interaction: Branch-guard sites with Heap and protocol theory . . . 58
0.23.41 Interaction: Branch-guard sites with Tensor shape theory . . . 58
0.23.42 Interaction: Branch-guard sites with Tensor order theory . . . 58
0.23.43 Interaction: Branch-guard sites with Tensor indexing theory . . . 59
0.23.44 Interaction: Branch-guard sites with Device and dtype compatibility theory . 59
0.23.45 Interaction: Branch-guard sites with Sequence and collection theory . . . 60
0.23.46 Interaction: Branch-guard sites with Exception theory . . . 60
0.23.47 Interaction: Branch-guard sites with Witness and equality transport theory . 61
0.23.48 Interaction: Branch-guard sites with Loop invariant and decreases theory . . 61
0.23.49 Interaction: Call-result sites with Linear arithmetic and intervals . . . 62
0.23.50 Interaction: Call-result sites with Boolean guard theory . . . 62
0.23.51 Interaction: Call-result sites with Nullability and optionality . . . 62
0.23.52 Interaction: Call-result sites with Heap and protocol theory . . . 63
0.23.53 Interaction: Call-result sites with Tensor shape theory . . . 63
0.23.54 Interaction: Call-result sites with Tensor order theory . . . 64
0.23.55 Interaction: Call-result sites with Tensor indexing theory . . . 64
0.23.56 Interaction: Call-result sites with Device and dtype compatibility theory . . . 65
0.23.57 Interaction: Call-result sites with Sequence and collection theory . . . 65
0.23.58 Interaction: Call-result sites with Exception theory . . . 65
0.23.59 Interaction: Call-result sites with Witness and equality transport theory . . . 66
0.23.60 Interaction: Call-result sites with Loop invariant and decreases theory . . . 66
0.23.61 Interaction: Tensor-shape sites with Linear arithmetic and intervals . . . 67
0.23.62 Interaction: Tensor-shape sites with Boolean guard theory . . . 67
0.23.63 Interaction: Tensor-shape sites with Nullability and optionality . . . 68
0.23.64 Interaction: Tensor-shape sites with Heap and protocol theory . . . 68
0.23.65 Interaction: Tensor-shape sites with Tensor shape theory . . . 68
0.23.66 Interaction: Tensor-shape sites with Tensor order theory . . . 69
0.23.67 Interaction: Tensor-shape sites with Tensor indexing theory . . . 69
0.23.68 Interaction: Tensor-shape sites with Device and dtype compatibility theory . 70
0.23.69 Interaction: Tensor-shape sites with Sequence and collection theory . . . 70
0.23.70 Interaction: Tensor-shape sites with Exception theory . . . 71

0.24 Evaluation, benchmarks, and success criteria . . . 71
0.25 Staged migration plan . . . 72
0.26 Extended implementation profiles . . . 72
0.26.1 Package `src/deppy/assist` with Linear arithmetic and intervals . . . 72
0.26.2 Package `src/deppy/assist` with Boolean guard theory . . . 73
0.26.3 Package `src/deppy/assist` with Nullability and optionality . . . 73
0.26.4 Package `src/deppy/assist` with Heap and protocol theory . . . 74
0.26.5 Package `src/deppy/assist` with Tensor shape theory . . . 74
0.26.6 Package `src/deppy/assist` with Tensor order theory . . . 74
0.26.7 Package `src/deppy/assist` with Tensor indexing theory . . . 75
0.26.8 Package `src/deppy/assist` with Device and dtype compatibility theory . . . 75
0.26.9 Package `src/deppy/assist` with Sequence and collection theory . . . 76
0.26.10 Package `src/deppy/assist` with Exception theory . . . 76
0.26.11 Package `src/deppy/assist` with Witness and equality transport theory . . . 77
0.26.12 Package `src/deppy/assist` with Loop invariant and decreases theory . . . 77
0.26.13 Package `src/deppy/contracts` with Linear arithmetic and intervals . . . 77
0.26.14 Package `src/deppy/contracts` with Boolean guard theory . . . 78
0.26.15 Package `src/deppy/contracts` with Nullability and optionality . . . 78
0.26.16 Package `src/deppy/contracts` with Heap and protocol theory . . . 79
0.26.17 Package `src/deppy/contracts` with Tensor shape theory . . . 79
0.26.18 Package `src/deppy/contracts` with Tensor order theory . . . 79
0.26.19 Package `src/deppy/contracts` with Tensor indexing theory . . . 80
0.26.20 Package `src/deppy/contracts` with Device and dtype compatibility theory . . . 80
0.26.21 Package `src/deppy/contracts` with Sequence and collection theory . . . 81
0.26.22 Package `src/deppy/contracts` with Exception theory . . . 81
0.26.23 Package `src/deppy/contracts` with Witness and equality transport theory . . . 81
0.26.24 Package `src/deppy/contracts` with Loop invariant and decreases theory . . . 82
0.26.25 Package `src/deppy/core` with Linear arithmetic and intervals . . . 82
0.26.26 Package `src/deppy/core` with Boolean guard theory . . . 83
0.26.27 Package `src/deppy/core` with Nullability and optionality . . . 83
0.26.28 Package `src/deppy/core` with Heap and protocol theory . . . 84
0.26.29 Package `src/deppy/core` with Tensor shape theory . . . 84
0.26.30 Package `src/deppy/core` with Tensor order theory . . . 84
0.26.31 Package `src/deppy/core` with Tensor indexing theory . . . 85
0.26.32 Package `src/deppy/core` with Device and dtype compatibility theory . . . 85
0.26.33 Package `src/deppy/core` with Sequence and collection theory . . . 86
0.26.34 Package `src/deppy/core` with Exception theory . . . 86
0.26.35 Package `src/deppy/core` with Witness and equality transport theory . . . 86
0.26.36 Package `src/deppy/core` with Loop invariant and decreases theory . . . 87
0.26.37 Package `src/deppy/frontend` with Linear arithmetic and intervals . . . 87
0.26.38 Package `src/deppy/frontend` with Boolean guard theory . . . 88
0.26.39 Package `src/deppy/frontend` with Nullability and optionality . . . 88
0.26.40 Package `src/deppy/frontend` with Heap and protocol theory . . . 89
0.26.41 Package `src/deppy/frontend` with Tensor shape theory . . . 89
0.26.42 Package `src/deppy/frontend` with Tensor order theory . . . 89
0.26.43 Package `src/deppy/frontend` with Tensor indexing theory . . . 90
0.26.44 Package `src/deppy/frontend` with Device and dtype compatibility theory . . . 90
0.26.45 Package `src/deppy/frontend` with Sequence and collection theory . . . 91

0.26.46 Package `src/deppy/frontend` with Exception theory . . . 91
0.26.47 Package `src/deppy/frontend` with Witness and equality transport theory . 91
0.26.48 Package `src/deppy/frontend` with Loop invariant and decreases theory . . . 92
0.26.49 Package `src/deppy/generate` with Linear arithmetic and intervals . . . 92
0.26.50 Package `src/deppy/generate` with Boolean guard theory . . . 93
0.26.51 Package `src/deppy/generate` with Nullability and optionality . . . 93
0.26.52 Package `src/deppy/generate` with Heap and protocol theory . . . 94
0.26.53 Package `src/deppy/generate` with Tensor shape theory . . . 94
0.26.54 Package `src/deppy/generate` with Tensor order theory . . . 94
0.26.55 Package `src/deppy/generate` with Tensor indexing theory . . . 95
0.26.56 Package `src/deppy/generate` with Device and dtype compatibility theory . 95
0.26.57 Package `src/deppy/generate` with Sequence and collection theory . . . 96
0.26.58 Package `src/deppy/generate` with Exception theory . . . 96
0.26.59 Package `src/deppy/generate` with Witness and equality transport theory . 96
0.26.60 Package `src/deppy/generate` with Loop invariant and decreases theory . . . 97
0.27 Deep dive: list semantics, bounds safety, and length-dependent contracts . . . 97
0.27.1 Length as a first-class refinement coordinate . . . 97
0.27.2 Element-type refinement and homogeneity . . . 98
0.27.3 Mutation, aliasing, and frame conditions for lists . . . 98
0.27.4 Sorting, searching, and order-sensitive list operations . . . 99
0.27.5 List-specific error sites . . . 99
0.27.6 Interaction with the broader sheaf architecture . . . 99
0.28 Deep dive: dict semantics, key safety, and structural contracts . . . 99
0.28.1 Key domain as a refinement coordinate . . . 100
0.28.2 Value-type refinement per key . . . 100
0.28.3 Dict-specific error sites . . . 100
0.28.4 TypedDict and structural dict contracts . . . 101
0.28.5 Dict comprehensions and transformations . . . 101
0.28.6 Interaction with JSON, serialization, and API boundaries . . . 101
0.29 Deep dive: heap semantics, object state, and mutation tracking . . . 101
0.29.1 Object state as a local section . . . 101
0.29.2 Mutation frames and aliasing . . . 102
0.29.3 Ownership and resource semantics . . . 102
0.29.4 Class hierarchies and method resolution . . . 102
0.29.5 Dataclasses, named tuples, and structured constructors . . . 103
0.29.6 Heap-related error sites . . . 103
0.30 Deep dive: protocols, structural typing, and duck typing . . . 103
0.30.1 Protocol conformance as a local section . . . 103
0.30.2 Explicit Protocol annotations as boundary seeds . . . 104
0.30.3 Protocol subtyping and refinement . . . 104
0.30.4 Duck typing without annotations . . . 104
0.30.5 Protocol-related error sites . . . 104
0.30.6 Iterable, iterator, and sequence protocols . . . 104
0.30.7 Callable protocols and higher-order functions . . . 105
0.31 Deep dive: decorators, metaprogramming, and dynamic class construction . . . 105
0.31.1 Decorators as semantic transformers . . . 105
0.31.2 Metaclasses and dynamic class construction . . . 106
0.31.3 Dynamic attribute assignment and `__getattr__` . . . 106

0.31.4 Monkey-patching and module-level mutation . . . 106
0.31.5 Decorator stacking and composition . . . 107
0.32 Deep dive: the optional annotation system and annotation economy . . . 107
0.32.1 Zero-annotation mode . . . 107
0.32.2 Lightweight annotation: type hints . . . 107
0.32.3 Medium annotation: deppy decorators . . . 108
0.32.4 Heavy annotation: theorem decorators and proof modules . . . 108
0.32.5 Annotation economy: measuring the value of each annotation . . . 108
0.32.6 LLM-authored annotations . . . 109
0.32.7 Gradual adoption path . . . 109
0.33 Deep dive: PyTorch shape theory and symbolic shape inference . . . 109
0.33.1 Shape representation . . . 109
0.33.2 Shape rules for core operations . . . 110
0.33.3 Symbolic shape propagation through a module . . . 111
0.33.4 Output shape as a refinement type . . . 111
0.33.5 Shape error sites and diagnostics . . . 111
0.33.6 Broadcasting rules and their subtleties . . . 112
0.33.7 Device and dtype as secondary shape coordinates . . . 112
0.34 Deep dive: library-specific theories beyond PyTorch . . . 112
0.34.1 NumPy shape and broadcasting theory . . . 112
0.34.2 Pandas DataFrame and Series theory . . . 113
0.34.3 Standard library: pathlib, os, and file I/O . . . 113
0.34.4 Standard library: collections . . . 113
0.34.5 Standard library: itertools and functools . . . 114
0.34.6 SQLAlchemy and database ORM theory . . . 114
0.34.7 Requests and HTTP client theory . . . 114
0.34.8 Theory-pack extensibility and composition . . . 115
0.35 Deep dive: exception handling, control flow, and the Optional/None theory . . . 115
0.35.1 Exceptional control flow as site structure . . . 115
0.35.2 Exception type hierarchies . . . 115
0.35.3 Exception-raising operations as error sites . . . 116
0.35.4 The Optional/None theory . . . 116
0.35.5 Control-flow refinement across branches . . . 117
0.35.6 Match statements (Python 3.10+) . . . 117
0.35.7 Generators, yield, and async/await . . . 117
0.36 Deep dive: numeric safety—overflow, NaN, division by zero, and floating-point contracts 117
0.36.1 The numeric-safety site family . . . 118
0.36.2 NaN propagation theory . . . 118
0.36.3 Infinity tracking . . . 118
0.36.4 Division-by-zero analysis . . . 119
0.36.5 Overflow and underflow in fixed-width arithmetic . . . 119
0.36.6 Numeric-safety contracts for machine learning . . . 119
0.36.7 Interaction with the solver boundary . . . 119
0.37 Deep dive: comprehensions, iteration semantics, and string safety . . . 120
0.37.1 List, dict, set, and generator comprehensions . . . 120
0.37.2 For-loop semantics and loop-carried invariants . . . 120
0.37.3 Iterator protocol and lazy evaluation . . . 121
0.37.4 String operations and encoding safety . . . 121

0.37.5 Zip, enumerate, and iteration combinators . . . 122
0.38 Deep dive: module-level state, imports, closures, and the class/object theory . . . 122
0.38.1 Module-level state and import semantics . . . 122
0.38.2 Class definitions and instance semantics . . . 123
0.38.3 Closures and nested functions . . . 124
0.38.4 Dataclasses and attrs . . . 124
0.38.5 Metaclasses and dynamic class creation . . . 125
0.38.6 Global state and singleton patterns . . . 125
0.39 Deep dive: union types, type narrowing, pattern matching, and the complete type algebra . . . 125
0.39.1 Union types as disjunctive sections . . . 125
0.39.2 Literal types and constant propagation . . . 126
0.39.3 TypeVar, Generic, and parametric polymorphism . . . 126
0.39.4 Overloaded functions . . . 127
0.39.5 NewType, TypeAlias, and nominal distinctions . . . 127
0.39.6 Final, ClassVar, and read-only refinements . . . 127
0.39.7 TypedDict and structural record types . . . 127
0.39.8 Self type and recursive types . . . 128
0.39.9 Intersection types and protocol conjunction . . . 128
0.39.10 Type algebra summary . . . 128
0.40 Deep dive: end-to-end worked example of sheaf-descent analysis . . . 128
0.40.1 The target program . . . 129
0.40.2 Site cover construction . . . 130
0.40.3 Backward synthesis: input requirements . . . 131
0.40.4 Forward synthesis: output guarantees . . . 131
0.40.5 Runtime-error freedom certificate . . . 131
0.40.6 Residual obligations . . . 131
0.40.7 Interprocedural context . . . 132

I Full Dependent Typing and Proof Mode **133**

1 The Proof-Mode Philosophy: From Inference to Verification **134**
1.1 The annotation economy: cherry-picking depth . . . 134
1.2 Why this is not a separate system . . . 135
1.3 The annotation surface: Python-native proof syntax . . . 135
1.3.1 Preconditions and postconditions . . . 136
1.3.2 Dependent function types . . . 136
1.3.3 Quantified properties . . . 136
1.3.4 Proof blocks . . . 137
1.4 The trust gradient: how annotations flow through inference . . . 137

2 Dependent Types in Python: The Full Theory **139**
2.1 The type universe . . . 139
2.2 Propositions as types, proofs as programs . . . 140
2.3 The decidable core and the proof frontier . . . 140
2.4 Inductive types and recursive proofs . . . 141

3 Cherry-Picked Annotation: The Selective Verification Workflow **143**
3.1 Identifying verification targets . . . 143
3.2 The annotation workflow . . . 143
3.2.1 Step 1: Run inference on unannotated code . . . 143
3.2.2 Step 2: Triage residuals . . . 143
3.2.3 Step 3: Add targeted annotations . . . 144
3.2.4 Step 4: Iterate . . . 144
3.3 Example: selective verification of a binary search . . . 144
3.4 Cost-benefit analysis of annotation depth . . . 145

4 The Proof Term Language: Agda-Style Reasoning in Python **146**
4.1 Proof terms as Python expressions . . . 146
4.2 Tactic-style proofs . . . 147
4.3 Proof by calculation . . . 147
4.4 Lemma libraries and reuse . . . 148
4.5 Proof automation levels . . . 148
4.6 Proof-carrying code: embedding proofs in functions . . . 149
4.7 The interaction between proof terms and inference . . . 150

5 Class Invariants and Object-Oriented Proofs **151**
5.1 Class invariants as persistent sections . . . 151
5.2 The `old()` function for state change reasoning . . . 152
5.3 Inheritance and invariant strengthening . . . 152
5.4 Protocol proofs: verifying structural contracts . . . 152

6 Proving Properties of Data Structures **154**
6.1 Sorted lists and binary search trees . . . 154
6.1.1 Proving sortedness preservation . . . 154
6.1.2 Binary search tree invariant . . . 155
6.2 Hash maps and amortized complexity . . . 156

7 Proving Properties of Algorithms **157**
7.1 Sorting algorithms: from specification to proof . . . 157
7.1.1 The sorting specification . . . 157
7.1.2 Merge sort with full proof . . . 157
7.1.3 The merge function . . . 158
7.2 Graph algorithms . . . 159
7.2.1 Breadth-first search: reachability proof . . . 159
7.2.2 Topological sort: acyclicity and ordering . . . 160
7.3 Numeric algorithms . . . 161
7.3.1 Newton's method: convergence and precision . . . 161
7.3.2 Matrix multiplication: shape correctness with value proof . . . 162
7.4 String and parsing algorithms . . . 162
7.4.1 Balanced parentheses checker . . . 162
7.5 Annotation cost summary . . . 163

8 The Proof Engine: How Proofs Flow Through the Sheaf Machinery **164**
8.1 Proof obligations as section constraints . . . 164
8.2 The obligation lifecycle . . . 164
8.3 Obligation propagation through the site cover . . . 165
8.4 The proof context stack . . . 166
8.5 Interaction with the inference engine . . . 166
8.6 The solver interface . . . 167
8.7 Proof caching and incrementality . . . 167
8.8 Error reporting and diagnostics . . . 167

9 Proving Properties of PyTorch Programs **169**
9.1 Shape-correct neural network architectures . . . 169
9.1.1 Proving shape compatibility of a multi-layer model . . . 169
9.1.2 Attention mechanism shape proof . . . 170
9.2 Gradient correctness . . . 171
9.2.1 Proving that gradients are well-defined . . . 171
9.3 Training loop correctness . . . 172
9.3.1 Proving that a training loop maintains model invariants . . . 172
9.4 Model architecture validation . . . 173
9.4.1 Proving that a ResNet skip connection is shape-compatible . . . 173

10 Proof Mode for Real-World Python: Effects, I/O, and the Trusted Computing Base **175**
10.1 The effect discipline . . . 175
10.2 Resource tracking and linear types . . . 176
10.3 State-dependent types for mutable objects . . . 176
10.4 Proving properties across module boundaries . . . 177
10.5 The trusted computing base . . . 178
10.6 Soundness and the escape hatches . . . 178

11 Advanced Proof Techniques: Abstraction, Refinement, and Parametricity **180**
11.1 Data abstraction and representation invariants . . . 180
11.2 Simulation and refinement proofs . . . 182
11.3 Parametricity and generic proofs . . . 182
11.4 Proof by refinement: stepwise development . . . 183
11.5 Proof composition and modular verification . . . 183
11.5.1 Assume-guarantee reasoning . . . 183
11.6 Proof of absence of common vulnerability patterns . . . 184

12 The Annotation Economy: Maximizing Proof Value with Minimal Effort **185**
12.1 The information-theoretic view . . . 185
12.2 The annotation advisor . . . 185
12.3 Progressive verification: a worked example . . . 186
12.4 When not to prove . . . 187

II Advanced Sheaf Theory, Optimization, and the Relevance Problem **188**

13 The Full Sheaf-Theoretic Framework for Type Inference **189**
13.1 Sites, covers, and the Grothendieck topology . . . 189
13.2 The type presheaf and the sheaf condition . . . 189
13.3 Cohomology of the type sheaf . . . 190
13.4 Descent data and effective descent . . . 191
13.5 Stalks and the local-to-global principle . . . 191
13.6 Sheaf morphisms and type transformations . . . 192

14 The Optimization Problem: Maximizing Type Information **193**
14.1 The information lattice . . . 193
14.2 The global optimization objective . . . 193
14.3 Reformulation as a constraint satisfaction problem . . . 194
14.4 The lattice structure of solutions . . . 194
14.5 Approximation algorithms . . . 194
14.6 Duality: the backward optimization problem . . . 195

15 The Relevance Problem: Which Propositions Matter for Bug Detection? **196**
15.1 The relevance criterion . . . 196
15.2 Computing relevance: the backward slice . . . 196
15.3 The predicate abstraction framework . . . 197
15.4 Automatic predicate discovery . . . 197
15.5 Predicate domain refinement . . . 198
15.6 Relevance scoring . . . 198

16 User-Guided Relevance and the Interactive Query Interface **199**
16.1 The `@track` annotation: user-declared relevance . . . 199
16.2 Property queries: asking the system what it knows . . . 200
16.3 The exploration mode: discovering unknown properties . . . 200
16.4 Relevance propagation across module boundaries . . . 201
16.5 The relevance graph . . . 201
16.6 Relevance under uncertainty: probabilistic relevance . . . 202

17 Formal Verification Integration: Connecting to External Provers **203**
17.1 The verification backend interface . . . 203
17.2 Obligation routing . . . 204
17.3 Translation to Lean 4 . . . 204
17.4 Proof portability and the semantic bridge . . . 204
17.5 Hybrid verification workflows . . . 205

18 Information-Theoretic Optimization of Type Inference **206**
18.1 Types as probability distributions . . . 206
18.2 Mutual information between sites . . . 207
18.3 Rate-distortion theory for type precision . . . 207
18.4 The predicate selection problem . . . 208
18.5 Pareto-optimal type assignments . . . 208
18.6 Online optimization: incremental type inference . . . 209
18.7 The Lagrangian relaxation of the combined problem . . . 209

19 Obstruction Theory for Type Errors **210**
19.1 Obstructions as cohomology classes . . . 210
19.2 Minimal obstruction representatives . . . 210
19.3 Error classification by obstruction type . . . 211
19.4 Obstruction-guided repair . . . 211
19.5 Spectral sequences for complex type errors . . . 212

20 Topos-Theoretic Semantics: The Internal Logic of the Type Topos **213**
20.1 The type topos . . . 213
20.2 The subobject classifier and propositions . . . 213
20.3 Propositions as subobjects of the type sheaf . . . 214
20.4 The internal logic for type reasoning . . . 214
20.5 The forcing semantics: what is "true at a site"? . . . 215
20.6 Relevance in the internal logic . . . 215
20.7 The Lawvere-Tierney topology for decidable propositions . . . 215
20.8 Modalities for annotation levels . . . 216

21 Automated Deduction Strategies for Richer Dependent Types **217**
21.1 Abductive inference: hypothesizing missing predicates . . . 217
21.2 Craig interpolation for interface synthesis . . . 217
21.3 Quantifier instantiation for bounded domains . . . 218
21.4 Abstract interpretation with predicate abstraction . . . 218
21.5 Symbolic execution for path-sensitive inference . . . 219
21.6 Liquid types: type inference as predicate abduction . . . 219
21.7 Machine-learning-guided predicate suggestion . . . 219

22 Enriched Category Theory for Type Propagation and Kan Extensions **221**
22.1 Enriched categories of types . . . 221
22.2 Weighted colimits for type synthesis . . . 221
22.3 Kan extensions for interprocedural transport . . . 222
22.4 The pointwise formula and computational realization . . . 223
22.5 Profunctors for type interfaces . . . 223
22.6 The fibered category of type families . . . 223

23 The Relevance Optimization Problem: Formal Treatment **225**
23.1 The space of decorations . . . 225
23.2 Minimal useful decoration sets . . . 225
23.3 The relevance lattice . . . 226
23.4 Relevance propagation as a Galois connection . . . 226
23.5 The relevance fixpoint . . . 226
23.6 User interaction with the relevance system . . . 227
23.7 Relevance-weighted MaxSMT . . . 227

24 Connections to Homotopy Type Theory and Higher-Dimensional Type Structure **229**
24.1 Types as spaces, programs as paths . . . 229
24.2 The univalence principle for type equivalence . . . 229
24.3 Higher inductive types for recursive data structures . . . 230
24.4 Transport along type paths . . . 230

24.5 Identity types and equality reasoning 231
24.6 The fundamental groupoid of a type 231
24.7 Truncation levels and proof irrelevance 232

25 Synthesis of Relevant Propositions: Algorithms and Complexity 233
25.1 The proposition synthesis pipeline 233
25.2 Complexity analysis 233
25.3 Optimizations for large programs 234
25.4 The relevance oracle: user-system interaction protocol 234
25.5 Relevance for proof obligations 235

Abstract

This monograph rewrites the DepPy story around a different answer to the question "what is a type?" A type is not a flat set of predicates chosen from a candidate pool. A type is a globally glued semantic section obtained from local evidence distributed across a cover of program sites: argument-boundary sites, control-flow sites, SSA-value sites, call and wrapper sites, theory-trigger sites, dynamic-trace sites, proof-module sites, heap/protocol sites, and runtime-error sites. The practical motivation is the same as before but made more ambitious and more precise. We want ordinary native Python and native PyTorch code, written with minimal or zero additional annotation, to support the automatic inference of the richest safe information available: input requirements, output guarantees, intermediate local types, witness-carrying facts, library-specific obligations, proof goals, and predicted runtime errors. We also want the same system to admit an explicit proof-first surface that feels closer to F* or theorem-carrying programming, yet still elaborates back into the same semantic core and the same explicit trusted boundary.

The sheaf-descent perspective makes the slogan "minimal annotation, maximum information" mathematically concrete. Minimal annotation is not merely fewer clauses; it is the smallest amount of boundary information needed to eliminate global ambiguity. Maximum information is not merely a high score on a frontier of atoms; it is the richest globally glueable section compatible with local solvability, proof transport, and library semantics. This change unlocks a unified account of four things that are usually separated: the synthesis of safe input refinements, the synthesis of informative output refinements, the typing of local intermediate values, and the anticipation of runtime errors that remain reachable under current assumptions. Backward propagation through the site cover computes the weakest safe input neighborhoods that keep error-sensitive local sites nonempty. Forward descent computes the strongest output semantics and the richest local contracts justified by those safe neighborhoods together with the available evidence. Residual ambiguity is represented not as an unstructured list of failed obligations but as obstruction data that points to the exact site or overlap where more information would be most valuable.

The implementation thesis of this document is intentionally concrete. It maps the mathematics to a staged DepPy architecture grounded in the current code base: AST lowering, CFG and SSA construction, value-lineage recovery, library-trigger normalization, theory-pack decomposition, interprocedural summary transport, frontier extraction, repair guidance, and renderer-based user surfaces. The existing modules such as `src/deppy/pipeline_monograph.py`, `src/deppy/surface/python_ssa.py`, `src/deppy/interprocedural/frontier_propagation.py`, and the split PyTorch packs become the seed of a more principled implementation in which theory packs create sites and compatibility maps rather than merely proposing atoms. The monograph is therefore both a semantic proposal and a theory of implementation: a blueprint for a system that can verify that a Torch module avoids library-specific errors such as invalid reshape, out-of-bounds indexing, broadcast mismatch, sort/index misuse, or device inconsistencies, while also supporting proof-oriented capabilities reminiscent of F* but expressed in native Python.

0.1 Problem statement, target behavior, and acceptance criteria

The problem is to build a dependent environment for native Python that infers as much semantically useful information as possible without forcing users to leave ordinary Python syntax. A successful system should analyze a module and return a high-value semantic object describing at least six classes of information. First, it should synthesize input requirements strong enough to avoid reachable runtime errors and weak enough to remain useful. Second, it should synthesize output guarantees strong enough to describe the actual semantic value of the module: sortedness, permutation preservation, output shape, non-emptiness, protocol availability, witness export, ownership, or algorithm-specific correctness coordinates. Third, it should infer local-site types for intermediate values rather than treating function boundaries as the only place where meaning lives. Fourth, it should classify which runtime errors remain reachable and under what input neighborhoods. Fifth, it should distinguish facts that are solver-checked, proof-checked, dynamically observed, heuristically suggested, or still residual. Sixth, it should do all of this under a library model rich enough for real PyTorch code and a proof surface rich enough for F*-like theorem-carrying development.

The user experience target is deliberately bimodal. A regular Python user should be able to write a small Torch helper using plain control flow, local bindings, wrappers, and a few natural assertions. Running the analyzer should surface a module summary containing inferred preconditions, inferred postconditions, local intermediate semantics, and concrete explanations of why certain runtime errors have been eliminated or remain possible. A proof-oriented user should be able to add theorem modules, transport lemmas, or witness-carrying helper functions in native Python syntax, with optional decorators or helper combinators, and have those artifacts tighten the same underlying semantic object rather than enter a parallel tool. The trust boundary must remain explicit: every imported proof or suggested refinement is either checked, reduced to a decidable fragment, classified as bounded automation, or retained as a residual obligation.

This monograph therefore uses a stronger acceptance criterion than "the tool finds some useful refinements." The correct target is a contract transformer that maps sparse boundary evidence to maximal safe semantic structure. For a module `f`, the system should compute a pair (ρ_{in}, ρ_{out}) plus a family of local sections and a residual runtime-error map. The input refinement ρ_{in} should be minimal among safe refinements in the selected lattice of admissible input neighborhoods. The output refinement ρ_{out} should be maximal among globally glued postconditions compatible with ρ_{in} and the local evidence. The local-site family should explain why those boundary facts hold, and the runtime-error map should explain which error-sensitive sites are empty or nonempty under the synthesized assumptions. The result should be renderable as native-Python contracts, diagnostics, proof goals, benchmark telemetry, and semantic frontiers.

A final constraint is migration realism. The current DepPy code base already contains a serious prototype: semantic signatures, theory packs, split PyTorch reasoning, CFG and SSA lowering, frontier enumeration, interprocedural propagation, repair suggestions, and showcase examples. The rewrite must not pretend that implementation begins from zero. Instead, it should explain how to reorganize existing modules into a sheaf-descent architecture so that current capabilities become special cases of a more coherent semantic core.

0.2 Design stance: native Python first, rich semantics second, silence never

The design stance has three parts. The first is syntactic humility. Users should write native Python, not a pseudo-Python proof language that happens to borrow Python keywords. This means

normal functions, classes, local variables, comprehensions, exceptions, higher-order helpers, and PyTorch operator calls must remain first-class. Optional decorators such as `@deppy.requires`, `@deppy.ensures`, `@deppy.theorem`, or `@deppy.decreases` are allowed, but the architecture must treat them as seeds, not as a precondition for precision. The second is semantic ambition. Internally the system should support full dependent structure: dependent functions, witness-carrying results, indexed families, equality transports, module-level theorems, and rich library contracts. The third is epistemic honesty. Any fact that cannot be justified by the selected local logic, proof kernel, or marked empirical evidence must remain visible as a residual. The system should never silently collapse unknowns into weak defaults if doing so would mislead users about safety.

This stance immediately rules out two common failure modes. The first is the "annotation cliff," where rich typing is available only once the user has already rewritten the program in a theorem-prover style. The second is the "heuristic mirage," where a tool produces attractive contracts but hides which parts come from actual proof, which from a bounded SMT slice, and which from a best-effort guess or observed execution. A sheaf-descent system can avoid both because it records provenance at the site level. Local sections carry not only semantic content but also support and trust metadata. When the global section is glued, the renderer can explain which regions were settled by theory packs, which by proofs, which by dynamic traces, which by imported call summaries, and which remain ambiguous.

The native-Python stance also shapes the implementation vocabulary. Instead of pretending that users manipulate global proof terms directly, the system should harvest meaning from ordinary control-flow regions, SSA values, local assertions, library operators, and wrapper structure. This is why the site notion is so important. A local variable carrying the result of `torch.sort(xs).values`, a branch guard such as `if xs.numel() > 0`, a proof-carrying helper returning a witness pair, and a call to a protocol method are all legitimate places where semantics becomes locally clear. A global contract is then something the system glues from those local clarities, not something the user must narrate from scratch.

0.3 Running examples and the target shape of answers

A useful monograph should pin the theory to a few representative programs. The first running example is a Torch ranking helper that sorts scores, extracts the top element, and then feeds the result through a lightweight wrapper chain. A current prototype can already infer sortedness and permutation-like facts for certain direct patterns. The rewritten system should do more. It should infer a safe input requirement stating the tensor is nonempty if the function indexes its top element. It should infer output guarantees stating that the returned value comes from the original tensor, that the associated index lives in the admissible source range, and that the intermediate sorted tensor satisfies a local order section. It should explain why the index-out-of-range error site is empty under the synthesized precondition and why the sortedness site remains nonempty under the imported sort semantics.

The second running example is a proof-oriented algorithm module written in native Python but using theorem decorators and ghost helpers. Think of a stable partition helper or a graph relaxation routine. The user writes a few explicit lemmas: perhaps one transport lemma about how a partition step preserves a permutation witness, or one invariant lemma about a loop maintaining a monotone potential. The sheaf-descent system should interpret these as transport maps between proof-local sites and ordinary program sites. The user should not need to restate the full correctness theorem on every wrapper. The system should instead import the proof-carrying local sections, propagate them through the cover, and tighten both input requirements and output guarantees.

If some step still exceeds the decidable fragment, the residual should be attached to the specific overlap or transport site that failed, not dumped as an opaque theorem hole.

The third running example is a library-verification use case. Suppose the goal is simply to certify that a Torch module does not hit library-specific errors on a selected input family. The user may care less about theorem-level postconditions and more about excluding invalid reshape, mismatched indexing, empty reductions, device mismatches, or unsupported dtype/operator combinations. The rewritten architecture should treat these errors as first-class sites whose viability can be checked locally and pulled back to the input boundary. The output of analysis should then include a compact synthesized requirement such as "rank at least one, first dimension positive, target view cardinality equal to source cardinality, and selector domain included in source extent." This is a highly practical deliverable: a theorem-free user still gets a strong precondition that prevents library-specific runtime failures.

These three examples fix the standard for the rest of the document. A good implementation theory must explain how local semantics is extracted, how it is represented, how it is checked, how it is glued, how it is pulled back to safe inputs, how it is pushed forward to rich outputs, and how it is rendered back to native Python users.

0.4 Semantic kernel: site categories, presheaves, boundaries, and errors

Definition 0.4.1 (Observation site category)**.** Given a Python module, let $\mathcal{S}$ be a category whose objects are observation sites and whose morphisms are restriction or reindexing maps. Objects include module sites, function sites, CFG-block sites, SSA-value sites, call sites, return sites, argument-boundary sites, output-boundary sites, dynamic-trace sites, proof-module sites, heap/protocol sites, and error-sensitive sites induced by partial operators or exceptional transitions.

Definition 0.4.2 (Semantic presheaf)**.** A semantic presheaf is a functor $\mathsf{Sem} : \mathcal{S}^{op} \to \mathsf{Poset}$ assigning to each site a partially ordered set of local semantic sections. Each section contains a carrier type, a local refinement family, provenance metadata, a proof-boundary classification, and an optional witness or transport payload. Restriction maps preserve soundness and drop only those coordinates that are not visible in the smaller site.

The decisive shift from earlier formulations is that candidate atoms are no longer primary. A flat atom is only a view of a richer local section. For a Torch sort site, the local section may contain not only `sorted(result)` but also a permutation witness, index-domain constraints, projected result types, site provenance, and transport hooks to downstream indexing or view sites. For a heap/protocol site, the local section may contain ownership, field consistency, protocol availability, mutation frame information, and exception viability. When the system renders a contract surface or a frontier, it projects from this richer object rather than mistaking the projection for the semantic core.

Two boundaries are special. The input boundary ∂_{in} contains sites for formal parameters, receiver objects, ambient protocol assumptions, ghost inputs, module configuration, and any dynamic assumptions the user or test trace is willing to mark as admissible. The output boundary ∂_{out} contains sites for return values, raised exceptions, mutated heap regions, exported witnesses, and theorem-level guarantees. Runtime errors are represented by a family Err of error-sensitive sites. Each such site has a local notion of viability; it is nonempty if the operation can proceed without entering a library-specific or general runtime error state under the current boundary assumptions.

Definition 0.4.3 (Bidirectional refinement problem)**.** For a module or function f, the bidirectional refinement problem is to compute an input section ρ_{in} on ∂_{in}, an output section ρ_{out} on ∂_{out}, and a compatible global section σ such that: (1) every error-sensitive site reachable from ρ_{in} is nonempty; (2) ρ_{out} is maximal among output sections extendable from ρ_{in}; (3) ρ_{in} is minimal among safe input sections in the chosen admissible lattice; and (4) any remaining ambiguity is explained by explicit obstruction data.

This definition intentionally merges typing, verification, and runtime error anticipation. A dependent environment should not force users to ask separate tools for precondition synthesis, postcondition synthesis, intermediate typing, and error prediction. All four are shadows cast by one semantic object viewed from different boundaries.

0.5 The information order and the optimization objective

Although the sheaf-descent rewrite moves away from "choose atoms" as the primary optimization variable, optimization remains central. There are usually many compatible global sections, many possible input refinements, and many ways to summarize local evidence. We therefore need an explicit information order and a cost model.

For input refinements, the order is reverse permissiveness: a stronger precondition is lower in the lattice because it excludes more behaviors. For output refinements, the order is semantic informativeness: a richer postcondition is higher because it settles more coordinates, exports more witnesses, or fixes more relations. For local sections, the order combines ambiguity reduction, proof status, and runtime-safety value. A useful implementation does not search all these orders blindly. It uses site-local solvers, compatibility constraints, and ranking heuristics to identify a finite frontier of globally meaningful options.

A convenient objective for implementation is a lexicographic or weighted tuple: first minimize residual runtime-error reachability; then minimize input assumption cost; then maximize output information; then maximize local semantic coverage; then minimize proof debt; then minimize explanation complexity. This ordering matches the intended user experience. A system that infers a gorgeous postcondition while leaving an index error reachable under ordinary inputs is not acceptable. A system that eliminates the error only by inventing absurdly strong preconditions is also not acceptable. A system that leaves huge residuals where cheap local theory reasoning would suffice is wasting automation budget.

The existing DepPy frontier machinery is therefore not discarded; it is reinterpreted. Frontiers should now range over compact summaries of glued sections rather than subsets of atoms. A frontier point may represent one safe input refinement together with one maximal output summary and a small list of key local witnesses or residual obstructions. This is a richer object than the current packed signature, but it remains finite and renderable because the heavy lifting stays local and the global glueing data is compressed before rendering.

0.6 From native Python source to site covers

A precise implementation theory must say exactly how the site category is built from real Python. The correct pipeline begins with existing frontend and surface infrastructure rather than bypassing it. The current code already exposes AST, CFG, SSA, heap-sensitive lowering, and interprocedural summaries. The rewrite should formalize those stages as cover synthesis.

First, parse the module and record ordinary syntax sites: function definitions, class bodies, comprehensions, attribute accesses, literal values, call expressions, assertions, exception edges, and

return nodes. Second, lower to a CFG so that control-dependent sites become explicit. Branch conditions, loop headers, exceptional successors, and join points each matter because they define where local facts are introduced, merged, or invalidated. Third, construct SSA and heap-SSA. The current module `src/deppy/surface/python_ssa.py` already computes versioned names, phi nodes, final returns, and value lineage. The sheaf-descent rewrite should reinterpret these as indices for SSA-value sites and overlap maps. A phi node is not merely a bookkeeping device; it is an overlap site at which sections from predecessor blocks must agree or expose an obstruction. A lineage chain is not merely a convenience for wrapper detection; it is a witness that two textual values live in the same semantic neighborhood.

Fourth, normalize theory-triggering expressions. The shared normalization helper `src/deppy/library_theories` already shows the direction: theory packs should see normalized return or lineage views, not shallow syntax. Fifth, generate library-specific sites. A call to `torch.sort` should create order, permutation, index, and alias sites; a `view` or `reshape` call should create shape-compatibility and output-shape sites; an indexing operator should create selector-domain and source-target relation sites; a protocol method access should create method-availability and frame sites. Sixth, add proof sites for any theorem modules, lemma decorators, proof combinators, or checked witness exports. Finally, add trace sites if the user provides representative executions or recorded observations.

The output of this phase should be a first-class cover artifact. Concretely it should be a data structure containing site identifiers, site kinds, local carrier schemas, incoming and outgoing restriction maps, provenance handles back to AST and SSA positions, and a dependency index for error-sensitive operations. The monograph recommends introducing an explicit cover package under something like `src/deppy/core/site_cover.py` or `src/deppy/observe/site_graph.py`. This artifact would become the new substrate over which theory packs, local solvers, descent, and rendering operate.

0.7 Local stalk theories and decidable kernels

The sheaf formulation does not grant license to build one gigantic global logic. Quite the opposite: it makes local decidability more important. Every site kind must declare a local logic or "stalk theory" appropriate to the neighborhood. The following families are central.

Arithmetic and sequence sites support linear arithmetic, interval reasoning, length equalities, non-emptiness, and simple finite-domain facts. Heap/protocol sites support field availability, ownership frames, alias exclusions, optional nullability, and effect summaries. Tensor-shape sites support rank, dimension, shape tuples, reshape/view cardinality, broadcast legality, and reduction preconditions. Tensor-order and permutation sites support sortedness, monotone projection, permutation witnesses, argsort relations, and sorted-value/index coupling. Indexing sites support selector-domain inclusion and source-target value provenance. Proof sites support theorem statements, transport obligations, residual lemmas, and imported witness objects. Trace sites support empirical contraction of otherwise ambiguous neighborhoods without pretending to establish a proof.

A precise implementation rule is that every theory pack must declare three things. First, how it creates sites and what local carriers those sites use. Second, what formulas or witness objects its local solver can discharge. Third, how facts restrict across overlaps. This is the sheaf equivalent of the current theory-pack schema in `src/deppy/core/theory_pack.py`. The schema should therefore be extended from atom-generation callbacks to a richer interface containing site constructors, local logic descriptors, viability predicates, forward refinement extractors, backward precondition pullbacks, and proof-boundary classifiers. The implementation should still allow pack composition and dependencies, but the unit of composition becomes site families and transports

rather than isolated atoms.

Decidability remains explicit. A tensor-shape site may live in a solver-friendly semialgebraic fragment. A permutation witness site may only support finite or symbolic template reasoning. A proof transport site may require a checked theorem module or stay residual. The renderer must preserve these distinctions. The system should not claim that a theorem-level permutation property is as trusted as a simple index-bound fact unless both truly pass the same kernel.

0.8 Bidirectional synthesis: safe input requirements, rich local sections, and maximal output guarantees

Bidirectional synthesis is the central algorithmic novelty. The backward pass starts from error-sensitive sites and from user-prioritized output desires. Each partial operator exports a viability predicate over the local incoming neighborhood. For example, an indexing site says that a selector domain must lie inside the source extent; a reshape site says that source and target cardinality must match; a reduction site may require a nonempty dimension unless an identity value is supplied; a protocol dispatch site says that the receiver neighborhood must implement the requested method or field. These viability predicates are transported backward along restriction maps toward the input boundary. The meet of all pulled-back viability requirements is the synthesized safe input refinement ρ_{in}, provided the meet is consistent. If the meet is inconsistent, the system reports the obstruction class rather than silently weakening the claim.

The forward pass begins from ρ_{in} together with local syntax, theory-pack, trace, and proof evidence. It solves each site locally, propagates solved sections across overlaps, and glues the richest compatible family. Local sites for intermediate values therefore receive precise semantics even when the user never annotated them. The same pass yields the maximal output section ρ_{out} extendable from the safe input boundary. In practice this means the analyzer can return both a synthesized `requires` surface and a synthesized `ensures` surface, along with a catalogue of local intermediate facts and a runtime-error report. The output side is not second-class. A user wants to know not only what must hold for safety, but also what is gained in return.

The implementation should run these passes to a fixed point because they inform one another. A stronger synthesized output section may reveal that some input constraint is redundant because the gluing data already forces a stronger local relation upstream. Conversely, a sharper input requirement may unlock stronger local solves and therefore stronger outputs. The fixed-point engine should stop when both the backward safe-input lattice element and the forward output frontier stabilize.

A useful design principle is that every synthesized contract should be grounded in a minimal explanation set. When the system proposes an input requirement, it should attach the local error-sensitive sites that forced it. When it proposes an output guarantee, it should attach the sites and overlaps that glued to justify it. This gives users both trust and leverage: they can decide whether to accept the synthesized contract, weaken it manually, or add one proof or assertion that changes the fixed point.

0.9 Runtime errors as first-class semantic sites

Traditional type systems talk primarily about accepted and rejected programs. For native Python, especially library-rich Python, this is not enough. We need a structured account of runtime errors that can be anticipated, localized, classified, and linked to synthesized preconditions. The sheaf-

descent rewrite therefore models runtime errors as first-class semantic sites rather than as an afterthought.

Every error-sensitive operator introduces one or more error sites. Standard Python introduces sites for `IndexError`, `KeyError`, `AttributeError`, `TypeError`, `ValueError`, `ZeroDivisionError`, assertion failure, iteration exhaustion in contexts that expect a value, and protocol violations. PyTorch introduces sites for invalid reshape or view, broadcast mismatch, out-of-range gather or index-select, invalid reduction over an empty dimension, device mismatch, unsupported dtype/-operator combinations, invalid matrix shape alignment, and library-specific semantic mismatches such as confusing sorted values with sorted indices. Heapful object code introduces sites for missing field initialization, stale alias use after mutation, frame escape, or constructor invariant failure. Proof-oriented code introduces sites for missing transport lemmas, unverified witness exports, or decreases obligations that fail to justify recursion.

Each error site must provide four methods to the implementation. First, a constructor that identifies where the site comes from and which operands it sees. Second, a local viability predicate describing when the site is safe. Third, a local explanation generator that can render the site to the user in ordinary language. Fourth, a pullback rule that transforms the site's viability into a constraint on upstream neighborhoods. The correct module boundary for this logic is a mix of theory packs and a centralized error-site registry. The monograph recommends adding a package such as `src/deppy/kernel/error_sites.py` or `src/deppy/observe/error_catalog.py` so that runtime-error treatment is uniform across standard Python and library-specific packs.

This approach has a subtle but important effect on the overall story. A module analysis is no longer judged only by the postconditions it discovers. It is also judged by how completely it explains the absence or presence of runtime errors. A user verifying a Torch module for library-specific safety may care primarily about the error-site frontier, not about full functional correctness. The same sheaf-descent architecture supports both use cases.

0.10 Interprocedural transport, wrappers, and local transparency

Interprocedural behavior is where many prototypes lose semantic information. The current DepPy work already contains an important lesson: direct-return wrapper propagation is too weak, but SSA lineage and alias-aware wrapper recognition make interprocedural transport much more robust. The sheaf-descent rewrite turns that lesson into a general rule.

A function summary should not be a single exported atom set. It should be a boundary section plus an interior miniature cover containing especially valuable local sites and transport maps. When a caller invokes the function, the system instantiates the callee cover with an argument reindexing map, imports the boundary and selected interior sections, and glues them into the caller cover at the call-result and wrapper sites. The existing logic in `src/deppy/interprocedural/frontier_propagation.py` already performs a weaker version of this idea for direct-return and alias-forwarding wrappers. The next step is to lift it from atom import to section import.

Wrapper transparency is especially important for native-Python ergonomics. Users constantly write tiny helpers, local wrappers, decorator-generated functions, output extractors, and alias-preserving reorganizations. If the semantics does not survive such structure, the system will feel fragile and non-Pythonic. A site-based formulation provides a precise criterion for transparency: a wrapper is semantically transparent to the extent that its induced reindexing map preserves or refines the relevant local sections without introducing new obstructions. This criterion is stronger and cleaner than a bag of textual pattern recognizers, even though pattern recognizers may remain the first implementation strategy.

The implementation plan should therefore extend interprocedural summaries in three directions. First, summaries must retain local-site provenance so that imported sections can explain themselves. Second, summaries must support bidirectional transport: callee error sites should contribute to caller input requirements, and caller input assumptions may tighten callee outputs. Third, summary import must be lineage-aware so that wrappers built out of local aliases, projections, and shape-preserving operators remain semantically visible.

0.11 Proof-oriented capabilities in native Python: an F*-like surface without leaving Python

The user explicitly asked for F*-like capabilities but in native Python and with minimal extra annotation. The sheaf-descent design makes this plausible because proof artifacts become just another family of local sites and transport maps. The proof surface should expose optional tools, not a mandatory second language. At minimum the surface should support theorem decorators, explicit ghost or proof parameters, witness-returning helpers, local `assert` statements lifted into proof seeds, optional `@deppy.requires`/`@deppy.ensures` decorators, optional `@deppy.decreases` or `@deppy.variant` markers for recursive and loop reasoning, and helper combinators for equality transport or witness projection.

A key design choice is that proof artifacts should not merely add finished facts. They should define transport maps. A theorem about a sorting helper should not just inject `sorted(result)` as a leaf proposition. It should also describe how that fact behaves under wrappers, views, projections, and index consumers. A loop invariant should not just be checked at a function boundary. It should be attached to loop-header and loop-body sites so that the global section can glue across iterations. A witness-returning helper should create both a value site and a proof site connected by a dependency relation. This is exactly how F*-like proof terms acquire practical value for ordinary code: they change the geometry of future inference rather than remaining isolated certificates.

The current proof modules in `src/deppy/surface/proof_dsl.py`, `src/deppy/surface/proof_modules.py`, and related elaboration files are an ideal seed. The rewrite should recast them as frontends to a proof-site constructor and a transport compiler. Residual proof obligations should remain visible when the kernel or bounded solver fragment cannot discharge them. The renderer should be able to show both proof-first users and ordinary users what semantic leverage a given proof artifact provides. For example: "this lemma resolves the only remaining obstruction between the sort-index site and the downstream selector site, which strengthens the output contract from range-safety to permutation preservation." This kind of explanation is more compelling than a raw theorem list.

0.12 Torch as the flagship library theory

PyTorch remains the flagship because it combines rich local semantics, real user value, and the need to reason about library-specific runtime errors. A sheaf- descent implementation should treat Torch packs as factories for semantic covers. A call to `torch.sort` should create order, permutation, and index sites plus compatibility maps to any downstream consumers. A `view` or `reshape` call should create shape-compatibility, cardinality, alias, and output-shape sites. A subscript or `gather` call should create selector-domain and source-target value-relation sites. A device move or mixed-device operator should create compatibility sites whose viability rules generate either safe input constraints or concrete diagnostics.

The current split packs in `src/deppy/library_theories/pytorch_sort.py`, `src/deppy/library_theories/p` and `src/deppy/library_theories/pytorch_indexing.py` already point in the right direction.

The rewrite should keep the decomposition but strengthen the interface. Each pack should export: site constructors, local solver hooks, backward viability functions, forward output summarizers, residual explanation templates, and transport declarations. This allows Torch support to grow monotonically. Adding `topk`, `gather`, `take_along_dim`, `argmax`, or broadcasting packs should create more local semantic neighborhoods without forcing a global rearchitecture.

The practical target is strong and concrete. For a Torch module, the analyzer should be able to synthesize a useful input contract explaining how to avoid library-specific runtime errors, infer local semantics strong enough to catch values-versus-indices confusion or illegal shape transformations, and emit an output contract describing properties such as sortedness, range safety, output-shape, or selector provenance. This is already a compelling verification story even for users who do not care about full theorem-level proofs.

0.13 General Python, protocols, heaps, classes, and exceptions

Although Torch is the showcase, the architecture must be broader. Native Python programs are full of protocols, attribute access, constructors, mutable objects, control-dependent initialization, and exception-heavy APIs. A sheaf-descent system should therefore treat object and protocol reasoning as first-class local semantics. A constructor body defines sites for field initialization and invariant establishment. Attribute reads and writes define heap/protocol sites. Method dispatch defines protocol-availability sites. Branches or guards that check `is None`, `hasattr`, membership, or length define dynamic-local refinements. Exception edges define error-sensitive sites and possibly exceptional output sections.

This model yields a practical way to support Pythonic object-oriented code without pretending that Python's heap is simple. A function may be safe only if a receiver object satisfies a frame condition or owns a certain initialized field. Instead of encoding the entire heap globally, the system records local heap sections where they matter and glues them through explicit overlaps and frame maps. This keeps the logic modular and helps explanation: when an attribute error remains reachable, the renderer can point to the exact protocol or initialization site that failed to glue.

The existing example corpus already contains files such as `examples/heap_field_mismatch.py`, `examples/protocol_mismatch.py`, and `examples/constructor_usage_bug.py`. The rewritten monograph should treat them not as toy extras but as proof that the same site-based architecture used for Torch can handle ordinary Python safety, protocol conformance, and constructor invariants.

0.14 Trust boundary, solver boundary, and explicit residuals

The sheaf-descent rewrite should be more ambitious semantically but no less strict about trust. Every local solver and every imported proof artifact must advertise its status. The recommended status lattice is still: trusted automatic, bounded automatic, dynamically observed, proof-checked, assumed, and residual. However, the classifier now lives on local sections rather than on standalone atoms. This is important because one local section may mix carriers settled by parsing, arithmetic facts settled by Z3, a witness settled by a theorem module, and an unresolved transport dependency.

The implementation should centralize this classification in the kernel layer. The current proof-boundary logic can be refactored into something like a section-boundary auditor. Whenever a site is solved, the auditor records which coordinates are trusted and which remain residual. During gluing, the global section accumulates this information instead of flattening it. Renderers should therefore be able to say "the output sortedness claim is solver-backed, the permutation witness is imported from a checked theorem module, and the alias- preservation step through a custom wrapper remains

residual." This level of clarity is essential if the system is to support both theorem-oriented users and ordinary developers.

A sheaf semantics also clarifies what not to do. The system must not infer a safe input requirement from purely empirical evidence and then present it as a hard theorem. It may offer that requirement as an observed or suggested boundary section, but the provenance must remain explicit. Similarly, a proof authored by an LLM should count only after elaboration and kernel checking. The fact that a rich local section exists at all does not mean every coordinate of that section has the same trust status.

0.15 Precise implementation blueprint and migration from the current code base

The fastest way to make this rewrite useful is to build it as a disciplined reorganization of the current DepPy code base. The present prototype already contains almost every conceptual ingredient, but they are organized around signatures, atoms, and frontiers rather than site covers and descended sections. A precise migration should therefore proceed in layers.

Layer one introduces an explicit cover artifact. The frontend and surface layers already parse Python, lower CFGs, and build SSA plus lineage. Those outputs should feed a new cover builder that creates site nodes and overlap edges. Layer two enriches theory packs so that each pack builds site families instead of only returning atoms. The existing sort, shape, and indexing packs can be migrated first because they already expose semantically meaningful decomposition points. Layer three adds local solver interfaces and error-site viability interfaces. Layer four replaces or wraps current frontier extraction so that it operates on section summaries. Layer five upgrades renderers to display synthesized input requirements, output guarantees, local-site facts, and residual obstructions. Layer six strengthens proof-surface integration by compiling theorem modules into transport maps and proof sites.

The migration must be incremental. The old `DepPyPipeline` and the current `MonographPipeline` should stay alive while the new site-cover pipeline is introduced in parallel. Every step should come with tests and with corpus examples that exercise exactly one new capability. This is especially important for performance and explanation quality. A site cover can explode if constructed naively. The implementation therefore needs locality heuristics, summary sites, and pack-specific sparsification rules from the beginning.

0.16 Package-by-package implementation blueprint

The rest of this document is deliberately implementation-heavy. The goal is not merely to describe a beautiful semantic object, but to pin that object to concrete packages, data structures, and migration steps. The following subsections map the sheaf-descent architecture package by package.

0.16.1 Package `src/deppy/assist`

Current and continuing responsibility. The package `src/deppy/assist` should continue to own interactive explanation, prompt-shaping, and recommendation helpers. The rewrite does not erase this role; it makes the role more explicit by connecting it to site construction, local section solving, gluing, or rendering.

Role under sheaf-descent semantics. Under the rewrite, the package should Convert obstruction and frontier objects into concise user guidance that points to exact sites, overlaps, and candidate annotations rather than generic advice. This means its public interfaces must be rephrased in terms of site covers, boundary sections, local sections, transports, error viability, or audited frontiers rather than opaque atom sets.

Required additions. Add site-aware explanation templates, support for contrastive explanations between two frontiers, and active-query helpers that ask for the smallest boundary seed resolving a chosen obstruction.

Acceptance criteria. A package migration is complete when tests can exercise its behavior without appealing to the full pipeline. In other words, the package should expose artifacts that are meaningful on their own: a cover builder, a pack-local site constructor, a local solver result, a transport map, a frontier summary, or a renderer output whose provenance is site-aware.

0.16.2 Package `src/deppy/contracts`

Current and continuing responsibility. The package `src/deppy/contracts` should continue to own surface syntax for explicit user contracts. The rewrite does not erase this role; it makes the role more explicit by connecting it to site construction, local section solving, gluing, or rendering.

Role under sheaf-descent semantics. Under the rewrite, the package should Continue hosting requires, ensures, witness, and theorem-facing contract objects, but redefine them as boundary seeds and local section constraints instead of global atom bags. This means its public interfaces must be rephrased in terms of site covers, boundary sections, local sections, transports, error viability, or audited frontiers rather than opaque atom sets.

Required additions. Add explicit input-boundary and output-boundary seed classes, transport-seed descriptors, and provenance fields that record whether a seed came from source text, proof code, or synthesized repair.

Acceptance criteria. A package migration is complete when tests can exercise its behavior without appealing to the full pipeline. In other words, the package should expose artifacts that are meaningful on their own: a cover builder, a pack-local site constructor, a local solver result, a transport map, a frontier summary, or a renderer output whose provenance is site-aware.

0.16.3 Package `src/deppy/core`

Current and continuing responsibility. The package `src/deppy/core` should continue to own semantic datatypes and the stable trusted substrate. The rewrite does not erase this role; it makes the role more explicit by connecting it to site construction, local section solving, gluing, or rendering.

Role under sheaf-descent semantics. Under the rewrite, the package should Own the site, section, boundary, obstruction, and frontier summary data structures that every other package manipulates. This means its public interfaces must be rephrased in terms of site covers, boundary sections, local sections, transports, error viability, or audited frontiers rather than opaque atom sets.

Required additions. Create modules such as `site_cover`, `local_section`, `obstruction_basis`, `boundary_section`, and `frontier_summary` while keeping compatibility shims for current semantic signatures.

Acceptance criteria. A package migration is complete when tests can exercise its behavior without appealing to the full pipeline. In other words, the package should expose artifacts that are meaningful on their own: a cover builder, a pack-local site constructor, a local solver result, a transport map, a frontier summary, or a renderer output whose provenance is site-aware.

0.16.4 Package `src/deppy/frontend`

Current and continuing responsibility. The package `src/deppy/frontend` should continue to own module loading and source normalization. The rewrite does not erase this role; it makes the role more explicit by connecting it to site construction, local section solving, gluing, or rendering.

Role under sheaf-descent semantics. Under the rewrite, the package should Remain responsible for turning Python modules, proof files, and example corpora into normalized function and module models. This means its public interfaces must be rephrased in terms of site covers, boundary sections, local sections, transports, error viability, or audited frontiers rather than opaque atom sets.

Required additions. Expose richer hooks so the cover builder sees decorators, theorem markers, docstring hints, and inline assertions as possible seed sources.

Acceptance criteria. A package migration is complete when tests can exercise its behavior without appealing to the full pipeline. In other words, the package should expose artifacts that are meaningful on their own: a cover builder, a pack-local site constructor, a local solver result, a transport map, a frontier summary, or a renderer output whose provenance is site-aware.

0.16.5 Package `src/deppy/generate`

Current and continuing responsibility. The package `src/deppy/generate` should continue to own candidate and suggestion generation. The rewrite does not erase this role; it makes the role more explicit by connecting it to site construction, local section solving, gluing, or rendering.

Role under sheaf-descent semantics. Under the rewrite, the package should Shift from raw candidate-atom generation to candidate-site and candidate-transport generation. This means its public interfaces must be rephrased in terms of site covers, boundary sections, local sections, transports, error viability, or audited frontiers rather than opaque atom sets.

Required additions. Preserve current candidate-bank ideas but make them produce boundary-seed options, local witness templates, and transport hypotheses ranked by obstruction impact.

Acceptance criteria. A package migration is complete when tests can exercise its behavior without appealing to the full pipeline. In other words, the package should expose artifacts that are meaningful on their own: a cover builder, a pack-local site constructor, a local solver result, a transport map, a frontier summary, or a renderer output whose provenance is site-aware.

0.16.6 Package `src/deppy/harvest`

Current and continuing responsibility. The package `src/deppy/harvest` should continue to own extracting guards and latent semantic hints from source. The rewrite does not erase this role; it makes the role more explicit by connecting it to site construction, local section solving, gluing, or rendering.

Role under sheaf-descent semantics. Under the rewrite, the package should Harvest input-side guards, output-side assertions, loop invariants, protocol checks, and assertion sites from ordinary Python as first-class local evidence. This means its public interfaces must be rephrased in terms of site covers, boundary sections, local sections, transports, error viability, or audited frontiers rather than opaque atom sets.

Required additions. Add structured harvesting of guard scopes, traceability to SSA values, and canonicalization so harvested evidence becomes a section constructor rather than an untyped string.

Acceptance criteria. A package migration is complete when tests can exercise its behavior without appealing to the full pipeline. In other words, the package should expose artifacts that are meaningful on their own: a cover builder, a pack-local site constructor, a local solver result, a transport map, a frontier summary, or a renderer output whose provenance is site-aware.

0.16.7 Package `src/deppy/interprocedural`

Current and continuing responsibility. The package `src/deppy/interprocedural` should continue to own summary propagation across call graphs. The rewrite does not erase this role; it makes the role more explicit by connecting it to site construction, local section solving, gluing, or rendering.

Role under sheaf-descent semantics. Under the rewrite, the package should Promote existing call summaries and frontier propagation into section transport across caller and callee covers. This means its public interfaces must be rephrased in terms of site covers, boundary sections, local sections, transports, error viability, or audited frontiers rather than opaque atom sets.

Required additions. Extend summary objects so they carry boundary sections, selected interior sites, local provenance, and backward error pullbacks.

Acceptance criteria. A package migration is complete when tests can exercise its behavior without appealing to the full pipeline. In other words, the package should expose artifacts that are meaningful on their own: a cover builder, a pack-local site constructor, a local solver result, a transport map, a frontier summary, or a renderer output whose provenance is site-aware.

0.16.8 Package `src/deppy/kernel`

Current and continuing responsibility. The package `src/deppy/kernel` should continue to own trusted classification and checking. The rewrite does not erase this role; it makes the role more explicit by connecting it to site construction, local section solving, gluing, or rendering.

Role under sheaf-descent semantics. Under the rewrite, the package should Centralize section-boundary classification, local proof checking, solver bridges, and residual marking. This means its public interfaces must be rephrased in terms of site covers, boundary sections, local sections, transports, error viability, or audited frontiers rather than opaque atom sets.

Required additions. Introduce a section auditor that classifies each coordinate of a local section, stores proof obligations, and exposes audited gluing results to renderers.

Acceptance criteria. A package migration is complete when tests can exercise its behavior without appealing to the full pipeline. In other words, the package should expose artifacts that are meaningful on their own: a cover builder, a pack-local site constructor, a local solver result, a transport map, a frontier summary, or a renderer output whose provenance is site-aware.

0.16.9 Package `src/deppy/library_theories`

Current and continuing responsibility. The package `src/deppy/library_theories` should continue to own pack-specific site generation and local solvers. The rewrite does not erase this role; it makes the role more explicit by connecting it to site construction, local section solving, gluing, or rendering.

Role under sheaf-descent semantics. Under the rewrite, the package should Remain the home of PyTorch and future library reasoning, but elevate theory packs from atom factories to cover factories. This means its public interfaces must be rephrased in terms of site covers, boundary sections, local sections, transports, error viability, or audited frontiers rather than opaque atom sets.

Required additions. Each pack should export site constructors, viability predicates, forward summarizers, local solver adapters, and transport declarations.

Acceptance criteria. A package migration is complete when tests can exercise its behavior without appealing to the full pipeline. In other words, the package should expose artifacts that are meaningful on their own: a cover builder, a pack-local site constructor, a local solver result, a transport map, a frontier summary, or a renderer output whose provenance is site-aware.

0.16.10 Package `src/deppy/native_python`

Current and continuing responsibility. The package `src/deppy/native_python` should continue to own Pythonic user surface and diagnostics. The rewrite does not erase this role; it makes the role more explicit by connecting it to site construction, local section solving, gluing, or rendering.

Role under sheaf-descent semantics. Under the rewrite, the package should Own lightweight decorator APIs, diagnostics integration, and user-facing translations of boundary sections into native-Python syntax. This means its public interfaces must be rephrased in terms of site covers, boundary sections, local sections, transports, error viability, or audited frontiers rather than opaque atom sets.

Required additions. Add utilities that render synthesized requires and ensures clauses, optional theorem stubs, and precise runtime-error narratives into forms familiar to Python developers.

Acceptance criteria. A package migration is complete when tests can exercise its behavior without appealing to the full pipeline. In other words, the package should expose artifacts that are meaningful on their own: a cover builder, a pack-local site constructor, a local solver result, a transport map, a frontier summary, or a renderer output whose provenance is site-aware.

0.16.11 Package `src/deppy/normalize`

Current and continuing responsibility. The package `src/deppy/normalize` should continue to own canonical expression and contract normalization. The rewrite does not erase this role; it makes the role more explicit by connecting it to site construction, local section solving, gluing, or rendering.

Role under sheaf-descent semantics. Under the rewrite, the package should Provide canonicalized expressions, lineage-aware names, theorem skeleton normalization, and error-site normalization. This means its public interfaces must be rephrased in terms of site covers, boundary sections, local sections, transports, error viability, or audited frontiers rather than opaque atom sets.

Required additions. The success criterion is that theory packs, error sites, and proof transport all consume one normalization vocabulary instead of ad hoc AST walkers.

Acceptance criteria. A package migration is complete when tests can exercise its behavior without appealing to the full pipeline. In other words, the package should expose artifacts that are meaningful on their own: a cover builder, a pack-local site constructor, a local solver result, a transport map, a frontier summary, or a renderer output whose provenance is site-aware.

0.16.12 Package `src/deppy/observe`

Current and continuing responsibility. The package `src/deppy/observe` should continue to own dynamic trace ingestion and empirical contraction. The rewrite does not erase this role; it makes the role more explicit by connecting it to site construction, local section solving, gluing, or rendering.

Role under sheaf-descent semantics. Under the rewrite, the package should Treat traces, logged tensor shapes, observed branch outcomes, and replayed assertions as trace-site constructors and local section contractors. This means its public interfaces must be rephrased in terms of site covers, boundary sections, local sections, transports, error viability, or audited frontiers rather than opaque atom sets.

Required additions. Add explicit provenance tags so empirical contractions are useful without being confused with proof-backed facts.

Acceptance criteria. A package migration is complete when tests can exercise its behavior without appealing to the full pipeline. In other words, the package should expose artifacts that are meaningful on their own: a cover builder, a pack-local site constructor, a local solver result, a transport map, a frontier summary, or a renderer output whose provenance is site-aware.

0.16.13 Package `src/deppy/optimize`

Current and continuing responsibility. The package `src/deppy/optimize` should continue to own frontiers, repair, and cost-aware search. The rewrite does not erase this role; it makes the role more explicit by connecting it to site construction, local section solving, gluing, or rendering.

Role under sheaf-descent semantics. Under the rewrite, the package should Reframe optimization over summaries of descended sections and obstruction-resolving actions. This means its public interfaces must be rephrased in terms of site covers, boundary sections, local sections, transports, error viability, or audited frontiers rather than opaque atom sets.

Required additions. Existing MaxSMT or enumeration machinery should be retained where useful, but the optimized object becomes a frontier point over input sections, output sections, and residual debt.

Acceptance criteria. A package migration is complete when tests can exercise its behavior without appealing to the full pipeline. In other words, the package should expose artifacts that are meaningful on their own: a cover builder, a pack-local site constructor, a local solver result, a transport map, a frontier summary, or a renderer output whose provenance is site-aware.

0.16.14 Package `src/deppy/predicates`

Current and continuing responsibility. The package `src/deppy/predicates` should continue to own reusable logical vocabularies. The rewrite does not erase this role; it makes the role more explicit by connecting it to site construction, local section solving, gluing, or rendering.

Role under sheaf-descent semantics. Under the rewrite, the package should Keep reusable predicates, but attach them to site families and local logic descriptors rather than storing them as context-free strings. This means its public interfaces must be rephrased in terms of site covers, boundary sections, local sections, transports, error viability, or audited frontiers rather than opaque atom sets.

Required additions. This package should become the vocabulary layer from which packs and renderers derive printable contracts.

Acceptance criteria. A package migration is complete when tests can exercise its behavior without appealing to the full pipeline. In other words, the package should expose artifacts that are meaningful on their own: a cover builder, a pack-local site constructor, a local solver result, a transport map, a frontier summary, or a renderer output whose provenance is site-aware.

0.16.15 Package `src/deppy/proof`

Current and continuing responsibility. The package `src/deppy/proof` should continue to own theorem-oriented APIs and proof transport compilation. The rewrite does not erase this role; it makes the role more explicit by connecting it to site construction, local section solving, gluing, or rendering.

Role under sheaf-descent semantics. Under the rewrite, the package should Compile theorem modules, witness combinators, and explicit proof annotations into proof sites and transport maps. This means its public interfaces must be rephrased in terms of site covers, boundary sections, local sections, transports, error viability, or audited frontiers rather than opaque atom sets.

Required additions. Implement proof elaboration as site construction plus kernel obligations instead of as direct fact insertion.

Acceptance criteria. A package migration is complete when tests can exercise its behavior without appealing to the full pipeline. In other words, the package should expose artifacts that are meaningful on their own: a cover builder, a pack-local site constructor, a local solver result, a transport map, a frontier summary, or a renderer output whose provenance is site-aware.

0.16.16 Package `src/deppy/render`

Current and continuing responsibility. The package `src/deppy/render` should continue to own contract, diagnostic, and proof-goal views. The rewrite does not erase this role; it makes the role more explicit by connecting it to site construction, local section solving, gluing, or rendering.

Role under sheaf-descent semantics. Under the rewrite, the package should Render synthesized input requirements, output guarantees, local-site sections, error-site reports, and obstruction explanations in multiple user-facing forms. This means its public interfaces must be rephrased in terms of site covers, boundary sections, local sections, transports, error viability, or audited frontiers rather than opaque atom sets.

Required additions. Add sheaf-aware views such as overlap conflicts, site-level provenance cards, and frontier comparisons.

Acceptance criteria. A package migration is complete when tests can exercise its behavior without appealing to the full pipeline. In other words, the package should expose artifacts that are meaningful on their own: a cover builder, a pack-local site constructor, a local solver result, a transport map, a frontier summary, or a renderer output whose provenance is site-aware.

0.16.17 Package `src/deppy/solver`

Current and continuing responsibility. The package `src/deppy/solver` should continue to own bridges to SMT and other local engines. The rewrite does not erase this role; it makes the role more explicit by connecting it to site construction, local section solving, gluing, or rendering.

Role under sheaf-descent semantics. Under the rewrite, the package should Stay local: solve arithmetic, shape, and finite witness subproblems attached to specific site families. This means its public interfaces must be rephrased in terms of site covers, boundary sections, local sections, transports, error viability, or audited frontiers rather than opaque atom sets.

Required additions. Expose per-site kernels and explicit failure reasons so gluing can distinguish solver failure from genuine semantic ambiguity.

Acceptance criteria. A package migration is complete when tests can exercise its behavior without appealing to the full pipeline. In other words, the package should expose artifacts that are meaningful on their own: a cover builder, a pack-local site constructor, a local solver result, a transport map, a frontier summary, or a renderer output whose provenance is site-aware.

0.16.18 Package `src/deppy/static_analysis`

Current and continuing responsibility. The package `src/deppy/static_analysis` should continue to own supporting classical analyses reused by cover synthesis. The rewrite does not erase this role; it makes the role more explicit by connecting it to site construction, local section solving, gluing, or rendering.

Role under sheaf-descent semantics. Under the rewrite, the package should Retain conventional analyses that feed the cover, such as call graph construction, simple data flow, and control-dependence summaries. This means its public interfaces must be rephrased in terms of site covers, boundary sections, local sections, transports, error viability, or audited frontiers rather than opaque atom sets.

Required additions. The key addition is stable identifiers that let site construction and renderers refer back to traditional analysis results.

Acceptance criteria. A package migration is complete when tests can exercise its behavior without appealing to the full pipeline. In other words, the package should expose artifacts that are meaningful on their own: a cover builder, a pack-local site constructor, a local solver result, a transport map, a frontier summary, or a renderer output whose provenance is site-aware.

0.16.19 Package `src/deppy/surface`

Current and continuing responsibility. The package `src/deppy/surface` should continue to own ordinary Python and proof-surface lowering. The rewrite does not erase this role; it makes the role more explicit by connecting it to site construction, local section solving, gluing, or rendering.

Role under sheaf-descent semantics. Under the rewrite, the package should Remain the main lowering entry point from syntax into CFG, SSA, heap-SSA, proof syntax, and now site covers. This means its public interfaces must be rephrased in terms of site covers, boundary sections, local sections, transports, error viability, or audited frontiers rather than opaque atom sets.

Required additions. This package should expose a cover builder API built on top of current AST, CFG, and SSA datatypes so tests can target it directly.

Acceptance criteria. A package migration is complete when tests can exercise its behavior without appealing to the full pipeline. In other words, the package should expose artifacts that are meaningful on their own: a cover builder, a pack-local site constructor, a local solver result, a transport map, a frontier summary, or a renderer output whose provenance is site-aware.

0.16.20 Package `src/deppy/types`

Current and continuing responsibility. The package `src/deppy/types` should continue to own base carrier and type-language definitions. The rewrite does not erase this role; it makes the role more explicit by connecting it to site construction, local section solving, gluing, or rendering.

Role under sheaf-descent semantics. Under the rewrite, the package should Hold the base carrier language, indexed families, witness-bearing result forms, and nominal/structural type descriptors used by local sections. This means its public interfaces must be rephrased in terms of site covers, boundary sections, local sections, transports, error viability, or audited frontiers rather than opaque atom sets.

Required additions. The rewrite should distinguish carrier types from local section refinements, allowing the same carrier to participate in many site-specific sections.

Acceptance criteria. A package migration is complete when tests can exercise its behavior without appealing to the full pipeline. In other words, the package should expose artifacts that are meaningful on their own: a cover builder, a pack-local site constructor, a local solver result, a transport map, a frontier summary, or a renderer output whose provenance is site-aware.

0.17 Site-family catalogue

The site-family catalogue below is implementation guidance. Each family should have an explicit constructor, a local carrier schema, and clear rules for backward and forward propagation.

0.17.1 Argument-boundary sites

Program fragment represented. Argument-boundary sites represent formal parameters, receiver objects, ambient module state, and optional ghost inputs. They matter because the cover must preserve the places where meaning becomes locally obvious or locally dangerous.

Primary semantic purpose. Their main role is input admissibility and safe precondition synthesis. In the sheaf-descent view a site family is justified only if it reduces annotation demand, improves runtime-safety prediction, or enables stronger output synthesis.

Concrete representation. The natural implementation home is surface, contracts, and harvest layers. The local payload should contain at least parameter carriers, harvested guards, protocol assumptions, and call-context projections together with provenance back to source and an audited trust classification.

Backward and forward behavior. Every family must specify how viability requirements pull back toward the input boundary and how solved local sections push forward toward outputs. Families that do neither are not mature enough for inclusion in the core architecture.

0.17.2 Return and output-boundary sites

Program fragment represented. Return and output-boundary sites represent returns, raised exceptions, mutated heap regions, and exported witnesses. They matter because the cover must preserve the places where meaning becomes locally obvious or locally dangerous.

Primary semantic purpose. Their main role is maximal output refinement synthesis. In the sheaf-descent view a site family is justified only if it reduces annotation demand, improves runtime-safety prediction, or enables stronger output synthesis.

Concrete representation. The natural implementation home is core and render layers. The local payload should contain at least boundary sections, postcondition summaries, and exceptional outputs together with provenance back to source and an audited trust classification.

Backward and forward behavior. Every family must specify how viability requirements pull back toward the input boundary and how solved local sections push forward toward outputs. Families that do neither are not mature enough for inclusion in the core architecture.

0.17.3 SSA-value sites

Program fragment represented. SSA-value sites represent versioned locals, tuple projections, alias chains, and phi merges. They matter because the cover must preserve the places where meaning becomes locally obvious or locally dangerous.

Primary semantic purpose. Their main role is local intermediate typing and wrapper transparency. In the sheaf-descent view a site family is justified only if it reduces annotation demand, improves runtime-safety prediction, or enables stronger output synthesis.

Concrete representation. The natural implementation home is `surface/python_ssa.py` and interprocedural transport. The local payload should contain at least carrier types, lineage identifiers, local refinements, and overlap maps together with provenance back to source and an audited trust classification.

Backward and forward behavior. Every family must specify how viability requirements pull back toward the input boundary and how solved local sections push forward toward outputs. Families that do neither are not mature enough for inclusion in the core architecture.

0.17.4 Branch-guard sites

Program fragment represented. Branch-guard sites represent control-dependent refinements introduced by if-statements, assertions, and match-like structures. They matter because the cover must preserve the places where meaning becomes locally obvious or locally dangerous.

Primary semantic purpose. Their main role is guard harvesting and local narrowing. In the sheaf-descent view a site family is justified only if it reduces annotation demand, improves runtime-safety prediction, or enables stronger output synthesis.

Concrete representation. The natural implementation home is harvest plus `static_analysis`. The local payload should contain at least predicate implications, branch polarity, and merge-side obstruction hints together with provenance back to source and an audited trust classification.

Backward and forward behavior. Every family must specify how viability requirements pull back toward the input boundary and how solved local sections push forward toward outputs. Families that do neither are not mature enough for inclusion in the core architecture.

0.17.5 Call-result sites

Program fragment represented. Call-result sites represent semantic neighborhoods created at function and method calls. They matter because the cover must preserve the places where meaning becomes locally obvious or locally dangerous.

Primary semantic purpose. Their main role is summary import and error propagation. In the sheaf-descent view a site family is justified only if it reduces annotation demand, improves runtime-safety prediction, or enables stronger output synthesis.

Concrete representation. The natural implementation home is interprocedural summaries and library packs. The local payload should contain at least callee boundary import, actual-to-formal reindexing, and local result sections together with provenance back to source and an audited trust classification.

Backward and forward behavior. Every family must specify how viability requirements pull back toward the input boundary and how solved local sections push forward toward outputs. Families that do neither are not mature enough for inclusion in the core architecture.

0.17.6 Tensor-shape sites

Program fragment represented. Tensor-shape sites represent rank, shape tuple, broadcasting, reshape/view legality, and reduction preconditions. They matter because the cover must preserve the places where meaning becomes locally obvious or locally dangerous.

Primary semantic purpose. Their main role is Torch-specific safety and rich tensor outputs. In the sheaf-descent view a site family is justified only if it reduces annotation demand, improves runtime-safety prediction, or enables stronger output synthesis.

Concrete representation. The natural implementation home is `library_theories/pytorch_shapes.py` and solver bridges. The local payload should contain at least shape equations, cardinality facts, rank bounds, and view legality witnesses together with provenance back to source and an audited trust classification.

Backward and forward behavior. Every family must specify how viability requirements pull back toward the input boundary and how solved local sections push forward toward outputs. Families that do neither are not mature enough for inclusion in the core architecture.

0.17.7 Tensor-order sites

Program fragment represented. Tensor-order sites represent sortedness, argsort, permutation, monotone slices, and value-index coupling. They matter because the cover must preserve the places where meaning becomes locally obvious or locally dangerous.

Primary semantic purpose. Their main role is semantics around sort-like operators. In the sheaf-descent view a site family is justified only if it reduces annotation demand, improves runtime-safety prediction, or enables stronger output synthesis.

Concrete representation. The natural implementation home is `library_theories/pytorch_sort.py`. The local payload should contain at least order predicates, permutation witnesses, and transport hooks to selectors together with provenance back to source and an audited trust classification.

Backward and forward behavior. Every family must specify how viability requirements pull back toward the input boundary and how solved local sections push forward toward outputs. Families that do neither are not mature enough for inclusion in the core architecture.

0.17.8 Tensor-indexing sites

Program fragment represented. Tensor-indexing sites represent selectors, slices, gather/index-select domains, and source-target relations. They matter because the cover must preserve the places where meaning becomes locally obvious or locally dangerous.

Primary semantic purpose. Their main role is index safety and provenance-preserving outputs. In the sheaf-descent view a site family is justified only if it reduces annotation demand, improves runtime-safety prediction, or enables stronger output synthesis.

Concrete representation. The natural implementation home is `library_theories/pytorch_indexing.py`. The local payload should contain at least selector-domain sections, source extent constraints, and target-origin facts together with provenance back to source and an audited trust classification.

Backward and forward behavior. Every family must specify how viability requirements pull back toward the input boundary and how solved local sections push forward toward outputs. Families that do neither are not mature enough for inclusion in the core architecture.

0.17.9 Heap/protocol sites

Program fragment represented. Heap/protocol sites represent attribute existence, field initialization, alias frames, protocol method availability, and ownership. They matter because the cover must preserve the places where meaning becomes locally obvious or locally dangerous.

Primary semantic purpose. Their main role is ordinary Python object safety. In the sheaf-descent view a site family is justified only if it reduces annotation demand, improves runtime-safety prediction, or enables stronger output synthesis.

Concrete representation. The natural implementation home is surface, predicates, and kernel support. The local payload should contain at least field maps, protocol obligations, frame conditions, and constructor invariants together with provenance back to source and an audited trust classification.

Backward and forward behavior. Every family must specify how viability requirements pull back toward the input boundary and how solved local sections push forward toward outputs. Families that do neither are not mature enough for inclusion in the core architecture.

0.17.10 Proof sites

Program fragment represented. Proof sites represent theorem declarations, lemma bodies, witness exports, decreases obligations, and transport seeds. They matter because the cover must preserve the places where meaning becomes locally obvious or locally dangerous.

Primary semantic purpose. Their main role is F*-like capabilities in native Python. In the sheaf-descent view a site family is justified only if it reduces annotation demand, improves runtime-safety prediction, or enables stronger output synthesis.

Concrete representation. The natural implementation home is proof and kernel packages. The local payload should contain at least proof obligations, witness carriers, and checked transport maps together with provenance back to source and an audited trust classification.

Backward and forward behavior. Every family must specify how viability requirements pull back toward the input boundary and how solved local sections push forward toward outputs. Families that do neither are not mature enough for inclusion in the core architecture.

0.17.11 Trace sites

Program fragment represented. Trace sites represent observed shapes, branch outcomes, assertion passes, and sampled value relations. They matter because the cover must preserve the places where meaning becomes locally obvious or locally dangerous.

Primary semantic purpose. Their main role is empirical contraction of ambiguity classes. In the sheaf-descent view a site family is justified only if it reduces annotation demand, improves runtime-safety prediction, or enables stronger output synthesis.

Concrete representation. The natural implementation home is observe package. The local payload should contain at least observation sets, provenance labels, and non-theorem trust markers together with provenance back to source and an audited trust classification.

Backward and forward behavior. Every family must specify how viability requirements pull back toward the input boundary and how solved local sections push forward toward outputs. Families that do neither are not mature enough for inclusion in the core architecture.

0.17.12 Error sites

Program fragment represented. Error sites represent partial operations or exceptional transitions that may fail. They matter because the cover must preserve the places where meaning becomes locally obvious or locally dangerous.

Primary semantic purpose. Their main role is runtime-error prediction and safe-input synthesis. In the sheaf-descent view a site family is justified only if it reduces annotation demand, improves runtime-safety prediction, or enables stronger output synthesis.

Concrete representation. The natural implementation home is kernel error catalog and packs. The local payload should contain at least viability predicates, explanation templates, and upstream pullback rules together with provenance back to source and an audited trust classification.

Backward and forward behavior. Every family must specify how viability requirements pull back toward the input boundary and how solved local sections push forward toward outputs. Families that do neither are not mature enough for inclusion in the core architecture.

0.17.13 Loop-header and invariant sites

Program fragment represented. Loop-header and invariant sites represent repeated states at loop heads, candidate invariants, and ranking data. They matter because the cover must preserve the places where meaning becomes locally obvious or locally dangerous.

Primary semantic purpose. Their main role is local-to-global reasoning for iterative algorithms. In the sheaf-descent view a site family is justified only if it reduces annotation demand, improves runtime-safety prediction, or enables stronger output synthesis.

Concrete representation. The natural implementation home is surface, proof, and optimize. The local payload should contain at least candidate invariant sections, decreases facts, and merge compatibility constraints together with provenance back to source and an audited trust classification.

Backward and forward behavior. Every family must specify how viability requirements pull back toward the input boundary and how solved local sections push forward toward outputs. Families that do neither are not mature enough for inclusion in the core architecture.

0.17.14 Module-summary sites

Program fragment represented. Module-summary sites represent module-wide exports, public API contracts, and cached summaries. They matter because the cover must preserve the places where meaning becomes locally obvious or locally dangerous.

Primary semantic purpose. Their main role is cross-module reuse and distribution. In the sheaf-descent view a site family is justified only if it reduces annotation demand, improves runtime-safety prediction, or enables stronger output synthesis.

Concrete representation. The natural implementation home is pipeline and render packages. The local payload should contain at least public boundary sections, exposed residuals, and reusable transport assets together with provenance back to source and an audited trust classification.

Backward and forward behavior. Every family must specify how viability requirements pull back toward the input boundary and how solved local sections push forward toward outputs. Families that do neither are not mature enough for inclusion in the core architecture.

0.18 Theory-family catalogue

Theories remain open-world and additive, but the unit of addition is now a site-aware local calculus rather than an atom bank.

0.18.1 Linear arithmetic and intervals

Logical vocabulary. This theory family should cover non-negativity, bounds, lengths, simple affine equalities. The vocabulary must be printable to users, normalizable for local solving, and explicit about provenance.

Program evidence source. It is triggered primarily by argument bounds, list lengths, loop counters, and tensor extents. The trigger mechanism should use the shared normalization and lineage vocabulary so that aliasing and wrappers do not erase semantics.

Local solving discipline. The recommended local engine is local SMT over arithmetic fragments. Even when heuristic ranking or proof suggestions help, the trust boundary should be expressed through a precise local kernel judgment.

Why it belongs in the core story. The semantic payoff is IndexError, ZeroDivisionError, and shape-side precondition pullback. A theory family that cannot influence safe input synthesis, maximal output synthesis, local typing quality, or residual explanation quality should be treated as optional rather than foundational.

0.18.2 Boolean guard theory

Logical vocabulary. This theory family should cover guard polarity, assertion scope, and branch-conditioned refinement. The vocabulary must be printable to users, normalizable for local solving, and explicit about provenance.

Program evidence source. It is triggered primarily by if-statements, asserts, early returns, and exception guards. The trigger mechanism should use the shared normalization and lineage vocabulary so that aliasing and wrappers do not erase semantics.

Local solving discipline. The recommended local engine is normalization plus implication checks. Even when heuristic ranking or proof suggestions help, the trust boundary should be expressed through a precise local kernel judgment.

Why it belongs in the core story. The semantic payoff is control-dependent safety and lightweight proof seed extraction. A theory family that cannot influence safe input synthesis, maximal output synthesis, local typing quality, or residual explanation quality should be treated as optional rather than foundational.

0.18.3 Nullability and optionality

Logical vocabulary. This theory family should cover None checks, existence guards, and optional protocol availability. The vocabulary must be printable to users, normalizable for local solving, and explicit about provenance.

Program evidence source. It is triggered primarily by ordinary Python branches and constructor code. The trigger mechanism should use the shared normalization and lineage vocabulary so that aliasing and wrappers do not erase semantics.

Local solving discipline. The recommended local engine is simple symbolic narrowing. Even when heuristic ranking or proof suggestions help, the trust boundary should be expressed through a precise local kernel judgment.

Why it belongs in the core story. The semantic payoff is AttributeError prevention and stronger post-branch outputs. A theory family that cannot influence safe input synthesis, maximal output synthesis, local typing quality, or residual explanation quality should be treated as optional rather than foundational.

0.18.4 Heap and protocol theory

Logical vocabulary. This theory family should cover field initialization, protocol method existence, frame conditions, and alias exclusions. The vocabulary must be printable to users, normalizable for local solving, and explicit about provenance.

Program evidence source. It is triggered primarily by classes, protocols, mutable objects, and resource wrappers. The trigger mechanism should use the shared normalization and lineage vocabulary so that aliasing and wrappers do not erase semantics.

Local solving discipline. The recommended local engine is bounded symbolic checks plus explicit residuals. Even when heuristic ranking or proof suggestions help, the trust boundary should be expressed through a precise local kernel judgment.

Why it belongs in the core story. The semantic payoff is constructor safety, attribute access, and method-dispatch correctness. A theory family that cannot influence safe input synthesis, maximal output synthesis, local typing quality, or residual explanation quality should be treated as optional rather than foundational.

0.18.5 Tensor shape theory

Logical vocabulary. This theory family should cover rank, dimension, shape tuples, cardinality, broadcast legality, and reduction support. The vocabulary must be printable to users, normalizable for local solving, and explicit about provenance.

Program evidence source. It is triggered primarily by Torch tensor operators and shape-sensitive wrappers. The trigger mechanism should use the shared normalization and lineage vocabulary so that aliasing and wrappers do not erase semantics.

Local solving discipline. The recommended local engine is solver-backed local shape kernels. Even when heuristic ranking or proof suggestions help, the trust boundary should be expressed through a precise local kernel judgment.

Why it belongs in the core story. The semantic payoff is invalid reshape, broadcast mismatch, and exact output-shape synthesis. A theory family that cannot influence safe input synthesis, maximal output synthesis, local typing quality, or residual explanation quality should be treated as optional rather than foundational.

0.18.6 Tensor order theory

Logical vocabulary. This theory family should cover sortedness, monotonicity, permutation witnesses, argsort relations. The vocabulary must be printable to users, normalizable for local solving, and explicit about provenance.

Program evidence source. It is triggered primarily by sort-like operators and order-preserving wrappers. The trigger mechanism should use the shared normalization and lineage vocabulary so that aliasing and wrappers do not erase semantics.

Local solving discipline. The recommended local engine is finite witness templates plus solver-backed subchecks. Even when heuristic ranking or proof suggestions help, the trust boundary should be expressed through a precise local kernel judgment.

Why it belongs in the core story. The semantic payoff is values-vs-indices confusion and richer sorted outputs. A theory family that cannot influence safe input synthesis, maximal output synthesis, local typing quality, or residual explanation quality should be treated as optional rather than foundational.

0.18.7 Tensor indexing theory

Logical vocabulary. This theory family should cover selector domains, slice extents, gather/index-select legality, and value provenance. The vocabulary must be printable to users, normalizable for local solving, and explicit about provenance.

Program evidence source. It is triggered primarily by subscripts, slices, and advanced indexing. The trigger mechanism should use the shared normalization and lineage vocabulary so that aliasing and wrappers do not erase semantics.

Local solving discipline. The recommended local engine is arithmetic plus pack-specific rules. Even when heuristic ranking or proof suggestions help, the trust boundary should be expressed through a precise local kernel judgment.

Why it belongs in the core story. The semantic payoff is out-of-range selector detection and source-target output relations. A theory family that cannot influence safe input synthesis, maximal output synthesis, local typing quality, or residual explanation quality should be treated as optional rather than foundational.

0.18.8 Device and dtype compatibility theory

Logical vocabulary. This theory family should cover device equality, admissible implicit casts, and operator support matrices. The vocabulary must be printable to users, normalizable for local solving, and explicit about provenance.

Program evidence source. It is triggered primarily by mixed-device or mixed-dtype Torch programs. The trigger mechanism should use the shared normalization and lineage vocabulary so that aliasing and wrappers do not erase semantics.

Local solving discipline. The recommended local engine is table-driven local checks. Even when heuristic ranking or proof suggestions help, the trust boundary should be expressed through a precise local kernel judgment.

Why it belongs in the core story. The semantic payoff is device mismatch and unsupported operator combinations. A theory family that cannot influence safe input synthesis, maximal output synthesis, local typing quality, or residual explanation quality should be treated as optional rather than foundational.

0.18.9 Sequence and collection theory

Logical vocabulary. This theory family should cover non-emptiness, membership, stable length relations, and head/tail legality. The vocabulary must be printable to users, normalizable for local solving, and explicit about provenance.

Program evidence source. It is triggered primarily by lists, tuples, deques, and iterator adapters. The trigger mechanism should use the shared normalization and lineage vocabulary so that aliasing and wrappers do not erase semantics.

Local solving discipline. The recommended local engine is simple algebraic checks. Even when heuristic ranking or proof suggestions help, the trust boundary should be expressed through a precise local kernel judgment.

Why it belongs in the core story. The semantic payoff is head-of-empty and unpacking safety. A theory family that cannot influence safe input synthesis, maximal output synthesis, local typing quality, or residual explanation quality should be treated as optional rather than foundational.

0.18.10 Exception theory

Logical vocabulary. This theory family should cover exceptional output channels, throw sites, and caught-vs-uncaught distinctions. The vocabulary must be printable to users, normalizable for local solving, and explicit about provenance.

Program evidence source. It is triggered primarily by try/except blocks, library raises, and proof obligations that may fail. The trigger mechanism should use the shared normalization and lineage vocabulary so that aliasing and wrappers do not erase semantics.

Local solving discipline. The recommended local engine is control-flow plus error-site hooks. Even when heuristic ranking or proof suggestions help, the trust boundary should be expressed through a precise local kernel judgment.

Why it belongs in the core story. The semantic payoff is precise reporting of exceptional contracts and remaining failures. A theory family that cannot influence safe input synthesis, maximal output synthesis, local typing quality, or residual explanation quality should be treated as optional rather than foundational.

0.18.11 Witness and equality transport theory

Logical vocabulary. This theory family should cover dependent pairs, equality witnesses, and explicit transports. The vocabulary must be printable to users, normalizable for local solving, and explicit about provenance.

Program evidence source. It is triggered primarily by proof-oriented code and witness-returning helpers. The trigger mechanism should use the shared normalization and lineage vocabulary so that aliasing and wrappers do not erase semantics.

Local solving discipline. The recommended local engine is proof-kernel backed local checking. Even when heuristic ranking or proof suggestions help, the trust boundary should be expressed through a precise local kernel judgment.

Why it belongs in the core story. The semantic payoff is proof reuse and output enrichment. A theory family that cannot influence safe input synthesis, maximal output synthesis, local typing quality, or residual explanation quality should be treated as optional rather than foundational.

0.18.12 Loop invariant and decreases theory

Logical vocabulary. This theory family should cover candidate invariants, ranking relations, and preservation constraints. The vocabulary must be printable to users, normalizable for local solving, and explicit about provenance.

Program evidence source. It is triggered primarily by loop-heavy algorithms and recursive helpers. The trigger mechanism should use the shared normalization and lineage vocabulary so that aliasing and wrappers do not erase semantics.

Local solving discipline. The recommended local engine is template generation plus bounded checks. Even when heuristic ranking or proof suggestions help, the trust boundary should be expressed through a precise local kernel judgment.

Why it belongs in the core story. The semantic payoff is F*-like capability without abandoning native Python. A theory family that cannot influence safe input synthesis, maximal output synthesis, local typing quality, or residual explanation quality should be treated as optional rather than foundational.

0.19 Algorithmic realization

The algorithms below should exist as explicit pipeline stages, each with standalone tests and telemetry.

0.19.1 Cover synthesis

Algorithm 1 Sheaf-descent stage: Cover synthesis

1: Parse the module and lower every function to CFG plus SSA plus heap-sensitive summaries.
2: Create site nodes for arguments, returns, SSA values, branch guards, calls, library triggers, proof artifacts, traces, and error-sensitive operations.
3: Insert overlap edges whenever two sites share lineage, control provenance, or pack-declared transport structure.
4: Audit the resulting cover for sparsity so that low-value sites can be summarized before local solving begins.

Implementation note. The stage *Cover synthesis* should emit a durable artifact rather than merely mutating hidden pipeline state. This is essential for debugging, benchmarking, and regression testing against the existing example corpus.

0.19.2 Local solving

Algorithm 2 Sheaf-descent stage: Local solving

1: Dispatch each site to its declared local solver or checker.
2: Store both solved coordinates and unsolved residual coordinates with trust classification.
3: Normalize witness objects so that downstream transports can consume them without pack-specific decoding.
4: Emit viability information for every error-sensitive or partially defined local site.

Implementation note. The stage *Local solving* should emit a durable artifact rather than merely mutating hidden pipeline state. This is essential for debugging, benchmarking, and regression testing against the existing example corpus.

0.19.3 Backward safe-input synthesis

Algorithm 3 Sheaf-descent stage: Backward safe-input synthesis

1: Initialize a requirement set from every nontrivial error-sensitive site and every user-prioritized output guarantee that depends on an input-side support condition.
2: Pull those requirements backward through overlap and restriction maps toward the input boundary.
3: Meet the pulled-back requirements in the input lattice and detect inconsistency as an obstruction.
4: Render the surviving meet as synthesized requires-style contracts plus a provenance report.

Implementation note. The stage *Backward safe-input synthesis* should emit a durable artifact rather than merely mutating hidden pipeline state. This is essential for debugging, benchmarking, and regression testing against the existing example corpus.

0.19.4 Forward output synthesis

Algorithm 4 Sheaf-descent stage: Forward output synthesis

1: Solve local sites under the synthesized safe input section.
2: Propagate solved sections forward through overlaps and wrapper transparencies.
3: Compute maximal output summaries consistent with the global glueing constraints.
4: Render postconditions, witness exports, local-site facts, and residuals separately by trust class.

Implementation note. The stage *Forward output synthesis* should emit a durable artifact rather than merely mutating hidden pipeline state. This is essential for debugging, benchmarking, and regression testing against the existing example corpus.

0.19.5 Obstruction extraction

Algorithm 5 Sheaf-descent stage: Obstruction extraction

1: When gluing fails or remains ambiguous, collect the precise overlaps whose restrictions disagree or whose transports remain unproven.
2: Compress these disagreements into an obstruction basis that is small enough to explain to users.
3: Rank candidate seeds, proofs, or theory activations by how many obstruction coordinates they resolve.
4: Expose the ranked repairs through the assist and render layers.

Implementation note. The stage *Obstruction extraction* should emit a durable artifact rather than merely mutating hidden pipeline state. This is essential for debugging, benchmarking, and regression testing against the existing example corpus.

0.19.6 Interprocedural section transport

Algorithm 6 Sheaf-descent stage: Interprocedural section transport

1: Import callee boundary and selected interior sections at call sites using actual-to-formal reindexing.
2: Push back callee error viability to caller inputs when the caller has not already enforced stronger conditions.
3: Track alias-forwarding wrappers via SSA lineage so that helper structure does not erase semantics.
4: Cache imported summaries at module-summary sites for reuse and benchmarking.

Implementation note. The stage *Interprocedural section transport* should emit a durable artifact rather than merely mutating hidden pipeline state. This is essential for debugging, benchmarking, and regression testing against the existing example corpus.

0.20 Native-Python contract and proof surface

A practical system needs a surface story that ordinary Python developers can recognize. The best design is sparse and layered: zero-annotation mode should be fully supported; decorator-based seeds should be optional; and theorem-style helpers should be available for users who want them.

```
import deppy
import torch

@deppy.requires(lambda scores: scores.dim() == 1)
@deppy.ensures(lambda result, scores: deppy.sorted_tensor(result.values))
def rank_scores(scores: torch.Tensor):
    values, idx = torch.sort(scores)
    return values, idx
```

The rewrite, however, should make clear that such decorators are only one way to seed the cover. The same module should be analyzable without them, perhaps using only local assertions or representative traces. For proof-oriented users, the surface may look like the following.

```
import deppy

@deppy.theorem
def sort_projection_transport(xs):
    values, idx = torch.sort(xs)
    deppy.assume(deppy.sorted_tensor(values))
    return deppy.transport(
        source=deppy.sorted_tensor(values),
        target=deppy.argsort_relation(idx, xs),
    )
```

The theorem syntax remains native Python, but the elaborator should convert it into proof sites, local obligations, and transport objects. Users who never want such features can ignore them entirely. Users who do want them should receive leverage from surprisingly small annotations because the proof artifacts change future descent rather than merely satisfying one immediate goal.

0.21 Example corpus and benchmark expectations

The existing example corpus is a strategic asset. The rewrite should use it as a standing regression suite for semantic ambition, not just for parser correctness.

0.21.1 `examples/protocol_mismatch.py`

Primary focus. This example stresses structural protocol availability and attribute safety. It is valuable because it forces the cover, theory packs, and renderer to cooperate on a task that ordinary Python users actually care about.

Desired success condition. The analyzer should synthesize receiver-side requirements or report the exact protocol site that remains unglued. The example should produce synthesized boundary contracts, local-site sections, and residual explanations that are stable enough to be snapshot-tested.

0.21.2 `examples/heap_field_mismatch.py`

Primary focus. This example stresses field initialization and mutation-sensitive heap sections. It is valuable because it forces the cover, theory packs, and renderer to cooperate on a task that ordinary Python users actually care about.

Desired success condition. The output should explain which constructor or branch failed to establish a required heap section. The example should produce synthesized boundary contracts, local-site sections, and residual explanations that are stable enough to be snapshot-tested.

0.21.3 `examples/safe_guarded_head.py`

Primary focus. This example stresses guard harvesting for head-of-sequence safety. It is valuable because it forces the cover, theory packs, and renderer to cooperate on a task that ordinary Python users actually care about.

Desired success condition. The backward pass should recover the minimal non-emptiness requirement when the guard is absent and discharge the error site when the guard is present. The example should produce synthesized boundary contracts, local-site sections, and residual explanations that are stable enough to be snapshot-tested.

0.21.4 `examples/pytorch_shape_bug_from_tensorguard.py`

Primary focus. This example stresses invalid shape transformations in tensor code. It is valuable because it forces the cover, theory packs, and renderer to cooperate on a task that ordinary Python users actually care about.

Desired success condition. A good run synthesizes cardinality or rank requirements and a precise shape-oriented error explanation. The example should produce synthesized boundary contracts, local-site sections, and residual explanations that are stable enough to be snapshot-tested.

0.21.5 `examples/pytorch_sort_bad_return.py`

Primary focus. This example stresses values-versus-indices confusion after sort. It is valuable because it forces the cover, theory packs, and renderer to cooperate on a task that ordinary Python users actually care about.

Desired success condition. The local order and selector sites should separate the semantic roles of values and index outputs. The example should produce synthesized boundary contracts, local-site sections, and residual explanations that are stable enough to be snapshot-tested.

0.21.6 `examples/pytorch_semantic_showcase.py`

Primary focus. This example stresses interprocedural sort, indexing, and shape composition. It is valuable because it forces the cover, theory packs, and renderer to cooperate on a task that ordinary Python users actually care about.

Desired success condition. This file is the showcase for imported transport across realistic wrapper chains. The example should produce synthesized boundary contracts, local-site sections, and residual explanations that are stable enough to be snapshot-tested.

0.21.7 `examples/pytorch_long_capability_audit.py`

Primary focus. This example stresses long composed pipelines with nested shape and index operations. It is valuable because it forces the cover, theory packs, and renderer to cooperate on a task that ordinary Python users actually care about.

Desired success condition. The benchmark should show how far local semantics survives under aliasing and composition. The example should produce synthesized boundary contracts, local-site sections, and residual explanations that are stable enough to be snapshot-tested.

0.21.8 `examples/dependent_sigma_witness.py`

Primary focus. This example stresses witness-carrying results and sigma-like exports. It is valuable because it forces the cover, theory packs, and renderer to cooperate on a task that ordinary Python users actually care about.

Desired success condition. The proof and output-boundary sites should surface explicit witness structure rather than flattening it to text. The example should produce synthesized boundary contracts, local-site sections, and residual explanations that are stable enough to be snapshot-tested.

0.21.9 `examples/constructor_usage_bug.py`

Primary focus. This example stresses constructor invariants and object misuse. It is valuable because it forces the cover, theory packs, and renderer to cooperate on a task that ordinary Python users actually care about.

Desired success condition. A sheaf-descent implementation should pinpoint the constructor or usage site that leaves the object state unsafe. The example should produce synthesized boundary contracts, local-site sections, and residual explanations that are stable enough to be snapshot-tested.

0.21.10 `examples/heap_protocol_branch_bug.py`

Primary focus. This example stresses branch-sensitive protocol and heap reasoning. It is valuable because it forces the cover, theory packs, and renderer to cooperate on a task that ordinary Python users actually care about.

Desired success condition. Guard sites and heap/protocol sites should cooperate rather than existing in separate analyses. The example should produce synthesized boundary contracts, local-site sections, and residual explanations that are stable enough to be snapshot-tested.

0.21.11 `examples/mixed_mode_sort_equivalence.py`

Primary focus. This example stresses proof-oriented and ordinary-Python semantics meeting in one pipeline. It is valuable because it forces the cover, theory packs, and renderer to cooperate on a task that ordinary Python users actually care about.

Desired success condition. The example should demonstrate that theorem-level transport improves unannotated client typing. The example should produce synthesized boundary contracts, local-site sections, and residual explanations that are stable enough to be snapshot-tested.

0.21.12 `examples/theorem_carrying_benchmark.proof`

Primary focus. This example stresses proof-first transport reuse. It is valuable because it forces the cover, theory packs, and renderer to cooperate on a task that ordinary Python users actually care about.

Desired success condition. This benchmark should quantify how much ordinary-user annotation rate drops once proof sites are available. The example should produce synthesized boundary contracts, local-site sections, and residual explanations that are stable enough to be snapshot-tested.

0.22 Runtime-error catalogue and safety synthesis

Error handling is central because safe-input synthesis is one of the main user goals.

0.22.1 IndexError on standard sequences

Local cause. The site becomes problematic when selector-domain exceeds sequence extent In the site catalogue this should be represented explicitly rather than buried in an exception string.

Backward viability strategy. The correct response is to arithmetically pull back range bounds to the input boundary or to upstream guards. This is the mechanism by which runtime error prediction becomes contract synthesis.

Expected rendered outcome. The final user-facing artifact should include input non-emptiness or index-range synthesis plus local provenance explanations. This is how the same semantic machinery serves both verification-heavy and diagnostics-heavy use cases.

0.22.2 KeyError on mappings

Local cause. The site becomes problematic when key-membership cannot be established at a mapping access site In the site catalogue this should be represented explicitly rather than buried in an exception string.

Backward viability strategy. The correct response is to harvest membership guards and propagate key-domain requirements. This is the mechanism by which runtime error prediction becomes contract synthesis.

Expected rendered outcome. The final user-facing artifact should include synthesized input membership requirements or explicit residual key assumptions. This is how the same semantic machinery serves both verification-heavy and diagnostics-heavy use cases.

0.22.3 AttributeError

Local cause. The site becomes problematic when receiver protocol or initialization site lacks the requested member In the site catalogue this should be represented explicitly rather than buried in an exception string.

Backward viability strategy. The correct response is to pull back method-availability or field-initialization requirements. This is the mechanism by which runtime error prediction becomes contract synthesis.

Expected rendered outcome. The final user-facing artifact should include protocol-aware diagnostics and constructor-side repairs. This is how the same semantic machinery serves both verification-heavy and diagnostics-heavy use cases.

0.22.4 TypeError from wrong callable or wrong operand

Local cause. The site becomes problematic when carrier mismatch at a call or operator site In the site catalogue this should be represented explicitly rather than buried in an exception string.

Backward viability strategy. The correct response is to strengthen input carrier assumptions or propagate proof-backed coercion transports. This is the mechanism by which runtime error prediction becomes contract synthesis.

Expected rendered outcome. The final user-facing artifact should include clear distinction between type-carrier failure and refinement failure. This is how the same semantic machinery serves both verification-heavy and diagnostics-heavy use cases.

0.22.5 ValueError from invalid reshape or view

Local cause. The site becomes problematic when shape cardinality or contiguity assumptions fail locally In the site catalogue this should be represented explicitly rather than buried in an exception string.

Backward viability strategy. The correct response is to derive cardinality, rank, or layout requirements on tensor inputs. This is the mechanism by which runtime error prediction becomes contract synthesis.

Expected rendered outcome. The final user-facing artifact should include library-specific preconditions and exact offending site reports. This is how the same semantic machinery serves both verification-heavy and diagnostics-heavy use cases.

0.22.6 RuntimeError from broadcast mismatch

Local cause. The site becomes problematic when tensor extents are not broadcast-compatible In the site catalogue this should be represented explicitly rather than buried in an exception string.

Backward viability strategy. The correct response is to solve local broadcast equations and pull them back to argument shapes. This is the mechanism by which runtime error prediction becomes contract synthesis.

Expected rendered outcome. The final user-facing artifact should include strong shape requirements and output-shape guarantees when safe. This is how the same semantic machinery serves both verification-heavy and diagnostics-heavy use cases.

0.22.7 Out-of-range gather/index-select

Local cause. The site becomes problematic when advanced selector domains exceed tensor extents In the site catalogue this should be represented explicitly rather than buried in an exception string.

Backward viability strategy. The correct response is to combine indexing theory with lineage-aware source extents. This is the mechanism by which runtime error prediction becomes contract synthesis.

Expected rendered outcome. The final user-facing artifact should include selector-range requirements and source-target provenance. This is how the same semantic machinery serves both verification-heavy and diagnostics-heavy use cases.

0.22.8 Reduction on an empty dimension

Local cause. The site becomes problematic when an operator assumes nonempty input along a reduced axis In the site catalogue this should be represented explicitly rather than buried in an exception string.

Backward viability strategy. The correct response is to infer non-emptiness on the relevant dimension or track identity-value exceptions. This is the mechanism by which runtime error prediction becomes contract synthesis.

Expected rendered outcome. The final user-facing artifact should include safer tensor analytics and better top-k style checks. This is how the same semantic machinery serves both verification-heavy and diagnostics-heavy use cases.

0.22.9 Device mismatch

Local cause. The site becomes problematic when operators expect tensors on compatible devices but receive mixed inputs In the site catalogue this should be represented explicitly rather than buried in an exception string.

Backward viability strategy. The correct response is to pull back device equality or admissible transfer assumptions. This is the mechanism by which runtime error prediction becomes contract synthesis.

Expected rendered outcome. The final user-facing artifact should include device contracts that ordinary users can actually read. This is how the same semantic machinery serves both verification-heavy and diagnostics-heavy use cases.

0.22.10 Unsupported dtype/operator combination

Local cause. The site becomes problematic when local operator support matrix excludes the current dtype family In the site catalogue this should be represented explicitly rather than buried in an exception string.

Backward viability strategy. The correct response is to derive dtype compatibility or explicit cast requirements. This is the mechanism by which runtime error prediction becomes contract synthesis.

Expected rendered outcome. The final user-facing artifact should include library-aware contracts instead of opaque runtime surprises. This is how the same semantic machinery serves both verification-heavy and diagnostics-heavy use cases.

0.22.11 ZeroDivisionError

Local cause. The site becomes problematic when denominator site admits zero In the site catalogue this should be represented explicitly rather than buried in an exception string.

Backward viability strategy. The correct response is to backward viability over arithmetic sites. This is the mechanism by which runtime error prediction becomes contract synthesis.

Expected rendered outcome. The final user-facing artifact should include small but essential safety contracts for numeric Python. This is how the same semantic machinery serves both verification-heavy and diagnostics-heavy use cases.

0.22.12 Assertion failure

Local cause. The site becomes problematic when user assertion site remains nontrivial after local solving In the site catalogue this should be represented explicitly rather than buried in an exception string.

Backward viability strategy. The correct response is to treat assertions as either proven local sections or residual error sites. This is the mechanism by which runtime error prediction becomes contract synthesis.

Expected rendered outcome. The final user-facing artifact should include precise proof-goal rendering rather than generic assertion text. This is how the same semantic machinery serves both verification-heavy and diagnostics-heavy use cases.

0.22.13 Constructor invariant failure

Local cause. The site becomes problematic when object fields do not glue to a safe initialized state In the site catalogue this should be represented explicitly rather than buried in an exception string.

Backward viability strategy. The correct response is to pull back field-initialization and branch constraints to constructor entry. This is the mechanism by which runtime error prediction becomes contract synthesis.

Expected rendered outcome. The final user-facing artifact should include ordinary OO safety with explicit field provenance. This is how the same semantic machinery serves both verification-heavy and diagnostics-heavy use cases.

0.22.14 Protocol misuse in method dispatch

Local cause. The site becomes problematic when receiver lacks a required method family or semantic contract In the site catalogue this should be represented explicitly rather than buried in an exception string.

Backward viability strategy. The correct response is to synthesize protocol requirements or residual obligations. This is the mechanism by which runtime error prediction becomes contract synthesis.

Expected rendered outcome. The final user-facing artifact should include structural typing that remains native to Python. This is how the same semantic machinery serves both verification-heavy and diagnostics-heavy use cases.

0.22.15 Proof transport failure

Local cause. The site becomes problematic when a theorem site exists but the transport relation remains unverified In the site catalogue this should be represented explicitly rather than buried in an exception string.

Backward viability strategy. The correct response is to emit a residual proof site rather than pretending a guarantee exists. This is the mechanism by which runtime error prediction becomes contract synthesis.

Expected rendered outcome. The final user-facing artifact should include clear separation of proof debt from ordinary runtime errors. This is how the same semantic machinery serves both verification-heavy and diagnostics-heavy use cases.

0.22.16 Decreases or invariant failure

Local cause. The site becomes problematic when recursive or loop reasoning cannot justify progress or preservation In the site catalogue this should be represented explicitly rather than buried in an exception string.

Backward viability strategy. The correct response is to generate proof-side obligations and optionally strengthen input assumptions or invariants. This is the mechanism by which runtime error prediction becomes contract synthesis.

Expected rendered outcome. The final user-facing artifact should include F*-like feedback within native Python syntax. This is how the same semantic machinery serves both verification-heavy and diagnostics-heavy use cases.

0.23 Cross-component interaction profiles

The following interaction notes are intentionally concrete. They describe how particular site families and theory families should cooperate in an actual implementation.

0.23.1 Interaction: Argument-boundary sites with Linear arithmetic and intervals

Why the interaction matters. Argument-boundary sites are one of the places where linear arithmetic and intervals can become genuinely useful to ordinary users. If the interaction is modeled poorly, either the system loses semantic information or it asks for annotations that should have been avoidable.

Concrete implementation rule. The cover builder should create enough overlap structure that the local vocabulary of linear arithmetic and intervals can be restricted onto the relevant coordinates of argument-boundary sites. The local solver should emit not only solved facts but also viability and transport metadata. The backward pass should then decide whether the interaction contributes to safe input synthesis, while the forward pass decides whether it contributes to output enrichment.

Typical failure mode if omitted. Without this interaction the system is likely either to over-approximate and lose useful guarantees, or to under-approximate and miss a reachable runtime error. In both cases the user experience degrades because local evidence fails to become global semantics.

Suggested test shape. Add one corpus example in which a fact from linear arithmetic and intervals is only visible because of the presence of argument-boundary sites. The test should assert both a synthesized contract effect and a provenance explanation.

0.23.2 Interaction: Argument-boundary sites with Boolean guard theory

Why the interaction matters. Argument-boundary sites are one of the places where boolean guard theory can become genuinely useful to ordinary users. If the interaction is modeled poorly, either the system loses semantic information or it asks for annotations that should have been avoidable.

Concrete implementation rule. The cover builder should create enough overlap structure that the local vocabulary of boolean guard theory can be restricted onto the relevant coordinates of argument-boundary sites. The local solver should emit not only solved facts but also viability and transport metadata. The backward pass should then decide whether the interaction contributes to safe input synthesis, while the forward pass decides whether it contributes to output enrichment.

Typical failure mode if omitted. Without this interaction the system is likely either to over-approximate and lose useful guarantees, or to under-approximate and miss a reachable runtime error. In both cases the user experience degrades because local evidence fails to become global semantics.

Suggested test shape. Add one corpus example in which a fact from boolean guard theory is only visible because of the presence of argument-boundary sites. The test should assert both a synthesized contract effect and a provenance explanation.

0.23.3 Interaction: Argument-boundary sites with Nullability and optionality

Why the interaction matters. Argument-boundary sites are one of the places where nullability and optionality can become genuinely useful to ordinary users. If the interaction is modeled poorly, either the system loses semantic information or it asks for annotations that should have been avoidable.

Concrete implementation rule. The cover builder should create enough overlap structure that the local vocabulary of nullability and optionality can be restricted onto the relevant coordinates of argument-boundary sites. The local solver should emit not only solved facts but also viability and transport metadata. The backward pass should then decide whether the interaction contributes to safe input synthesis, while the forward pass decides whether it contributes to output enrichment.

Typical failure mode if omitted. Without this interaction the system is likely either to over-approximate and lose useful guarantees, or to under-approximate and miss a reachable runtime error. In both cases the user experience degrades because local evidence fails to become global semantics.

Suggested test shape. Add one corpus example in which a fact from nullability and optionality is only visible because of the presence of argument-boundary sites. The test should assert both a synthesized contract effect and a provenance explanation.

0.23.4 Interaction: Argument-boundary sites with Heap and protocol theory

Why the interaction matters. Argument-boundary sites are one of the places where heap and protocol theory can become genuinely useful to ordinary users. If the interaction is modeled poorly, either the system loses semantic information or it asks for annotations that should have been avoidable.

Concrete implementation rule. The cover builder should create enough overlap structure that the local vocabulary of heap and protocol theory can be restricted onto the relevant coordinates of argument-boundary sites. The local solver should emit not only solved facts but also viability and transport metadata. The backward pass should then decide whether the interaction contributes to safe input synthesis, while the forward pass decides whether it contributes to output enrichment.

Typical failure mode if omitted. Without this interaction the system is likely either to over-approximate and lose useful guarantees, or to under-approximate and miss a reachable runtime error. In both cases the user experience degrades because local evidence fails to become global semantics.

Suggested test shape. Add one corpus example in which a fact from heap and protocol theory is only visible because of the presence of argument-boundary sites. The test should assert both a synthesized contract effect and a provenance explanation.

0.23.5 Interaction: Argument-boundary sites with Tensor shape theory

Why the interaction matters. Argument-boundary sites are one of the places where tensor shape theory can become genuinely useful to ordinary users. If the interaction is modeled poorly, either the system loses semantic information or it asks for annotations that should have been avoidable.

Concrete implementation rule. The cover builder should create enough overlap structure that the local vocabulary of tensor shape theory can be restricted onto the relevant coordinates of argument-boundary sites. The local solver should emit not only solved facts but also viability and transport metadata. The backward pass should then decide whether the interaction contributes to safe input synthesis, while the forward pass decides whether it contributes to output enrichment.

Typical failure mode if omitted. Without this interaction the system is likely either to over-approximate and lose useful guarantees, or to under-approximate and miss a reachable runtime error. In both cases the user experience degrades because local evidence fails to become global semantics.

Suggested test shape. Add one corpus example in which a fact from tensor shape theory is only visible because of the presence of argument-boundary sites. The test should assert both a synthesized contract effect and a provenance explanation.

0.23.6 Interaction: Argument-boundary sites with Tensor order theory

Why the interaction matters. Argument-boundary sites are one of the places where tensor order theory can become genuinely useful to ordinary users. If the interaction is modeled poorly, either the system loses semantic information or it asks for annotations that should have been avoidable.

Concrete implementation rule. The cover builder should create enough overlap structure that the local vocabulary of tensor order theory can be restricted onto the relevant coordinates of argument-boundary sites. The local solver should emit not only solved facts but also viability and transport metadata. The backward pass should then decide whether the interaction contributes to safe input synthesis, while the forward pass decides whether it contributes to output enrichment.

Typical failure mode if omitted. Without this interaction the system is likely either to over-approximate and lose useful guarantees, or to under-approximate and miss a reachable runtime error. In both cases the user experience degrades because local evidence fails to become global semantics.

Suggested test shape. Add one corpus example in which a fact from tensor order theory is only visible because of the presence of argument-boundary sites. The test should assert both a synthesized contract effect and a provenance explanation.

0.23.7 Interaction: Argument-boundary sites with Tensor indexing theory

Why the interaction matters. Argument-boundary sites are one of the places where tensor indexing theory can become genuinely useful to ordinary users. If the interaction is modeled poorly, either the system loses semantic information or it asks for annotations that should have been avoidable.

Concrete implementation rule. The cover builder should create enough overlap structure that the local vocabulary of tensor indexing theory can be restricted onto the relevant coordinates of argument-boundary sites. The local solver should emit not only solved facts but also viability and transport metadata. The backward pass should then decide whether the interaction contributes to safe input synthesis, while the forward pass decides whether it contributes to output enrichment.

Typical failure mode if omitted. Without this interaction the system is likely either to over-approximate and lose useful guarantees, or to under-approximate and miss a reachable runtime error. In both cases the user experience degrades because local evidence fails to become global semantics.

Suggested test shape. Add one corpus example in which a fact from tensor indexing theory is only visible because of the presence of argument-boundary sites. The test should assert both a synthesized contract effect and a provenance explanation.

0.23.8 Interaction: Argument-boundary sites with Device and dtype compatibility theory

Why the interaction matters. Argument-boundary sites are one of the places where device and dtype compatibility theory can become genuinely useful to ordinary users. If the interaction is modeled poorly, either the system loses semantic information or it asks for annotations that should have been avoidable.

Concrete implementation rule. The cover builder should create enough overlap structure that the local vocabulary of device and dtype compatibility theory can be restricted onto the relevant coordinates of argument-boundary sites. The local solver should emit not only solved facts but also viability and transport metadata. The backward pass should then decide whether the interaction contributes to safe input synthesis, while the forward pass decides whether it contributes to output enrichment.

Typical failure mode if omitted. Without this interaction the system is likely either to over-approximate and lose useful guarantees, or to under-approximate and miss a reachable runtime error. In both cases the user experience degrades because local evidence fails to become global semantics.

Suggested test shape. Add one corpus example in which a fact from device and dtype compatibility theory is only visible because of the presence of argument-boundary sites. The test should assert both a synthesized contract effect and a provenance explanation.

0.23.9 Interaction: Argument-boundary sites with Sequence and collection theory

Why the interaction matters. Argument-boundary sites are one of the places where sequence and collection theory can become genuinely useful to ordinary users. If the interaction is modeled poorly, either the system loses semantic information or it asks for annotations that should have been avoidable.

Concrete implementation rule. The cover builder should create enough overlap structure that the local vocabulary of sequence and collection theory can be restricted onto the relevant coordinates of argument-boundary sites. The local solver should emit not only solved facts but also viability and transport metadata. The backward pass should then decide whether the interaction contributes to safe input synthesis, while the forward pass decides whether it contributes to output enrichment.

Typical failure mode if omitted. Without this interaction the system is likely either to over-approximate and lose useful guarantees, or to under-approximate and miss a reachable runtime error. In both cases the user experience degrades because local evidence fails to become global semantics.

Suggested test shape. Add one corpus example in which a fact from sequence and collection theory is only visible because of the presence of argument-boundary sites. The test should assert both a synthesized contract effect and a provenance explanation.

0.23.10 Interaction: Argument-boundary sites with Exception theory

Why the interaction matters. Argument-boundary sites are one of the places where exception theory can become genuinely useful to ordinary users. If the interaction is modeled poorly, either the system loses semantic information or it asks for annotations that should have been avoidable.

Concrete implementation rule. The cover builder should create enough overlap structure that the local vocabulary of exception theory can be restricted onto the relevant coordinates of argument-boundary sites. The local solver should emit not only solved facts but also viability and transport metadata. The backward pass should then decide whether the interaction contributes to safe input synthesis, while the forward pass decides whether it contributes to output enrichment.

Typical failure mode if omitted. Without this interaction the system is likely either to over-approximate and lose useful guarantees, or to under-approximate and miss a reachable runtime error. In both cases the user experience degrades because local evidence fails to become global semantics.

Suggested test shape. Add one corpus example in which a fact from exception theory is only visible because of the presence of argument-boundary sites. The test should assert both a synthesized contract effect and a provenance explanation.

0.23.11 Interaction: Argument-boundary sites with Witness and equality transport theory

Why the interaction matters. Argument-boundary sites are one of the places where witness and equality transport theory can become genuinely useful to ordinary users. If the interaction is

modeled poorly, either the system loses semantic information or it asks for annotations that should have been avoidable.

Concrete implementation rule. The cover builder should create enough overlap structure that the local vocabulary of witness and equality transport theory can be restricted onto the relevant coordinates of argument-boundary sites. The local solver should emit not only solved facts but also viability and transport metadata. The backward pass should then decide whether the interaction contributes to safe input synthesis, while the forward pass decides whether it contributes to output enrichment.

Typical failure mode if omitted. Without this interaction the system is likely either to over-approximate and lose useful guarantees, or to under-approximate and miss a reachable runtime error. In both cases the user experience degrades because local evidence fails to become global semantics.

Suggested test shape. Add one corpus example in which a fact from witness and equality transport theory is only visible because of the presence of argument-boundary sites. The test should assert both a synthesized contract effect and a provenance explanation.

0.23.12 Interaction: Argument-boundary sites with Loop invariant and decreases theory

Why the interaction matters. Argument-boundary sites are one of the places where loop invariant and decreases theory can become genuinely useful to ordinary users. If the interaction is modeled poorly, either the system loses semantic information or it asks for annotations that should have been avoidable.

Concrete implementation rule. The cover builder should create enough overlap structure that the local vocabulary of loop invariant and decreases theory can be restricted onto the relevant coordinates of argument-boundary sites. The local solver should emit not only solved facts but also viability and transport metadata. The backward pass should then decide whether the interaction contributes to safe input synthesis, while the forward pass decides whether it contributes to output enrichment.

Typical failure mode if omitted. Without this interaction the system is likely either to over-approximate and lose useful guarantees, or to under-approximate and miss a reachable runtime error. In both cases the user experience degrades because local evidence fails to become global semantics.

Suggested test shape. Add one corpus example in which a fact from loop invariant and decreases theory is only visible because of the presence of argument-boundary sites. The test should assert both a synthesized contract effect and a provenance explanation.

0.23.13 Interaction: Return and output-boundary sites with Linear arithmetic and intervals

Why the interaction matters. Return and output-boundary sites are one of the places where linear arithmetic and intervals can become genuinely useful to ordinary users. If the interaction is modeled poorly, either the system loses semantic information or it asks for annotations that should have been avoidable.

Concrete implementation rule. The cover builder should create enough overlap structure that the local vocabulary of linear arithmetic and intervals can be restricted onto the relevant coordinates of return and output-boundary sites. The local solver should emit not only solved facts but also viability and transport metadata. The backward pass should then decide whether the interaction contributes to safe input synthesis, while the forward pass decides whether it contributes to output enrichment.

Typical failure mode if omitted. Without this interaction the system is likely either to over-approximate and lose useful guarantees, or to under-approximate and miss a reachable runtime error. In both cases the user experience degrades because local evidence fails to become global semantics.

Suggested test shape. Add one corpus example in which a fact from linear arithmetic and intervals is only visible because of the presence of return and output-boundary sites. The test should assert both a synthesized contract effect and a provenance explanation.

0.23.14 Interaction: Return and output-boundary sites with Boolean guard theory

Why the interaction matters. Return and output-boundary sites are one of the places where boolean guard theory can become genuinely useful to ordinary users. If the interaction is modeled poorly, either the system loses semantic information or it asks for annotations that should have been avoidable.

Concrete implementation rule. The cover builder should create enough overlap structure that the local vocabulary of boolean guard theory can be restricted onto the relevant coordinates of return and output-boundary sites. The local solver should emit not only solved facts but also viability and transport metadata. The backward pass should then decide whether the interaction contributes to safe input synthesis, while the forward pass decides whether it contributes to output enrichment.

Typical failure mode if omitted. Without this interaction the system is likely either to over-approximate and lose useful guarantees, or to under-approximate and miss a reachable runtime error. In both cases the user experience degrades because local evidence fails to become global semantics.

Suggested test shape. Add one corpus example in which a fact from boolean guard theory is only visible because of the presence of return and output-boundary sites. The test should assert both a synthesized contract effect and a provenance explanation.

0.23.15 Interaction: Return and output-boundary sites with Nullability and optionality

Why the interaction matters. Return and output-boundary sites are one of the places where nullability and optionality can become genuinely useful to ordinary users. If the interaction is modeled poorly, either the system loses semantic information or it asks for annotations that should have been avoidable.

Concrete implementation rule. The cover builder should create enough overlap structure that the local vocabulary of nullability and optionality can be restricted onto the relevant coordinates of return and output-boundary sites. The local solver should emit not only solved facts but also viability and transport metadata. The backward pass should then decide whether the interaction contributes to safe input synthesis, while the forward pass decides whether it contributes to output enrichment.

Typical failure mode if omitted. Without this interaction the system is likely either to over-approximate and lose useful guarantees, or to under-approximate and miss a reachable runtime error. In both cases the user experience degrades because local evidence fails to become global semantics.

Suggested test shape. Add one corpus example in which a fact from nullability and optionality is only visible because of the presence of return and output-boundary sites. The test should assert both a synthesized contract effect and a provenance explanation.

0.23.16 Interaction: Return and output-boundary sites with Heap and protocol theory

Why the interaction matters. Return and output-boundary sites are one of the places where heap and protocol theory can become genuinely useful to ordinary users. If the interaction is modeled poorly, either the system loses semantic information or it asks for annotations that should have been avoidable.

Concrete implementation rule. The cover builder should create enough overlap structure that the local vocabulary of heap and protocol theory can be restricted onto the relevant coordinates of return and output-boundary sites. The local solver should emit not only solved facts but also viability and transport metadata. The backward pass should then decide whether the interaction contributes to safe input synthesis, while the forward pass decides whether it contributes to output enrichment.

Typical failure mode if omitted. Without this interaction the system is likely either to over-approximate and lose useful guarantees, or to under-approximate and miss a reachable runtime error. In both cases the user experience degrades because local evidence fails to become global semantics.

Suggested test shape. Add one corpus example in which a fact from heap and protocol theory is only visible because of the presence of return and output-boundary sites. The test should assert both a synthesized contract effect and a provenance explanation.

0.23.17 Interaction: Return and output-boundary sites with Tensor shape theory

Why the interaction matters. Return and output-boundary sites are one of the places where tensor shape theory can become genuinely useful to ordinary users. If the interaction is modeled poorly, either the system loses semantic information or it asks for annotations that should have been avoidable.

Concrete implementation rule. The cover builder should create enough overlap structure that the local vocabulary of tensor shape theory can be restricted onto the relevant coordinates of return and output-boundary sites. The local solver should emit not only solved facts but also viability and transport metadata. The backward pass should then decide whether the interaction contributes to safe input synthesis, while the forward pass decides whether it contributes to output enrichment.

Typical failure mode if omitted. Without this interaction the system is likely either to over-approximate and lose useful guarantees, or to under-approximate and miss a reachable runtime error. In both cases the user experience degrades because local evidence fails to become global semantics.

Suggested test shape. Add one corpus example in which a fact from tensor shape theory is only visible because of the presence of return and output-boundary sites. The test should assert both a synthesized contract effect and a provenance explanation.

0.23.18 Interaction: Return and output-boundary sites with Tensor order theory

Why the interaction matters. Return and output-boundary sites are one of the places where tensor order theory can become genuinely useful to ordinary users. If the interaction is modeled poorly, either the system loses semantic information or it asks for annotations that should have been avoidable.

Concrete implementation rule. The cover builder should create enough overlap structure that the local vocabulary of tensor order theory can be restricted onto the relevant coordinates of return and output-boundary sites. The local solver should emit not only solved facts but also viability and transport metadata. The backward pass should then decide whether the interaction contributes to safe input synthesis, while the forward pass decides whether it contributes to output enrichment.

Typical failure mode if omitted. Without this interaction the system is likely either to over-approximate and lose useful guarantees, or to under-approximate and miss a reachable runtime error. In both cases the user experience degrades because local evidence fails to become global semantics.

Suggested test shape. Add one corpus example in which a fact from tensor order theory is only visible because of the presence of return and output-boundary sites. The test should assert both a synthesized contract effect and a provenance explanation.

0.23.19 Interaction: Return and output-boundary sites with Tensor indexing theory

Why the interaction matters. Return and output-boundary sites are one of the places where tensor indexing theory can become genuinely useful to ordinary users. If the interaction is modeled poorly, either the system loses semantic information or it asks for annotations that should have been avoidable.

Concrete implementation rule. The cover builder should create enough overlap structure that the local vocabulary of tensor indexing theory can be restricted onto the relevant coordinates of return and output-boundary sites. The local solver should emit not only solved facts but also viability and transport metadata. The backward pass should then decide whether the interaction

contributes to safe input synthesis, while the forward pass decides whether it contributes to output enrichment.

Typical failure mode if omitted. Without this interaction the system is likely either to over-approximate and lose useful guarantees, or to under-approximate and miss a reachable runtime error. In both cases the user experience degrades because local evidence fails to become global semantics.

Suggested test shape. Add one corpus example in which a fact from tensor indexing theory is only visible because of the presence of return and output-boundary sites. The test should assert both a synthesized contract effect and a provenance explanation.

0.23.20 Interaction: Return and output-boundary sites with Device and dtype compatibility theory

Why the interaction matters. Return and output-boundary sites are one of the places where device and dtype compatibility theory can become genuinely useful to ordinary users. If the interaction is modeled poorly, either the system loses semantic information or it asks for annotations that should have been avoidable.

Concrete implementation rule. The cover builder should create enough overlap structure that the local vocabulary of device and dtype compatibility theory can be restricted onto the relevant coordinates of return and output-boundary sites. The local solver should emit not only solved facts but also viability and transport metadata. The backward pass should then decide whether the interaction contributes to safe input synthesis, while the forward pass decides whether it contributes to output enrichment.

Typical failure mode if omitted. Without this interaction the system is likely either to over-approximate and lose useful guarantees, or to under-approximate and miss a reachable runtime error. In both cases the user experience degrades because local evidence fails to become global semantics.

Suggested test shape. Add one corpus example in which a fact from device and dtype compatibility theory is only visible because of the presence of return and output-boundary sites. The test should assert both a synthesized contract effect and a provenance explanation.

0.23.21 Interaction: Return and output-boundary sites with Sequence and collection theory

Why the interaction matters. Return and output-boundary sites are one of the places where sequence and collection theory can become genuinely useful to ordinary users. If the interaction is modeled poorly, either the system loses semantic information or it asks for annotations that should have been avoidable.

Concrete implementation rule. The cover builder should create enough overlap structure that the local vocabulary of sequence and collection theory can be restricted onto the relevant coordinates of return and output-boundary sites. The local solver should emit not only solved facts but also viability and transport metadata. The backward pass should then decide whether the interaction contributes to safe input synthesis, while the forward pass decides whether it contributes to output enrichment.

Typical failure mode if omitted. Without this interaction the system is likely either to over-approximate and lose useful guarantees, or to under-approximate and miss a reachable runtime error. In both cases the user experience degrades because local evidence fails to become global semantics.

Suggested test shape. Add one corpus example in which a fact from sequence and collection theory is only visible because of the presence of return and output-boundary sites. The test should assert both a synthesized contract effect and a provenance explanation.

0.23.22 Interaction: Return and output-boundary sites with Exception theory

Why the interaction matters. Return and output-boundary sites are one of the places where exception theory can become genuinely useful to ordinary users. If the interaction is modeled poorly, either the system loses semantic information or it asks for annotations that should have been avoidable.

Concrete implementation rule. The cover builder should create enough overlap structure that the local vocabulary of exception theory can be restricted onto the relevant coordinates of return and output-boundary sites. The local solver should emit not only solved facts but also viability and transport metadata. The backward pass should then decide whether the interaction contributes to safe input synthesis, while the forward pass decides whether it contributes to output enrichment.

Typical failure mode if omitted. Without this interaction the system is likely either to over-approximate and lose useful guarantees, or to under-approximate and miss a reachable runtime error. In both cases the user experience degrades because local evidence fails to become global semantics.

Suggested test shape. Add one corpus example in which a fact from exception theory is only visible because of the presence of return and output-boundary sites. The test should assert both a synthesized contract effect and a provenance explanation.

0.23.23 Interaction: Return and output-boundary sites with Witness and equality transport theory

Why the interaction matters. Return and output-boundary sites are one of the places where witness and equality transport theory can become genuinely useful to ordinary users. If the interaction is modeled poorly, either the system loses semantic information or it asks for annotations that should have been avoidable.

Concrete implementation rule. The cover builder should create enough overlap structure that the local vocabulary of witness and equality transport theory can be restricted onto the relevant coordinates of return and output-boundary sites. The local solver should emit not only solved facts but also viability and transport metadata. The backward pass should then decide whether the interaction contributes to safe input synthesis, while the forward pass decides whether it contributes to output enrichment.

Typical failure mode if omitted. Without this interaction the system is likely either to over-approximate and lose useful guarantees, or to under-approximate and miss a reachable runtime error. In both cases the user experience degrades because local evidence fails to become global semantics.

Suggested test shape. Add one corpus example in which a fact from witness and equality transport theory is only visible because of the presence of return and output-boundary sites. The test should assert both a synthesized contract effect and a provenance explanation.

0.23.24 Interaction: Return and output-boundary sites with Loop invariant and decreases theory

Why the interaction matters. Return and output-boundary sites are one of the places where loop invariant and decreases theory can become genuinely useful to ordinary users. If the interaction is modeled poorly, either the system loses semantic information or it asks for annotations that should have been avoidable.

Concrete implementation rule. The cover builder should create enough overlap structure that the local vocabulary of loop invariant and decreases theory can be restricted onto the relevant coordinates of return and output-boundary sites. The local solver should emit not only solved facts but also viability and transport metadata. The backward pass should then decide whether the interaction contributes to safe input synthesis, while the forward pass decides whether it contributes to output enrichment.

Typical failure mode if omitted. Without this interaction the system is likely either to over-approximate and lose useful guarantees, or to under-approximate and miss a reachable runtime error. In both cases the user experience degrades because local evidence fails to become global semantics.

Suggested test shape. Add one corpus example in which a fact from loop invariant and decreases theory is only visible because of the presence of return and output-boundary sites. The test should assert both a synthesized contract effect and a provenance explanation.

0.23.25 Interaction: SSA-value sites with Linear arithmetic and intervals

Why the interaction matters. SSA-value sites are one of the places where linear arithmetic and intervals can become genuinely useful to ordinary users. If the interaction is modeled poorly, either the system loses semantic information or it asks for annotations that should have been avoidable.

Concrete implementation rule. The cover builder should create enough overlap structure that the local vocabulary of linear arithmetic and intervals can be restricted onto the relevant coordinates of ssa-value sites. The local solver should emit not only solved facts but also viability and transport metadata. The backward pass should then decide whether the interaction contributes to safe input synthesis, while the forward pass decides whether it contributes to output enrichment.

Typical failure mode if omitted. Without this interaction the system is likely either to over-approximate and lose useful guarantees, or to under-approximate and miss a reachable runtime error. In both cases the user experience degrades because local evidence fails to become global semantics.

Suggested test shape. Add one corpus example in which a fact from linear arithmetic and intervals is only visible because of the presence of ssa-value sites. The test should assert both a synthesized contract effect and a provenance explanation.

0.23.26 Interaction: SSA-value sites with Boolean guard theory

Why the interaction matters. SSA-value sites are one of the places where boolean guard theory can become genuinely useful to ordinary users. If the interaction is modeled poorly, either the system loses semantic information or it asks for annotations that should have been avoidable.

Concrete implementation rule. The cover builder should create enough overlap structure that the local vocabulary of boolean guard theory can be restricted onto the relevant coordinates of ssa-value sites. The local solver should emit not only solved facts but also viability and transport metadata. The backward pass should then decide whether the interaction contributes to safe input synthesis, while the forward pass decides whether it contributes to output enrichment.

Typical failure mode if omitted. Without this interaction the system is likely either to over-approximate and lose useful guarantees, or to under-approximate and miss a reachable runtime error. In both cases the user experience degrades because local evidence fails to become global semantics.

Suggested test shape. Add one corpus example in which a fact from boolean guard theory is only visible because of the presence of ssa-value sites. The test should assert both a synthesized contract effect and a provenance explanation.

0.23.27 Interaction: SSA-value sites with Nullability and optionality

Why the interaction matters. SSA-value sites are one of the places where nullability and optionality can become genuinely useful to ordinary users. If the interaction is modeled poorly, either the system loses semantic information or it asks for annotations that should have been avoidable.

Concrete implementation rule. The cover builder should create enough overlap structure that the local vocabulary of nullability and optionality can be restricted onto the relevant coordinates of ssa-value sites. The local solver should emit not only solved facts but also viability and transport metadata. The backward pass should then decide whether the interaction contributes to safe input synthesis, while the forward pass decides whether it contributes to output enrichment.

Typical failure mode if omitted. Without this interaction the system is likely either to over-approximate and lose useful guarantees, or to under-approximate and miss a reachable runtime error. In both cases the user experience degrades because local evidence fails to become global semantics.

Suggested test shape. Add one corpus example in which a fact from nullability and optionality is only visible because of the presence of ssa-value sites. The test should assert both a synthesized contract effect and a provenance explanation.

0.23.28 Interaction: SSA-value sites with Heap and protocol theory

Why the interaction matters. SSA-value sites are one of the places where heap and protocol theory can become genuinely useful to ordinary users. If the interaction is modeled poorly, either the system loses semantic information or it asks for annotations that should have been avoidable.

Concrete implementation rule. The cover builder should create enough overlap structure that the local vocabulary of heap and protocol theory can be restricted onto the relevant coordinates of ssa-value sites. The local solver should emit not only solved facts but also viability and transport metadata. The backward pass should then decide whether the interaction contributes to safe input synthesis, while the forward pass decides whether it contributes to output enrichment.

Typical failure mode if omitted. Without this interaction the system is likely either to over-approximate and lose useful guarantees, or to under-approximate and miss a reachable runtime error. In both cases the user experience degrades because local evidence fails to become global semantics.

Suggested test shape. Add one corpus example in which a fact from heap and protocol theory is only visible because of the presence of ssa-value sites. The test should assert both a synthesized contract effect and a provenance explanation.

0.23.29 Interaction: SSA-value sites with Tensor shape theory

Why the interaction matters. SSA-value sites are one of the places where tensor shape theory can become genuinely useful to ordinary users. If the interaction is modeled poorly, either the system loses semantic information or it asks for annotations that should have been avoidable.

Concrete implementation rule. The cover builder should create enough overlap structure that the local vocabulary of tensor shape theory can be restricted onto the relevant coordinates of ssa-value sites. The local solver should emit not only solved facts but also viability and transport metadata. The backward pass should then decide whether the interaction contributes to safe input synthesis, while the forward pass decides whether it contributes to output enrichment.

Typical failure mode if omitted. Without this interaction the system is likely either to over-approximate and lose useful guarantees, or to under-approximate and miss a reachable runtime error. In both cases the user experience degrades because local evidence fails to become global semantics.

Suggested test shape. Add one corpus example in which a fact from tensor shape theory is only visible because of the presence of ssa-value sites. The test should assert both a synthesized contract effect and a provenance explanation.

0.23.30 Interaction: SSA-value sites with Tensor order theory

Why the interaction matters. SSA-value sites are one of the places where tensor order theory can become genuinely useful to ordinary users. If the interaction is modeled poorly, either the system loses semantic information or it asks for annotations that should have been avoidable.

Concrete implementation rule. The cover builder should create enough overlap structure that the local vocabulary of tensor order theory can be restricted onto the relevant coordinates of ssa-value sites. The local solver should emit not only solved facts but also viability and transport metadata. The backward pass should then decide whether the interaction contributes to safe input synthesis, while the forward pass decides whether it contributes to output enrichment.

Typical failure mode if omitted. Without this interaction the system is likely either to over-approximate and lose useful guarantees, or to under-approximate and miss a reachable runtime error. In both cases the user experience degrades because local evidence fails to become global semantics.

Suggested test shape. Add one corpus example in which a fact from tensor order theory is only visible because of the presence of ssa-value sites. The test should assert both a synthesized contract effect and a provenance explanation.

0.23.31 Interaction: SSA-value sites with Tensor indexing theory

Why the interaction matters. SSA-value sites are one of the places where tensor indexing theory can become genuinely useful to ordinary users. If the interaction is modeled poorly, either the system loses semantic information or it asks for annotations that should have been avoidable.

Concrete implementation rule. The cover builder should create enough overlap structure that the local vocabulary of tensor indexing theory can be restricted onto the relevant coordinates of ssa-value sites. The local solver should emit not only solved facts but also viability and transport metadata. The backward pass should then decide whether the interaction contributes to safe input synthesis, while the forward pass decides whether it contributes to output enrichment.

Typical failure mode if omitted. Without this interaction the system is likely either to over-approximate and lose useful guarantees, or to under-approximate and miss a reachable runtime error. In both cases the user experience degrades because local evidence fails to become global semantics.

Suggested test shape. Add one corpus example in which a fact from tensor indexing theory is only visible because of the presence of ssa-value sites. The test should assert both a synthesized contract effect and a provenance explanation.

0.23.32 Interaction: SSA-value sites with Device and dtype compatibility theory

Why the interaction matters. SSA-value sites are one of the places where device and dtype compatibility theory can become genuinely useful to ordinary users. If the interaction is modeled poorly, either the system loses semantic information or it asks for annotations that should have been avoidable.

Concrete implementation rule. The cover builder should create enough overlap structure that the local vocabulary of device and dtype compatibility theory can be restricted onto the relevant coordinates of ssa-value sites. The local solver should emit not only solved facts but also viability and transport metadata. The backward pass should then decide whether the interaction contributes to safe input synthesis, while the forward pass decides whether it contributes to output enrichment.

Typical failure mode if omitted. Without this interaction the system is likely either to over-approximate and lose useful guarantees, or to under-approximate and miss a reachable runtime error. In both cases the user experience degrades because local evidence fails to become global semantics.

Suggested test shape. Add one corpus example in which a fact from device and dtype compatibility theory is only visible because of the presence of ssa-value sites. The test should assert both a synthesized contract effect and a provenance explanation.

0.23.33 Interaction: SSA-value sites with Sequence and collection theory

Why the interaction matters. SSA-value sites are one of the places where sequence and collection theory can become genuinely useful to ordinary users. If the interaction is modeled poorly, either the system loses semantic information or it asks for annotations that should have been avoidable.

Concrete implementation rule. The cover builder should create enough overlap structure that the local vocabulary of sequence and collection theory can be restricted onto the relevant coordinates of ssa-value sites. The local solver should emit not only solved facts but also viability and transport metadata. The backward pass should then decide whether the interaction contributes to safe input synthesis, while the forward pass decides whether it contributes to output enrichment.

Typical failure mode if omitted. Without this interaction the system is likely either to over-approximate and lose useful guarantees, or to under-approximate and miss a reachable runtime error. In both cases the user experience degrades because local evidence fails to become global semantics.

Suggested test shape. Add one corpus example in which a fact from sequence and collection theory is only visible because of the presence of ssa-value sites. The test should assert both a synthesized contract effect and a provenance explanation.

0.23.34 Interaction: SSA-value sites with Exception theory

Why the interaction matters. SSA-value sites are one of the places where exception theory can become genuinely useful to ordinary users. If the interaction is modeled poorly, either the system loses semantic information or it asks for annotations that should have been avoidable.

Concrete implementation rule. The cover builder should create enough overlap structure that the local vocabulary of exception theory can be restricted onto the relevant coordinates of ssa-value sites. The local solver should emit not only solved facts but also viability and transport metadata. The backward pass should then decide whether the interaction contributes to safe input synthesis, while the forward pass decides whether it contributes to output enrichment.

Typical failure mode if omitted. Without this interaction the system is likely either to over-approximate and lose useful guarantees, or to under-approximate and miss a reachable runtime error. In both cases the user experience degrades because local evidence fails to become global semantics.

Suggested test shape. Add one corpus example in which a fact from exception theory is only visible because of the presence of ssa-value sites. The test should assert both a synthesized contract effect and a provenance explanation.

0.23.35 Interaction: SSA-value sites with Witness and equality transport theory

Why the interaction matters. SSA-value sites are one of the places where witness and equality transport theory can become genuinely useful to ordinary users. If the interaction is modeled poorly, either the system loses semantic information or it asks for annotations that should have been avoidable.

Concrete implementation rule. The cover builder should create enough overlap structure that the local vocabulary of witness and equality transport theory can be restricted onto the relevant coordinates of ssa-value sites. The local solver should emit not only solved facts but also viability and transport metadata. The backward pass should then decide whether the interaction contributes to safe input synthesis, while the forward pass decides whether it contributes to output enrichment.

Typical failure mode if omitted. Without this interaction the system is likely either to over-approximate and lose useful guarantees, or to under-approximate and miss a reachable runtime error. In both cases the user experience degrades because local evidence fails to become global semantics.

Suggested test shape. Add one corpus example in which a fact from witness and equality transport theory is only visible because of the presence of ssa-value sites. The test should assert both a synthesized contract effect and a provenance explanation.

0.23.36 Interaction: SSA-value sites with Loop invariant and decreases theory

Why the interaction matters. SSA-value sites are one of the places where loop invariant and decreases theory can become genuinely useful to ordinary users. If the interaction is modeled poorly, either the system loses semantic information or it asks for annotations that should have been avoidable.

Concrete implementation rule. The cover builder should create enough overlap structure that the local vocabulary of loop invariant and decreases theory can be restricted onto the relevant coordinates of ssa-value sites. The local solver should emit not only solved facts but also viability and transport metadata. The backward pass should then decide whether the interaction contributes to safe input synthesis, while the forward pass decides whether it contributes to output enrichment.

Typical failure mode if omitted. Without this interaction the system is likely either to over-approximate and lose useful guarantees, or to under-approximate and miss a reachable runtime error. In both cases the user experience degrades because local evidence fails to become global semantics.

Suggested test shape. Add one corpus example in which a fact from loop invariant and decreases theory is only visible because of the presence of ssa-value sites. The test should assert both a synthesized contract effect and a provenance explanation.

0.23.37 Interaction: Branch-guard sites with Linear arithmetic and intervals

Why the interaction matters. Branch-guard sites are one of the places where linear arithmetic and intervals can become genuinely useful to ordinary users. If the interaction is modeled poorly, either the system loses semantic information or it asks for annotations that should have been avoidable.

Concrete implementation rule. The cover builder should create enough overlap structure that the local vocabulary of linear arithmetic and intervals can be restricted onto the relevant coordinates of branch-guard sites. The local solver should emit not only solved facts but also viability and transport metadata. The backward pass should then decide whether the interaction contributes to safe input synthesis, while the forward pass decides whether it contributes to output enrichment.

Typical failure mode if omitted. Without this interaction the system is likely either to over-approximate and lose useful guarantees, or to under-approximate and miss a reachable runtime error. In both cases the user experience degrades because local evidence fails to become global semantics.

Suggested test shape. Add one corpus example in which a fact from linear arithmetic and intervals is only visible because of the presence of branch-guard sites. The test should assert both a synthesized contract effect and a provenance explanation.

0.23.38 Interaction: Branch-guard sites with Boolean guard theory

Why the interaction matters. Branch-guard sites are one of the places where boolean guard theory can become genuinely useful to ordinary users. If the interaction is modeled poorly, either the system loses semantic information or it asks for annotations that should have been avoidable.

Concrete implementation rule. The cover builder should create enough overlap structure that the local vocabulary of boolean guard theory can be restricted onto the relevant coordinates of branch-guard sites. The local solver should emit not only solved facts but also viability and transport metadata. The backward pass should then decide whether the interaction contributes to safe input synthesis, while the forward pass decides whether it contributes to output enrichment.

Typical failure mode if omitted. Without this interaction the system is likely either to over-approximate and lose useful guarantees, or to under-approximate and miss a reachable runtime error. In both cases the user experience degrades because local evidence fails to become global semantics.

Suggested test shape. Add one corpus example in which a fact from boolean guard theory is only visible because of the presence of branch-guard sites. The test should assert both a synthesized contract effect and a provenance explanation.

0.23.39 Interaction: Branch-guard sites with Nullability and optionality

Why the interaction matters. Branch-guard sites are one of the places where nullability and optionality can become genuinely useful to ordinary users. If the interaction is modeled poorly, either the system loses semantic information or it asks for annotations that should have been avoidable.

Concrete implementation rule. The cover builder should create enough overlap structure that the local vocabulary of nullability and optionality can be restricted onto the relevant coordinates of branch-guard sites. The local solver should emit not only solved facts but also viability and transport metadata. The backward pass should then decide whether the interaction contributes to safe input synthesis, while the forward pass decides whether it contributes to output enrichment.

Typical failure mode if omitted. Without this interaction the system is likely either to over-approximate and lose useful guarantees, or to under-approximate and miss a reachable runtime error. In both cases the user experience degrades because local evidence fails to become global semantics.

Suggested test shape. Add one corpus example in which a fact from nullability and optionality is only visible because of the presence of branch-guard sites. The test should assert both a synthesized contract effect and a provenance explanation.

0.23.40 Interaction: Branch-guard sites with Heap and protocol theory

Why the interaction matters. Branch-guard sites are one of the places where heap and protocol theory can become genuinely useful to ordinary users. If the interaction is modeled poorly, either the system loses semantic information or it asks for annotations that should have been avoidable.

Concrete implementation rule. The cover builder should create enough overlap structure that the local vocabulary of heap and protocol theory can be restricted onto the relevant coordinates of branch-guard sites. The local solver should emit not only solved facts but also viability and transport metadata. The backward pass should then decide whether the interaction contributes to safe input synthesis, while the forward pass decides whether it contributes to output enrichment.

Typical failure mode if omitted. Without this interaction the system is likely either to over-approximate and lose useful guarantees, or to under-approximate and miss a reachable runtime error. In both cases the user experience degrades because local evidence fails to become global semantics.

Suggested test shape. Add one corpus example in which a fact from heap and protocol theory is only visible because of the presence of branch-guard sites. The test should assert both a synthesized contract effect and a provenance explanation.

0.23.41 Interaction: Branch-guard sites with Tensor shape theory

Why the interaction matters. Branch-guard sites are one of the places where tensor shape theory can become genuinely useful to ordinary users. If the interaction is modeled poorly, either the system loses semantic information or it asks for annotations that should have been avoidable.

Concrete implementation rule. The cover builder should create enough overlap structure that the local vocabulary of tensor shape theory can be restricted onto the relevant coordinates of branch-guard sites. The local solver should emit not only solved facts but also viability and transport metadata. The backward pass should then decide whether the interaction contributes to safe input synthesis, while the forward pass decides whether it contributes to output enrichment.

Typical failure mode if omitted. Without this interaction the system is likely either to over-approximate and lose useful guarantees, or to under-approximate and miss a reachable runtime error. In both cases the user experience degrades because local evidence fails to become global semantics.

Suggested test shape. Add one corpus example in which a fact from tensor shape theory is only visible because of the presence of branch-guard sites. The test should assert both a synthesized contract effect and a provenance explanation.

0.23.42 Interaction: Branch-guard sites with Tensor order theory

Why the interaction matters. Branch-guard sites are one of the places where tensor order theory can become genuinely useful to ordinary users. If the interaction is modeled poorly, either the system loses semantic information or it asks for annotations that should have been avoidable.

Concrete implementation rule. The cover builder should create enough overlap structure that the local vocabulary of tensor order theory can be restricted onto the relevant coordinates of branch-guard sites. The local solver should emit not only solved facts but also viability and transport metadata. The backward pass should then decide whether the interaction contributes to safe input synthesis, while the forward pass decides whether it contributes to output enrichment.

Typical failure mode if omitted. Without this interaction the system is likely either to over-approximate and lose useful guarantees, or to under-approximate and miss a reachable runtime error. In both cases the user experience degrades because local evidence fails to become global semantics.

Suggested test shape. Add one corpus example in which a fact from tensor order theory is only visible because of the presence of branch-guard sites. The test should assert both a synthesized contract effect and a provenance explanation.

0.23.43 Interaction: Branch-guard sites with Tensor indexing theory

Why the interaction matters. Branch-guard sites are one of the places where tensor indexing theory can become genuinely useful to ordinary users. If the interaction is modeled poorly, either the system loses semantic information or it asks for annotations that should have been avoidable.

Concrete implementation rule. The cover builder should create enough overlap structure that the local vocabulary of tensor indexing theory can be restricted onto the relevant coordinates of branch-guard sites. The local solver should emit not only solved facts but also viability and transport metadata. The backward pass should then decide whether the interaction contributes to safe input synthesis, while the forward pass decides whether it contributes to output enrichment.

Typical failure mode if omitted. Without this interaction the system is likely either to over-approximate and lose useful guarantees, or to under-approximate and miss a reachable runtime error. In both cases the user experience degrades because local evidence fails to become global semantics.

Suggested test shape. Add one corpus example in which a fact from tensor indexing theory is only visible because of the presence of branch-guard sites. The test should assert both a synthesized contract effect and a provenance explanation.

0.23.44 Interaction: Branch-guard sites with Device and dtype compatibility theory

Why the interaction matters. Branch-guard sites are one of the places where device and dtype compatibility theory can become genuinely useful to ordinary users. If the interaction is modeled poorly, either the system loses semantic information or it asks for annotations that should have been avoidable.

Concrete implementation rule. The cover builder should create enough overlap structure that the local vocabulary of device and dtype compatibility theory can be restricted onto the relevant coordinates of branch-guard sites. The local solver should emit not only solved facts but also viability and transport metadata. The backward pass should then decide whether the interaction contributes to safe input synthesis, while the forward pass decides whether it contributes to output enrichment.

Typical failure mode if omitted. Without this interaction the system is likely either to over-approximate and lose useful guarantees, or to under-approximate and miss a reachable runtime error. In both cases the user experience degrades because local evidence fails to become global semantics.

Suggested test shape. Add one corpus example in which a fact from device and dtype compatibility theory is only visible because of the presence of branch-guard sites. The test should assert both a synthesized contract effect and a provenance explanation.

0.23.45 Interaction: Branch-guard sites with Sequence and collection theory

Why the interaction matters. Branch-guard sites are one of the places where sequence and collection theory can become genuinely useful to ordinary users. If the interaction is modeled poorly, either the system loses semantic information or it asks for annotations that should have been avoidable.

Concrete implementation rule. The cover builder should create enough overlap structure that the local vocabulary of sequence and collection theory can be restricted onto the relevant coordinates of branch-guard sites. The local solver should emit not only solved facts but also viability and transport metadata. The backward pass should then decide whether the interaction contributes to safe input synthesis, while the forward pass decides whether it contributes to output enrichment.

Typical failure mode if omitted. Without this interaction the system is likely either to over-approximate and lose useful guarantees, or to under-approximate and miss a reachable runtime error. In both cases the user experience degrades because local evidence fails to become global semantics.

Suggested test shape. Add one corpus example in which a fact from sequence and collection theory is only visible because of the presence of branch-guard sites. The test should assert both a synthesized contract effect and a provenance explanation.

0.23.46 Interaction: Branch-guard sites with Exception theory

Why the interaction matters. Branch-guard sites are one of the places where exception theory can become genuinely useful to ordinary users. If the interaction is modeled poorly, either the system loses semantic information or it asks for annotations that should have been avoidable.

Concrete implementation rule. The cover builder should create enough overlap structure that the local vocabulary of exception theory can be restricted onto the relevant coordinates of branch-guard sites. The local solver should emit not only solved facts but also viability and transport metadata. The backward pass should then decide whether the interaction contributes to safe input synthesis, while the forward pass decides whether it contributes to output enrichment.

Typical failure mode if omitted. Without this interaction the system is likely either to over-approximate and lose useful guarantees, or to under-approximate and miss a reachable runtime error. In both cases the user experience degrades because local evidence fails to become global semantics.

Suggested test shape. Add one corpus example in which a fact from exception theory is only visible because of the presence of branch-guard sites. The test should assert both a synthesized contract effect and a provenance explanation.

0.23.47 Interaction: Branch-guard sites with Witness and equality transport theory

Why the interaction matters. Branch-guard sites are one of the places where witness and equality transport theory can become genuinely useful to ordinary users. If the interaction is modeled poorly, either the system loses semantic information or it asks for annotations that should have been avoidable.

Concrete implementation rule. The cover builder should create enough overlap structure that the local vocabulary of witness and equality transport theory can be restricted onto the relevant coordinates of branch-guard sites. The local solver should emit not only solved facts but also viability and transport metadata. The backward pass should then decide whether the interaction contributes to safe input synthesis, while the forward pass decides whether it contributes to output enrichment.

Typical failure mode if omitted. Without this interaction the system is likely either to over-approximate and lose useful guarantees, or to under-approximate and miss a reachable runtime error. In both cases the user experience degrades because local evidence fails to become global semantics.

Suggested test shape. Add one corpus example in which a fact from witness and equality transport theory is only visible because of the presence of branch-guard sites. The test should assert both a synthesized contract effect and a provenance explanation.

0.23.48 Interaction: Branch-guard sites with Loop invariant and decreases theory

Why the interaction matters. Branch-guard sites are one of the places where loop invariant and decreases theory can become genuinely useful to ordinary users. If the interaction is modeled poorly, either the system loses semantic information or it asks for annotations that should have been avoidable.

Concrete implementation rule. The cover builder should create enough overlap structure that the local vocabulary of loop invariant and decreases theory can be restricted onto the relevant coordinates of branch-guard sites. The local solver should emit not only solved facts but also viability and transport metadata. The backward pass should then decide whether the interaction contributes to safe input synthesis, while the forward pass decides whether it contributes to output enrichment.

Typical failure mode if omitted. Without this interaction the system is likely either to over-approximate and lose useful guarantees, or to under-approximate and miss a reachable runtime error. In both cases the user experience degrades because local evidence fails to become global semantics.

Suggested test shape. Add one corpus example in which a fact from loop invariant and decreases theory is only visible because of the presence of branch-guard sites. The test should assert both a synthesized contract effect and a provenance explanation.

0.23.49 Interaction: Call-result sites with Linear arithmetic and intervals

Why the interaction matters. Call-result sites are one of the places where linear arithmetic and intervals can become genuinely useful to ordinary users. If the interaction is modeled poorly, either the system loses semantic information or it asks for annotations that should have been avoidable.

Concrete implementation rule. The cover builder should create enough overlap structure that the local vocabulary of linear arithmetic and intervals can be restricted onto the relevant coordinates of call-result sites. The local solver should emit not only solved facts but also viability and transport metadata. The backward pass should then decide whether the interaction contributes to safe input synthesis, while the forward pass decides whether it contributes to output enrichment.

Typical failure mode if omitted. Without this interaction the system is likely either to over-approximate and lose useful guarantees, or to under-approximate and miss a reachable runtime error. In both cases the user experience degrades because local evidence fails to become global semantics.

Suggested test shape. Add one corpus example in which a fact from linear arithmetic and intervals is only visible because of the presence of call-result sites. The test should assert both a synthesized contract effect and a provenance explanation.

0.23.50 Interaction: Call-result sites with Boolean guard theory

Why the interaction matters. Call-result sites are one of the places where boolean guard theory can become genuinely useful to ordinary users. If the interaction is modeled poorly, either the system loses semantic information or it asks for annotations that should have been avoidable.

Concrete implementation rule. The cover builder should create enough overlap structure that the local vocabulary of boolean guard theory can be restricted onto the relevant coordinates of call-result sites. The local solver should emit not only solved facts but also viability and transport metadata. The backward pass should then decide whether the interaction contributes to safe input synthesis, while the forward pass decides whether it contributes to output enrichment.

Typical failure mode if omitted. Without this interaction the system is likely either to over-approximate and lose useful guarantees, or to under-approximate and miss a reachable runtime error. In both cases the user experience degrades because local evidence fails to become global semantics.

Suggested test shape. Add one corpus example in which a fact from boolean guard theory is only visible because of the presence of call-result sites. The test should assert both a synthesized contract effect and a provenance explanation.

0.23.51 Interaction: Call-result sites with Nullability and optionality

Why the interaction matters. Call-result sites are one of the places where nullability and optionality can become genuinely useful to ordinary users. If the interaction is modeled poorly, either the system loses semantic information or it asks for annotations that should have been avoidable.

Concrete implementation rule. The cover builder should create enough overlap structure that the local vocabulary of nullability and optionality can be restricted onto the relevant coordinates of call-result sites. The local solver should emit not only solved facts but also viability and transport metadata. The backward pass should then decide whether the interaction contributes to safe input synthesis, while the forward pass decides whether it contributes to output enrichment.

Typical failure mode if omitted. Without this interaction the system is likely either to over-approximate and lose useful guarantees, or to under-approximate and miss a reachable runtime error. In both cases the user experience degrades because local evidence fails to become global semantics.

Suggested test shape. Add one corpus example in which a fact from nullability and optionality is only visible because of the presence of call-result sites. The test should assert both a synthesized contract effect and a provenance explanation.

0.23.52 Interaction: Call-result sites with Heap and protocol theory

Why the interaction matters. Call-result sites are one of the places where heap and protocol theory can become genuinely useful to ordinary users. If the interaction is modeled poorly, either the system loses semantic information or it asks for annotations that should have been avoidable.

Concrete implementation rule. The cover builder should create enough overlap structure that the local vocabulary of heap and protocol theory can be restricted onto the relevant coordinates of call-result sites. The local solver should emit not only solved facts but also viability and transport metadata. The backward pass should then decide whether the interaction contributes to safe input synthesis, while the forward pass decides whether it contributes to output enrichment.

Typical failure mode if omitted. Without this interaction the system is likely either to over-approximate and lose useful guarantees, or to under-approximate and miss a reachable runtime error. In both cases the user experience degrades because local evidence fails to become global semantics.

Suggested test shape. Add one corpus example in which a fact from heap and protocol theory is only visible because of the presence of call-result sites. The test should assert both a synthesized contract effect and a provenance explanation.

0.23.53 Interaction: Call-result sites with Tensor shape theory

Why the interaction matters. Call-result sites are one of the places where tensor shape theory can become genuinely useful to ordinary users. If the interaction is modeled poorly, either the system loses semantic information or it asks for annotations that should have been avoidable.

Concrete implementation rule. The cover builder should create enough overlap structure that the local vocabulary of tensor shape theory can be restricted onto the relevant coordinates of call-result sites. The local solver should emit not only solved facts but also viability and transport metadata. The backward pass should then decide whether the interaction contributes to safe input synthesis, while the forward pass decides whether it contributes to output enrichment.

Typical failure mode if omitted. Without this interaction the system is likely either to over-approximate and lose useful guarantees, or to under-approximate and miss a reachable runtime error. In both cases the user experience degrades because local evidence fails to become global semantics.

Suggested test shape. Add one corpus example in which a fact from tensor shape theory is only visible because of the presence of call-result sites. The test should assert both a synthesized contract effect and a provenance explanation.

0.23.54 Interaction: Call-result sites with Tensor order theory

Why the interaction matters. Call-result sites are one of the places where tensor order theory can become genuinely useful to ordinary users. If the interaction is modeled poorly, either the system loses semantic information or it asks for annotations that should have been avoidable.

Concrete implementation rule. The cover builder should create enough overlap structure that the local vocabulary of tensor order theory can be restricted onto the relevant coordinates of call-result sites. The local solver should emit not only solved facts but also viability and transport metadata. The backward pass should then decide whether the interaction contributes to safe input synthesis, while the forward pass decides whether it contributes to output enrichment.

Typical failure mode if omitted. Without this interaction the system is likely either to over-approximate and lose useful guarantees, or to under-approximate and miss a reachable runtime error. In both cases the user experience degrades because local evidence fails to become global semantics.

Suggested test shape. Add one corpus example in which a fact from tensor order theory is only visible because of the presence of call-result sites. The test should assert both a synthesized contract effect and a provenance explanation.

0.23.55 Interaction: Call-result sites with Tensor indexing theory

Why the interaction matters. Call-result sites are one of the places where tensor indexing theory can become genuinely useful to ordinary users. If the interaction is modeled poorly, either the system loses semantic information or it asks for annotations that should have been avoidable.

Concrete implementation rule. The cover builder should create enough overlap structure that the local vocabulary of tensor indexing theory can be restricted onto the relevant coordinates of call-result sites. The local solver should emit not only solved facts but also viability and transport metadata. The backward pass should then decide whether the interaction contributes to safe input synthesis, while the forward pass decides whether it contributes to output enrichment.

Typical failure mode if omitted. Without this interaction the system is likely either to over-approximate and lose useful guarantees, or to under-approximate and miss a reachable runtime error. In both cases the user experience degrades because local evidence fails to become global semantics.

Suggested test shape. Add one corpus example in which a fact from tensor indexing theory is only visible because of the presence of call-result sites. The test should assert both a synthesized contract effect and a provenance explanation.

0.23.56 Interaction: Call-result sites with Device and dtype compatibility theory

Why the interaction matters. Call-result sites are one of the places where device and dtype compatibility theory can become genuinely useful to ordinary users. If the interaction is modeled poorly, either the system loses semantic information or it asks for annotations that should have been avoidable.

Concrete implementation rule. The cover builder should create enough overlap structure that the local vocabulary of device and dtype compatibility theory can be restricted onto the relevant coordinates of call-result sites. The local solver should emit not only solved facts but also viability and transport metadata. The backward pass should then decide whether the interaction contributes to safe input synthesis, while the forward pass decides whether it contributes to output enrichment.

Typical failure mode if omitted. Without this interaction the system is likely either to over-approximate and lose useful guarantees, or to under-approximate and miss a reachable runtime error. In both cases the user experience degrades because local evidence fails to become global semantics.

Suggested test shape. Add one corpus example in which a fact from device and dtype compatibility theory is only visible because of the presence of call-result sites. The test should assert both a synthesized contract effect and a provenance explanation.

0.23.57 Interaction: Call-result sites with Sequence and collection theory

Why the interaction matters. Call-result sites are one of the places where sequence and collection theory can become genuinely useful to ordinary users. If the interaction is modeled poorly, either the system loses semantic information or it asks for annotations that should have been avoidable.

Concrete implementation rule. The cover builder should create enough overlap structure that the local vocabulary of sequence and collection theory can be restricted onto the relevant coordinates of call-result sites. The local solver should emit not only solved facts but also viability and transport metadata. The backward pass should then decide whether the interaction contributes to safe input synthesis, while the forward pass decides whether it contributes to output enrichment.

Typical failure mode if omitted. Without this interaction the system is likely either to over-approximate and lose useful guarantees, or to under-approximate and miss a reachable runtime error. In both cases the user experience degrades because local evidence fails to become global semantics.

Suggested test shape. Add one corpus example in which a fact from sequence and collection theory is only visible because of the presence of call-result sites. The test should assert both a synthesized contract effect and a provenance explanation.

0.23.58 Interaction: Call-result sites with Exception theory

Why the interaction matters. Call-result sites are one of the places where exception theory can become genuinely useful to ordinary users. If the interaction is modeled poorly, either the system loses semantic information or it asks for annotations that should have been avoidable.

Concrete implementation rule. The cover builder should create enough overlap structure that the local vocabulary of exception theory can be restricted onto the relevant coordinates of call-result sites. The local solver should emit not only solved facts but also viability and transport metadata. The backward pass should then decide whether the interaction contributes to safe input synthesis, while the forward pass decides whether it contributes to output enrichment.

Typical failure mode if omitted. Without this interaction the system is likely either to over-approximate and lose useful guarantees, or to under-approximate and miss a reachable runtime error. In both cases the user experience degrades because local evidence fails to become global semantics.

Suggested test shape. Add one corpus example in which a fact from exception theory is only visible because of the presence of call-result sites. The test should assert both a synthesized contract effect and a provenance explanation.

0.23.59 Interaction: Call-result sites with Witness and equality transport theory

Why the interaction matters. Call-result sites are one of the places where witness and equality transport theory can become genuinely useful to ordinary users. If the interaction is modeled poorly, either the system loses semantic information or it asks for annotations that should have been avoidable.

Concrete implementation rule. The cover builder should create enough overlap structure that the local vocabulary of witness and equality transport theory can be restricted onto the relevant coordinates of call-result sites. The local solver should emit not only solved facts but also viability and transport metadata. The backward pass should then decide whether the interaction contributes to safe input synthesis, while the forward pass decides whether it contributes to output enrichment.

Typical failure mode if omitted. Without this interaction the system is likely either to over-approximate and lose useful guarantees, or to under-approximate and miss a reachable runtime error. In both cases the user experience degrades because local evidence fails to become global semantics.

Suggested test shape. Add one corpus example in which a fact from witness and equality transport theory is only visible because of the presence of call-result sites. The test should assert both a synthesized contract effect and a provenance explanation.

0.23.60 Interaction: Call-result sites with Loop invariant and decreases theory

Why the interaction matters. Call-result sites are one of the places where loop invariant and decreases theory can become genuinely useful to ordinary users. If the interaction is modeled poorly, either the system loses semantic information or it asks for annotations that should have been avoidable.

Concrete implementation rule. The cover builder should create enough overlap structure that the local vocabulary of loop invariant and decreases theory can be restricted onto the relevant coordinates of call-result sites. The local solver should emit not only solved facts but also viability and transport metadata. The backward pass should then decide whether the interaction contributes to safe input synthesis, while the forward pass decides whether it contributes to output enrichment.

Typical failure mode if omitted. Without this interaction the system is likely either to over-approximate and lose useful guarantees, or to under-approximate and miss a reachable runtime error. In both cases the user experience degrades because local evidence fails to become global semantics.

Suggested test shape. Add one corpus example in which a fact from loop invariant and decreases theory is only visible because of the presence of call-result sites. The test should assert both a synthesized contract effect and a provenance explanation.

0.23.61 Interaction: Tensor-shape sites with Linear arithmetic and intervals

Why the interaction matters. Tensor-shape sites are one of the places where linear arithmetic and intervals can become genuinely useful to ordinary users. If the interaction is modeled poorly, either the system loses semantic information or it asks for annotations that should have been avoidable.

Concrete implementation rule. The cover builder should create enough overlap structure that the local vocabulary of linear arithmetic and intervals can be restricted onto the relevant coordinates of tensor-shape sites. The local solver should emit not only solved facts but also viability and transport metadata. The backward pass should then decide whether the interaction contributes to safe input synthesis, while the forward pass decides whether it contributes to output enrichment.

Typical failure mode if omitted. Without this interaction the system is likely either to over-approximate and lose useful guarantees, or to under-approximate and miss a reachable runtime error. In both cases the user experience degrades because local evidence fails to become global semantics.

Suggested test shape. Add one corpus example in which a fact from linear arithmetic and intervals is only visible because of the presence of tensor-shape sites. The test should assert both a synthesized contract effect and a provenance explanation.

0.23.62 Interaction: Tensor-shape sites with Boolean guard theory

Why the interaction matters. Tensor-shape sites are one of the places where boolean guard theory can become genuinely useful to ordinary users. If the interaction is modeled poorly, either the system loses semantic information or it asks for annotations that should have been avoidable.

Concrete implementation rule. The cover builder should create enough overlap structure that the local vocabulary of boolean guard theory can be restricted onto the relevant coordinates of tensor-shape sites. The local solver should emit not only solved facts but also viability and transport metadata. The backward pass should then decide whether the interaction contributes to safe input synthesis, while the forward pass decides whether it contributes to output enrichment.

Typical failure mode if omitted. Without this interaction the system is likely either to over-approximate and lose useful guarantees, or to under-approximate and miss a reachable runtime error. In both cases the user experience degrades because local evidence fails to become global semantics.

Suggested test shape. Add one corpus example in which a fact from boolean guard theory is only visible because of the presence of tensor-shape sites. The test should assert both a synthesized contract effect and a provenance explanation.

0.23.63 Interaction: Tensor-shape sites with Nullability and optionality

Why the interaction matters. Tensor-shape sites are one of the places where nullability and optionality can become genuinely useful to ordinary users. If the interaction is modeled poorly, either the system loses semantic information or it asks for annotations that should have been avoidable.

Concrete implementation rule. The cover builder should create enough overlap structure that the local vocabulary of nullability and optionality can be restricted onto the relevant coordinates of tensor-shape sites. The local solver should emit not only solved facts but also viability and transport metadata. The backward pass should then decide whether the interaction contributes to safe input synthesis, while the forward pass decides whether it contributes to output enrichment.

Typical failure mode if omitted. Without this interaction the system is likely either to over-approximate and lose useful guarantees, or to under-approximate and miss a reachable runtime error. In both cases the user experience degrades because local evidence fails to become global semantics.

Suggested test shape. Add one corpus example in which a fact from nullability and optionality is only visible because of the presence of tensor-shape sites. The test should assert both a synthesized contract effect and a provenance explanation.

0.23.64 Interaction: Tensor-shape sites with Heap and protocol theory

Why the interaction matters. Tensor-shape sites are one of the places where heap and protocol theory can become genuinely useful to ordinary users. If the interaction is modeled poorly, either the system loses semantic information or it asks for annotations that should have been avoidable.

Concrete implementation rule. The cover builder should create enough overlap structure that the local vocabulary of heap and protocol theory can be restricted onto the relevant coordinates of tensor-shape sites. The local solver should emit not only solved facts but also viability and transport metadata. The backward pass should then decide whether the interaction contributes to safe input synthesis, while the forward pass decides whether it contributes to output enrichment.

Typical failure mode if omitted. Without this interaction the system is likely either to over-approximate and lose useful guarantees, or to under-approximate and miss a reachable runtime error. In both cases the user experience degrades because local evidence fails to become global semantics.

Suggested test shape. Add one corpus example in which a fact from heap and protocol theory is only visible because of the presence of tensor-shape sites. The test should assert both a synthesized contract effect and a provenance explanation.

0.23.65 Interaction: Tensor-shape sites with Tensor shape theory

Why the interaction matters. Tensor-shape sites are one of the places where tensor shape theory can become genuinely useful to ordinary users. If the interaction is modeled poorly, either the system loses semantic information or it asks for annotations that should have been avoidable.

Concrete implementation rule. The cover builder should create enough overlap structure that the local vocabulary of tensor shape theory can be restricted onto the relevant coordinates of tensor-shape sites. The local solver should emit not only solved facts but also viability and transport metadata. The backward pass should then decide whether the interaction contributes to safe input synthesis, while the forward pass decides whether it contributes to output enrichment.

Typical failure mode if omitted. Without this interaction the system is likely either to over-approximate and lose useful guarantees, or to under-approximate and miss a reachable runtime error. In both cases the user experience degrades because local evidence fails to become global semantics.

Suggested test shape. Add one corpus example in which a fact from tensor shape theory is only visible because of the presence of tensor-shape sites. The test should assert both a synthesized contract effect and a provenance explanation.

0.23.66 Interaction: Tensor-shape sites with Tensor order theory

Why the interaction matters. Tensor-shape sites are one of the places where tensor order theory can become genuinely useful to ordinary users. If the interaction is modeled poorly, either the system loses semantic information or it asks for annotations that should have been avoidable.

Concrete implementation rule. The cover builder should create enough overlap structure that the local vocabulary of tensor order theory can be restricted onto the relevant coordinates of tensor-shape sites. The local solver should emit not only solved facts but also viability and transport metadata. The backward pass should then decide whether the interaction contributes to safe input synthesis, while the forward pass decides whether it contributes to output enrichment.

Typical failure mode if omitted. Without this interaction the system is likely either to over-approximate and lose useful guarantees, or to under-approximate and miss a reachable runtime error. In both cases the user experience degrades because local evidence fails to become global semantics.

Suggested test shape. Add one corpus example in which a fact from tensor order theory is only visible because of the presence of tensor-shape sites. The test should assert both a synthesized contract effect and a provenance explanation.

0.23.67 Interaction: Tensor-shape sites with Tensor indexing theory

Why the interaction matters. Tensor-shape sites are one of the places where tensor indexing theory can become genuinely useful to ordinary users. If the interaction is modeled poorly, either the system loses semantic information or it asks for annotations that should have been avoidable.

Concrete implementation rule. The cover builder should create enough overlap structure that the local vocabulary of tensor indexing theory can be restricted onto the relevant coordinates of tensor-shape sites. The local solver should emit not only solved facts but also viability and transport metadata. The backward pass should then decide whether the interaction contributes to safe input synthesis, while the forward pass decides whether it contributes to output enrichment.

Typical failure mode if omitted. Without this interaction the system is likely either to over-approximate and lose useful guarantees, or to under-approximate and miss a reachable runtime error. In both cases the user experience degrades because local evidence fails to become global semantics.

Suggested test shape. Add one corpus example in which a fact from tensor indexing theory is only visible because of the presence of tensor-shape sites. The test should assert both a synthesized contract effect and a provenance explanation.

0.23.68 Interaction: Tensor-shape sites with Device and dtype compatibility theory

Why the interaction matters. Tensor-shape sites are one of the places where device and dtype compatibility theory can become genuinely useful to ordinary users. If the interaction is modeled poorly, either the system loses semantic information or it asks for annotations that should have been avoidable.

Concrete implementation rule. The cover builder should create enough overlap structure that the local vocabulary of device and dtype compatibility theory can be restricted onto the relevant coordinates of tensor-shape sites. The local solver should emit not only solved facts but also viability and transport metadata. The backward pass should then decide whether the interaction contributes to safe input synthesis, while the forward pass decides whether it contributes to output enrichment.

Typical failure mode if omitted. Without this interaction the system is likely either to over-approximate and lose useful guarantees, or to under-approximate and miss a reachable runtime error. In both cases the user experience degrades because local evidence fails to become global semantics.

Suggested test shape. Add one corpus example in which a fact from device and dtype compatibility theory is only visible because of the presence of tensor-shape sites. The test should assert both a synthesized contract effect and a provenance explanation.

0.23.69 Interaction: Tensor-shape sites with Sequence and collection theory

Why the interaction matters. Tensor-shape sites are one of the places where sequence and collection theory can become genuinely useful to ordinary users. If the interaction is modeled poorly, either the system loses semantic information or it asks for annotations that should have been avoidable.

Concrete implementation rule. The cover builder should create enough overlap structure that the local vocabulary of sequence and collection theory can be restricted onto the relevant coordinates of tensor-shape sites. The local solver should emit not only solved facts but also viability and transport metadata. The backward pass should then decide whether the interaction contributes to safe input synthesis, while the forward pass decides whether it contributes to output enrichment.

Typical failure mode if omitted. Without this interaction the system is likely either to over-approximate and lose useful guarantees, or to under-approximate and miss a reachable runtime error. In both cases the user experience degrades because local evidence fails to become global semantics.

Suggested test shape. Add one corpus example in which a fact from sequence and collection theory is only visible because of the presence of tensor-shape sites. The test should assert both a synthesized contract effect and a provenance explanation.

0.23.70 Interaction: Tensor-shape sites with Exception theory

Why the interaction matters. Tensor-shape sites are one of the places where exception theory can become genuinely useful to ordinary users. If the interaction is modeled poorly, either the system loses semantic information or it asks for annotations that should have been avoidable.

Concrete implementation rule. The cover builder should create enough overlap structure that the local vocabulary of exception theory can be restricted onto the relevant coordinates of tensor-shape sites. The local solver should emit not only solved facts but also viability and transport metadata. The backward pass should then decide whether the interaction contributes to safe input synthesis, while the forward pass decides whether it contributes to output enrichment.

Typical failure mode if omitted. Without this interaction the system is likely either to over-approximate and lose useful guarantees, or to under-approximate and miss a reachable runtime error. In both cases the user experience degrades because local evidence fails to become global semantics.

Suggested test shape. Add one corpus example in which a fact from exception theory is only visible because of the presence of tensor-shape sites. The test should assert both a synthesized contract effect and a provenance explanation.

0.24 Evaluation, benchmarks, and success criteria

A monograph of this scope should end with measurable criteria. The rewrite succeeds if it improves four things simultaneously. First, annotation economy: ordinary users should obtain strong contracts and strong error explanations with little or no added text. Second, semantic richness: the analyzer should recover input refinements, output guarantees, local intermediate sections, and theorem- level transport effects that the current atom-centric architecture would either miss or flatten. Third, honesty: the system must clearly separate solver-backed, proof-backed, trace-backed, and residual coordinates. Fourth, migration plausibility: new artifacts such as site covers, local sections, and obstruction bases must fit naturally around the current packages and example corpus.

Benchmarking should therefore record more than pass/fail. Useful metrics include: number of constructed sites after sparsification; overlap density; number of error-sensitive sites proved empty under synthesized preconditions; size of the minimal safe input section; information score of the maximal output section; number of local intermediate values carrying nontrivial sections; count of residual obstruction coordinates before and after user seeds; leverage of proof artifacts on unannotated client modules; stability of synthesized contracts under small wrapper refactorings; and runtime overhead of cover construction, local solving, and gluing. For Torch specifically, the benchmark suite should measure how often the system prevents library-specific errors without manual annotation and how often it recovers semantically rich output summaries such as sortedness, selector provenance, or exact shape.

The final success criterion is conceptual: users should come away believing that native Python can support serious dependent reasoning without ceasing to be native Python. If the sheaf-descent rewrite works, the explanation will be simple. Rich semantics becomes possible not because the

tool guessed a lucky set of predicates, but because the program, proofs, library calls, and traces jointly determine a global semantic object whose ambiguity can be measured, reduced, and rendered honestly.

0.25 Staged migration plan

The migration plan should be staged so that every phase lands with stable tests and without invalidating the current user-facing pipeline.

Phase 1: Introduce explicit site-cover and local-section datatypes in the core, with compatibility shims from current semantic signatures.

Phase 2: Build cover synthesis from the existing AST, CFG, SSA, and lineage infrastructure, and snapshot the resulting cover on selected examples.

Phase 3: Upgrade split PyTorch packs to export site constructors and viability hooks while preserving current example behavior.

Phase 4: Add backward safe-input synthesis and forward output synthesis on a small family of site kinds: sort, shape, and indexing.

Phase 5: Generalize runtime-error sites across standard Python and heap/protocol reasoning.

Phase 6: Compile theorem modules into transport maps and proof sites.

Phase 7: Replace or wrap current frontier extraction with section-summary frontiers and obstruction-aware repairs.

Phase 8: Migrate renderers so every analysis can be viewed as synthesized Python contracts, diagnostics, proof goals, or telemetry.

Phase 9: Expand the corpus and benchmark suite until the new architecture is the default path for both Torch-heavy and ordinary-Python examples.

Each phase should preserve one invariant: if the new architecture makes a claim, that claim must be at least as explicit about trust status as the current one. This invariant matters more than temporary performance regressions, because once trust becomes blurry the rest of the design loses its point.

0.26 Extended implementation profiles

This appendix exists to make the implementation story extremely explicit. It records package-theory pairings that matter for a realistic build.

0.26.1 Package `src/deppy/assist` with Linear arithmetic and intervals

Integration objective. The package `src/deppy/assist` must interact cleanly with the theory family linear arithmetic and intervals. The reason is practical: the package owns part of the data path through which non-negativity, bounds, lengths, simple affine equalities becomes visible, audited, and eventually rendered.

Required interface. At minimum the integration should expose normalized inputs, site-aware outputs, provenance, and failure reasons. The package should neither hide theory-relevant structure from the cover builder nor flatten theory results so aggressively that renderers lose meaning.

Typical evidence source. In practice the evidence will come from argument bounds, list lengths, loop counters, and tensor extents. The integration should preserve enough of that source structure that local solvers using local SMT over arithmetic fragments can operate without pack-specific hacks.

Expected user-facing payoff. If implemented well, the payoff is IndexError, ZeroDivisionError, and shape-side precondition pullback. This should appear not only in benchmark metrics but also in concrete rendered contracts, diagnostics, or proof goals.

0.26.2 Package `src/deppy/assist` with Boolean guard theory

Integration objective. The package `src/deppy/assist` must interact cleanly with the theory family boolean guard theory. The reason is practical: the package owns part of the data path through which guard polarity, assertion scope, and branch-conditioned refinement becomes visible, audited, and eventually rendered.

Required interface. At minimum the integration should expose normalized inputs, site-aware outputs, provenance, and failure reasons. The package should neither hide theory-relevant structure from the cover builder nor flatten theory results so aggressively that renderers lose meaning.

Typical evidence source. In practice the evidence will come from if-statements, asserts, early returns, and exception guards. The integration should preserve enough of that source structure that local solvers using normalization plus implication checks can operate without pack-specific hacks.

Expected user-facing payoff. If implemented well, the payoff is control-dependent safety and lightweight proof seed extraction. This should appear not only in benchmark metrics but also in concrete rendered contracts, diagnostics, or proof goals.

0.26.3 Package `src/deppy/assist` with Nullability and optionality

Integration objective. The package `src/deppy/assist` must interact cleanly with the theory family nullability and optionality. The reason is practical: the package owns part of the data path through which none checks, existence guards, and optional protocol availability becomes visible, audited, and eventually rendered.

Required interface. At minimum the integration should expose normalized inputs, site-aware outputs, provenance, and failure reasons. The package should neither hide theory-relevant structure from the cover builder nor flatten theory results so aggressively that renderers lose meaning.

Typical evidence source. In practice the evidence will come from ordinary Python branches and constructor code. The integration should preserve enough of that source structure that local solvers using simple symbolic narrowing can operate without pack-specific hacks.

Expected user-facing payoff. If implemented well, the payoff is AttributeError prevention and stronger post-branch outputs. This should appear not only in benchmark metrics but also in concrete rendered contracts, diagnostics, or proof goals.

0.26.4 Package `src/deppy/assist` with Heap and protocol theory

Integration objective. The package `src/deppy/assist` must interact cleanly with the theory family heap and protocol theory. The reason is practical: the package owns part of the data path through which field initialization, protocol method existence, frame conditions, and alias exclusions becomes visible, audited, and eventually rendered.

Required interface. At minimum the integration should expose normalized inputs, site-aware outputs, provenance, and failure reasons. The package should neither hide theory-relevant structure from the cover builder nor flatten theory results so aggressively that renderers lose meaning.

Typical evidence source. In practice the evidence will come from classes, protocols, mutable objects, and resource wrappers. The integration should preserve enough of that source structure that local solvers using bounded symbolic checks plus explicit residuals can operate without pack-specific hacks.

Expected user-facing payoff. If implemented well, the payoff is constructor safety, attribute access, and method-dispatch correctness. This should appear not only in benchmark metrics but also in concrete rendered contracts, diagnostics, or proof goals.

0.26.5 Package `src/deppy/assist` with Tensor shape theory

Integration objective. The package `src/deppy/assist` must interact cleanly with the theory family tensor shape theory. The reason is practical: the package owns part of the data path through which rank, dimension, shape tuples, cardinality, broadcast legality, and reduction support becomes visible, audited, and eventually rendered.

Required interface. At minimum the integration should expose normalized inputs, site-aware outputs, provenance, and failure reasons. The package should neither hide theory-relevant structure from the cover builder nor flatten theory results so aggressively that renderers lose meaning.

Typical evidence source. In practice the evidence will come from Torch tensor operators and shape-sensitive wrappers. The integration should preserve enough of that source structure that local solvers using solver-backed local shape kernels can operate without pack-specific hacks.

Expected user-facing payoff. If implemented well, the payoff is invalid reshape, broadcast mismatch, and exact output-shape synthesis. This should appear not only in benchmark metrics but also in concrete rendered contracts, diagnostics, or proof goals.

0.26.6 Package `src/deppy/assist` with Tensor order theory

Integration objective. The package `src/deppy/assist` must interact cleanly with the theory family tensor order theory. The reason is practical: the package owns part of the data path through

which sortedness, monotonicity, permutation witnesses, argsort relations becomes visible, audited, and eventually rendered.

Required interface. At minimum the integration should expose normalized inputs, site-aware outputs, provenance, and failure reasons. The package should neither hide theory-relevant structure from the cover builder nor flatten theory results so aggressively that renderers lose meaning.

Typical evidence source. In practice the evidence will come from sort-like operators and order-preserving wrappers. The integration should preserve enough of that source structure that local solvers using finite witness templates plus solver-backed subchecks can operate without pack-specific hacks.

Expected user-facing payoff. If implemented well, the payoff is values-vs-indices confusion and richer sorted outputs. This should appear not only in benchmark metrics but also in concrete rendered contracts, diagnostics, or proof goals.

0.26.7 Package `src/deppy/assist` with Tensor indexing theory

Integration objective. The package `src/deppy/assist` must interact cleanly with the theory family tensor indexing theory. The reason is practical: the package owns part of the data path through which selector domains, slice extents, gather/index-select legality, and value provenance becomes visible, audited, and eventually rendered.

Required interface. At minimum the integration should expose normalized inputs, site-aware outputs, provenance, and failure reasons. The package should neither hide theory-relevant structure from the cover builder nor flatten theory results so aggressively that renderers lose meaning.

Typical evidence source. In practice the evidence will come from subscripts, slices, and advanced indexing. The integration should preserve enough of that source structure that local solvers using arithmetic plus pack-specific rules can operate without pack-specific hacks.

Expected user-facing payoff. If implemented well, the payoff is out-of-range selector detection and source-target output relations. This should appear not only in benchmark metrics but also in concrete rendered contracts, diagnostics, or proof goals.

0.26.8 Package `src/deppy/assist` with Device and dtype compatibility theory

Integration objective. The package `src/deppy/assist` must interact cleanly with the theory family device and dtype compatibility theory. The reason is practical: the package owns part of the data path through which device equality, admissible implicit casts, and operator support matrices becomes visible, audited, and eventually rendered.

Required interface. At minimum the integration should expose normalized inputs, site-aware outputs, provenance, and failure reasons. The package should neither hide theory-relevant structure from the cover builder nor flatten theory results so aggressively that renderers lose meaning.

Typical evidence source. In practice the evidence will come from mixed-device or mixed-dtype Torch programs. The integration should preserve enough of that source structure that local solvers using table-driven local checks can operate without pack-specific hacks.

Expected user-facing payoff. If implemented well, the payoff is device mismatch and unsupported operator combinations. This should appear not only in benchmark metrics but also in concrete rendered contracts, diagnostics, or proof goals.

0.26.9 Package `src/deppy/assist` with Sequence and collection theory

Integration objective. The package `src/deppy/assist` must interact cleanly with the theory family sequence and collection theory. The reason is practical: the package owns part of the data path through which non-emptiness, membership, stable length relations, and head/tail legality becomes visible, audited, and eventually rendered.

Required interface. At minimum the integration should expose normalized inputs, site-aware outputs, provenance, and failure reasons. The package should neither hide theory-relevant structure from the cover builder nor flatten theory results so aggressively that renderers lose meaning.

Typical evidence source. In practice the evidence will come from lists, tuples, deques, and iterator adapters. The integration should preserve enough of that source structure that local solvers using simple algebraic checks can operate without pack-specific hacks.

Expected user-facing payoff. If implemented well, the payoff is head-of-empty and unpacking safety. This should appear not only in benchmark metrics but also in concrete rendered contracts, diagnostics, or proof goals.

0.26.10 Package `src/deppy/assist` with Exception theory

Integration objective. The package `src/deppy/assist` must interact cleanly with the theory family exception theory. The reason is practical: the package owns part of the data path through which exceptional output channels, throw sites, and caught-vs-uncaught distinctions becomes visible, audited, and eventually rendered.

Required interface. At minimum the integration should expose normalized inputs, site-aware outputs, provenance, and failure reasons. The package should neither hide theory-relevant structure from the cover builder nor flatten theory results so aggressively that renderers lose meaning.

Typical evidence source. In practice the evidence will come from try/except blocks, library raises, and proof obligations that may fail. The integration should preserve enough of that source structure that local solvers using control-flow plus error-site hooks can operate without pack-specific hacks.

Expected user-facing payoff. If implemented well, the payoff is precise reporting of exceptional contracts and remaining failures. This should appear not only in benchmark metrics but also in concrete rendered contracts, diagnostics, or proof goals.

0.26.11 Package `src/deppy/assist` with Witness and equality transport theory

Integration objective. The package `src/deppy/assist` must interact cleanly with the theory family witness and equality transport theory. The reason is practical: the package owns part of the data path through which dependent pairs, equality witnesses, and explicit transports becomes visible, audited, and eventually rendered.

Required interface. At minimum the integration should expose normalized inputs, site-aware outputs, provenance, and failure reasons. The package should neither hide theory-relevant structure from the cover builder nor flatten theory results so aggressively that renderers lose meaning.

Typical evidence source. In practice the evidence will come from proof-oriented code and witness-returning helpers. The integration should preserve enough of that source structure that local solvers using proof-kernel backed local checking can operate without pack-specific hacks.

Expected user-facing payoff. If implemented well, the payoff is proof reuse and output enrichment. This should appear not only in benchmark metrics but also in concrete rendered contracts, diagnostics, or proof goals.

0.26.12 Package `src/deppy/assist` with Loop invariant and decreases theory

Integration objective. The package `src/deppy/assist` must interact cleanly with the theory family loop invariant and decreases theory. The reason is practical: the package owns part of the data path through which candidate invariants, ranking relations, and preservation constraints becomes visible, audited, and eventually rendered.

Required interface. At minimum the integration should expose normalized inputs, site-aware outputs, provenance, and failure reasons. The package should neither hide theory-relevant structure from the cover builder nor flatten theory results so aggressively that renderers lose meaning.

Typical evidence source. In practice the evidence will come from loop-heavy algorithms and recursive helpers. The integration should preserve enough of that source structure that local solvers using template generation plus bounded checks can operate without pack-specific hacks.

Expected user-facing payoff. If implemented well, the payoff is F*-like capability without abandoning native Python. This should appear not only in benchmark metrics but also in concrete rendered contracts, diagnostics, or proof goals.

0.26.13 Package `src/deppy/contracts` with Linear arithmetic and intervals

Integration objective. The package `src/deppy/contracts` must interact cleanly with the theory family linear arithmetic and intervals. The reason is practical: the package owns part of the data path through which non-negativity, bounds, lengths, simple affine equalities becomes visible, audited, and eventually rendered.

Required interface. At minimum the integration should expose normalized inputs, site-aware outputs, provenance, and failure reasons. The package should neither hide theory-relevant structure from the cover builder nor flatten theory results so aggressively that renderers lose meaning.

Typical evidence source. In practice the evidence will come from argument bounds, list lengths, loop counters, and tensor extents. The integration should preserve enough of that source structure that local solvers using local SMT over arithmetic fragments can operate without pack-specific hacks.

Expected user-facing payoff. If implemented well, the payoff is IndexError, ZeroDivisionError, and shape-side precondition pullback. This should appear not only in benchmark metrics but also in concrete rendered contracts, diagnostics, or proof goals.

0.26.14 Package `src/deppy/contracts` with Boolean guard theory

Integration objective. The package `src/deppy/contracts` must interact cleanly with the theory family boolean guard theory. The reason is practical: the package owns part of the data path through which guard polarity, assertion scope, and branch-conditioned refinement becomes visible, audited, and eventually rendered.

Required interface. At minimum the integration should expose normalized inputs, site-aware outputs, provenance, and failure reasons. The package should neither hide theory-relevant structure from the cover builder nor flatten theory results so aggressively that renderers lose meaning.

Typical evidence source. In practice the evidence will come from if-statements, asserts, early returns, and exception guards. The integration should preserve enough of that source structure that local solvers using normalization plus implication checks can operate without pack-specific hacks.

Expected user-facing payoff. If implemented well, the payoff is control-dependent safety and lightweight proof seed extraction. This should appear not only in benchmark metrics but also in concrete rendered contracts, diagnostics, or proof goals.

0.26.15 Package `src/deppy/contracts` with Nullability and optionality

Integration objective. The package `src/deppy/contracts` must interact cleanly with the theory family nullability and optionality. The reason is practical: the package owns part of the data path through which none checks, existence guards, and optional protocol availability becomes visible, audited, and eventually rendered.

Required interface. At minimum the integration should expose normalized inputs, site-aware outputs, provenance, and failure reasons. The package should neither hide theory-relevant structure from the cover builder nor flatten theory results so aggressively that renderers lose meaning.

Typical evidence source. In practice the evidence will come from ordinary Python branches and constructor code. The integration should preserve enough of that source structure that local solvers using simple symbolic narrowing can operate without pack-specific hacks.

Expected user-facing payoff. If implemented well, the payoff is AttributeError prevention and stronger post-branch outputs. This should appear not only in benchmark metrics but also in concrete rendered contracts, diagnostics, or proof goals.

0.26.16 Package `src/deppy/contracts` with Heap and protocol theory

Integration objective. The package `src/deppy/contracts` must interact cleanly with the theory family heap and protocol theory. The reason is practical: the package owns part of the data path through which field initialization, protocol method existence, frame conditions, and alias exclusions becomes visible, audited, and eventually rendered.

Required interface. At minimum the integration should expose normalized inputs, site-aware outputs, provenance, and failure reasons. The package should neither hide theory-relevant structure from the cover builder nor flatten theory results so aggressively that renderers lose meaning.

Typical evidence source. In practice the evidence will come from classes, protocols, mutable objects, and resource wrappers. The integration should preserve enough of that source structure that local solvers using bounded symbolic checks plus explicit residuals can operate without pack-specific hacks.

Expected user-facing payoff. If implemented well, the payoff is constructor safety, attribute access, and method-dispatch correctness. This should appear not only in benchmark metrics but also in concrete rendered contracts, diagnostics, or proof goals.

0.26.17 Package `src/deppy/contracts` with Tensor shape theory

Integration objective. The package `src/deppy/contracts` must interact cleanly with the theory family tensor shape theory. The reason is practical: the package owns part of the data path through which rank, dimension, shape tuples, cardinality, broadcast legality, and reduction support becomes visible, audited, and eventually rendered.

Required interface. At minimum the integration should expose normalized inputs, site-aware outputs, provenance, and failure reasons. The package should neither hide theory-relevant structure from the cover builder nor flatten theory results so aggressively that renderers lose meaning.

Typical evidence source. In practice the evidence will come from Torch tensor operators and shape-sensitive wrappers. The integration should preserve enough of that source structure that local solvers using solver-backed local shape kernels can operate without pack-specific hacks.

Expected user-facing payoff. If implemented well, the payoff is invalid reshape, broadcast mismatch, and exact output-shape synthesis. This should appear not only in benchmark metrics but also in concrete rendered contracts, diagnostics, or proof goals.

0.26.18 Package `src/deppy/contracts` with Tensor order theory

Integration objective. The package `src/deppy/contracts` must interact cleanly with the theory family tensor order theory. The reason is practical: the package owns part of the data path through which sortedness, monotonicity, permutation witnesses, argsort relations becomes visible, audited, and eventually rendered.

Required interface. At minimum the integration should expose normalized inputs, site-aware outputs, provenance, and failure reasons. The package should neither hide theory-relevant structure from the cover builder nor flatten theory results so aggressively that renderers lose meaning.

Typical evidence source. In practice the evidence will come from sort-like operators and order-preserving wrappers. The integration should preserve enough of that source structure that local solvers using finite witness templates plus solver-backed subchecks can operate without pack-specific hacks.

Expected user-facing payoff. If implemented well, the payoff is values-vs-indices confusion and richer sorted outputs. This should appear not only in benchmark metrics but also in concrete rendered contracts, diagnostics, or proof goals.

0.26.19 Package `src/deppy/contracts` with Tensor indexing theory

Integration objective. The package `src/deppy/contracts` must interact cleanly with the theory family tensor indexing theory. The reason is practical: the package owns part of the data path through which selector domains, slice extents, gather/index-select legality, and value provenance becomes visible, audited, and eventually rendered.

Required interface. At minimum the integration should expose normalized inputs, site-aware outputs, provenance, and failure reasons. The package should neither hide theory-relevant structure from the cover builder nor flatten theory results so aggressively that renderers lose meaning.

Typical evidence source. In practice the evidence will come from subscripts, slices, and advanced indexing. The integration should preserve enough of that source structure that local solvers using arithmetic plus pack-specific rules can operate without pack-specific hacks.

Expected user-facing payoff. If implemented well, the payoff is out-of-range selector detection and source-target output relations. This should appear not only in benchmark metrics but also in concrete rendered contracts, diagnostics, or proof goals.

0.26.20 Package `src/deppy/contracts` with Device and dtype compatibility theory

Integration objective. The package `src/deppy/contracts` must interact cleanly with the theory family device and dtype compatibility theory. The reason is practical: the package owns part of the data path through which device equality, admissible implicit casts, and operator support matrices becomes visible, audited, and eventually rendered.

Required interface. At minimum the integration should expose normalized inputs, site-aware outputs, provenance, and failure reasons. The package should neither hide theory-relevant structure from the cover builder nor flatten theory results so aggressively that renderers lose meaning.

Typical evidence source. In practice the evidence will come from mixed-device or mixed-dtype Torch programs. The integration should preserve enough of that source structure that local solvers using table-driven local checks can operate without pack-specific hacks.

Expected user-facing payoff. If implemented well, the payoff is device mismatch and unsupported operator combinations. This should appear not only in benchmark metrics but also in concrete rendered contracts, diagnostics, or proof goals.

0.26.21 Package `src/deppy/contracts` with Sequence and collection theory

Integration objective. The package `src/deppy/contracts` must interact cleanly with the theory family sequence and collection theory. The reason is practical: the package owns part of the data path through which non-emptiness, membership, stable length relations, and head/tail legality becomes visible, audited, and eventually rendered.

Required interface. At minimum the integration should expose normalized inputs, site-aware outputs, provenance, and failure reasons. The package should neither hide theory-relevant structure from the cover builder nor flatten theory results so aggressively that renderers lose meaning.

Typical evidence source. In practice the evidence will come from lists, tuples, deques, and iterator adapters. The integration should preserve enough of that source structure that local solvers using simple algebraic checks can operate without pack-specific hacks.

Expected user-facing payoff. If implemented well, the payoff is head-of-empty and unpacking safety. This should appear not only in benchmark metrics but also in concrete rendered contracts, diagnostics, or proof goals.

0.26.22 Package `src/deppy/contracts` with Exception theory

Integration objective. The package `src/deppy/contracts` must interact cleanly with the theory family exception theory. The reason is practical: the package owns part of the data path through which exceptional output channels, throw sites, and caught-vs-uncaught distinctions becomes visible, audited, and eventually rendered.

Required interface. At minimum the integration should expose normalized inputs, site-aware outputs, provenance, and failure reasons. The package should neither hide theory-relevant structure from the cover builder nor flatten theory results so aggressively that renderers lose meaning.

Typical evidence source. In practice the evidence will come from try/except blocks, library raises, and proof obligations that may fail. The integration should preserve enough of that source structure that local solvers using control-flow plus error-site hooks can operate without pack-specific hacks.

Expected user-facing payoff. If implemented well, the payoff is precise reporting of exceptional contracts and remaining failures. This should appear not only in benchmark metrics but also in concrete rendered contracts, diagnostics, or proof goals.

0.26.23 Package `src/deppy/contracts` with Witness and equality transport theory

Integration objective. The package `src/deppy/contracts` must interact cleanly with the theory family witness and equality transport theory. The reason is practical: the package owns part of

the data path through which dependent pairs, equality witnesses, and explicit transports becomes visible, audited, and eventually rendered.

Required interface. At minimum the integration should expose normalized inputs, site-aware outputs, provenance, and failure reasons. The package should neither hide theory-relevant structure from the cover builder nor flatten theory results so aggressively that renderers lose meaning.

Typical evidence source. In practice the evidence will come from proof-oriented code and witness-returning helpers. The integration should preserve enough of that source structure that local solvers using proof-kernel backed local checking can operate without pack-specific hacks.

Expected user-facing payoff. If implemented well, the payoff is proof reuse and output enrichment. This should appear not only in benchmark metrics but also in concrete rendered contracts, diagnostics, or proof goals.

0.26.24 Package `src/deppy/contracts` with Loop invariant and decreases theory

Integration objective. The package `src/deppy/contracts` must interact cleanly with the theory family loop invariant and decreases theory. The reason is practical: the package owns part of the data path through which candidate invariants, ranking relations, and preservation constraints becomes visible, audited, and eventually rendered.

Required interface. At minimum the integration should expose normalized inputs, site-aware outputs, provenance, and failure reasons. The package should neither hide theory-relevant structure from the cover builder nor flatten theory results so aggressively that renderers lose meaning.

Typical evidence source. In practice the evidence will come from loop-heavy algorithms and recursive helpers. The integration should preserve enough of that source structure that local solvers using template generation plus bounded checks can operate without pack-specific hacks.

Expected user-facing payoff. If implemented well, the payoff is F*-like capability without abandoning native Python. This should appear not only in benchmark metrics but also in concrete rendered contracts, diagnostics, or proof goals.

0.26.25 Package `src/deppy/core` with Linear arithmetic and intervals

Integration objective. The package `src/deppy/core` must interact cleanly with the theory family linear arithmetic and intervals. The reason is practical: the package owns part of the data path through which non-negativity, bounds, lengths, simple affine equalities becomes visible, audited, and eventually rendered.

Required interface. At minimum the integration should expose normalized inputs, site-aware outputs, provenance, and failure reasons. The package should neither hide theory-relevant structure from the cover builder nor flatten theory results so aggressively that renderers lose meaning.

Typical evidence source. In practice the evidence will come from argument bounds, list lengths, loop counters, and tensor extents. The integration should preserve enough of that source structure that local solvers using local SMT over arithmetic fragments can operate without pack-specific hacks.

Expected user-facing payoff. If implemented well, the payoff is IndexError, ZeroDivisionError, and shape-side precondition pullback. This should appear not only in benchmark metrics but also in concrete rendered contracts, diagnostics, or proof goals.

0.26.26 Package `src/deppy/core` with Boolean guard theory

Integration objective. The package `src/deppy/core` must interact cleanly with the theory family boolean guard theory. The reason is practical: the package owns part of the data path through which guard polarity, assertion scope, and branch-conditioned refinement becomes visible, audited, and eventually rendered.

Required interface. At minimum the integration should expose normalized inputs, site-aware outputs, provenance, and failure reasons. The package should neither hide theory-relevant structure from the cover builder nor flatten theory results so aggressively that renderers lose meaning.

Typical evidence source. In practice the evidence will come from if-statements, asserts, early returns, and exception guards. The integration should preserve enough of that source structure that local solvers using normalization plus implication checks can operate without pack-specific hacks.

Expected user-facing payoff. If implemented well, the payoff is control-dependent safety and lightweight proof seed extraction. This should appear not only in benchmark metrics but also in concrete rendered contracts, diagnostics, or proof goals.

0.26.27 Package `src/deppy/core` with Nullability and optionality

Integration objective. The package `src/deppy/core` must interact cleanly with the theory family nullability and optionality. The reason is practical: the package owns part of the data path through which none checks, existence guards, and optional protocol availability becomes visible, audited, and eventually rendered.

Required interface. At minimum the integration should expose normalized inputs, site-aware outputs, provenance, and failure reasons. The package should neither hide theory-relevant structure from the cover builder nor flatten theory results so aggressively that renderers lose meaning.

Typical evidence source. In practice the evidence will come from ordinary Python branches and constructor code. The integration should preserve enough of that source structure that local solvers using simple symbolic narrowing can operate without pack-specific hacks.

Expected user-facing payoff. If implemented well, the payoff is AttributeError prevention and stronger post-branch outputs. This should appear not only in benchmark metrics but also in concrete rendered contracts, diagnostics, or proof goals.

0.26.28 Package `src/deppy/core` with Heap and protocol theory

Integration objective. The package `src/deppy/core` must interact cleanly with the theory family heap and protocol theory. The reason is practical: the package owns part of the data path through which field initialization, protocol method existence, frame conditions, and alias exclusions becomes visible, audited, and eventually rendered.

Required interface. At minimum the integration should expose normalized inputs, site-aware outputs, provenance, and failure reasons. The package should neither hide theory-relevant structure from the cover builder nor flatten theory results so aggressively that renderers lose meaning.

Typical evidence source. In practice the evidence will come from classes, protocols, mutable objects, and resource wrappers. The integration should preserve enough of that source structure that local solvers using bounded symbolic checks plus explicit residuals can operate without pack-specific hacks.

Expected user-facing payoff. If implemented well, the payoff is constructor safety, attribute access, and method-dispatch correctness. This should appear not only in benchmark metrics but also in concrete rendered contracts, diagnostics, or proof goals.

0.26.29 Package `src/deppy/core` with Tensor shape theory

Integration objective. The package `src/deppy/core` must interact cleanly with the theory family tensor shape theory. The reason is practical: the package owns part of the data path through which rank, dimension, shape tuples, cardinality, broadcast legality, and reduction support becomes visible, audited, and eventually rendered.

Required interface. At minimum the integration should expose normalized inputs, site-aware outputs, provenance, and failure reasons. The package should neither hide theory-relevant structure from the cover builder nor flatten theory results so aggressively that renderers lose meaning.

Typical evidence source. In practice the evidence will come from Torch tensor operators and shape-sensitive wrappers. The integration should preserve enough of that source structure that local solvers using solver-backed local shape kernels can operate without pack-specific hacks.

Expected user-facing payoff. If implemented well, the payoff is invalid reshape, broadcast mismatch, and exact output-shape synthesis. This should appear not only in benchmark metrics but also in concrete rendered contracts, diagnostics, or proof goals.

0.26.30 Package `src/deppy/core` with Tensor order theory

Integration objective. The package `src/deppy/core` must interact cleanly with the theory family tensor order theory. The reason is practical: the package owns part of the data path through which sortedness, monotonicity, permutation witnesses, argsort relations becomes visible, audited, and eventually rendered.

Required interface. At minimum the integration should expose normalized inputs, site-aware outputs, provenance, and failure reasons. The package should neither hide theory-relevant structure from the cover builder nor flatten theory results so aggressively that renderers lose meaning.

Typical evidence source. In practice the evidence will come from sort-like operators and order-preserving wrappers. The integration should preserve enough of that source structure that local solvers using finite witness templates plus solver-backed subchecks can operate without pack-specific hacks.

Expected user-facing payoff. If implemented well, the payoff is values-vs-indices confusion and richer sorted outputs. This should appear not only in benchmark metrics but also in concrete rendered contracts, diagnostics, or proof goals.

0.26.31 Package `src/deppy/core` with Tensor indexing theory

Integration objective. The package `src/deppy/core` must interact cleanly with the theory family tensor indexing theory. The reason is practical: the package owns part of the data path through which selector domains, slice extents, gather/index-select legality, and value provenance becomes visible, audited, and eventually rendered.

Required interface. At minimum the integration should expose normalized inputs, site-aware outputs, provenance, and failure reasons. The package should neither hide theory-relevant structure from the cover builder nor flatten theory results so aggressively that renderers lose meaning.

Typical evidence source. In practice the evidence will come from subscripts, slices, and advanced indexing. The integration should preserve enough of that source structure that local solvers using arithmetic plus pack-specific rules can operate without pack-specific hacks.

Expected user-facing payoff. If implemented well, the payoff is out-of-range selector detection and source-target output relations. This should appear not only in benchmark metrics but also in concrete rendered contracts, diagnostics, or proof goals.

0.26.32 Package `src/deppy/core` with Device and dtype compatibility theory

Integration objective. The package `src/deppy/core` must interact cleanly with the theory family device and dtype compatibility theory. The reason is practical: the package owns part of the data path through which device equality, admissible implicit casts, and operator support matrices becomes visible, audited, and eventually rendered.

Required interface. At minimum the integration should expose normalized inputs, site-aware outputs, provenance, and failure reasons. The package should neither hide theory-relevant structure from the cover builder nor flatten theory results so aggressively that renderers lose meaning.

Typical evidence source. In practice the evidence will come from mixed-device or mixed-dtype Torch programs. The integration should preserve enough of that source structure that local solvers using table-driven local checks can operate without pack-specific hacks.

Expected user-facing payoff. If implemented well, the payoff is device mismatch and unsupported operator combinations. This should appear not only in benchmark metrics but also in concrete rendered contracts, diagnostics, or proof goals.

0.26.33 Package `src/deppy/core` with Sequence and collection theory

Integration objective. The package `src/deppy/core` must interact cleanly with the theory family sequence and collection theory. The reason is practical: the package owns part of the data path through which non-emptiness, membership, stable length relations, and head/tail legality becomes visible, audited, and eventually rendered.

Required interface. At minimum the integration should expose normalized inputs, site-aware outputs, provenance, and failure reasons. The package should neither hide theory-relevant structure from the cover builder nor flatten theory results so aggressively that renderers lose meaning.

Typical evidence source. In practice the evidence will come from lists, tuples, deques, and iterator adapters. The integration should preserve enough of that source structure that local solvers using simple algebraic checks can operate without pack-specific hacks.

Expected user-facing payoff. If implemented well, the payoff is head-of-empty and unpacking safety. This should appear not only in benchmark metrics but also in concrete rendered contracts, diagnostics, or proof goals.

0.26.34 Package `src/deppy/core` with Exception theory

Integration objective. The package `src/deppy/core` must interact cleanly with the theory family exception theory. The reason is practical: the package owns part of the data path through which exceptional output channels, throw sites, and caught-vs-uncaught distinctions becomes visible, audited, and eventually rendered.

Required interface. At minimum the integration should expose normalized inputs, site-aware outputs, provenance, and failure reasons. The package should neither hide theory-relevant structure from the cover builder nor flatten theory results so aggressively that renderers lose meaning.

Typical evidence source. In practice the evidence will come from try/except blocks, library raises, and proof obligations that may fail. The integration should preserve enough of that source structure that local solvers using control-flow plus error-site hooks can operate without pack-specific hacks.

Expected user-facing payoff. If implemented well, the payoff is precise reporting of exceptional contracts and remaining failures. This should appear not only in benchmark metrics but also in concrete rendered contracts, diagnostics, or proof goals.

0.26.35 Package `src/deppy/core` with Witness and equality transport theory

Integration objective. The package `src/deppy/core` must interact cleanly with the theory family witness and equality transport theory. The reason is practical: the package owns part of

the data path through which dependent pairs, equality witnesses, and explicit transports becomes visible, audited, and eventually rendered.

Required interface. At minimum the integration should expose normalized inputs, site-aware outputs, provenance, and failure reasons. The package should neither hide theory-relevant structure from the cover builder nor flatten theory results so aggressively that renderers lose meaning.

Typical evidence source. In practice the evidence will come from proof-oriented code and witness-returning helpers. The integration should preserve enough of that source structure that local solvers using proof-kernel backed local checking can operate without pack-specific hacks.

Expected user-facing payoff. If implemented well, the payoff is proof reuse and output enrichment. This should appear not only in benchmark metrics but also in concrete rendered contracts, diagnostics, or proof goals.

0.26.36 Package `src/deppy/core` with Loop invariant and decreases theory

Integration objective. The package `src/deppy/core` must interact cleanly with the theory family loop invariant and decreases theory. The reason is practical: the package owns part of the data path through which candidate invariants, ranking relations, and preservation constraints becomes visible, audited, and eventually rendered.

Required interface. At minimum the integration should expose normalized inputs, site-aware outputs, provenance, and failure reasons. The package should neither hide theory-relevant structure from the cover builder nor flatten theory results so aggressively that renderers lose meaning.

Typical evidence source. In practice the evidence will come from loop-heavy algorithms and recursive helpers. The integration should preserve enough of that source structure that local solvers using template generation plus bounded checks can operate without pack-specific hacks.

Expected user-facing payoff. If implemented well, the payoff is F*-like capability without abandoning native Python. This should appear not only in benchmark metrics but also in concrete rendered contracts, diagnostics, or proof goals.

0.26.37 Package `src/deppy/frontend` with Linear arithmetic and intervals

Integration objective. The package `src/deppy/frontend` must interact cleanly with the theory family linear arithmetic and intervals. The reason is practical: the package owns part of the data path through which non-negativity, bounds, lengths, simple affine equalities becomes visible, audited, and eventually rendered.

Required interface. At minimum the integration should expose normalized inputs, site-aware outputs, provenance, and failure reasons. The package should neither hide theory-relevant structure from the cover builder nor flatten theory results so aggressively that renderers lose meaning.

Typical evidence source. In practice the evidence will come from argument bounds, list lengths, loop counters, and tensor extents. The integration should preserve enough of that source structure that local solvers using local SMT over arithmetic fragments can operate without pack-specific hacks.

Expected user-facing payoff. If implemented well, the payoff is IndexError, ZeroDivisionError, and shape-side precondition pullback. This should appear not only in benchmark metrics but also in concrete rendered contracts, diagnostics, or proof goals.

0.26.38 Package `src/deppy/frontend` with Boolean guard theory

Integration objective. The package `src/deppy/frontend` must interact cleanly with the theory family boolean guard theory. The reason is practical: the package owns part of the data path through which guard polarity, assertion scope, and branch-conditioned refinement becomes visible, audited, and eventually rendered.

Required interface. At minimum the integration should expose normalized inputs, site-aware outputs, provenance, and failure reasons. The package should neither hide theory-relevant structure from the cover builder nor flatten theory results so aggressively that renderers lose meaning.

Typical evidence source. In practice the evidence will come from if-statements, asserts, early returns, and exception guards. The integration should preserve enough of that source structure that local solvers using normalization plus implication checks can operate without pack-specific hacks.

Expected user-facing payoff. If implemented well, the payoff is control-dependent safety and lightweight proof seed extraction. This should appear not only in benchmark metrics but also in concrete rendered contracts, diagnostics, or proof goals.

0.26.39 Package `src/deppy/frontend` with Nullability and optionality

Integration objective. The package `src/deppy/frontend` must interact cleanly with the theory family nullability and optionality. The reason is practical: the package owns part of the data path through which none checks, existence guards, and optional protocol availability becomes visible, audited, and eventually rendered.

Required interface. At minimum the integration should expose normalized inputs, site-aware outputs, provenance, and failure reasons. The package should neither hide theory-relevant structure from the cover builder nor flatten theory results so aggressively that renderers lose meaning.

Typical evidence source. In practice the evidence will come from ordinary Python branches and constructor code. The integration should preserve enough of that source structure that local solvers using simple symbolic narrowing can operate without pack-specific hacks.

Expected user-facing payoff. If implemented well, the payoff is AttributeError prevention and stronger post-branch outputs. This should appear not only in benchmark metrics but also in concrete rendered contracts, diagnostics, or proof goals.

0.26.40 Package `src/deppy/frontend` with Heap and protocol theory

Integration objective. The package `src/deppy/frontend` must interact cleanly with the theory family heap and protocol theory. The reason is practical: the package owns part of the data path through which field initialization, protocol method existence, frame conditions, and alias exclusions becomes visible, audited, and eventually rendered.

Required interface. At minimum the integration should expose normalized inputs, site-aware outputs, provenance, and failure reasons. The package should neither hide theory-relevant structure from the cover builder nor flatten theory results so aggressively that renderers lose meaning.

Typical evidence source. In practice the evidence will come from classes, protocols, mutable objects, and resource wrappers. The integration should preserve enough of that source structure that local solvers using bounded symbolic checks plus explicit residuals can operate without pack-specific hacks.

Expected user-facing payoff. If implemented well, the payoff is constructor safety, attribute access, and method-dispatch correctness. This should appear not only in benchmark metrics but also in concrete rendered contracts, diagnostics, or proof goals.

0.26.41 Package `src/deppy/frontend` with Tensor shape theory

Integration objective. The package `src/deppy/frontend` must interact cleanly with the theory family tensor shape theory. The reason is practical: the package owns part of the data path through which rank, dimension, shape tuples, cardinality, broadcast legality, and reduction support becomes visible, audited, and eventually rendered.

Required interface. At minimum the integration should expose normalized inputs, site-aware outputs, provenance, and failure reasons. The package should neither hide theory-relevant structure from the cover builder nor flatten theory results so aggressively that renderers lose meaning.

Typical evidence source. In practice the evidence will come from Torch tensor operators and shape-sensitive wrappers. The integration should preserve enough of that source structure that local solvers using solver-backed local shape kernels can operate without pack-specific hacks.

Expected user-facing payoff. If implemented well, the payoff is invalid reshape, broadcast mismatch, and exact output-shape synthesis. This should appear not only in benchmark metrics but also in concrete rendered contracts, diagnostics, or proof goals.

0.26.42 Package `src/deppy/frontend` with Tensor order theory

Integration objective. The package `src/deppy/frontend` must interact cleanly with the theory family tensor order theory. The reason is practical: the package owns part of the data path through which sortedness, monotonicity, permutation witnesses, argsort relations becomes visible, audited, and eventually rendered.

Required interface. At minimum the integration should expose normalized inputs, site-aware outputs, provenance, and failure reasons. The package should neither hide theory-relevant structure from the cover builder nor flatten theory results so aggressively that renderers lose meaning.

Typical evidence source. In practice the evidence will come from sort-like operators and order-preserving wrappers. The integration should preserve enough of that source structure that local solvers using finite witness templates plus solver-backed subchecks can operate without pack-specific hacks.

Expected user-facing payoff. If implemented well, the payoff is values-vs-indices confusion and richer sorted outputs. This should appear not only in benchmark metrics but also in concrete rendered contracts, diagnostics, or proof goals.

0.26.43 Package `src/deppy/frontend` with Tensor indexing theory

Integration objective. The package `src/deppy/frontend` must interact cleanly with the theory family tensor indexing theory. The reason is practical: the package owns part of the data path through which selector domains, slice extents, gather/index-select legality, and value provenance becomes visible, audited, and eventually rendered.

Required interface. At minimum the integration should expose normalized inputs, site-aware outputs, provenance, and failure reasons. The package should neither hide theory-relevant structure from the cover builder nor flatten theory results so aggressively that renderers lose meaning.

Typical evidence source. In practice the evidence will come from subscripts, slices, and advanced indexing. The integration should preserve enough of that source structure that local solvers using arithmetic plus pack-specific rules can operate without pack-specific hacks.

Expected user-facing payoff. If implemented well, the payoff is out-of-range selector detection and source-target output relations. This should appear not only in benchmark metrics but also in concrete rendered contracts, diagnostics, or proof goals.

0.26.44 Package `src/deppy/frontend` with Device and dtype compatibility theory

Integration objective. The package `src/deppy/frontend` must interact cleanly with the theory family device and dtype compatibility theory. The reason is practical: the package owns part of the data path through which device equality, admissible implicit casts, and operator support matrices becomes visible, audited, and eventually rendered.

Required interface. At minimum the integration should expose normalized inputs, site-aware outputs, provenance, and failure reasons. The package should neither hide theory-relevant structure from the cover builder nor flatten theory results so aggressively that renderers lose meaning.

Typical evidence source. In practice the evidence will come from mixed-device or mixed-dtype Torch programs. The integration should preserve enough of that source structure that local solvers using table-driven local checks can operate without pack-specific hacks.

Expected user-facing payoff. If implemented well, the payoff is device mismatch and unsupported operator combinations. This should appear not only in benchmark metrics but also in concrete rendered contracts, diagnostics, or proof goals.

0.26.45 Package `src/deppy/frontend` with Sequence and collection theory

Integration objective. The package `src/deppy/frontend` must interact cleanly with the theory family sequence and collection theory. The reason is practical: the package owns part of the data path through which non-emptiness, membership, stable length relations, and head/tail legality becomes visible, audited, and eventually rendered.

Required interface. At minimum the integration should expose normalized inputs, site-aware outputs, provenance, and failure reasons. The package should neither hide theory-relevant structure from the cover builder nor flatten theory results so aggressively that renderers lose meaning.

Typical evidence source. In practice the evidence will come from lists, tuples, deques, and iterator adapters. The integration should preserve enough of that source structure that local solvers using simple algebraic checks can operate without pack-specific hacks.

Expected user-facing payoff. If implemented well, the payoff is head-of-empty and unpacking safety. This should appear not only in benchmark metrics but also in concrete rendered contracts, diagnostics, or proof goals.

0.26.46 Package `src/deppy/frontend` with Exception theory

Integration objective. The package `src/deppy/frontend` must interact cleanly with the theory family exception theory. The reason is practical: the package owns part of the data path through which exceptional output channels, throw sites, and caught-vs-uncaught distinctions becomes visible, audited, and eventually rendered.

Required interface. At minimum the integration should expose normalized inputs, site-aware outputs, provenance, and failure reasons. The package should neither hide theory-relevant structure from the cover builder nor flatten theory results so aggressively that renderers lose meaning.

Typical evidence source. In practice the evidence will come from try/except blocks, library raises, and proof obligations that may fail. The integration should preserve enough of that source structure that local solvers using control-flow plus error-site hooks can operate without pack-specific hacks.

Expected user-facing payoff. If implemented well, the payoff is precise reporting of exceptional contracts and remaining failures. This should appear not only in benchmark metrics but also in concrete rendered contracts, diagnostics, or proof goals.

0.26.47 Package `src/deppy/frontend` with Witness and equality transport theory

Integration objective. The package `src/deppy/frontend` must interact cleanly with the theory family witness and equality transport theory. The reason is practical: the package owns part of

the data path through which dependent pairs, equality witnesses, and explicit transports becomes visible, audited, and eventually rendered.

Required interface. At minimum the integration should expose normalized inputs, site-aware outputs, provenance, and failure reasons. The package should neither hide theory-relevant structure from the cover builder nor flatten theory results so aggressively that renderers lose meaning.

Typical evidence source. In practice the evidence will come from proof-oriented code and witness-returning helpers. The integration should preserve enough of that source structure that local solvers using proof-kernel backed local checking can operate without pack-specific hacks.

Expected user-facing payoff. If implemented well, the payoff is proof reuse and output enrichment. This should appear not only in benchmark metrics but also in concrete rendered contracts, diagnostics, or proof goals.

0.26.48 Package `src/deppy/frontend` with Loop invariant and decreases theory

Integration objective. The package `src/deppy/frontend` must interact cleanly with the theory family loop invariant and decreases theory. The reason is practical: the package owns part of the data path through which candidate invariants, ranking relations, and preservation constraints becomes visible, audited, and eventually rendered.

Required interface. At minimum the integration should expose normalized inputs, site-aware outputs, provenance, and failure reasons. The package should neither hide theory-relevant structure from the cover builder nor flatten theory results so aggressively that renderers lose meaning.

Typical evidence source. In practice the evidence will come from loop-heavy algorithms and recursive helpers. The integration should preserve enough of that source structure that local solvers using template generation plus bounded checks can operate without pack-specific hacks.

Expected user-facing payoff. If implemented well, the payoff is F*-like capability without abandoning native Python. This should appear not only in benchmark metrics but also in concrete rendered contracts, diagnostics, or proof goals.

0.26.49 Package `src/deppy/generate` with Linear arithmetic and intervals

Integration objective. The package `src/deppy/generate` must interact cleanly with the theory family linear arithmetic and intervals. The reason is practical: the package owns part of the data path through which non-negativity, bounds, lengths, simple affine equalities becomes visible, audited, and eventually rendered.

Required interface. At minimum the integration should expose normalized inputs, site-aware outputs, provenance, and failure reasons. The package should neither hide theory-relevant structure from the cover builder nor flatten theory results so aggressively that renderers lose meaning.

Typical evidence source. In practice the evidence will come from argument bounds, list lengths, loop counters, and tensor extents. The integration should preserve enough of that source structure that local solvers using local SMT over arithmetic fragments can operate without pack-specific hacks.

Expected user-facing payoff. If implemented well, the payoff is IndexError, ZeroDivisionError, and shape-side precondition pullback. This should appear not only in benchmark metrics but also in concrete rendered contracts, diagnostics, or proof goals.

0.26.50 Package `src/deppy/generate` with Boolean guard theory

Integration objective. The package `src/deppy/generate` must interact cleanly with the theory family boolean guard theory. The reason is practical: the package owns part of the data path through which guard polarity, assertion scope, and branch-conditioned refinement becomes visible, audited, and eventually rendered.

Required interface. At minimum the integration should expose normalized inputs, site-aware outputs, provenance, and failure reasons. The package should neither hide theory-relevant structure from the cover builder nor flatten theory results so aggressively that renderers lose meaning.

Typical evidence source. In practice the evidence will come from if-statements, asserts, early returns, and exception guards. The integration should preserve enough of that source structure that local solvers using normalization plus implication checks can operate without pack-specific hacks.

Expected user-facing payoff. If implemented well, the payoff is control-dependent safety and lightweight proof seed extraction. This should appear not only in benchmark metrics but also in concrete rendered contracts, diagnostics, or proof goals.

0.26.51 Package `src/deppy/generate` with Nullability and optionality

Integration objective. The package `src/deppy/generate` must interact cleanly with the theory family nullability and optionality. The reason is practical: the package owns part of the data path through which none checks, existence guards, and optional protocol availability becomes visible, audited, and eventually rendered.

Required interface. At minimum the integration should expose normalized inputs, site-aware outputs, provenance, and failure reasons. The package should neither hide theory-relevant structure from the cover builder nor flatten theory results so aggressively that renderers lose meaning.

Typical evidence source. In practice the evidence will come from ordinary Python branches and constructor code. The integration should preserve enough of that source structure that local solvers using simple symbolic narrowing can operate without pack-specific hacks.

Expected user-facing payoff. If implemented well, the payoff is AttributeError prevention and stronger post-branch outputs. This should appear not only in benchmark metrics but also in concrete rendered contracts, diagnostics, or proof goals.

0.26.52 Package `src/deppy/generate` with Heap and protocol theory

Integration objective. The package `src/deppy/generate` must interact cleanly with the theory family heap and protocol theory. The reason is practical: the package owns part of the data path through which field initialization, protocol method existence, frame conditions, and alias exclusions becomes visible, audited, and eventually rendered.

Required interface. At minimum the integration should expose normalized inputs, site-aware outputs, provenance, and failure reasons. The package should neither hide theory-relevant structure from the cover builder nor flatten theory results so aggressively that renderers lose meaning.

Typical evidence source. In practice the evidence will come from classes, protocols, mutable objects, and resource wrappers. The integration should preserve enough of that source structure that local solvers using bounded symbolic checks plus explicit residuals can operate without pack-specific hacks.

Expected user-facing payoff. If implemented well, the payoff is constructor safety, attribute access, and method-dispatch correctness. This should appear not only in benchmark metrics but also in concrete rendered contracts, diagnostics, or proof goals.

0.26.53 Package `src/deppy/generate` with Tensor shape theory

Integration objective. The package `src/deppy/generate` must interact cleanly with the theory family tensor shape theory. The reason is practical: the package owns part of the data path through which rank, dimension, shape tuples, cardinality, broadcast legality, and reduction support becomes visible, audited, and eventually rendered.

Required interface. At minimum the integration should expose normalized inputs, site-aware outputs, provenance, and failure reasons. The package should neither hide theory-relevant structure from the cover builder nor flatten theory results so aggressively that renderers lose meaning.

Typical evidence source. In practice the evidence will come from Torch tensor operators and shape-sensitive wrappers. The integration should preserve enough of that source structure that local solvers using solver-backed local shape kernels can operate without pack-specific hacks.

Expected user-facing payoff. If implemented well, the payoff is invalid reshape, broadcast mismatch, and exact output-shape synthesis. This should appear not only in benchmark metrics but also in concrete rendered contracts, diagnostics, or proof goals.

0.26.54 Package `src/deppy/generate` with Tensor order theory

Integration objective. The package `src/deppy/generate` must interact cleanly with the theory family tensor order theory. The reason is practical: the package owns part of the data path through which sortedness, monotonicity, permutation witnesses, argsort relations becomes visible, audited, and eventually rendered.

Required interface. At minimum the integration should expose normalized inputs, site-aware outputs, provenance, and failure reasons. The package should neither hide theory-relevant structure from the cover builder nor flatten theory results so aggressively that renderers lose meaning.

Typical evidence source. In practice the evidence will come from sort-like operators and order-preserving wrappers. The integration should preserve enough of that source structure that local solvers using finite witness templates plus solver-backed subchecks can operate without pack-specific hacks.

Expected user-facing payoff. If implemented well, the payoff is values-vs-indices confusion and richer sorted outputs. This should appear not only in benchmark metrics but also in concrete rendered contracts, diagnostics, or proof goals.

0.26.55 Package `src/deppy/generate` with Tensor indexing theory

Integration objective. The package `src/deppy/generate` must interact cleanly with the theory family tensor indexing theory. The reason is practical: the package owns part of the data path through which selector domains, slice extents, gather/index-select legality, and value provenance becomes visible, audited, and eventually rendered.

Required interface. At minimum the integration should expose normalized inputs, site-aware outputs, provenance, and failure reasons. The package should neither hide theory-relevant structure from the cover builder nor flatten theory results so aggressively that renderers lose meaning.

Typical evidence source. In practice the evidence will come from subscripts, slices, and advanced indexing. The integration should preserve enough of that source structure that local solvers using arithmetic plus pack-specific rules can operate without pack-specific hacks.

Expected user-facing payoff. If implemented well, the payoff is out-of-range selector detection and source-target output relations. This should appear not only in benchmark metrics but also in concrete rendered contracts, diagnostics, or proof goals.

0.26.56 Package `src/deppy/generate` with Device and dtype compatibility theory

Integration objective. The package `src/deppy/generate` must interact cleanly with the theory family device and dtype compatibility theory. The reason is practical: the package owns part of the data path through which device equality, admissible implicit casts, and operator support matrices becomes visible, audited, and eventually rendered.

Required interface. At minimum the integration should expose normalized inputs, site-aware outputs, provenance, and failure reasons. The package should neither hide theory-relevant structure from the cover builder nor flatten theory results so aggressively that renderers lose meaning.

Typical evidence source. In practice the evidence will come from mixed-device or mixed-dtype Torch programs. The integration should preserve enough of that source structure that local solvers using table-driven local checks can operate without pack-specific hacks.

Expected user-facing payoff. If implemented well, the payoff is device mismatch and unsupported operator combinations. This should appear not only in benchmark metrics but also in concrete rendered contracts, diagnostics, or proof goals.

0.26.57 Package `src/deppy/generate` with Sequence and collection theory

Integration objective. The package `src/deppy/generate` must interact cleanly with the theory family sequence and collection theory. The reason is practical: the package owns part of the data path through which non-emptiness, membership, stable length relations, and head/tail legality becomes visible, audited, and eventually rendered.

Required interface. At minimum the integration should expose normalized inputs, site-aware outputs, provenance, and failure reasons. The package should neither hide theory-relevant structure from the cover builder nor flatten theory results so aggressively that renderers lose meaning.

Typical evidence source. In practice the evidence will come from lists, tuples, deques, and iterator adapters. The integration should preserve enough of that source structure that local solvers using simple algebraic checks can operate without pack-specific hacks.

Expected user-facing payoff. If implemented well, the payoff is head-of-empty and unpacking safety. This should appear not only in benchmark metrics but also in concrete rendered contracts, diagnostics, or proof goals.

0.26.58 Package `src/deppy/generate` with Exception theory

Integration objective. The package `src/deppy/generate` must interact cleanly with the theory family exception theory. The reason is practical: the package owns part of the data path through which exceptional output channels, throw sites, and caught-vs-uncaught distinctions becomes visible, audited, and eventually rendered.

Required interface. At minimum the integration should expose normalized inputs, site-aware outputs, provenance, and failure reasons. The package should neither hide theory-relevant structure from the cover builder nor flatten theory results so aggressively that renderers lose meaning.

Typical evidence source. In practice the evidence will come from try/except blocks, library raises, and proof obligations that may fail. The integration should preserve enough of that source structure that local solvers using control-flow plus error-site hooks can operate without pack-specific hacks.

Expected user-facing payoff. If implemented well, the payoff is precise reporting of exceptional contracts and remaining failures. This should appear not only in benchmark metrics but also in concrete rendered contracts, diagnostics, or proof goals.

0.26.59 Package `src/deppy/generate` with Witness and equality transport theory

Integration objective. The package `src/deppy/generate` must interact cleanly with the theory family witness and equality transport theory. The reason is practical: the package owns part of

the data path through which dependent pairs, equality witnesses, and explicit transports becomes visible, audited, and eventually rendered.

Required interface. At minimum the integration should expose normalized inputs, site-aware outputs, provenance, and failure reasons. The package should neither hide theory-relevant structure from the cover builder nor flatten theory results so aggressively that renderers lose meaning.

Typical evidence source. In practice the evidence will come from proof-oriented code and witness-returning helpers. The integration should preserve enough of that source structure that local solvers using proof-kernel backed local checking can operate without pack-specific hacks.

Expected user-facing payoff. If implemented well, the payoff is proof reuse and output enrichment. This should appear not only in benchmark metrics but also in concrete rendered contracts, diagnostics, or proof goals.

0.26.60 Package `src/deppy/generate` with Loop invariant and decreases theory

Integration objective. The package `src/deppy/generate` must interact cleanly with the theory family loop invariant and decreases theory. The reason is practical: the package owns part of the data path through which candidate invariants, ranking relations, and preservation constraints becomes visible, audited, and eventually rendered.

Required interface. At minimum the integration should expose normalized inputs, site-aware outputs, provenance, and failure reasons. The package should neither hide theory-relevant structure from the cover builder nor flatten theory results so aggressively that renderers lose meaning.

Typical evidence source. In practice the evidence will come from loop-heavy algorithms and recursive helpers. The integration should preserve enough of that source structure that local solvers using template generation plus bounded checks can operate without pack-specific hacks.

Expected user-facing payoff. If implemented well, the payoff is F*-like capability without abandoning native Python. This should appear not only in benchmark metrics but also in concrete rendered contracts, diagnostics, or proof goals.

0.27 Deep dive: list semantics, bounds safety, and length-dependent contracts

Python lists are the most common mutable sequence and one of the richest sources of runtime errors in everyday code. A sheaf-descent system must model lists not as opaque containers but as semantic neighborhoods carrying length, element-type, ordering, and mutability information at every site where a list value is live.

0.27.1 Length as a first-class refinement coordinate

The single most valuable fact about a list is its length. An enormous fraction of Python runtime errors reduce to a length violation: indexing past the end, unpacking the wrong number of elements, calling `pop()` on an empty list, slicing with out-of-range bounds, or passing a list of unexpected

size to a library function that silently assumes a particular shape. A sheaf-descent system should therefore treat length as a first-class coordinate on every list-valued site.

Concretely, every SSA value whose carrier type includes `list` should have a local section containing at least a symbolic length variable. When a list is constructed via a literal, the length is exactly known. When a list is constructed via comprehension, the length may depend on the length of the source iterable and the filtering predicate. When a list is received as a function argument, the length is initially unconstrained but may be refined by guards, assertions, or downstream usage patterns.

Guard harvesting for length. The harvest layer should recognize the following patterns as length evidence:

- `if len(xs) > 0`, `if xs`, `if not xs` — non-emptiness or emptiness guards that refine the length coordinate to > 0 or $= 0$.
- `if len(xs) == n`, `if len(xs) >= k` — exact or bounded length guards.
- `assert len(xs) > 0` — assertion-based refinement.
- `for x in xs: ...` — the loop body site sees a non-exhausted iterator, which implies the original list was nonempty if the body executes at all.
- `xs = list(range(n))` or `xs = [0] * n` — construction sites where the length is directly computable.

Backward synthesis of length requirements. When a downstream site indexes a list at position i, the error site for `IndexError` has viability predicate $0 \leq i < \text{len}(xs)$. The backward pass should pull this requirement to the input boundary. If i is a constant, the requirement becomes $\text{len}(xs) > i$. If i is symbolic but bounded by an upstream guard or computation, the requirement becomes the appropriate symbolic inequality.

The same logic applies to slice bounds, unpacking sites, and reduction calls. For example, `xs[0]` requires $\text{len}(xs) \geq 1$; `a, b, c = xs` requires $\text{len}(xs) = 3$; `min(xs)` requires $\text{len}(xs) \geq 1$ unless an explicit default is provided.

Forward synthesis of length guarantees. After a list comprehension `ys = [f(x) for x in xs]`, the output site should carry $\text{len}(ys) = \text{len}(xs)$. After a filter comprehension `ys = [x for x in xs if p(x)]`, the output site carries $\text{len}(ys) \leq \text{len}(xs)$. After `xs.append(v)`, the post-mutation site carries $\text{len}(xs') = \text{len}(xs) + 1$. These forward facts become part of the output section and flow into downstream sites.

0.27.2 Element-type refinement and homogeneity

Python lists are heterogeneous in principle, but in practice most lists are homogeneous. The system should track element-type information when it is statically visible or dynamically observed. A list literal `[1, 2, 3]` has element type `int`. A typed comprehension `[int(x) for x in xs]` has element type `int`. A list received as a parameter with annotation `list[str]` has element type `str`. Element-type information flows into downstream operations: `sum(xs)` requires numeric elements; `"".join(xs)` requires string elements; `sorted(xs)` requires comparable elements.

0.27.3 Mutation, aliasing, and frame conditions for lists

Lists are mutable, so the sheaf-descent system must track mutation sites. An `xs.append(v)` call creates a post-mutation site where the length has increased by one and the last element is `v`. An

`xs.pop()` call creates a post-mutation site where the length has decreased by one, with an error site requiring $\text{len}(xs) \geq 1$. An `xs[i] = v` call creates a mutation site where the element at position i has changed, with an error site requiring $0 \leq i < \text{len}(xs)$.

Aliasing matters because two variables may refer to the same list object. If `ys = xs` and then `ys.append(v)`, the mutation is visible through `xs`. The heap-SSA layer should track this aliasing so that the sheaf-descent system can propagate mutation effects through aliases. The frame condition for a function that receives a list should state which list arguments may be mutated and which are treated as read-only.

0.27.4 Sorting, searching, and order-sensitive list operations

The list method `xs.sort()` is an in-place mutation that establishes a sortedness invariant on the post-mutation site. The built-in `sorted(xs)` returns a new sorted list. In both cases, the output site should carry a `sorted` predicate. Downstream operations like `bisect.bisect_left(xs, v)` require a sorted input; the backward pass should synthesize a sortedness requirement or verify that the upstream site already carries the invariant.

0.27.5 List-specific error sites

The following error sites should be registered for list operations:

- `IndexError`: access at index i when $i \notin [0, \text{len})$ or $i \notin [-\text{len}, -1]$ for negative indexing.
- `ValueError`: `xs.remove(v)` when $v \notin xs$.
- `IndexError`: `xs.pop(i)` when i is out of range.
- Unpacking mismatch: `a, b = xs` when $\text{len}(xs) \neq 2$.
- Empty-sequence reduction: `min(xs)`, `max(xs)` when $\text{len}(xs) = 0$ and no default is provided.

Each error site should have an explicit viability predicate, a backward pullback rule, and an explanation template so that the renderer can say exactly why an `IndexError` is possible or has been excluded.

0.27.6 Interaction with the broader sheaf architecture

List sites participate in the same cover as all other program sites. A function that receives a list, guards its length, indexes it, and returns an element should produce a global section where the input boundary carries a length requirement, the guard site narrows the local section, the indexing site is verified safe, and the output boundary carries an element-type guarantee. The same backward and forward passes that handle tensor shapes should handle list lengths, because both are instances of the same viability and extension machinery. The only difference is the local theory: tensor shapes use semialgebraic arithmetic over multi-dimensional extents, while list lengths use simpler linear arithmetic over a single dimension.

0.28 Deep dive: dict semantics, key safety, and structural contracts

Python dictionaries are the backbone of configuration, serialization, keyword dispatch, and ad hoc record types. They are also a major source of `KeyError`, `TypeError`, and silent logic bugs when expected keys are missing or carry unexpected types. A sheaf-descent system should model dictionaries with enough structure to synthesize key-membership requirements, value-type guarantees, and structural contracts.

0.28.1 Key domain as a refinement coordinate

The most valuable fact about a dictionary is its key domain: the set of keys that are definitely present. A dict literal `\{"a": 1, "b": 2\}` has exact key domain {"a", "b"}. A dict built by successive `d[k] = v` assignments accumulates keys. A dict received as a parameter may have an unknown key domain, but guards like `if "x" in d` or `d.get("x", default)` refine the local section.

The system should distinguish three levels of key-domain knowledge:

1. *Exact domain*: all keys are known, as in a literal or a fully traced construction.
2. *Lower-bounded domain*: certain keys are known present due to guards, construction patterns, or TypedDict annotations, but additional keys may exist.
3. *Unknown domain*: no key-membership information is available.

Guard harvesting for key membership. The harvest layer should recognize:

- `if k in d` — establishes key presence in the true branch.
- `if k not in d` — establishes key absence in the true branch.
- `d.get(k, default)` — safe access that avoids `KeyError` but introduces an optionality section on the result.
- `d.setdefault(k, v)` — establishes key presence after the call.
- `try: v = d[k] except KeyError: ...` — the try body site sees key presence; the except body sees key absence.

0.28.2 Value-type refinement per key

For dictionaries used as records or configuration objects, the system should track per-key value types. A `TypedDict` annotation provides this directly. Without annotations, the system can infer per-key types from construction patterns: `d = \{"name": "Alice", "age": 30\}` gives d["name"] : `str` and d["age"] : `int`.

When a function receives a dict and accesses specific keys with type-sensitive operations, the backward pass should synthesize per-key type requirements. For example, if the function calls `int(d["count"])`, the backward pass should require that `"count"` is present and that its value supports conversion to `int`.

0.28.3 Dict-specific error sites

The following error sites should be registered:

- `KeyError`: `d[k]` when k is not in the key domain and no guard or `get` alternative is used.
- `TypeError`: using an unhashable key type.
- `AttributeError`: calling dict-specific methods on a non-dict value that was expected to be a dict.
- Structural mismatch: passing a dict that lacks required keys to a function expecting a particular record shape.

Backward synthesis of key requirements. When the function accesses `d["x"]`, `d["y"]`, and `d["z"]`, the backward pass should synthesize the input requirement {"x", "y", "z"} $\subseteq$ keys(d). If any access is guarded, only the unguarded accesses contribute to the mandatory requirement.

Forward synthesis of structural guarantees. When a function constructs and returns a dict, the output section should describe the key domain and per-key value types. This is especially valuable for JSON-like pipelines and configuration builders where downstream consumers depend on specific keys being present.

0.28.4 TypedDict and structural dict contracts

Python's `TypedDict` provides a static annotation for dict structure. The sheaf-descent system should treat `TypedDict` annotations as boundary seeds that pin the key domain and per-key types. But the system should also be able to infer TypedDict-like structural contracts from unannotated code. If a function always constructs a dict with keys `"status"`, `"data"`, and `"error"`, the output section should reflect that structure even without an explicit `TypedDict` annotation.

0.28.5 Dict comprehensions and transformations

Dict comprehensions like `\{k: f(v) for k, v in items\}` should propagate key-domain information from the source. If the source has a known key domain, the result has the same domain with transformed values. Filtering comprehensions like `\{k: v for k, v in d.items() if p(k)\}` produce a subset domain.

The `dict.update()` method and the `|` merge operator should be modeled as domain-union operations. After `d1.update(d2)`, the post-mutation site has $\text{keys}(d1') \supseteq \text{keys}(d1) \cup \text{keys}(d2)$ and value types from $d2$ override those from $d1$ for shared keys.

0.28.6 Interaction with JSON, serialization, and API boundaries

Many Python programs receive dicts from JSON parsing, API responses, or configuration files. These are precisely the places where key-membership errors are most common and most painful. The sheaf-descent system should treat deserialization boundaries as input sites with rich structural requirements. If downstream code accesses specific keys, the backward pass should synthesize the structural contract that the input JSON must satisfy. This is a high-value practical deliverable that requires no theorem-level sophistication—just careful key-domain tracking and backward propagation.

0.29 Deep dive: heap semantics, object state, and mutation tracking

Python is a heap-heavy language. Objects carry mutable fields, classes define initialization protocols, methods read and write instance state, and aliasing is pervasive. A serious dependent environment must model enough heap structure to catch real bugs—missing field initialization, stale alias reads, frame violations—without attempting a full separation-logic encoding that would be impractical for a Python tool.

0.29.1 Object state as a local section

Every object-valued SSA site should carry a local section describing the object's known state: which fields are definitely initialized, what types those fields carry, whether the object satisfies a particular protocol, and what mutation frame applies. This is the heap analog of carrying a length coordinate on a list site or a shape coordinate on a tensor site.

Field initialization tracking. The constructor body (`__init__`) is the primary site where fields are established. Each `self.field = value` statement creates a post-assignment site where the field is marked initialized with a particular type. At the end of the constructor, the object should carry a section listing all initialized fields. If a field is conditionally initialized (inside an `if` branch), the post-constructor section should reflect that the field is only *possibly* initialized, creating an obstruction that the backward pass can convert into an input requirement on the branch condition.

Field read safety. An `AttributeError` error site should be created at every field read `self.field` or `obj.field`. The viability predicate requires that the field is initialized in the current heap section. The backward pass should pull this requirement to the constructor or to the call site that created the object.

Example: conditional initialization bug. Consider a class whose constructor initializes `self.data` only when a flag is true. Any method that reads `self.data` unconditionally has a reachable `AttributeError`. The sheaf-descent system should detect this by noting that the field-initialization site is branch-dependent while the field-read site is unconditional, producing an obstruction at the overlap between constructor output and method input.

0.29.2 Mutation frames and aliasing

When a method mutates an object's field, the post-mutation site should update the field's type and value information. When two variables alias the same object (`y = x`), mutations through one alias must be visible at the other. The heap-SSA layer should track aliasing so that the cover builder can insert appropriate overlap edges between sites that share aliased heap locations.

A practical simplification is to track *frame conditions* rather than full alias analysis. A frame condition for a function states which heap locations may be modified and which are preserved. If a function's frame condition says it does not modify `obj.data`, then callers can assume that `obj.data` retains its pre-call section after the call returns.

0.29.3 Ownership and resource semantics

Some Python objects represent resources with ownership semantics: file handles, database connections, locks, GPU memory allocations. The sheaf-descent system should support ownership sections that track whether a resource is open, closed, acquired, or released. An error site should be created at every use of a resource that requires it to be in a particular state. For example, calling `f.read()` after `f.close()` should produce an error site with viability predicate "file is open."

Context managers (`with` statements) provide natural ownership boundaries. The `__enter__` site establishes the resource as acquired; the `__exit__` site marks it as released. Between them, the resource is available. After the `with` block, the resource should not be used. The system should synthesize these ownership transitions automatically for standard context-manager patterns.

0.29.4 Class hierarchies and method resolution

When a method is called on an object, the system must resolve which implementation will execute. For concrete classes this is straightforward. For abstract base classes or protocol-typed parameters, the resolution depends on the runtime type. The sheaf-descent system should model method resolution as a dispatch site where the viability predicate requires that the receiver's type supports the called method. This integrates naturally with protocol reasoning (covered in the next section) because protocol conformance is exactly the statement that the dispatch site is viable.

0.29.5 Dataclasses, named tuples, and structured constructors

Python's `dataclasses`, `typing.NamedTuple`, and `attrs` classes provide structured constructors with known fields. These should be treated as special cases where the constructor section is fully determined by the class definition. Every field is initialized, every field has a declared type, and the post-construction section is exact. This makes dataclass-heavy code especially amenable to automatic analysis.

0.29.6 Heap-related error sites

- `AttributeError`: field not initialized or not present on the receiver's type.
- Stale alias: reading a field through an alias after it was mutated through another path, leading to unexpected values.
- Resource use after release: calling methods on a closed file, released lock, or deallocated buffer.
- Frame violation: a function modifies heap state that its caller assumed was preserved.
- Constructor invariant failure: an object is used before its constructor has established the required field set.
- Slot mismatch: accessing a field not declared in `__slots__`.

Each of these should have a viability predicate, a backward pullback rule to the constructor or caller boundary, and an explanation template suitable for rendering to ordinary Python developers.

0.30 Deep dive: protocols, structural typing, and duck typing

Python's type system is fundamentally structural in practice: if an object has the right methods and attributes, it works, regardless of its nominal class hierarchy. The `typing.Protocol` mechanism formalizes this, but most Python code relies on duck typing without explicit protocol annotations. A sheaf-descent system must handle both annotated and unannotated structural typing with enough precision to catch protocol mismatches and synthesize structural requirements.

0.30.1 Protocol conformance as a local section

When a function calls `obj.read()`, `obj.write(data)`, or `len(obj)`, it implicitly requires that the receiver conforms to a protocol: the object must have a `read` method, a `write` method, or a `__len__` method. Each such method access creates a protocol-dispatch site with a viability predicate requiring the method's existence and correct signature.

The sheaf-descent system should infer these protocol requirements automatically from method calls, attribute accesses, and special-method invocations (`__iter__`, `__getitem__`, `__contains__`, `__enter__`, `__exit__`, etc.). The backward pass should collect all required methods and attributes into a synthesized protocol section on the input boundary.

Example: synthesized protocol requirement. Given a function:

```
def process(stream):
    header = stream.readline()
    data = stream.read()
    stream.close()
    return header, data
```

The system should synthesize the input requirement that `stream` must have methods `readline() -> str`, `read() -> str`, and `close() -> None`. This is equivalent to conformance with a protocol containing those three methods.

0.30.2 Explicit Protocol annotations as boundary seeds

When a user provides `typing.Protocol` annotations, those should be treated as boundary seeds that pin the protocol section. The system should verify that the function body's usage is consistent with the declared protocol and that callers provide objects conforming to it. If the function uses methods beyond those declared in the protocol, the system should report an obstruction: the declared protocol is weaker than what the body requires.

0.30.3 Protocol subtyping and refinement

Protocol subtyping is structural: a protocol P is a subtype of Q if P has all the methods of Q with compatible signatures. The sheaf-descent system should compute protocol subtyping at overlap sites where a value flows from a site with one protocol assumption to a site with another. If the downstream protocol is not a subtype of the upstream protocol, the overlap produces an obstruction.

Protocol refinement goes further: a refined protocol may not only require methods but also attach semantic contracts to them. For example, an `OrderedContainer` protocol might require not just `__getitem__` but also that the elements are accessible in sorted order. This is where protocol reasoning connects to the broader dependent typing story.

0.30.4 Duck typing without annotations

Most Python code uses duck typing without explicit `Protocol` declarations. The sheaf-descent system should handle this by inferring implicit protocols from usage patterns. If a function calls `obj.fit(X, y)` and `obj.predict(X)`, the system infers a "sklearn-like estimator" protocol even without an explicit annotation. This inferred protocol becomes a local section on the input boundary.

The quality of this inference depends on how completely the system harvests method calls, attribute accesses, and special-method invocations from the function body. The harvest layer should be thorough, and the resulting protocol section should be renderable as a concrete `Protocol` class definition that the user could paste into their code if they wanted to make the requirement explicit.

0.30.5 Protocol-related error sites

- `AttributeError`: the receiver does not have the required method or attribute.
- `TypeError`: the method exists but has an incompatible signature (wrong number of arguments, wrong argument types).
- Protocol mismatch: a caller passes an object that satisfies some but not all of the required protocol methods.
- Semantic protocol violation: the method exists and has the right signature but does not satisfy a semantic contract (e.g., a `__lt__` that does not define a total order).

0.30.6 Iterable, iterator, and sequence protocols

The iteration protocol (`__iter__`, `__next__`) deserves special treatment because it is used in `for` loops, comprehensions, unpacking, and many library functions. The system should model:

- Whether an object is iterable (has `__iter__`).

- Whether iteration is finite (known or bounded length) or potentially infinite.
- The element type yielded by iteration.
- Whether the object supports random access (`__getitem__` with integer keys), which upgrades it from `Iterable` to `Sequence`.
- Whether the object supports `__len__`, which enables length-based reasoning.

These protocol facts should flow into downstream sites. A `for x in xs` loop body site should see that `x` has the element type of `xs`. A `list(xs)` call should produce a list with the same element type and, if the length is known, the same length.

0.30.7 Callable protocols and higher-order functions

Functions are first-class objects in Python, and higher-order patterns are common: callbacks, decorators, `map`, `filter`, `functools.reduce`. The system should model callable protocols:

- The number and types of parameters.
- The return type.
- Side effects (mutation frame, I/O, exceptions).
- Semantic contracts on the callable (e.g., a comparator must define a total order; a predicate must return a boolean).

When a higher-order function receives a callable argument, the backward pass should synthesize a callable-protocol requirement. When the callable is applied inside the function, the forward pass should propagate the callable's return type and semantic properties into the result section.

0.31 Deep dive: decorators, metaprogramming, and dynamic class construction

Python's decorator mechanism, metaclasses, and dynamic class construction patterns are among the most challenging features for any static analysis tool. A sheaf-descent system cannot pretend they do not exist; it must model them well enough to handle common patterns and degrade gracefully on exotic ones.

0.31.1 Decorators as semantic transformers

A decorator `@deco` applied to a function `f` replaces `f` with `deco(f)`. The system must model how this transformation affects the function's semantic section. Several important cases arise:

Identity-preserving decorators. Some decorators (`@staticmethod`, `@classmethod`, `@property`, `@functools.wraps`, many logging or timing decorators) preserve the essential signature and semantics of the wrapped function. The system should recognize these as transparent wrappers and propagate the inner function's semantic section through the decorator site.

Signature-changing decorators. Some decorators change the function's signature: adding parameters, removing parameters, changing return types, or wrapping the return value. For example, `@click.command()` transforms a function into a CLI command object; `@torch.no_grad()` wraps a function to disable gradient computation; retry decorators may change the exception profile. The system should model these through decorator-specific theory packs that describe how the output section relates to the input section.

Validation decorators. Decorators like `@pydantic.validate_call`, `@beartype.beartype`, or custom input-validation decorators add runtime checks. These should be treated as guard-insertion sites: the decorated function's body site sees inputs that have passed the validation checks, strengthening the local section.

DepPy's own decorators. The `@deppy.requires`, `@deppy.ensures`, `@deppy.theorem`, and `@deppy.decreases` decorators are boundary seeds in the sheaf-descent architecture. They should be parsed by the frontend, converted into boundary section constraints, and removed from the runtime execution path. The system should verify that the function body is consistent with the declared contracts and that callers satisfy the declared requirements.

0.31.2 Metaclasses and dynamic class construction

Python supports metaclasses (`type` subclasses), `__init_subclass__`, `__set_name__`, and dynamic class creation via `type("Name", bases, namespace)`. These features are used by frameworks like Django, SQLAlchemy, and dataclasses. A full treatment is beyond the scope of a first implementation, but the system should handle the common cases:

- `@dataclass`: treat as a structured constructor with known fields and types.
- `NamedTuple`: treat as an immutable record with known fields.
- `Enum`: treat as a finite value domain with known members.
- Django/SQLAlchemy models: treat as objects with fields declared in the class body, analogous to dataclasses but with ORM-specific semantics.

For unrecognized metaclass patterns, the system should emit a residual explaining that the class's field structure cannot be determined statically. This is honest and useful: the user learns exactly where the analysis reaches its limits.

0.31.3 Dynamic attribute assignment and `__getattr__`

Python allows dynamic attribute assignment (`obj.new_field = value`) and custom attribute resolution (`__getattr__`, `__getattribute__`). These make heap reasoning harder because the set of fields is not statically bounded. The system should handle three tiers:

1. Classes with `__slots__`: the field set is exactly known.
2. Classes without `__slots__` but with a standard `__init__`: the field set is determined by analyzing the constructor.
3. Classes with `__getattr__` or metaclass magic: the field set is unknown and field accesses should generate residual protocol requirements rather than concrete field-initialization checks.

0.31.4 Monkey-patching and module-level mutation

Python allows monkey-patching: modifying classes or modules at runtime. The system should not attempt to track arbitrary monkey-patches, but it should recognize common patterns:

- Module-level configuration: `config.DEBUG = True` at import time.
- Test fixtures that replace methods or attributes for testing.
- Plugin systems that register handlers dynamically.

For these patterns, the system should model the mutation effect at the module-initialization site and propagate the resulting state to downstream sites. If a monkey-patch invalidates a previously established section, the system should report the obstruction.

0.31.5 Decorator stacking and composition

Multiple decorators can be stacked:

```
@deppy.ensures(lambda r: r >= 0)
@functools.lru_cache(maxsize=128)
@deppy.requires(lambda n: n >= 0)
def fibonacci(n: int) -> int:
    ...
```

The system should process decorators from innermost to outermost. The `@deppy.requires` creates an input boundary seed. The `@lru_cache` wraps the function but preserves its semantic section (caching does not change the function's logical contract). The `@deppy.ensures` creates an output boundary seed. The resulting section should carry both the input requirement and the output guarantee.

0.32 Deep dive: the optional annotation system and annotation economy

A central thesis of this monograph is that annotations should be optional seeds, not mandatory prerequisites. The sheaf-descent architecture gives this thesis a precise meaning: an annotation is a user-supplied local section or boundary constraint that eliminates ambiguity, strengthens the global section, or resolves an obstruction. The system should work at every annotation level from zero to fully explicit, with smooth and predictable improvements as the user adds more.

0.32.1 Zero-annotation mode

In zero-annotation mode, the system relies entirely on syntax, control flow, library semantics, and optional dynamic traces. The cover is built from AST and SSA structure. Theory packs create sites from recognized library patterns. Guard harvesting extracts implicit refinements from branches and assertions. The backward and forward passes run to compute synthesized input requirements and output guarantees.

The quality of zero-annotation analysis depends on three factors:

1. How completely the harvest layer extracts guards, assertions, and construction patterns.
2. How rich the library theory packs are for the libraries actually used.
3. How deeply interprocedural transport preserves semantics across call boundaries.

For simple Torch modules, zero-annotation analysis should already catch shape errors, index errors, and values-versus-indices confusion. For complex algorithm code, zero-annotation analysis may leave significant residuals, but it should still identify which sites carry ambiguity and what kind of seed would resolve it.

0.32.2 Lightweight annotation: type hints

Python's standard type hints (`def f(x: int) -> str`) provide carrier-type information. These are already seeds: they pin the carrier coordinate of the input and output boundary sections. The system should use them to:

- Narrow element types for generic containers: `list[int]`, `dict[str, float]`, `Optional[Tensor]`.
- Establish protocol conformance: `Iterable[T]`, `Callable[[int], str]`, `Sequence[float]`.
- Enable richer downstream inference: knowing that `x: int` allows arithmetic-site solvers to fire; knowing that `xs: list[int]` enables length and element-type reasoning.

Standard type hints are cheap and familiar. The system should make the most of them by treating them as lightweight but valuable boundary seeds rather than ignoring them because they are not full dependent types.

0.32.3 Medium annotation: deppy decorators

The `@deppy.requires` and `@deppy.ensures` decorators allow users to state refinement predicates:

```
@deppy.requires(lambda xs: len(xs) > 0)
@deppy.ensures(lambda result, xs: result in xs)
def first_element(xs: list[int]) -> int:
    return xs[0]
```

These decorators are boundary seeds that resolve specific obstructions. The `requires` clause pins the input boundary section so that the `IndexError` site at `xs[0]` becomes empty. The `ensures` clause pins the output boundary section so that downstream consumers know the result is an element of the original list.

The system should not require both. If only `requires` is provided, the system should synthesize the strongest output section justified by the requirement. If only `ensures` is provided, the system should synthesize the weakest input section needed to support the guarantee. If neither is provided, both are synthesized. If both are provided, the system should verify consistency and report any obstruction between them.

0.32.4 Heavy annotation: theorem decorators and proof modules

For proof-oriented users, the system supports theorem decorators, ghost parameters, witness-returning functions, and explicit transport lemmas. These are heavy annotations in the sense that they carry high semantic payload, but they should still be expressed in native Python syntax:

```
@deppy.theorem
def sort_preserves_permutation(xs: list[int]):
    result = sorted(xs)
    deppy.witness(deppy.permutation(result, xs))
    return result
```

The theorem decorator tells the system to treat this function as a proof artifact rather than a runtime computation. The `deppy.witness` call exports a witness section. The proof elaborator converts this into a proof site with a transport map that downstream consumers can import.

0.32.5 Annotation economy: measuring the value of each annotation

The sheaf-descent architecture enables a precise measurement of annotation value. When a user adds an annotation, the system can compute:

- How many obstruction coordinates the annotation resolves.
- How many error sites become provably safe.
- How many output-section coordinates become stronger.
- How many downstream functions benefit through interprocedural transport.

This information should be exposed to the user through the renderer. A diagnostic like "adding `@deppy.requires(lambda xs: len(xs) > 0)` would resolve 3 error sites and strengthen 2 output contracts" is far more useful than a generic "add a precondition." This is annotation economy: helping users invest their annotation effort where it provides the most semantic return.

0.32.6 LLM-authored annotations

Large language models can generate annotations, contracts, and even theorem modules. The system should treat LLM-generated annotations the same as user-written ones: they are boundary seeds that must be verified. If an LLM suggests `@deppy.ensures(lambda r: sorted(r))`, the system should check whether the function body actually supports this claim. If verification succeeds, the annotation is accepted. If it fails, the system should report the specific obstruction that the LLM's suggestion did not address.

This creates a natural feedback loop: the LLM suggests, the system verifies, and unresolved obstructions guide the next suggestion. The key architectural point is that LLM outputs never bypass the trust boundary. They are treated as unverified seeds until the kernel or solver confirms them.

0.32.7 Gradual adoption path

The annotation system should support gradual adoption:

1. Start with zero annotations; the system infers what it can.
2. Add standard type hints to improve carrier-type reasoning.
3. Add `@deppy.requires`/`@deppy.ensures` where the system reports high-value obstructions.
4. Add theorem modules for algorithmic correctness where full dependent guarantees are desired.

At each step, the system should report what has improved and what remains. The user should never feel that they must annotate everything or nothing. The sheaf-descent architecture makes this possible because every annotation is a local operation on the cover, not a global mode switch.

0.33 Deep dive: PyTorch shape theory and symbolic shape inference

Shape errors are the most common runtime failure mode in tensor programs. A reshape with incompatible cardinality, a matmul with misaligned inner dimensions, a broadcast between incompatible extents, or a reduction over a nonexistent axis—these all produce opaque `RuntimeError` messages that could have been caught statically. The sheaf-descent system should provide a comprehensive shape theory that can infer output shapes as symbolic functions of input shapes and synthesize the weakest input shape requirements that prevent all shape-related runtime errors.

0.33.1 Shape representation

A tensor's shape is a tuple of non-negative integers. The system should represent shapes symbolically so that relationships between input and output shapes can be expressed as equations and inequalities.

Definition 0.33.1 (Symbolic shape)**.** A symbolic shape is a tuple $(d_1, d_2, \ldots, d_r)$ where each d_i is either a concrete non-negative integer, a symbolic variable, or a symbolic expression over variables. The rank r may itself be symbolic or concrete.

For example, if a function receives a tensor `x` with unknown shape, the system represents its shape as $(d_1, d_2, \ldots, d_r)$ with symbolic variables. After `y = x.view(d_1, -1)`, the output shape is $(d_1, \prod_{i=2}^{r} d_i)$, which can be expressed symbolically as $(d_1, d_1^{-1} \cdot \prod_i d_i)$.

0.33.2 Shape rules for core operations

Each tensor operation has a shape rule that relates input shapes to output shapes. The system should encode these rules as site-local viability predicates and output-shape functions.

Element-wise operations. For element-wise operations (`+`, `-`, `*`, `/`, `torch.relu`, etc.), the output shape equals the broadcast-compatible shape of the inputs. The viability predicate requires that the inputs are broadcast-compatible: for each dimension, either the extents match, or one of them is 1. The output extent in each dimension is the maximum of the input extents.

Matrix multiplication. For `torch.matmul(A, B)` or `A @ B`:

- If A has shape $(\ldots, m, k)$ and B has shape $(\ldots, k, n)$, the output has shape $(\ldots, m, n)$.
- The viability predicate requires that the inner dimensions match: $A.\text{shape}[-1] = B.\text{shape}[-2]$.
- Batch dimensions must be broadcast-compatible.

Reshape and view. For `x.view(s_1, s_2, \ldots , s_k)` or `x.reshape(s_1, \ldots , s_k)`:

- The viability predicate requires $\prod_i s_i = \prod_j d_j$ where d_j are the input dimensions (with at most one $s_i = -1$, representing the inferred dimension).
- For `view` specifically, contiguity of the input tensor is also required; if the input is not contiguous, a `RuntimeError` occurs.
- The output shape is $(s_1, s_2, \ldots, s_k)$ with the -1 dimension resolved to $\prod_j d_j / \prod_{i \neq -1} s_i$.

Transpose and permute. For `x.transpose(dim0, dim1)`, the output shape swaps dimensions `dim0` and `dim1`. The viability predicate requires that both dimension indices are valid: $0 \leq \texttt{dim0}, \texttt{dim1} < r$. For `x.permute(dims)`, the output shape is the input shape permuted according to `dims`, with viability requiring that `dims` is a valid permutation of $\{0, \ldots, r-1\}$.

Reduction operations. For `x.sum(dim=d)`, `x.mean(dim=d)`, etc.:

- If `keepdim=False`, the output shape is the input shape with dimension d removed.
- If `keepdim=True`, the output shape has dimension d set to 1.
- The viability predicate requires $0 \leq d < r$.
- For operations that require nonempty input along the reduced dimension (e.g., `argmax`, `argmin`), the viability predicate additionally requires $d_{\texttt{dim}} > 0$.

Concatenation and stacking. For `torch.cat(tensors, dim=d)`:

- All tensors must have the same number of dimensions.
- All dimensions except d must match across all tensors.
- The output dimension d is the sum of the input dimensions at d.

For `torch.stack(tensors, dim=d)`, a new dimension is inserted at position d with extent equal to the number of tensors.

Indexing and gathering. For `x[index]`, `torch.gather(x, dim, index)`, and `torch.index_select(x, dim, index)`:

- The viability predicate requires that all index values lie within $[0, x.\text{shape}[\texttt{dim}])$.
- The output shape depends on the indexing mode: basic indexing may reduce dimensions; advanced indexing may reshape according to the index tensor's shape; `gather` preserves the index tensor's shape.

Convolution. For `torch.nn.Conv2d(in_channels, out_channels, kernel_size, ...)` applied to input of shape (N, C_{in}, H, W):

- Viability requires C_{in} matches the layer's `in_channels`.
- Output shape: $(N, C_{out}, H_{out}, W_{out})$ where $H_{out} = \lfloor (H + 2p - k)/s \rfloor + 1$ with padding p, kernel size k, and stride s.
- Viability requires $H + 2p - k \geq 0$ and similarly for W.

0.33.3 Symbolic shape propagation through a module

A PyTorch module typically chains several operations. The shape theory should propagate symbolic shapes through the entire chain, accumulating equations and viability predicates at each step. For example:

```
def forward(self, x): # x: (batch, channels, height, width)
    x = self.conv1(x) # (batch, 32, h1, w1)
    x = x.view(x.size(0), -1) # (batch, 32 * h1 * w1)
    x = self.fc(x) # (batch, num_classes)
    return x
```

The shape theory should:

1. Start with symbolic input shape (b, c, h, w).
2. After `conv1`: output $(b, 32, h_1, w_1)$ where h_1, w_1 are functions of h, w and the convolution parameters.
3. After `view`: output $(b, 32 \cdot h_1 \cdot w_1)$. The viability predicate is automatically satisfied because the product is preserved.
4. After `fc`: output (b, n_c) with viability requiring $32 \cdot h_1 \cdot w_1 =$ `fc.in_features`.

The backward pass should pull the viability requirement at `fc` back to the input: the input dimensions (h, w) must satisfy $32 \cdot h_1(h) \cdot w_1(w) =$ `fc.in_features`. This is the synthesized input shape requirement for the module.

0.33.4 Output shape as a refinement type

The output section should express the output shape as a symbolic function of the input shape. For the module above, the output refinement type would be something like:

$$\texttt{result} : \texttt{Tensor}[\text{shape} = (b, n_c)]$$

where b is the batch dimension from the input. This is a genuine dependent type: the output shape depends on the input shape.

For more complex modules, the output shape may be a piecewise function (if there are shape-dependent branches) or may contain symbolic expressions involving input dimensions, layer parameters, and arithmetic operations.

0.33.5 Shape error sites and diagnostics

Each shape-violating operation should create an error site with:

- The exact viability predicate that was violated.
- The symbolic shape values at the point of failure.
- A backward trace showing which input shape constraint would prevent the error.
- A suggested fix in terms of input shape requirements, layer parameter changes, or inserted reshape operations.

This is the flagship deliverable for Torch users: run the analyzer on a module and get a precise report of all potential shape errors, together with the exact input shape constraints that would prevent them.

0.33.6 Broadcasting rules and their subtleties

Broadcasting is one of the most error-prone aspects of tensor programming because it is implicit and can mask dimension mismatches. The shape theory should model broadcasting precisely:

1. Right-align the shapes of the operands.
2. For each dimension, check that the extents are either equal or one of them is 1.
3. If neither condition holds, report a broadcast mismatch error site.
4. The output extent in each dimension is the maximum of the operand extents.

The backward pass should synthesize input shape requirements that guarantee broadcast compatibility. If a function adds tensors of shapes (a, b) and (c, d), the requirement is $a = c \vee a = 1 \vee c = 1$ and $b = d \vee b = 1 \vee d = 1$.

0.33.7 Device and dtype as secondary shape coordinates

While shape is the primary concern, device and dtype are closely related coordinates that affect runtime safety:

- Operations on tensors from different devices produce a `RuntimeError`.
- Some operations are not supported for certain dtypes (e.g., integer tensors do not support `torch.sin`).
- Mixed-precision operations may produce unexpected results.

The shape theory should carry device and dtype as secondary coordinates on tensor sites. The viability predicate for any binary operation should include device equality, and the output section should specify the result dtype according to PyTorch's promotion rules.

0.34 Deep dive: library-specific theories beyond PyTorch

The sheaf-descent architecture is not limited to PyTorch. Any library whose operators have structured semantics—shape rules, protocol requirements, ordering assumptions, domain constraints—can be modeled through theory packs that create local sites, viability predicates, and transport maps. This section sketches theories for several important Python libraries.

0.34.1 NumPy shape and broadcasting theory

NumPy shares PyTorch's shape-oriented semantics but with its own operator vocabulary and broadcasting rules. A NumPy theory pack should create sites for:

- `np.reshape`, `np.resize`: cardinality viability and output shape.
- `np.dot`, `np.matmul`, `@`: inner-dimension alignment.
- `np.concatenate`, `np.stack`, `np.split`: dimension matching and output shape synthesis.
- `np.sum`, `np.mean`, `np.argmax`: reduction axis validity and non-emptiness.
- Broadcasting across all element-wise operations.
- `np.linalg.inv`, `np.linalg.solve`: square-matrix and full-rank requirements.

The backward pass should synthesize array shape requirements, and the forward pass should express output shapes as symbolic functions of input shapes. Because NumPy and PyTorch

share many shape rules, the implementation should factor common shape reasoning into a shared tensor-shape kernel that both theory packs use.

0.34.2 Pandas DataFrame and Series theory

Pandas introduces a different kind of structural semantics: column schemas, index alignment, and dtype-dependent operations.

Column-presence theory. A DataFrame has a column schema: a set of column names with associated dtypes. Accessing a column that does not exist produces a `KeyError`. The theory pack should:

- Create column-presence sites at every column access (`df["col"]`, `df.col`, `df[["col1", "col2"]]`).
- Pull back column-presence requirements to the input boundary.
- Propagate column schemas through transformations: `df.rename`, `df.drop`, `df.merge`, `df.assign`.
- Synthesize output column schemas for operations that add or remove columns.

Index alignment theory. Pandas operations align on indices, which can produce `NaN` values when indices do not match. The theory should track whether operations require index alignment and whether misaligned indices produce silent data loss or explicit errors.

GroupBy and aggregation theory. `df.groupby("col").agg(func)` requires that `"col"` exists and that `func` is applicable to the grouped columns' dtypes. The theory should create sites for these requirements and propagate the resulting output schema (grouped columns become the index; aggregated columns have modified dtypes).

Dtype-sensitive operations. Many Pandas operations are dtype-sensitive: `df.sum()` on string columns produces concatenation rather than arithmetic; `df.mean()` on non-numeric columns raises an error. The theory should track column dtypes and create error sites for dtype-incompatible operations.

0.34.3 Standard library: pathlib, os, and file I/O

File I/O introduces resource semantics and path-validity requirements:

- `open(path, mode)`: creates a file handle with open/closed state.
- `f.read()`, `f.write()`: require the file to be open and in the correct mode (read vs. write).
- `f.close()`: transitions the resource to closed state; subsequent reads/writes should produce error sites.
- `pathlib.Path`: methods like `.read_text()`, `.exists()`, `.mkdir()` have implicit filesystem preconditions.

The theory should model file handles as owned resources with state transitions and create error sites for use-after-close patterns. Context managers (`with open(...) as f`) should be recognized as safe boundaries.

0.34.4 Standard library: collections

The `collections` module provides specialized containers:

- `defaultdict`: eliminates `KeyError` for missing keys by providing a factory. The theory should recognize that `defaultdict` access sites are always viable but may produce default values.
- `OrderedDict`: maintains insertion order. Iteration sites should carry ordering semantics.
- `deque`: supports efficient append/pop from both ends. Length reasoning applies as with lists, but with additional methods (`appendleft`, `popleft`, `rotate`).
- `Counter`: maps elements to counts. The theory should track that values are non-negative integers and that `most_common()` returns sorted results.
- `namedtuple`: provides structured access by name. The theory should treat named tuples as immutable records with known fields.

0.34.5 Standard library: itertools and functools

The `itertools` module creates lazy iterators with specific semantic properties:

- `itertools.chain(*iterables)`: concatenation; output length is sum of input lengths.
- `itertools.product(*iterables)`: Cartesian product; output length is the product of input lengths.
- `itertools.islice(it, n)`: truncation; output length is $\min(n, \text{len}(it))$.
- `itertools.groupby(it, key)`: requires sorted input for correct grouping; the theory should create a sortedness requirement site.
- `itertools.accumulate(it, func)`: output has same length as input; func must be a binary operation compatible with element types.

The `functools` module provides higher-order utilities:

- `functools.reduce(func, it, init)`: requires the iterable to be nonempty if `init` is not provided; `func` must accept two arguments of compatible types.
- `functools.partial(f, *args)`: modifies the callable's signature; the theory should compute the resulting parameter list.
- `functools.lru_cache`: preserves the function's semantic section (caching does not change the logical contract).

0.34.6 SQLAlchemy and database ORM theory

Database ORMs introduce schema-dependent operations:

- Model classes define column schemas. Accessing a non-existent column should produce an error site.
- Query construction (`session.query(Model).filter(...)`) should verify that filter conditions reference valid columns.
- Result materialization (`.all()`, `.first()`, `.one()`) has different nullability profiles: `.one()` raises if zero or multiple results exist; `.first()` may return `None`.

The theory should model ORM columns, query construction, and result types so that the backward pass synthesizes valid query requirements and the forward pass describes result types and nullability.

0.34.7 Requests and HTTP client theory

The `requests` library introduces network-boundary semantics:

- `requests.get(url)`: returns a `Response` object whose `.status_code` may indicate failure.
- `response.json()`: may raise `ValueError` if the response is not valid JSON; the theory should create an error site.

- `response.raise_for_status()`: raises an `HTTPError` for non-2xx status codes; this acts as a guard that refines the subsequent section to assume success.

0.34.8 Theory-pack extensibility and composition

A critical architectural property is that theory packs compose. When a function uses both PyTorch and NumPy, or Pandas and SQLAlchemy, the site cover should contain sites from all relevant packs, and overlaps should be created at values that flow between libraries. For example, `torch.from_numpy(arr)` creates an overlap between a NumPy shape site and a PyTorch shape site; the shapes must agree. The descent engine should handle cross-library overlaps the same way it handles within-library overlaps.

Adding a new theory pack should be additive: it creates new site families, new viability predicates, and new transport maps, but it should not invalidate existing sites or require re-analysis of code that does not use the new library. This monotonicity property is a direct consequence of the sheaf-descent architecture: a new pack extends the cover rather than replacing it.

0.35 Deep dive: exception handling, control flow, and the Optional/None theory

Python uses exceptions pervasively—not only for error conditions but also for control flow in iterators, context managers, and pattern matching. A sheaf-descent system must model exceptions as first-class semantic events rather than treating them as an afterthought that escapes the type system.

0.35.1 Exceptional control flow as site structure

Every `try/except/finally` block defines a family of sites:

- The try-body site, where execution may raise exceptions.
- One except-handler site per handler clause, refined by the caught exception type.
- The else-body site, which sees the post-try state under the assumption that no exception was raised.
- The finally-body site, which executes regardless of outcome.
- The post-block site, which merges the normal and exceptional flows.

The key insight is that except-handler sites act as guard sites: inside `except KeyError`, the system knows that a `KeyError` was raised, which implies that some upstream operation failed in a specific way. This information can refine the local section. For example, if a `KeyError` was caught after `d[k]`, the handler body can infer that $k \notin \mathrm{keys}(d)$.

0.35.2 Exception type hierarchies

Python's exception hierarchy matters for handler matching. A handler for `except Exception` catches almost everything; a handler for `except ValueError` catches only `ValueError` and its subclasses. The system should model the exception type hierarchy so that handler sites carry the correct type refinement.

Bare except and broad handlers. A bare `except:` or `except Exception:` handler is a catch-all. The system should recognize these as providing weak refinement—the handler body knows an

exception occurred but not which one. For safety analysis, the system should warn when a broad handler masks a specific error that the user might want to handle differently.

0.35.3 Exception-raising operations as error sites

Every operation that can raise an exception should create an error site. The system already does this for `IndexError`, `KeyError`, etc. But the exception theory should go further:

- User-defined `raise` statements create explicit error sites with the raised exception type.
- `assert condition` creates a conditional error site that raises `AssertionError` when the condition is false.
- Iterator exhaustion (`StopIteration`) creates error sites at `next()` calls without defaults.
- `NotImplementedError` in abstract methods creates error sites at dispatch sites when the receiver is abstract.

0.35.4 The Optional/None theory

`None` is the single most common source of `AttributeError` and `TypeError` in Python. The Optional theory should track nullability as a refinement coordinate on every site where a value might be `None`.

Sources of None.

- Functions that may return `None`: `dict.get()`, `list.pop()` with a default, `re.match()`, any function with an explicit `return None` or missing return path.
- Default parameter values: `def f(x=None)`.
- Optional type annotations: `x: Optional[int]`.
- Conditional initialization: a variable assigned only in one branch.

None guards and narrowing. The harvest layer should recognize None-checking patterns:

- `if x is not None`: narrows the true branch to non-None.
- `if x is None`: narrows the true branch to None.
- `if x`: for types where None is falsy, narrows to non-None in the true branch (with caveats for other falsy values like 0 or empty strings).
- `assert x is not None`: narrows the post-assertion site.
- `x = y if y is not None else default`: the assignment site carries non-None when the condition is true.

None-related error sites. An `AttributeError` site should be created at every attribute access or method call on a value that might be `None`. The viability predicate requires the value to be non-None. The backward pass should synthesize either a non-None input requirement or require that an upstream guard narrows away the None case.

Optional chaining and safe navigation. Python does not have a safe navigation operator like `?.` in other languages, so Optional handling requires explicit guards. The system should recognize common patterns:

```
result = x.method() if x is not None else default
```

In this pattern, the method-call site is protected by the guard, and the output site carries the union of the method's return type and the default's type.

0.35.5 Control-flow refinement across branches

Beyond exceptions and None, the general control-flow story should handle:

- `isinstance(x, T)` guards: narrow the type of `x` in the true branch to `T`.
- `hasattr(obj, "field")` guards: establish field presence.
- Truthiness guards: `if xs` on a list narrows to nonempty.
- Comparison guards: `if n > 0` narrows to positive.
- Early returns: `if not valid: return None` narrows the continuation to the valid case.

Each of these patterns should be recognized by the harvest layer and converted into site-level refinements that participate in the sheaf-descent computation. The key principle is that control flow is evidence: every branch taken is a local observation that narrows the admissible sections.

0.35.6 Match statements (Python 3.10+)

Python's `match`/`case` syntax provides structured pattern matching:

```
match command:
    case {"action": "move", "x": x, "y": y}:
        handle_move(x, y)
    case {"action": "quit"}:
        handle_quit()
    case _:
        handle_unknown(command)
```

Each case arm should create a site refined by the pattern: the first arm knows that `command` is a dict with keys `"action"`, `"x"`, `"y"` and that `command["action"] == "move"`. The system should use these pattern-derived refinements as local section constraints.

0.35.7 Generators, yield, and async/await

Generator functions (`yield`) and async functions (`async def`/`await`) introduce suspended-execution semantics:

- A generator function's "return type" is actually its yield type. The output section should describe the type of yielded values.
- `send()` and `throw()` introduce additional interaction sites between the generator and its consumer.
- `async` functions return coroutines; the actual return type is available only after `await`. The system should model the "unwrapped" return type at `await` sites.
- `async for` and `async with` combine iteration/context-manager protocols with async semantics.

These are important for real Python code but represent a stretch goal for the implementation. The system should handle basic generator yield types and async return types, with more complex interaction patterns generating residuals.

0.36 Deep dive: numeric safety—overflow, NaN, division by zero, and floating-point contracts

Numerical computation is the primary use case for both PyTorch and NumPy, and numerical errors—NaN propagation, division by zero, overflow, underflow—are among the most insidious bugs

in scientific and machine-learning code. This section describes how the sheaf-descent framework handles numeric safety as a first-class theory.

0.36.1 The numeric-safety site family

Every arithmetic operation creates a numeric-safety site. The site's viability predicate depends on the operation:

- **Division** (`x / y`, `x // y`, `x \% y`): viability requires $y \neq 0$. For floating-point division, the result may be $\pm\infty$ rather than raising `ZeroDivisionError`, but this is typically still a bug.
- **Square root** (`math.sqrt(x)`, `np.sqrt(x)`, `torch.sqrt(x)`): viability requires $x \geq 0$ for real-valued results.
- **Logarithm** (`math.log(x)`, `np.log(x)`, `torch.log(x)`): viability requires $x > 0$.
- **Power** (`x ** y`): when $x = 0$ and $y < 0$, produces `ZeroDivisionError` or ∞.
- **Integer overflow**: Python's arbitrary-precision integers cannot overflow, but NumPy and PyTorch fixed-width integers can. The theory should track whether operations on fixed-width types might exceed their range.

0.36.2 NaN propagation theory

NaN (Not a Number) is a floating-point value that propagates through arithmetic: any operation involving NaN produces NaN. This makes NaN a "contagion" that can silently corrupt entire computations.

NaN sources. Operations that can produce NaN include:

- `0.0 / 0.0`, `float('inf') - float('inf')`, `float('inf') * 0.0`.
- `math.sqrt(-1.0)` in NumPy/PyTorch (returns NaN rather than raising).
- `math.log(0.0)` or `math.log(-1.0)`.
- User input or data loading from corrupted sources.

NaN-freedom as a refinement. The system should track a NaN-freedom refinement coordinate: a value is *NaN-free* if it is guaranteed to contain no NaN values. Operations that preserve NaN-freedom (e.g., addition of two NaN-free values) should propagate this property. Operations that might introduce NaN should create error sites.

NaN checks as guards. Calls like `torch.isnan(x).any()`, `np.isnan(x)`, or `math.isnan(x)` should be recognized as guard sites. After a NaN check that confirms no NaN is present, the section should carry the NaN-free refinement.

0.36.3 Infinity tracking

Similar to NaN, $\pm\infty$ can propagate through computations and cause problems. The system should track finiteness as a refinement coordinate:

- `torch.isinf(x)` and `np.isinf(x)` are guard operations.
- Operations like `torch.clamp(x, min=-M, max=M)` establish finiteness and boundedness.
- `torch.softmax` outputs are always finite and in $[0, 1]$ when the input is finite, which the theory should capture.

0.36.4 Division-by-zero analysis

Division by zero is the most common numeric error. The theory should handle several patterns:

- Direct division: `a / b` creates an error site with viability $b \neq 0$.
- Denominator computation: `a / (b + eps)` is a common safe pattern; the theory should recognize that adding a positive epsilon to the denominator prevents division by zero when $b \geq 0$.
- Reduction denominators: `x.sum() / x.size(0)` is safe when the tensor is nonempty, connecting numeric safety to shape theory.
- Normalization: `x / x.norm()` requires `x.norm() > 0`, which is equivalent to $x \neq \mathbf{0}$.

Backward synthesis for division safety. The backward pass should synthesize input requirements that prevent division by zero. For `a / b`, the requirement is $b \neq 0$. For `a / f(x)`, the requirement is $f(x) \neq 0$, which should be pulled back through f to a requirement on x.

0.36.5 Overflow and underflow in fixed-width arithmetic

While Python integers are arbitrary-precision, NumPy and PyTorch use fixed-width types (`int32`, `float32`, etc.). The theory should track:

- **Integer overflow**: multiplication or addition of large values may wrap around. The theory should create overflow sites when operations on bounded types might exceed range.
- **Floating-point underflow**: very small values may be rounded to zero, which can interact with division-by-zero analysis.
- **Precision loss**: large floating-point values lose precision; `x + 1.0 == x` can be true for sufficiently large `x`.
- **Dtype promotion**: mixed-type arithmetic follows promotion rules (e.g., `int32 + float32 -> float64`). The theory should track dtype through computations and create sites where unexpected promotion occurs.

0.36.6 Numeric-safety contracts for machine learning

Machine learning code has domain-specific numeric safety patterns:

- **Loss functions**: cross-entropy loss requires probabilities in $(0, 1]$; the logarithm of zero produces $-\infty$. The theory should verify that softmax or sigmoid has been applied before log-loss.
- **Gradient computation**: gradients can be NaN or $\pm\infty$ when the loss function has singularities. Gradient clipping (`torch.nn.utils.clip_grad_norm_`) establishes boundedness.
- **Learning rate**: a learning rate of zero causes no parameter updates (not numerically unsafe but semantically problematic); a negative learning rate inverts the gradient direction.
- **Batch normalization**: requires batch size > 1 to compute variance; division by batch variance requires the batch to not be identical (variance > 0).
- **Softmax temperature**: `softmax(x / T)` requires $T > 0$; as $T \to 0$, the softmax approaches argmax and may cause numeric instability.

0.36.7 Interaction with the solver boundary

Numeric safety predicates are typically in decidable fragments:

- Linear arithmetic over integers: overflow bounds, array lengths.
- Nonlinear real arithmetic: square root domains, logarithm domains.

- Floating-point theory: Z3's FPA theory can reason about specific floating-point behaviors, though it is expensive.

The system should use the cheapest sufficient theory: linear integer arithmetic for most overflow and division-by-zero checks, nonlinear real arithmetic for domain constraints, and floating-point theory only when precise bit-level reasoning is required. When the numeric predicate exceeds the solver boundary, it should be surfaced as a residual obligation with an explanation.

0.37 Deep dive: comprehensions, iteration semantics, and string safety

Python's comprehension syntax, iterator protocol, and string operations are ubiquitous in real code. Each introduces semantic constraints that the sheaf-descent framework should capture.

0.37.1 List, dict, set, and generator comprehensions

Comprehensions are syntactic sugar for iteration with optional filtering. Each comprehension creates a family of sites:

Iterable sites. The source expression (`for x in source`) creates an iterable site. The viability predicate requires `source` to implement the iterator protocol (`__iter__` and `__next__`). The element type of the iteration variable x is determined by the source's yield type.

Filter sites. If the comprehension has an `if` clause (`[f(x) for x in xs if pred(x)]`), the filter condition creates a guard site. Inside the body, x is refined by the predicate.

Output length reasoning. The output of `[f(x) for x in xs]` has the same length as `xs` when there is no filter. With a filter, the output length is bounded above by the input length: $0 \leq |\text{output}| \leq |\text{input}|$. The system should track these bounds.

Nested comprehensions. `[f(x, y) for x in xs for y in ys]` iterates over the Cartesian product. The output length is $|\text{xs}| \times |\text{ys}|$. The system should compose length constraints multiplicatively.

Dict comprehensions. `\{k: v for k, v in items\}` creates a dict whose keys are determined by the comprehension. If the key expression can produce duplicates, later values overwrite earlier ones. The system should create a key-uniqueness site when the output dict's key set matters.

0.37.2 For-loop semantics and loop-carried invariants

For-loops are the general iteration mechanism. The system should handle:

Loop bounds. `for i in range(n)` creates a site where i ranges over $[0, n)$. Accesses like `xs[i]` inside the loop are safe when $n \leq \text{len}(xs)$. The system should synthesize this relationship as a backward requirement.

Loop-carried accumulation. A common pattern is accumulating a result:

```
total = 0
for x in xs:
    total += x
```

The loop body creates an accumulation site. The output refinement for `total` after the loop should express that it equals the sum of elements in `xs`. In general, loop-carried invariants are undecidable, but common patterns (summation, product, min, max, counting) should have dedicated invariant templates.

Loop-carried list construction.

```
results = []
for x in xs:
    results.append(f(x))
```

The system should recognize this as equivalent to `[f(x) for x in xs]` and apply the same length reasoning.

While-loop invariants. While-loops require explicit or synthesized invariants. The system should attempt to synthesize invariants from the loop condition, body mutations, and initialization. When synthesis fails, the invariant becomes a residual obligation. Common patterns:

- Countdown: `while n > 0:  n -= 1`—the invariant is $n \geq 0$.
- Search: `while not found:  ...`—the post-loop state has `found = True`.
- Convergence: `while error > tol:  ...`—the post-loop state has `error <= tol`.

0.37.3 Iterator protocol and lazy evaluation

Python's iterator protocol (`__iter__`, `__next__`) introduces state-dependent behavior:

- An iterator is consumed: after iterating once, calling `next()` again produces the next element or raises `StopIteration`.
- Iterators are single-pass unless they implement `__iter__` by returning a fresh iterator.
- `list(iterator)` materializes the iterator; calling `list()` on an already-consumed iterator produces an empty list.

The system should model iterator state as a resource: creating an iterator produces a fresh resource, consuming it depletes the resource, and using a depleted iterator produces unexpected empty results.

0.37.4 String operations and encoding safety

Strings in Python are sequences of Unicode codepoints. Key safety properties:

String indexing and slicing. `s[i]` requires $0 \leq i < \text{len}(s)$ (or $-\text{len}(s) \leq i < 0$ for negative indices). The same bounds reasoning as list indexing applies.

String formatting. `f"\{x:.2f\}"` requires `x` to be a number. `"\{0\} \{1\}".format(a, b)` requires at least two positional arguments. `"\{name\}".format(**d)` requires the key `"name"` to be present in `d`. The system should create error sites for format-string mismatches.

Encoding and decoding. `s.encode("utf-8")` is always safe for valid Python strings, but `b.decode("utf-8")` can raise `UnicodeDecodeError` when the bytes are not valid UTF-8. The system should create an error site for decoding operations and allow backward synthesis of encoding requirements.

String methods with semantic guarantees.

- `s.split(sep)` returns a list with at least one element.
- `s.strip()` returns a string no longer than `s`.
- `s.startswith(prefix)` and `s.endswith(suffix)` are boolean guards that refine the string's content.
- `s.replace(old, new)` preserves length when len(old) = len(new).
- `int(s)` and `float(s)` require `s` to represent a valid number; the system should create error sites for parsing operations.

Regular expressions. `re.match(pattern, s)` returns a match object or `None`. A successful match provides group access, where `m.group(i)` requires i to be a valid group index. The theory should model match results as optional-with-groups and create error sites for invalid group access.

0.37.5 Zip, enumerate, and iteration combinators

zip. `zip(xs, ys)` truncates to the shorter input. The output length is $\min(|\text{xs}|, |\text{ys}|)$. If the user expects equal-length inputs, they should use `zip(xs, ys, strict=True)` (Python 3.10+), which raises `ValueError` on length mismatch. The system should create a length-equality site when `strict=True` is used.

enumerate. `enumerate(xs, start=0)` yields pairs (i, x_i) where i starts at `start`. The system should track that the index i satisfies $\texttt{start} \leq i < \texttt{start} + |\text{xs}|$.

map and filter. `map(f, xs)` applies `f` to each element; the output has the same length as the input. `filter(pred, xs)` selects elements; the output length is bounded by the input length.

0.38 Deep dive: module-level state, imports, closures, and the class/object theory

Real Python programs are organized into modules, use imports extensively, define classes with mutable state, and exploit closures and nested functions. The sheaf-descent framework must handle these organizational structures to scale beyond single-function analysis.

0.38.1 Module-level state and import semantics

Python modules are objects with attributes. Module-level statements execute sequentially when the module is first imported. Key considerations:

Module-level configuration. Many libraries use module-level state for configuration:

```
import torch
torch.set_default_dtype(torch.float64)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

The system should model module-level assignments as establishing global state that subsequent functions can depend on. The device and dtype settings should flow into the sections of functions that use the default device or dtype.

Import graph and dependency ordering. Circular imports and conditional imports create challenges:

- Forward references (`from __future__ import annotations`) defer type evaluation.
- Conditional imports (`try: import numpy; HAS_NP = True except: HAS_NP = False`) create configuration-dependent code paths.
- Lazy imports (inside function bodies) defer module loading.

The system should treat the import graph as a dependency structure for theory-pack activation: importing `torch` activates the PyTorch theory pack; importing `pandas` activates the Pandas pack.

Module attributes and monkey-patching. Python allows setting arbitrary attributes on modules: `module.x = 42`. The system should model module-level attributes as a record type with the keys established by module-level assignments and `__all__` declarations.

0.38.2 Class definitions and instance semantics

Classes are the primary organization unit for stateful Python code. The system should handle several aspects:

Class body as a namespace. The class body defines attributes, methods, class variables, and nested classes. The system should extract:

- Instance attributes assigned in `__init__`.
- Class variables assigned at class scope.
- Methods with their signatures.
- Properties (`@property`) with their getter/setter contracts.
- Special methods (`__len__`, `__getitem__`, `__iter__`, etc.) that participate in protocol dispatch.

Constructor analysis. `__init__` establishes the initial state of an instance. The system should analyze `__init__` to determine:

- Which instance attributes are always set (and their types).
- Which attributes are conditionally set.
- Parameter requirements for construction.

This produces an initial-state section for the class that describes the guaranteed state after construction.

Method semantics. Each method operates on `self`, which carries the instance's current state section. The method's local analysis should use the instance state as context:

```
class Buffer:
    def __init__(self, capacity: int):
        self.capacity = capacity
        self.items: list = []

    def add(self, item):
        if len(self.items) >= self.capacity:
```

```
            raise OverflowError("full")
        self.items.append(item)
```

The `add` method has a precondition (len(self.items) < self.capacity), and a postcondition (the item is added and the length increases by one). The system should synthesize both.

Inheritance. Subclass methods may override parent methods. The system should verify that overrides satisfy the Liskov substitution principle: the subclass method's precondition should be no stronger than the parent's, and its postcondition should be no weaker.

Multiple inheritance and MRO. Python's method resolution order (MRO) determines which method is called in diamond-inheritance situations. The system should follow the MRO to determine which method definition is active at a given call site.

0.38.3 Closures and nested functions

Closures capture variables from enclosing scopes:

```
def make_multiplier(factor):
    def multiply(x):
        return x * factor
    return multiply
```

The inner function `multiply` captures `factor` from the enclosing scope. The system should:

- Track which variables are captured by closures.
- Include captured-variable refinements in the closure's section.
- Propagate the enclosing scope's state into the closure's analysis.

Mutable closures. When a closure captures a mutable variable (via `nonlocal`), the variable may be modified by both the enclosing scope and the closure. The system should create shared-state sites for such variables and track mutations through both scopes.

Closures as callbacks. A common pattern is passing closures as callbacks to higher-order functions:

```
def train(model, data, on_step=None):
    for batch in data:
        loss = model(batch)
        if on_step is not None:
            on_step(loss)
```

The system should propagate the type requirements on `on_step` (accepts a single argument of the same type as `loss`) into the callback's section.

0.38.4 Dataclasses and attrs

Dataclasses (`@dataclass`) and attrs (`@attrs`) generate boilerplate methods automatically. The system should:

- Recognize dataclass field declarations and extract the field schema.
- Model the auto-generated `__init__`, `__eq__`, `__repr__`, and `__hash__` methods.
- Handle `field(default=...)` and `field(default_factory=...)` for optional fields.

- Handle frozen dataclasses (`frozen=True`) as immutable records.
- Handle validators in attrs (`@x.validator`) as preconditions on field values.

0.38.5 Metaclasses and dynamic class creation

Metaclasses (`type(...)` or custom metaclasses) create classes dynamically. The system should handle common patterns:

- `type("Name", (Base,), {"method": lambda self: ...})` creates a class with known attributes.
- `__init_subclass__` hooks modify subclass behavior at definition time.
- Abstract base classes (`abc.ABC`) with `@abstractmethod` require subclasses to implement certain methods; instantiating an abstract class should produce an error site.

For highly dynamic metaclass usage, the system should generate residuals rather than attempting full static analysis. The key principle is graceful degradation: the system should analyze what it can and surface what it cannot.

0.38.6 Global state and singleton patterns

Global mutable state is common in Python (module-level dicts, registries, caches). The system should:

- Track which functions read or write global state.
- Create shared-state sites for globals that are accessed from multiple functions.
- Recognize singleton patterns (`_instance = None` with lazy initialization) and model the initialization state transition.
- Warn when global mutation creates implicit dependencies between otherwise independent functions.

0.39 Deep dive: union types, type narrowing, pattern matching, and the complete type algebra

Python's type system (as codified by PEP 484 and successors) includes union types, literal types, type guards, and increasingly powerful narrowing mechanisms. The sheaf-descent framework should subsume and extend these features.

0.39.1 Union types as disjunctive sections

A value of type `int | str` (or `Union[int, str]`) may be either an integer or a string. In the sheaf-descent framework, a union type corresponds to a disjunctive section: the value's stalk at a given site includes both alternatives until a guard narrows it.

Narrowing via isinstance.

```
def process(x: int | str) -> str:
    if isinstance(x, int):
        return str(x * 2) # x: int in this branch
    else:
        return x.upper() # x: str in this branch
```

The `isinstance` check creates a guard site that splits the section: the true branch restricts to `int`, the false branch to `str`. The system should automatically perform this narrowing for all union-typed variables.

Narrowing via type guards (PEP 647).

```
from typing import TypeGuard

def is_str_list(lst: list) -> TypeGuard[list[str]]:
    return all(isinstance(x, str) for x in lst)
```

When `is_str_list(xs)` returns `True`, the variable `xs` should be narrowed to `list[str]` in the true branch. The system should recognize `TypeGuard` annotations and use them as guard predicates.

Discriminated unions. A common pattern is using a discriminant field to select among alternatives:

```
@dataclass
class Circle:
    kind: Literal["circle"] = "circle"
    radius: float = 0.0

@dataclass
class Rectangle:
    kind: Literal["rect"] = "rect"
    width: float = 0.0
    height: float = 0.0

Shape = Circle | Rectangle
```

Checking `shape.kind == "circle"` should narrow `shape` to `Circle`. The system should recognize literal-typed discriminant fields and use them for narrowing.

0.39.2 Literal types and constant propagation

`Literal["read", "write"]` restricts a value to specific constants. The system should track literal types through assignments and function calls:

- `mode: Literal["r", "w"]` restricts the parameter to two values.
- Passing a literal string `"r"` satisfies the constraint.
- Conditional checks on literal-typed values narrow the section.
- `Literal[True]` and `Literal[False]` enable boolean narrowing.

0.39.3 TypeVar, Generic, and parametric polymorphism

Generic types use `TypeVar` to express parametric relationships:

```
T = TypeVar("T")

def first(xs: list[T]) -> T:
    return xs[0]
```

The system should instantiate `T` at each call site based on the actual argument type and propagate the instantiated return type. Bounded type variables (`TypeVar("T", bound=Comparable)`) should additionally check that the bound is satisfied.

ParamSpec and Callable types. `ParamSpec` (PEP 612) captures the parameter signature of a callable:

```
P = ParamSpec("P")
R = TypeVar("R")

def decorator(f: Callable[P, R]) -> Callable[P, R]:
    def wrapper(*args: P.args, **kwargs: P.kwargs) -> R:
        return f(*args, **kwargs)
    return wrapper
```

The system should propagate `P` through decorator application so that the wrapped function retains its original parameter types.

0.39.4 Overloaded functions

`@overload` (PEP 484) specifies different return types for different argument patterns:

```
@overload
def get(key: str, default: None = None) -> int | None: ...
@overload
def get(key: str, default: int) -> int: ...
```

The system should select the matching overload at each call site based on the argument types and use the corresponding return type.

0.39.5 NewType, TypeAlias, and nominal distinctions

`NewType` creates a distinct type at the type-checking level:

```
UserId = NewType("UserId", int)
```

A `UserId` is an `int` at runtime but should be treated as a distinct type by the analysis. This prevents accidentally using a raw integer where a `UserId` is expected. The system should respect these nominal distinctions.

0.39.6 Final, ClassVar, and read-only refinements

- `Final[int]` marks a variable as immutable after assignment. The system should create an error site for any subsequent assignment.
- `ClassVar[int]` marks an attribute as belonging to the class rather than instances. The system should track class-level vs. instance-level state separately.
- `ReadOnly` (PEP 705 for TypedDict) marks specific keys as read-only.

0.39.7 TypedDict and structural record types

`TypedDict` defines dicts with known keys and value types:

```
class Point(TypedDict):
    x: float
    y: float
```

The system should model TypedDicts as structural record types: accessing a declared key has the declared type; accessing an undeclared key produces an error site. `total=False` makes all keys optional; `Required` and `NotRequired` (PEP 655) fine-tune which keys are required.

0.39.8 Self type and recursive types

`Self` (PEP 673) refers to the current class in method return types:

```
class Builder:
    def set_name(self, name: str) -> Self:
        self.name = name
        return self
```

The system should resolve `Self` to the actual receiver type at each call site, enabling fluent-API type tracking through method chains.

Recursive types (e.g., `Tree = int | list[Tree]`) require the system to handle fixed-point type definitions. The sheaf-descent framework handles this naturally: the recursion creates a recursive site structure, and sections at each depth are consistent with the unfolding.

0.39.9 Intersection types and protocol conjunction

While Python's type system does not have explicit intersection types, the system should internally represent "implements both Protocol A and Protocol B" as a conjunction. This arises when a variable is narrowed by multiple `isinstance` checks or when a function parameter requires multiple protocol implementations.

0.39.10 Type algebra summary

The complete type algebra supported by the system should include:

- Base types: `int`, `float`, `str`, `bool`, `bytes`, `None`.
- Container types: `list[T]`, `dict[K, V]`, `set[T]`, `tuple[T, ...]`, `tuple[T1, T2]`.
- Union types: `T1 | T2`.
- Optional types: `T | None`.
- Literal types: `Literal["a", "b"]`, `Literal[1, 2]`.
- Generic types: `Generic[T]`, bounded type variables.
- Callable types: `Callable[[T1, T2], R]`, `ParamSpec`.
- Protocol types: structural subtyping.
- TypedDict types: structural record types.
- NewType: nominal distinctions.
- Refinement types: $\{x : T \mid \phi(x)\}$ where ϕ is in a decidable fragment.
- Dependent types: $\Pi(x : A).B(x)$ where B may depend on the value of x.
- Section types: the full sheaf-descent section structure.

This hierarchy subsumes Python's standard type annotations while extending them with refinement and dependent types that capture the richer semantic information the system infers.

0.40 Deep dive: end-to-end worked example of sheaf-descent analysis

This final deep-dive section walks through a complete sheaf-descent analysis of a realistic Python program that uses multiple libraries, control flow, error handling, and dependent-type-level reasoning. The goal is to show how all the pieces fit together.

0.40.1 The target program

Consider a function that loads data, trains a model, and returns metrics:

```
import torch
import torch.nn as nn
import numpy as np
from typing import Optional

def train_classifier(
    features: torch.Tensor,
    labels: torch.Tensor,
    hidden_dim: int = 128,
    epochs: int = 10,
    lr: float = 0.01,
    device: Optional[str] = None,
) -> dict:
    if device is None:
        device = "cuda" if torch.cuda.is_available() else "cpu"

    assert features.dim() == 2, "features must be 2D"
    assert labels.dim() == 1, "labels must be 1D"
    assert features.shape[0] == labels.shape[0], "batch mismatch"
    assert hidden_dim > 0, "hidden_dim must be positive"
    assert epochs > 0, "epochs must be positive"
    assert lr > 0, "lr must be positive"

    n_samples, input_dim = features.shape
    n_classes = int(labels.max().item()) + 1

    model = nn.Sequential(
        nn.Linear(input_dim, hidden_dim),
        nn.ReLU(),
        nn.Linear(hidden_dim, n_classes),
    )
    model = model.to(device)
    features = features.to(device)
    labels = labels.to(device)

    optimizer = torch.optim.SGD(model.parameters(), lr=lr)
    criterion = nn.CrossEntropyLoss()

    losses = []
    for epoch in range(epochs):
        logits = model(features)
        loss = criterion(logits, labels)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        losses.append(loss.item())

    with torch.no_grad():
        final_logits = model(features)
        preds = final_logits.argmax(dim=1)
        accuracy = (preds == labels).float().mean().item()
```

```
    return {
        "accuracy": accuracy,
        "final_loss": losses[-1],
        "losses": losses,
        "n_samples": n_samples,
        "n_classes": n_classes,
    }
```

0.40.2 Site cover construction

The system constructs the following site families:

Guard sites from assertions. Each `assert` statement creates a guard site. After the assertion block, the section carries:

- $\text{features.dim}() = 2$
- $\text{labels.dim}() = 1$
- $\text{features.shape}[0] = \text{labels.shape}[0]$
- $\text{hidden_dim} > 0$
- $\text{epochs} > 0$
- $\text{lr} > 0$

None-narrowing site. The `if device is None` check creates a narrowing site. In the true branch, `device` is assigned a string value; in the false branch, the input `device` is used. After the merge, `device` is known to be a non-None string.

Shape sites. `features.shape` creates a shape site with value $(\text{n_samples}, \text{input_dim})$. `labels.shape` creates a shape site with value $(\text{labels.shape}[0],)$. The destructuring `n_samples, input_dim = features.shape` binds symbolic names to shape components.

PyTorch theory sites.

- `nn.Linear(input_dim, hidden_dim)`: requires $\text{input_dim} > 0$ and $\text{hidden_dim} > 0$; output shape is $(\text{batch}, \text{hidden_dim})$.
- `nn.Linear(hidden_dim, n_classes)`: requires $\text{hidden_dim} > 0$ and $\text{n_classes} > 0$; output shape is $(\text{batch}, \text{n_classes})$.
- `model(features)`: the Sequential applies layers in order; the system composes shape transformations to verify shape compatibility.
- `criterion(logits, labels)`: CrossEntropyLoss requires $\text{logits.shape}[0] = \text{labels.shape}[0]$ and $\text{logits.shape}[1] = \text{n_classes}$.
- `final_logits.argmax(dim=1)`: output shape is $(\text{n_samples},)$; values in $[0, \text{n_classes})$.

Numeric safety sites.

- `labels.max().item() + 1`: requires the labels tensor to be nonempty (otherwise `max()` raises `RuntimeError`).
- `losses[-1]`: requires the losses list to be nonempty, which is guaranteed when $\text{epochs} > 0$.
- `lr > 0`: prevents degenerate optimization.

0.40.3 Backward synthesis: input requirements

The backward pass pulls back all viability predicates to the function boundary, synthesizing the weakest precondition. The result:

```
@deppy.requires(
    features.dim() == 2,
    labels.dim() == 1,
    features.shape[0] == labels.shape[0],
    features.shape[0] > 0, # nonempty for labels.max()
    labels.max() >= 0, # n_classes >= 1
    hidden_dim > 0,
    epochs > 0,
    lr > 0,
    device is None or isinstance(device, str),
)
```

0.40.4 Forward synthesis: output guarantees

The forward pass computes the strongest postcondition. The synthesized output section:

```
@deppy.ensures(
    lambda result: isinstance(result, dict),
    lambda result: set(result.keys()) == {
        "accuracy", "final_loss", "losses", "n_samples", "n_classes"
    },
    lambda result: 0.0 <= result["accuracy"] <= 1.0,
    lambda result: isinstance(result["final_loss"], float),
    lambda result: len(result["losses"]) == epochs,
    lambda result: result["n_samples"] == features.shape[0],
    lambda result: result["n_classes"] == int(labels.max().item()) + 1,
)
```

0.40.5 Runtime-error freedom certificate

Given the synthesized preconditions, the system can certify that the function is free of the following runtime errors:

- No `AssertionError`: all assertions are implied by the preconditions.
- No `RuntimeError` from empty tensor reduction: features.shape[0] > 0 guarantees nonempty tensors.
- No `IndexError` from `losses[-1]`: epochs > 0 guarantees the list is nonempty after the loop.
- No shape mismatch in `nn.Linear` or `CrossEntropyLoss`: the shape sites are consistent with the assertions.

0.40.6 Residual obligations

Some properties cannot be verified statically:

- Whether the device string is valid (e.g., `"cuda"` when CUDA is not available). This depends on runtime hardware.
- Whether the model converges (the loss decreases over epochs). This is a dynamic property that depends on data and hyperparameters.

- Whether the numeric values remain finite throughout training. The system can verify that no obvious division-by-zero or log-of-zero occurs, but cannot rule out all numeric instability.

These are surfaced as explicit residuals with explanations, allowing the user to decide whether to add runtime checks, provide proofs, or accept the risk.

0.40.7 Interprocedural context

If `train_classifier` is called from another function, the caller inherits the preconditions as obligations:

```
def run_experiment(data_path: str) -> dict:
    data = np.load(data_path)
    features = torch.from_numpy(data["X"])
    labels = torch.from_numpy(data["y"])
    return train_classifier(features, labels, epochs=20)
```

The system pulls back the preconditions to `run_experiment`'s boundary: the loaded data must have keys `"X"` and `"y"`; `data["X"]` must be 2D; `data["y"]` must be 1D; their first dimensions must match; the data must be nonempty. These become requirements on the `.npz` file, surfaced as either annotations or diagnostics.

This end-to-end example demonstrates the full sheaf-descent pipeline: from source code through site construction, bidirectional synthesis, error-freedom certification, residual surfacing, and interprocedural composition.

Part I

Full Dependent Typing and Proof Mode

Chapter 1

The Proof-Mode Philosophy: From Inference to Verification

The preceding parts of this monograph describe a system that *infers* rich dependent types with minimal annotation. The system harvests guards, propagates sections, synthesizes preconditions and postconditions, and certifies runtime-error freedom—all without requiring the programmer to write a single type annotation beyond what Python already supports.

This part describes the *proof mode*: a direct, seamless extension of the inference engine that allows programmers to *selectively* escalate from inference to verification. The key design principle is:

> **The proof mode is not a separate system.** It is the same sheaf-descent engine with the same site covers, the same sections, and the same solver backend. The only difference is that the programmer provides additional annotations—preconditions, postconditions, invariants, lemmas, or full proofs—at chosen sites, and the engine uses these annotations as *axioms* that enrich the local sections. Everything else continues to be inferred.

This design produces a unique value proposition: the programmer gets useful static checking for *all* of their code (via inference), and they can invest annotation effort at any granularity, from a single assertion to a full Agda-style proof, and the investment pays off immediately by enriching the inferred sections of surrounding code.

1.1 The annotation economy: cherry-picking depth

In traditional proof assistants (Agda, Coq, Lean, F*), every function must be fully annotated and every proof must be complete. This creates an all-or-nothing barrier: either you commit to proving everything, or you prove nothing. The result is that proof assistants are used only for specialized, high-assurance code.

Our system inverts this economics. The programmer can operate at any point on a continuous spectrum:

Level 0: Zero annotation. The programmer writes plain Python. The system infers whatever it can: shapes, bounds, None-safety, basic type properties. This is the default mode described in Parts I–III.

Level 1: Lightweight contracts. The programmer adds `@requires` and `@ensures` decorators to selected functions. These are checked by the solver and used to enrich the sections of callers and callees. No proofs are needed; the solver discharges obligations automatically when they fall within decidable fragments.

Level 2: Invariant annotations. The programmer annotates loop invariants, class invariants, or module-level invariants. These are checked inductively (base case + preservation) and used to strengthen the sections of code that depends on the invariant.

Level 3: Lemma statements. The programmer states lemmas—properties they believe to be true—and the system attempts to prove them automatically. If the solver succeeds, the lemma becomes an axiom available to surrounding code. If the solver fails, the lemma becomes a proof obligation.

Level 4: Proof sketches. The programmer provides a proof sketch: a sequence of intermediate assertions that guide the solver. The system fills in the gaps between assertions automatically.

Level 5: Full proofs. The programmer writes complete, step-by-step proofs in an Agda-like syntax embedded in Python decorators or comments. Every step is verified.

The crucial property is that all five levels coexist in the same codebase. A function at Level 0 can call a function at Level 5, and vice versa. The system mediates between levels through the section mechanism: a fully-proved function exports a maximally-rich section, which enriches the inferred sections of its Level-0 callers.

1.2 Why this is not a separate system

Consider the alternative: building a separate proof language (like F* or Dafny) and then connecting it to Python via an FFI or a translation layer. This approach has fundamental problems:

1. **Semantic gap.** The proof language has its own semantics, which must be related to Python's semantics by a trusted translation. Any discrepancy between the two is a soundness hole.
2. **Duplication.** The programmer must maintain two representations of the same code: the Python implementation and the proof-language specification. Changes to one must be reflected in the other.
3. **Cliff.** There is no smooth transition from "Python with types" to "proven Python." The programmer must cross a cliff from one tool to another.
4. **Inference loss.** The proof language does not benefit from the inference engine's automatic section synthesis. The programmer must re-specify everything the engine already knows.

Our approach avoids all four problems. The proof mode operates on the same AST, the same SSA representation, the same site covers, and the same solver backend as the inference mode. Annotations are additional constraints in the same constraint language. Proofs are additional evidence in the same section structure. There is no translation, no duplication, and no cliff.

1.3 The annotation surface: Python-native proof syntax

Proofs and specifications are written in Python syntax using decorators, type annotations, and a small library of proof combinators. The design principle is that every proof construct should be valid Python that can be executed (for testing) or ignored (for production).

1.3.1 Preconditions and postconditions

```
from deppy import requires, ensures, returns

@requires(lambda n: n >= 0)
@ensures(lambda n, result: result >= 1)
@ensures(lambda n, result: n <= 20 or returns.unchecked())
def factorial(n: int) -> int:
    if n == 0:
        return 1
    return n * factorial(n - 1)
```

The `@requires` decorator specifies a precondition; `@ensures` specifies a postcondition. These are lambda expressions over the function's parameters and (for `@ensures`) the return value. The system checks:

- The precondition is satisfiable (not vacuously false).
- The postcondition holds on all execution paths that satisfy the precondition.
- Callers satisfy the precondition at every call site.

The `returns.unchecked()` escape hatch allows the programmer to leave some postcondition aspects unverified (here, the overflow behavior for $n > 20$).

1.3.2 Dependent function types

Python's type annotations can express dependent types when combined with `deppy.Refine`:

```
from deppy import Refine

def safe_divide(
    a: float,
    b: Refine[float, lambda x: x != 0],
) -> float:
    return a / b
```

`Refine[T, pred]` is a refinement type: the type $\{x : T \mid \text{pred}(x)\}$. At call sites, the system checks that the actual argument satisfies the refinement predicate. If the predicate cannot be verified statically, the system can optionally insert a runtime check.

1.3.3 Quantified properties

For properties that quantify over collections, the system provides `forall` and `exists`:

```
from deppy import ensures, forall

@ensures(lambda xs, result:
    forall(range(len(result) - 1),
           lambda i: result[i] <= result[i + 1]))
def sort(xs: list[int]) -> list[int]:
    return sorted(xs)
```

The `forall` combinator expresses a universally-quantified property. The solver can often discharge such properties for built-in operations (like `sorted`) using library-specific axioms. For user-defined loops, the programmer may need to provide an inductive argument.

1.3.4 Proof blocks

For complex properties that require multi-step reasoning, the system provides `proof` blocks:

```
from deppy import proof, have, hence, qed

@ensures(lambda xs, result: len(result) == len(xs))
@ensures(lambda xs, result:
    forall(range(len(xs)), lambda i: result[i] == xs[i] ** 2))
def square_all(xs: list[int]) -> list[int]:
    result = []
    for i, x in enumerate(xs):
        result.append(x ** 2)
        with proof("loop_invariant"):
            have(len(result) == i + 1)
            have(forall(range(i + 1),
                        lambda j: result[j] == xs[j] ** 2))
    with proof("postcondition"):
        have(len(result) == len(xs))
        hence(forall(range(len(xs)),
                     lambda j: result[j] == xs[j] ** 2))
        qed()
    return result
```

The `with proof(...)` block is a context manager that introduces a proof scope. Inside the scope, `have(...)` asserts an intermediate fact, `hence(...)` derives a consequence, and `qed()` closes the proof. The system checks that each `have` follows from previous facts and the program state, and that the final `hence` establishes the desired property.

At runtime, proof blocks are no-ops (they execute but do nothing). At analysis time, they provide the solver with a proof skeleton that guides verification.

1.4 The trust gradient: how annotations flow through inference

The most novel aspect of proof mode is how annotations interact with inference. When a programmer annotates one function with a rich contract, the benefit propagates to all code that interacts with that function:

Forward propagation. A function with a strong postcondition exports a rich section. Any caller that receives the function's result inherits the postcondition as a known fact, which may allow the caller's own postcondition to be stronger—even if the caller has no annotations at all.

Backward propagation. A function with a strong precondition creates obligations at call sites. If a caller cannot satisfy the precondition from its own inferred section, the system propagates the requirement further back, potentially all the way to the module boundary, where it becomes an input requirement.

Lateral propagation. A lemma proved about a data structure enriches the sections of all code that uses that data structure. For example, proving that a custom list maintains a sortedness invariant allows binary-search code elsewhere in the module to be verified without additional annotation.

The result is that a small amount of annotation effort can have an outsized impact on the quality of inference across the entire codebase. This is the *annotation leverage* property.

Chapter 2

Dependent Types in Python: The Full Theory

This chapter develops the full dependent type theory that underpins proof mode. The theory is designed to be (a) expressive enough to state and verify real correctness properties, (b) compatible with Python's dynamic semantics, and (c) decidable in common cases while allowing controlled escape to interactive proof for hard cases.

2.1 The type universe

The type universe $\mathcal{U}$ is stratified into levels:

$$
\begin{aligned}
\mathcal{U}_0 &= \{\texttt{int}, \texttt{float}, \texttt{str}, \texttt{bool}, \texttt{bytes}, \texttt{None}\} \\
\mathcal{U}_1 &= \mathcal{U}_0 \cup \{\texttt{list}[A], \texttt{dict}[A, B], \texttt{set}[A], \texttt{tuple}[A_1, \ldots, A_n] \mid A, B, A_i \in \mathcal{U}_0\} \\
\mathcal{U}_2 &= \mathcal{U}_1 \cup \{A \to B, \Pi(x : A).B(x), \Sigma(x : A).B(x) \mid A \in \mathcal{U}_1, B \in \mathcal{U}_1\} \\
\mathcal{U}_3 &= \mathcal{U}_2 \cup \{\{x : A \mid \phi(x)\} \mid A \in \mathcal{U}_2, \phi \in \mathcal{L}_{\text{dec}}\} \\
\mathcal{U}_\omega &= \bigcup_n \mathcal{U}_n
\end{aligned}
$$

where $\mathcal{L}_{\text{dec}}$ is the decidable predicate language (quantifier-free linear arithmetic, array theory, and uninterpreted functions—the QF_LIRA + arrays fragment of SMT-LIB).

Refinement types. $\{x : A \mid \phi(x)\}$ is the type of values of type A that satisfy predicate ϕ. This is the workhorse of the system: most dependent types are expressed as refinements.

Dependent function types. $\Pi(x : A).B(x)$ is the type of functions that take an argument x of type A and return a value of type $B(x)$, where the return type may depend on the argument value. In Python:

```
def lookup(xs: list[int], i: Refine[int, lambda i: 0 <= i < len(xs)]
          ) -> int:
    return xs[i]
```

The return type is simply `int`, but the system infers the stronger section: the result equals `xs[i]`, which depends on `i`.

Dependent pair types. $\Sigma(x : A).B(x)$ is the type of pairs (a, b) where $a : A$ and $b : B(a)$. This arises naturally when a function returns a value together with a proof of some property:

```
def find_min(xs: Refine[list[int], lambda l: len(l) > 0]
            ) -> tuple[int, int]:
    idx = 0
    for i in range(1, len(xs)):
        if xs[i] < xs[idx]:
            idx = i
    return idx, xs[idx]
    # inferred section: result[0] in range(len(xs))
    # result[1] == xs[result[0]]
    # forall j in range(len(xs)): result[1] <= xs[j]
```

2.2 Propositions as types, proofs as programs

Following the Curry-Howard correspondence, the system treats propositions as types and proofs as programs. This means that proving a property is the same as constructing a value of the corresponding type.

The proposition type. A proposition P is a type in $\mathcal{U}_\omega$ that is either inhabited (the proposition is true) or empty (the proposition is false). The system uses the notation `Prop[P]` for proposition types.

Proof terms. A proof of P is a value of type `Prop[P]`. The system provides several ways to construct proof terms:

- **Solver discharge.** If P is in the decidable fragment, the solver can construct the proof automatically.
- **Refl.** If P is an equality $a = b$ and a and b are definitionally equal, the proof is `refl`.
- **Assumption.** If P is in the current context (a precondition, a guard, or a previously-proved fact), the proof is the assumption itself.
- **Application.** If $f : P \to Q$ and $p : P$, then $f(p) : Q$.
- **Induction.** For properties of natural numbers or lists, the system supports inductive proofs.

The proof is erased at runtime. Proof terms exist only at analysis time. At runtime, they are erased to `None`. This means that proofs have zero runtime cost.

2.3 The decidable core and the proof frontier

Not every proposition can be decided automatically. The system maintains a clear boundary between what the solver can handle and what requires human guidance:

The decidable core. The solver can automatically discharge propositions in the following fragments:

- Quantifier-free linear integer arithmetic (QF_LIA).
- Quantifier-free linear real arithmetic (QF_LRA).
- Quantifier-free arrays with extensionality (QF_AX).
- Quantifier-free bitvectors (QF_BV).

- Quantifier-free uninterpreted functions (QF_UF).
- Boolean combinations of the above.
- Bounded quantification: $\forall i \in [0, n).\ P(i)$ where n is a known constant or a symbolically-bounded expression.

The proof frontier. When a proposition falls outside the decidable core, the system cannot discharge it automatically. Instead, it reports the proposition as a *proof obligation*—a residual that the programmer must address. The programmer can:

1. Provide a proof (Levels 4–5).
2. Provide intermediate lemmas that reduce the obligation to decidable sub-goals (Level 3).
3. Accept the obligation as a trusted axiom (`@assume`).
4. Downgrade to a runtime check (`@check`).

This design ensures that the system never silently gives up. Every unverified property is either proved, assumed, or checked at runtime.

2.4 Inductive types and recursive proofs

Many interesting properties require induction. The system supports inductive reasoning for:

Natural number induction. For a property $P(n)$ where $n : \texttt{int}$ and $n \geq 0$:

```
from deppy import induction, base_case, inductive_step

@ensures(lambda n, result: result == n * (n + 1) // 2)
def triangle(n: Refine[int, lambda n: n >= 0]) -> int:
    total = 0
    for i in range(n + 1):
        total += i
    with induction(n):
        with base_case(n == 0):
            have(total == 0 == 0 * 1 // 2)
        with inductive_step(n > 0):
            assume(triangle(n - 1) == (n - 1) * n // 2)
            have(total == triangle(n - 1) + n)
            hence(total == n * (n + 1) // 2)
    return total
```

Structural induction on lists. For a property $P(\text{xs})$ where $\text{xs} : \texttt{list}[A]$:

- Base case: $P([])$.
- Inductive step: for all $x : A$ and $\text{xs}' : \texttt{list}[A]$, if $P(\text{xs}')$ then $P([x] + \text{xs}')$.

Well-founded induction. For recursive functions that do not follow simple structural patterns, the programmer can specify a well-founded measure:

```
from deppy import decreases

@decreases(lambda n: n) # measure function
@ensures(lambda n, result: result >= 1)
def factorial(n: Refine[int, lambda n: n >= 0]) -> int:
    if n == 0:
```

```
        return 1
    return n * factorial(n - 1)
```

The `@decreases` decorator specifies a measure that strictly decreases at each recursive call. The system verifies that the measure is non-negative and decreasing, which ensures termination.

Chapter 3

Cherry-Picked Annotation: The Selective Verification Workflow

The power of proof mode lies not in forcing the programmer to prove everything, but in allowing them to prove *exactly what matters* while the inference engine handles the rest. This chapter describes the practical workflow for selective verification.

3.1 Identifying verification targets

Not all code deserves the same level of assurance. The programmer should invest proof effort where:

- **Correctness is critical**: financial calculations, security checks, safety-critical control.
- **Bugs are expensive**: core algorithms, data structure invariants, protocol implementations.
- **Testing is insufficient**: corner cases, concurrency, numeric precision.
- **The property is non-obvious**: algorithmic invariants, complex shape transformations, protocol compliance.

The system helps identify targets by reporting the *inferred section quality* at each site. Sites where the inferred section is weak (few known properties) are candidates for annotation investment.

3.2 The annotation workflow

3.2.1 Step 1: Run inference on unannotated code

The programmer runs the system on their existing Python code with zero annotations. The system produces:

- Inferred preconditions and postconditions for each function.
- Detected potential runtime errors with their viability predicates.
- A list of residual obligations that could not be discharged.
- A section-quality map showing where inference is strong and where it is weak.

3.2.2 Step 2: Triage residuals

The programmer reviews the residual obligations and decides for each one:

- **Accept.** The residual is not important; mark it as accepted.
- **Check.** The residual should be checked at runtime; the system inserts a runtime assertion.
- **Prove.** The residual is important and should be proved; the programmer adds annotations.

- **Strengthen.** The inferred section is too weak; the programmer adds a contract to strengthen it.

3.2.3 Step 3: Add targeted annotations

The programmer adds annotations *only* at the sites identified in Step 2. The system re-runs inference with the new annotations, which may discharge some residuals and strengthen some sections.

3.2.4 Step 4: Iterate

The programmer repeats Steps 2–3 until the remaining residuals are all accepted or checked. There is no requirement to reach zero residuals; the programmer chooses their own stopping point.

3.3 Example: selective verification of a binary search

Consider a binary search implementation. At Level 0 (no annotations), the system infers basic properties: the function takes a sorted list and a target, accesses list elements within bounds, and returns an integer. But it cannot verify that the returned index is correct.

```
def binary_search(xs: list[int], target: int) -> int:
    lo, hi = 0, len(xs)
    while lo < hi:
        mid = (lo + hi) // 2
        if xs[mid] < target:
            lo = mid + 1
        else:
            hi = mid
    return lo
```

Level 0 inference. The system infers:

- Input: `xs` is a list of int; `target` is int.
- The loop accesses `xs[mid]` where $0 \leq$ `mid` $<$ `len(xs)`—inferred safe when `len(xs)` > 0 and $0 \leq$ `lo` $\leq$ `hi` $\leq$ `len(xs)`.
- Output: `lo` is an int in $[0, \texttt{len(xs)}]$.
- Residual: cannot verify that the returned index is the correct insertion point.

Level 1: add contracts. The programmer adds preconditions and postconditions:

```
from deppy import requires, ensures, forall, Refine

@requires(lambda xs: forall(range(len(xs) - 1),
                            lambda i: xs[i] <= xs[i + 1]))
@ensures(lambda xs, target, result:
    0 <= result <= len(xs))
@ensures(lambda xs, target, result:
    forall(range(result), lambda i: xs[i] < target))
@ensures(lambda xs, target, result:
    forall(range(result, len(xs)), lambda i: xs[i] >= target))
def binary_search(xs: list[int], target: int) -> int:
    ...
```

Level 2: add the loop invariant. The postconditions require proving a loop invariant. The programmer annotates the loop:

```
while lo < hi:
    with invariant():
        have(0 <= lo <= hi <= len(xs))
        have(forall(range(lo), lambda i: xs[i] < target))
        have(forall(range(hi, len(xs)),
                    lambda i: xs[i] >= target))
    mid = (lo + hi) // 2
    if xs[mid] < target:
        lo = mid + 1
    else:
        hi = mid
```

The system verifies that:

1. The invariant holds on loop entry (from the initialization and the sorted-input precondition).
2. The invariant is preserved by each iteration.
3. The invariant plus the loop-exit condition (`lo` $\geq$ `hi`) implies the postconditions.

All three checks reduce to decidable linear arithmetic over bounded quantifiers, so the solver handles them automatically. The programmer needed to provide only the loop invariant; everything else was inferred or checked by the solver.

3.4 Cost-benefit analysis of annotation depth

The following table summarizes the annotation cost and verification benefit at each level, using binary search as the running example:

Level	Annotation cost	Verification benefit	What is checked
0	0 lines	Bounds safety	Array accesses in bounds, integer ranges
1	6 lines	Functional contract	Pre/postconditions stated; callers checked
2	5 lines	Full correctness	Loop invariant verified; postcondition proved
3–5	N/A	N/A	Not needed for this example

The key observation is that 11 lines of annotation (Levels 1+2) suffice to prove full functional correctness of binary search, while the inference engine handles all the mechanical work: bounds checking, variable tracking, invariant checking against the solver.

Chapter 4

The Proof Term Language: Agda-Style Reasoning in Python

This chapter defines the complete proof term language available in proof mode. The language is designed to be embeddable in Python (all constructs are valid Python expressions or statements) while supporting the full power of constructive dependent type theory.

4.1 Proof terms as Python expressions

Every proof term is a Python expression that, at analysis time, carries a type (the proposition being proved) and, at runtime, evaluates to `None` (proofs are erased). The proof term language is defined by the following grammar:

1. **Reflexivity.** `refl()` proves $a = a$ for any expression a. The system checks that both sides of the equality are definitionally equal after normalization.
2. **Symmetry.** `symm(p)` takes a proof $p : a = b$ and produces a proof of $b = a$.
3. **Transitivity.** `trans(p, q)` takes proofs $p : a = b$ and $q : b = c$ and produces a proof of $a = c$.
4. **Congruence.** `cong(f, p)` takes a function f and a proof $p : a = b$ and produces a proof of $f(a) = f(b)$.
5. **Substitution.** `subst(p, q)` takes a proof $p : a = b$ and a proof $q : P(a)$ and produces a proof of $P(b)$.
6. **Modus ponens.** `mp(p, q)` takes a proof $p : A \Rightarrow B$ and a proof $q : A$ and produces a proof of B.
7. **Introduction.** `intro(lambda x: ...)` introduces a universally-quantified variable or a function argument.
8. **Elimination.** `elim(p, a)` applies a universally-quantified proof $p : \forall x.\ P(x)$ to a specific value a, producing $P(a)$.
9. **And-introduction.** `and_intro(p, q)` takes proofs $p : A$ and $q : B$ and produces a proof of $A \wedge B$.
10. **And-elimination.** `fst(p)` and `snd(p)` project from a conjunction proof.
11. **Or-introduction.** `inl(p)` and `inr(q)` inject into a disjunction.
12. **Or-elimination.** `cases(p, f, g)` takes a proof $p : A \vee B$, a function $f : A \to C$, and a function $g : B \to C$, and produces a proof of C.
13. **Absurdity.** `absurd(p)` takes a proof of $\perp$ (False) and produces a proof of anything.
14. **Solver.** `auto()` asks the SMT solver to discharge the current goal. If the solver succeeds, the proof is complete.

15. **Assumption.** `by_assumption(name)` references a named assumption in the current context.

4.2 Tactic-style proofs

In addition to direct proof terms, the system supports a tactic-style interface where the programmer manipulates a proof goal:

```
from deppy.proof import Proof

with Proof("sum_zero") as p:
    p.goal("forall(range(n), lambda i: 0) == 0 * n")
    p.induction("n")
    with p.case("n == 0"):
        p.simp() # simplifies to 0 == 0
    with p.case("n > 0"):
        p.rewrite("forall(range(n), lambda i: 0)",
                  "forall(range(n-1), lambda i: 0) + 0")
        p.apply_ih() # applies induction hypothesis
        p.simp() # simplifies arithmetic
```

The tactic interface provides:

- `p.goal(expr)`: sets the proof goal.
- `p.induction(var)`: initiates induction on a variable.
- `p.case(cond)`: introduces a case split.
- `p.simp()`: applies arithmetic and boolean simplification.
- `p.rewrite(lhs, rhs)`: rewrites an expression using a known equality.
- `p.apply_ih()`: applies the induction hypothesis.
- `p.apply(lemma)`: applies a previously-proved lemma.
- `p.auto()`: asks the solver to close the goal.
- `p.unfold(name)`: unfolds a definition.
- `p.split()`: splits a conjunction goal into two sub-goals.
- `p.left()` / `p.right()`: selects the left/right side of a disjunction goal.
- `p.intro(name)`: introduces a universally-quantified variable.
- `p.exact(term)`: provides an exact proof term.
- `p.contradiction()`: derives False from contradictory hypotheses.

4.3 Proof by calculation

For equational reasoning, the system supports calculation proofs:

```
from deppy.proof import calc

@ensures(lambda n, result: result == n * (n + 1) * (2 * n + 1) // 6)
def sum_squares(n: Refine[int, lambda n: n >= 0]) -> int:
    total = 0
    for i in range(n + 1):
        total += i * i
    with calc("sum_squares_formula"):
        step("sum(i**2 for i in range(n+1))")
        eq("sum(i**2 for i in range(n)) + n**2",
           by="definition of range")
```

```
        eq("(n-1)*n*(2*(n-1)+1)//6 + n**2",
           by="induction hypothesis")
        eq("(n-1)*n*(2*n-1)//6 + n**2",
           by="arithmetic")
        eq("((n-1)*n*(2*n-1) + 6*n**2) // 6",
           by="common denominator")
        eq("(2*n**3 - 3*n**2 + n + 6*n**2) // 6",
           by="expansion")
        eq("(2*n**3 + 3*n**2 + n) // 6",
           by="simplification")
        eq("n * (n+1) * (2*n+1) // 6",
           by="factoring")
    return total
```

Each `eq(..., by=...)` step asserts that the previous expression equals the new one, with a justification. The solver verifies each step independently, so the programmer only needs to provide enough intermediate steps for each gap to be decidable.

4.4 Lemma libraries and reuse

Proved lemmas are first-class objects that can be stored, imported, and applied:

```
from deppy import lemma, Prop

@lemma
def add_comm(a: int, b: int) -> Prop["a + b == b + a"]:
    return auto()

@lemma
def add_assoc(a: int, b: int, c: int
             ) -> Prop["(a + b) + c == a + (b + c)"]:
    return auto()

@lemma
def mul_dist(a: int, b: int, c: int
            ) -> Prop["a * (b + c) == a * b + a * c"]:
    return auto()
```

These lemmas are proved once and available for use in any proof via `p.apply(add_comm)` or `mp(add_comm(x, y), ...)`.

Library lemma bundles. The system ships with pre-proved lemma bundles for common domains:

- `deppy.lemmas.arithmetic`: commutativity, associativity, distributivity, division properties.
- `deppy.lemmas.lists`: length properties, append, concatenation, reversal, sorting.
- `deppy.lemmas.sets`: membership, union, intersection, subset.
- `deppy.lemmas.order`: transitivity, antisymmetry, total order.
- `deppy.lemmas.tensors`: shape algebra, broadcasting rules, dimension arithmetic.

4.5 Proof automation levels

The system provides multiple levels of proof automation:

Level A: Full automation (`auto()`). The solver attempts to discharge the goal using its decidable theories. This handles most arithmetic, boolean, and array-theory goals.

Level B: Simplification (`simp()`). The simplifier applies rewriting rules (commutativity, associativity, identity elements, beta reduction) to normalize the goal. This often reduces complex goals to forms that `auto()` can handle.

Level C: Heuristic search (`search()`). The system searches for a proof by trying combinations of available lemmas, hypotheses, and proof strategies. This is more expensive but can find proofs that `auto()` misses.

Level D: Interactive (`sorry()`). When all automation fails, `sorry()` marks the goal as unproved but accepted. This allows the programmer to continue developing surrounding proofs while deferring a difficult sub-goal. The system tracks all `sorry()` uses and reports them as obligations.

4.6 Proof-carrying code: embedding proofs in functions

Proofs can be embedded directly in function bodies, interleaved with computation:

```
from deppy import requires, ensures, have, hence, Refine

@requires(lambda xs: len(xs) > 0)
@ensures(lambda xs, result: result in xs)
@ensures(lambda xs, result:
    forall(range(len(xs)), lambda i: result <= xs[i]))
def find_minimum(xs: list[int]) -> int:
    minimum = xs[0]
    have("minimum == xs[0]")

    for i in range(1, len(xs)):
        with invariant():
            have("minimum in xs[:i]")
            have("forall(range(i), lambda j: minimum <= xs[j])")
        if xs[i] < minimum:
            minimum = xs[i]
            have("minimum == xs[i]")
            have("minimum in xs[:i+1]")
        else:
            have("minimum <= xs[i]")
            have("minimum in xs[:i+1]")

    hence("minimum in xs")
    hence("forall(range(len(xs)), lambda j: minimum <= xs[j])")
    return minimum
```

The `have` and `hence` statements are proof checkpoints. Each one is verified by the solver in the context of the current program state. Between checkpoints, the system fills in the logical steps automatically. The result is that the programmer provides a human-readable proof outline, and the system verifies that each step follows.

4.7 The interaction between proof terms and inference

When proof mode is active, every proved property immediately enriches the sheaf-descent sections:

- A proved postcondition becomes a guaranteed output section.
- A proved loop invariant becomes a fixed-point section at the loop header.
- A proved lemma becomes an available axiom for all sites that can reference its variables.
- A proved class invariant becomes a guaranteed property at every method entry and exit.

This enrichment propagates through the interprocedural transport mechanism. If function f has a proved postcondition and function g calls f, then g's local section at the call return site is enriched with f's postcondition—even if g has no annotations at all. This is the *annotation leverage* effect.

Chapter 5

Class Invariants and Object-Oriented Proofs

Object-oriented code organizes state into objects with methods that maintain invariants. The proof mode supports class-level invariants that are checked at construction and preserved by every method.

5.1 Class invariants as persistent sections

A class invariant is a property that holds for every instance of the class at every method boundary. In the sheaf-descent framework, the invariant is a persistent section attached to the class's site family.

```
from deppy import invariant_of, requires, ensures

@invariant_of(lambda self:
    0 <= self.size <= self.capacity and
    len(self.items) == self.size)
class BoundedBuffer:
    def __init__(self, capacity: Refine[int, lambda c: c > 0]):
        self.capacity = capacity
        self.items: list = []
        self.size = 0

    @requires(lambda self: self.size < self.capacity)
    @ensures(lambda self: self.size == old(self.size) + 1)
    def push(self, item):
        self.items.append(item)
        self.size += 1

    @requires(lambda self: self.size > 0)
    @ensures(lambda self, result:
        self.size == old(self.size) - 1)
    def pop(self):
        self.size -= 1
        return self.items.pop()

    @ensures(lambda self, result: result == (self.size == 0))
    def is_empty(self) -> bool:
        return self.size == 0
```

```
    @ensures(lambda self, result: result == (self.size == self.capacity))
    def is_full(self) -> bool:
        return self.size == self.capacity
```

The system verifies:

1. **Establishment.** The invariant holds after `__init__` completes.
2. **Preservation.** Every public method preserves the invariant: if the invariant holds on entry (plus the method's precondition), it holds on exit.
3. **Usage.** Every method can assume the invariant on entry, which enriches the local section. For example, `pop` knows that `self.size` > 0 (from the precondition) and that `len(self.items)` $=$ `self.size` (from the invariant), so the `self.items.pop()` call is safe.

5.2 The `old()` function for state change reasoning

Postconditions often need to relate the post-state to the pre-state. The `old(expr)` function captures the value of an expression at method entry:

- `old(self.size)`: the size before the method executed.
- `old(self.items)`: a snapshot of the items list before mutation.
- `old(len(xs))`: the length of `xs` before the method.

The system implements `old` by creating a shadow copy of the referenced expressions at method entry and substituting these copies in the postcondition. At analysis time, the pre-state values are symbolic; at runtime (for checked contracts), they are actual snapshots.

5.3 Inheritance and invariant strengthening

When a subclass extends a class with an invariant:

```
@invariant_of(lambda self:
    all(x >= 0 for x in self.items))
class NonNegativeBuffer(BoundedBuffer):
    @requires(lambda self, item: item >= 0)
    def push(self, item):
        super().push(item)
```

The subclass invariant is the *conjunction* of the parent invariant and the subclass-specific invariant. The system verifies that:

- The subclass `__init__` establishes both invariants.
- Overridden methods preserve both invariants.
- The Liskov substitution principle holds: the subclass's preconditions are no stronger than the parent's (for inherited methods), and the postconditions are no weaker.

5.4 Protocol proofs: verifying structural contracts

Python protocols (from `typing.Protocol`) define structural interfaces. Proof mode can verify that a class correctly implements a protocol:

```
from typing import Protocol, runtime_checkable
from deppy import implements
```

```python
@runtime_checkable
class Stack(Protocol):
    @requires(lambda self: True)
    @ensures(lambda self: self.size() == old(self.size()) + 1)
    def push(self, item) -> None: ...

    @requires(lambda self: self.size() > 0)
    @ensures(lambda self, result:
        self.size() == old(self.size()) - 1)
    def pop(self) -> object: ...

    @ensures(lambda self, result: result >= 0)
    def size(self) -> int: ...

@implements(Stack)
class ListStack:
    def __init__(self):
        self.items: list = []

    def push(self, item) -> None:
        self.items.append(item)

    def pop(self):
        return self.items.pop()

    def size(self) -> int:
        return len(self.items)
```

The `@implements(Stack)` decorator instructs the system to verify that `ListStack` satisfies the `Stack` protocol's contracts. This includes signature compatibility, precondition weakening (the implementation's preconditions must be at least as weak as the protocol's), and postcondition strengthening (the implementation's postconditions must be at least as strong).

Chapter 6

Proving Properties of Data Structures

Data structures are the primary targets for proof-mode investment, because their invariants are critical for correctness and are often violated by subtle bugs that testing misses.

6.1 Sorted lists and binary search trees

6.1.1 Proving sortedness preservation

```
from deppy import requires, ensures, forall, Refine, lemma

Sorted = Refine[list[int], lambda xs:
    forall(range(len(xs) - 1), lambda i: xs[i] <= xs[i+1])]

@requires(lambda xs: True)
@ensures(lambda xs, result: isinstance(result, list))
@ensures(lambda xs, result: len(result) == len(xs))
@ensures(lambda xs, result:
    forall(range(len(result) - 1),
           lambda i: result[i] <= result[i+1]))
@ensures(lambda xs, result:
    forall(xs, lambda x: x in result))
def insertion_sort(xs: list[int]) -> Sorted:
    result = []
    for x in xs:
        with invariant():
            have("isinstance(result, Sorted)")
            have("len(result) == len(xs[:i])")
        result = insert_sorted(result, x)
    return result

@requires(lambda xs: isinstance(xs, Sorted))
@ensures(lambda xs, x, result: isinstance(result, Sorted))
@ensures(lambda xs, x, result: len(result) == len(xs) + 1)
@ensures(lambda xs, x, result: x in result)
def insert_sorted(xs: Sorted, x: int) -> Sorted:
    for i in range(len(xs)):
        if x <= xs[i]:
            return xs[:i] + [x] + xs[i:]
    return xs + [x]
```

The system verifies:

1. `insert_sorted` preserves sortedness: given a sorted list, the output is sorted. This is verified by case analysis on the insertion point.
2. `insertion_sort` produces a sorted list: the loop invariant states that `result` is always sorted, and `insert_sorted` preserves this.
3. The output is a permutation of the input: every element of the input appears in the output.

6.1.2 Binary search tree invariant

```
from deppy import invariant_of

@invariant_of(lambda self:
    (self.left is None or
     forall(self.left.all_values(),
            lambda v: v < self.value)) and
    (self.right is None or
     forall(self.right.all_values(),
            lambda v: v > self.value)) and
    (self.left is None or self.left.is_bst()) and
    (self.right is None or self.right.is_bst()))
class BST:
    def __init__(self, value: int):
        self.value = value
        self.left: BST | None = None
        self.right: BST | None = None

    def is_bst(self) -> bool:
        """Check the BST invariant (used in the invariant itself)."""
        ...

    def all_values(self) -> list[int]:
        """Return all values in the tree."""
        result = [self.value]
        if self.left is not None:
            result = self.left.all_values() + result
        if self.right is not None:
            result = result + self.right.all_values()
        return result

    @ensures(lambda self, key, result:
        key in self.all_values() or result is None)
    @ensures(lambda self, key, result:
        result is not None and result.value == key
        or result is None and key not in self.all_values())
    def search(self, key: int) -> "BST | None":
        if key == self.value:
            return self
        elif key < self.value and self.left is not None:
            return self.left.search(key)
        elif key > self.value and self.right is not None:
            return self.right.search(key)
        return None
```

The BST invariant is recursive: each subtree must also be a BST. The system handles this through structural induction on the tree, verifying that each method preserves the invariant at every level of the recursion.

6.2 Hash maps and amortized complexity

While the system focuses on functional correctness rather than complexity, some correctness properties depend on amortized arguments. For example, a hash map with linear probing is correct only if the load factor remains below 1. The proof mode can express this:

```
@invariant_of(lambda self:
    self.count < self.capacity and
    self.capacity > 0 and
    self.count >= 0)
class HashMap:
    def __init__(self, initial_capacity: Refine[int, lambda c: c > 0]):
        self.capacity = initial_capacity
        self.count = 0
        self.table = [None] * initial_capacity

    @requires(lambda self: self.count < self.capacity - 1)
    def put(self, key, value):
        # linear probing always terminates when count < capacity
        idx = hash(key) % self.capacity
        while self.table[idx] is not None:
            with invariant():
                have("exists an empty slot in self.table")
            if self.table[idx][0] == key:
                self.table[idx] = (key, value)
                return
            idx = (idx + 1) % self.capacity
        self.table[idx] = (key, value)
        self.count += 1
```

The invariant `self.count` $<$ `self.capacity` ensures that the linear probing loop always terminates (there is always at least one empty slot).

Chapter 7

Proving Properties of Algorithms

This chapter demonstrates proof mode on a range of algorithms, showing how the annotation effort scales with the complexity of the correctness property and how the inference engine reduces the burden.

7.1 Sorting algorithms: from specification to proof

7.1.1 The sorting specification

A sorting function must satisfy three properties:

1. **Sorted output.** The output is in non-decreasing order.
2. **Permutation.** The output contains exactly the same elements as the input (with the same multiplicities).
3. **Length preservation.** The output has the same length as the input.

In our notation:

```
from deppy import ensures, forall, Refine
from collections import Counter

@ensures(lambda xs, result: len(result) == len(xs))
@ensures(lambda xs, result:
    forall(range(len(result) - 1),
           lambda i: result[i] <= result[i+1]))
@ensures(lambda xs, result: Counter(result) == Counter(xs))
def sort_spec(xs: list[int]) -> list[int]:
    """Specification for a correct sorting function."""
    ...
```

7.1.2 Merge sort with full proof

```
from deppy import requires, ensures, forall, have, hence
from deppy import decreases, lemma, auto

@ensures(lambda xs, result: len(result) == len(xs))
@ensures(lambda xs, result:
    forall(range(len(result) - 1),
           lambda i: result[i] <= result[i+1]))
@ensures(lambda xs, result: Counter(result) == Counter(xs))
```

```
@decreases(lambda xs: len(xs))
def merge_sort(xs: list[int]) -> list[int]:
    if len(xs) <= 1:
        have("xs is trivially sorted")
        return xs[:]

    mid = len(xs) // 2
    have("0 < mid < len(xs)")

    left = merge_sort(xs[:mid])
    have("len(left) == mid")
    have("forall(range(len(left)-1), "
         "lambda i: left[i] <= left[i+1])")
    have("Counter(left) == Counter(xs[:mid])")

    right = merge_sort(xs[mid:])
    have("len(right) == len(xs) - mid")
    have("forall(range(len(right)-1), "
         "lambda i: right[i] <= right[i+1])")
    have("Counter(right) == Counter(xs[mid:])")

    result = merge(left, right)
    hence("len(result) == len(xs)")
    hence("Counter(result) == Counter(xs)")
    return result
```

The `@decreases(lambda xs: len(xs))` annotation ensures termination: at each recursive call, the input length strictly decreases. The proof checkpoints (`have`) document the key facts at each stage; the solver verifies each one from the postconditions of the recursive calls and the `merge` function.

7.1.3 The merge function

```
@requires(lambda a, b:
    forall(range(len(a)-1), lambda i: a[i] <= a[i+1]))
@requires(lambda a, b:
    forall(range(len(b)-1), lambda i: b[i] <= b[i+1]))
@ensures(lambda a, b, result: len(result) == len(a) + len(b))
@ensures(lambda a, b, result:
    forall(range(len(result)-1),
           lambda i: result[i] <= result[i+1]))
@ensures(lambda a, b, result:
    Counter(result) == Counter(a) + Counter(b))
def merge(a: list[int], b: list[int]) -> list[int]:
    result = []
    i, j = 0, 0
    while i < len(a) and j < len(b):
        with invariant():
            have("len(result) == i + j")
            have("forall(range(len(result)-1), "
                 "lambda k: result[k] <= result[k+1])")
            have("(len(result) == 0 or "
                 "(i < len(a) and result[-1] <= a[i]) or "
```

```
                "(j < len(b) and result[-1] <= b[j]))")
            have("Counter(result) == "
                "Counter(a[:i]) + Counter(b[:j])")
        if a[i] <= b[j]:
            result.append(a[i])
            i += 1
        else:
            result.append(b[j])
            j += 1
    result.extend(a[i:])
    result.extend(b[j:])
    return result
```

The loop invariant captures four properties:

1. The result length tracks the progress ($i + j$).
2. The result is always sorted.
3. The last element of the result is less than or equal to the next unconsumed element from either input.
4. The result is a merge of the consumed portions of both inputs.

The system verifies each invariant clause inductively: it holds initially (empty result, $i = j = 0$), and each iteration preserves it. This is within the decidable fragment (bounded quantification, linear arithmetic, array reasoning).

7.2 Graph algorithms

7.2.1 Breadth-first search: reachability proof

```
from deppy import requires, ensures, forall, have, invariant

Graph = dict[int, list[int]]

@requires(lambda graph, start: start in graph)
@ensures(lambda graph, start, result:
    start in result)
@ensures(lambda graph, start, result:
    forall(result,
           lambda v: is_reachable(graph, start, v)))
@ensures(lambda graph, start, result:
    forall(graph.keys(),
           lambda v: is_reachable(graph, start, v)
                    or v not in result))
def bfs(graph: Graph, start: int) -> set[int]:
    visited = {start}
    queue = [start]

    while queue:
        with invariant():
            have("start in visited")
            have("forall(visited, "
                "lambda v: is_reachable(graph, start, v))")
            have("forall(queue, lambda v: v in visited)")
            have("forall(visited, lambda v: "
```

```
                "forall(graph.get(v, []), lambda w: "
                "w in visited or w in queue or "
                "v in queue))")

        node = queue.pop(0)
        for neighbor in graph.get(node, []):
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)
                have("is_reachable(graph, start, neighbor)")

    hence("queue is empty, so all neighbors of visited "
          "nodes are in visited")
    hence("visited is the complete reachable set")
    return visited
```

The loop invariant captures the BFS frontier property: every visited node is reachable from start, every queued node is visited, and every neighbor of a visited node is either visited or queued. When the queue is empty, the last property implies that the visited set is closed under the neighbor relation, hence it contains all reachable nodes.

7.2.2 Topological sort: acyclicity and ordering

```
from deppy import requires, ensures, forall, Refine

DAG = Refine[Graph, lambda g: is_acyclic(g)]

@requires(lambda graph: is_acyclic(graph))
@ensures(lambda graph, result: len(result) == len(graph))
@ensures(lambda graph, result:
    set(result) == set(graph.keys()))
@ensures(lambda graph, result:
    forall(range(len(result)),
           lambda i: forall(graph.get(result[i], []),
                            lambda dep: result.index(dep) < i)))
def topological_sort(graph: DAG) -> list[int]:
    in_degree = {v: 0 for v in graph}
    for v in graph:
        for w in graph[v]:
            in_degree[w] = in_degree.get(w, 0) + 1

    queue = [v for v in graph if in_degree[v] == 0]
    result = []

    while queue:
        with invariant():
            have("len(result) + len(queue) + "
                 "sum(1 for v in graph if in_degree[v] > 0 "
                 "and v not in result) == len(graph)")
            have("forall(range(len(result)), "
                 "lambda i: forall(graph.get(result[i],[]),"
                 "lambda d: d in result[:i]))")
```

```
        v = queue.pop(0)
        result.append(v)
        for w in graph[v]:
            in_degree[w] -= 1
            if in_degree[w] == 0:
                queue.append(w)

    have("len(result) == len(graph)")
    return result
```

The postcondition asserts topological ordering: for every node v in the result, all of v's dependencies appear earlier. The acyclicity precondition is essential for Kahn's algorithm to process all nodes.

7.3 Numeric algorithms

7.3.1 Newton's method: convergence and precision

For numeric algorithms, the system can verify safety properties (no division by zero, no overflow) while leaving convergence properties as proof obligations:

```
from deppy import requires, ensures, Refine, have

@requires(lambda x: x > 0)
@requires(lambda tol: tol > 0)
@requires(lambda max_iter: max_iter > 0)
@ensures(lambda x, tol, max_iter, result:
    result > 0)
@ensures(lambda x, tol, max_iter, result:
    abs(result * result - x) <= tol
    or "max iterations reached")
def sqrt_newton(
    x: Refine[float, lambda v: v > 0],
    tol: Refine[float, lambda v: v > 0] = 1e-10,
    max_iter: Refine[int, lambda v: v > 0] = 100,
) -> float:
    guess = x / 2.0 if x > 1.0 else 1.0
    have("guess > 0")

    for _ in range(max_iter):
        with invariant():
            have("guess > 0")
        new_guess = (guess + x / guess) / 2.0
        have("guess > 0 implies x / guess is finite")
        have("new_guess > 0")
        if abs(new_guess - guess) < tol:
            return new_guess
        guess = new_guess

    return guess
```

The system verifies:

- No division by zero: `guess > 0` is maintained by the invariant, so `x / guess` is safe.

- The output is positive: follows from the invariant.
- Each iteration produces a finite result: follows from `guess > 0` and `x > 0`.

The convergence property ($|r^2 - x| \leq \epsilon$) is stated as a postcondition but qualified with "or max iterations reached." The system cannot prove convergence automatically (it depends on real analysis), but it can verify all the supporting numeric safety properties.

7.3.2 Matrix multiplication: shape correctness with value proof

```
from deppy import requires, ensures, forall

@requires(lambda A, B:
    len(A) > 0 and len(A[0]) > 0 and
    len(B) > 0 and len(B[0]) > 0 and
    len(A[0]) == len(B))
@ensures(lambda A, B, result:
    len(result) == len(A) and
    len(result[0]) == len(B[0]))
@ensures(lambda A, B, result:
    forall(range(len(A)), lambda i:
        forall(range(len(B[0])), lambda j:
            result[i][j] == sum(
                A[i][k] * B[k][j]
                for k in range(len(A[0]))))))
def matmul(
    A: list[list[float]],
    B: list[list[float]],
) -> list[list[float]]:
    m, n, p = len(A), len(A[0]), len(B[0])
    result = [[0.0] * p for _ in range(m)]

    for i in range(m):
        for j in range(p):
            total = 0.0
            for k in range(n):
                with invariant():
                    have("total == sum(A[i][l] * B[l][j] "
                         "for l in range(k))")
                total += A[i][k] * B[k][j]
            result[i][j] = total
            have("result[i][j] == sum(A[i][k] * B[k][j] "
                 "for k in range(n))")

    return result
```

The innermost loop invariant tracks the partial sum. The solver verifies that adding the k-th term extends the partial sum correctly. The outer postcondition asserts that each element of the result matrix equals the dot product of the corresponding row and column.

7.4 String and parsing algorithms

7.4.1 Balanced parentheses checker

```python
from deppy import ensures, have, invariant

@ensures(lambda s, result:
    result == is_balanced_spec(s))
def is_balanced(s: str) -> bool:
    depth = 0
    for ch in s:
        with invariant():
            have("depth >= 0")
            have("depth == count_open(s[:i]) - count_close(s[:i])")
        if ch == '(':
            depth += 1
        elif ch == ')':
            if depth == 0:
                return False
            depth -= 1
    return depth == 0
```

The invariant tracks that `depth` equals the number of unmatched open parentheses seen so far. The system verifies that returning `False` when `depth == 0` and `ch == ')'` is correct (an unmatched close paren), and that returning `depth == 0` at the end correctly identifies balanced strings.

7.5 Annotation cost summary

Algorithm	Lines of code	Annotation lines	Ratio	What is proved
Binary search	8	11	1.4x	Correct insertion point
Merge sort	12	18	1.5x	Sorted, permutation, length
Merge	14	12	0.9x	Sorted merge, permutation
BFS	12	15	1.3x	Complete reachability
Topological sort	14	10	0.7x	Topological ordering
Newton's method	10	8	0.8x	Numeric safety, positivity
Matrix multiply	10	12	1.2x	Value correctness
Balanced parens	10	6	0.6x	Specification equivalence

The annotation-to-code ratio ranges from 0.6x to 1.5x, with an average of about 1.0x. This is dramatically less than traditional proof assistants, where the proof-to-code ratio is typically 5–20x. The savings come from the solver handling most arithmetic and array reasoning automatically, and the inference engine providing the mechanical bookkeeping (variable tracking, control flow, shape propagation).

Chapter 8

The Proof Engine: How Proofs Flow Through the Sheaf Machinery

This chapter describes the internal architecture of the proof engine—how proof obligations are generated, propagated, simplified, and discharged within the sheaf-descent framework. The key insight is that proofs are *not* a separate layer bolted onto the inference engine; they are additional constraints in the same section-optimization problem.

8.1 Proof obligations as section constraints

Recall that the inference engine computes sections (type assignments) at each site by solving an optimization problem: maximize the information content of sections subject to the constraint that sections agree on overlaps. Proof mode adds a new kind of constraint: *proof obligations*.

A proof obligation is a proposition P that must hold at a specific site s. In the section-optimization framework, this corresponds to the constraint

$$\sigma(s) \models P$$

where $\sigma(s)$ is the section assigned to site s. The obligation may come from:

- A postcondition: P must hold at the function exit site.
- An invariant: P must hold at the loop header site on every iteration.
- A `have` statement: P must hold at the statement's site.
- A caller obligation: P must hold at a call site to satisfy the callee's precondition.

8.2 The obligation lifecycle

Each proof obligation goes through a lifecycle:

Generation. When the system encounters a `@ensures`, `@requires`, `with invariant()`, or `have()` construct, it generates an obligation. The obligation records:

- The proposition P.
- The site s where P must hold.
- The source location in the Python code.
- The available context: all sections known at s.

Simplification. Before sending the obligation to the solver, the system simplifies it:

1. **Context reduction.** If P is already implied by the current section at s (from inference), the obligation is trivially discharged.
2. **Substitution.** Symbolic values are substituted: if the section at s knows that $x = 42$, then $P(x)$ becomes $P(42)$.
3. **Normalization.** Arithmetic expressions are normalized (constant folding, associativity, commutativity).
4. **Decomposition.** Conjunctive obligations ($P \wedge Q$) are split into separate obligations for P and Q.

Discharge. The simplified obligation is sent to the SMT solver. If the solver returns SAT (satisfiable with a model showing the obligation holds) or UNSAT (the negation is unsatisfiable, meaning the obligation always holds), the obligation is discharged. In practice, the solver checks $\Gamma \models P$ where Γ is the conjunction of all known facts at site s.

Escalation. If the solver returns UNKNOWN or times out, the obligation is escalated:

1. First, the system attempts more expensive proof strategies (lemma search, case splitting, bounded unrolling).
2. If all automation fails, the obligation becomes a *residual* reported to the programmer.
3. The programmer can then provide a proof, a lemma, or mark the obligation as `sorry()` or `@assume`.

8.3 Obligation propagation through the site cover

Proof obligations propagate through the site cover via the same transport maps that propagate inferred sections:

Backward propagation of postconditions. A postcondition P at the function exit site generates obligations at each `return` site. At each return, the system checks whether the current section implies P. If not, the system traces backward through the control flow to find which earlier site must be strengthened.

Forward propagation of preconditions. A precondition P at a callee's entry site generates an obligation at the call site. The caller must ensure that P holds given the caller's section at the call site. If the caller's section is too weak, the obligation is propagated further backward to the caller's entry, becoming a precondition of the caller.

Loop invariant verification. A loop invariant I generates three obligations:

1. **Initialization**: I holds at the loop entry (before the first iteration), given the section at the loop's predecessor site.
2. **Preservation**: If I holds at the loop header and the loop body executes, then I holds at the end of the loop body (which is the loop header's predecessor in the next iteration).
3. **Usage**: After the loop exits, $I \wedge \neg C$ holds, where C is the loop condition. This conjunction enriches the section at the post-loop site.

Cross-function propagation. When a proved postcondition of function f enriches the section at a call site in function g, this enrichment may allow obligations in g to be discharged that previously could not be. The system re-checks pending obligations whenever a section is strengthened.

8.4 The proof context stack

At each program point, the proof engine maintains a *proof context*: an ordered list of known facts (hypotheses) that can be used to discharge obligations. The context is organized as a stack of scopes:

1. **Global scope**: axioms, proved lemmas, library facts.
2. **Module scope**: module-level assertions, import-derived facts.
3. **Function scope**: preconditions, parameter refinements.
4. **Block scope**: guard refinements, pattern-match bindings.
5. **Loop scope**: loop invariant (on each iteration).
6. **Proof scope**: facts established by `have` statements within a `with proof()` block.

When checking an obligation at site s, the system collects all hypotheses from all enclosing scopes and passes them to the solver as the context Γ.

8.5 Interaction with the inference engine

The proof engine and the inference engine share the same data structures and work in interleaved passes:

Pass 1: Inference. The inference engine computes initial sections for all sites, ignoring proof annotations. This establishes the baseline: what the system can determine without any programmer help.

Pass 2: Annotation integration. The system integrates programmer annotations (contracts, invariants, lemmas) as additional section constraints. This may strengthen some sections and create new obligations.

Pass 3: Obligation discharge. The proof engine attempts to discharge all obligations, using the sections from Pass 2 as context. Successfully discharged obligations confirm the annotations; failed obligations become residuals.

Pass 4: Section enrichment. Successfully verified annotations enrich the sections. Proved postconditions become guaranteed output sections. Proved invariants become fixed-point sections. The enriched sections propagate through the transport maps.

Pass 5: Re-inference. The inference engine re-runs with the enriched sections. This may infer additional properties that were not available in Pass 1 (because the programmer's annotations provided information the inference engine could not derive on its own).

This five-pass architecture means that a single annotation can have cascading effects: the annotation is verified (Pass 3), enriches a section (Pass 4), which enables new inference (Pass 5), which may discharge obligations in other functions that depend on the newly-inferred properties.

8.6 The solver interface

The proof engine communicates with the SMT solver through a typed interface that translates between the section representation and SMT-LIB:

Section to SMT-LIB translation.

- Integer refinements become QF_LIA constraints.
- Floating-point refinements become QF_LRA or QF_FPA constraints.
- Array/list refinements become array-theory constraints.
- Boolean refinements become propositional logic.
- Quantified properties become bounded-quantifier expansions or array-theory axioms.
- Uninterpreted functions model user-defined pure functions.

Solver result interpretation.

- **UNSAT**: The negation of the obligation is unsatisfiable under the context, meaning the obligation always holds. Discharge it.
- **SAT with counterexample**: The obligation can be violated. The counterexample provides a concrete input that triggers the violation. Report it as a verification failure with the counterexample.
- **UNKNOWN**: The solver could not determine the result within the timeout. Escalate to more expensive strategies or report as a residual.

Incremental solving. The system uses Z3's incremental solving features (`push()`/`pop()`) to share context across related obligations. This is critical for performance: when checking a loop invariant's preservation across many paths through the loop body, the common context (the invariant itself, the function's precondition, the class invariant) is shared, and only the path-specific facts are pushed and popped.

8.7 Proof caching and incrementality

The proof engine caches discharged obligations so that re-running the analysis after a small code change does not re-prove everything:

- Each obligation is keyed by its proposition and context hash.
- If the code at a site has not changed and the context sections are unchanged, the cached discharge result is reused.
- If the code changes but the obligation's SMT-LIB encoding is identical, the cached result is still valid.
- Proof cache invalidation is conservative: any change to a function invalidates all obligations in that function, but obligations in other functions are preserved.

This means that in a large codebase, re-verification after a local change is fast: only the changed function and its direct callers are re-checked.

8.8 Error reporting and diagnostics

When an obligation cannot be discharged, the system provides rich diagnostics:

```
PROOF OBLIGATION FAILED at line 42:
  Obligation: result[i] <= result[i+1]
  Context:
    - result is a list of int (inferred)
    - len(result) == i + 1 (from loop invariant)
    - result[i] == xs[idx] (from assignment)
  Missing: no relationship between xs[idx] and result[i-1]
  Suggestion: add a loop invariant relating result elements
              to the insertion position
  Counterexample: xs = [3, 1, 2], i = 1
    result = [3, 1], result[0] = 3 > result[1] = 1
```

The diagnostics include:

- The obligation that failed and its source location.
- The context: what the system knows at the failure point.
- The gap: what information is missing to discharge the obligation.
- A suggestion: what annotation might help.
- A counterexample (if the solver found one): a concrete input that demonstrates the failure.

Chapter 9

Proving Properties of PyTorch Programs

Machine learning programs present unique verification challenges: tensor shapes, numeric precision, gradient correctness, and model architecture properties. This chapter demonstrates how proof mode handles PyTorch-specific verification.

9.1 Shape-correct neural network architectures

9.1.1 Proving shape compatibility of a multi-layer model

```
from deppy import requires, ensures, have
import torch
import torch.nn as nn

@requires(lambda in_dim, hidden_dims, out_dim:
    in_dim > 0 and out_dim > 0 and
    len(hidden_dims) > 0 and
    forall(hidden_dims, lambda d: d > 0))
@ensures(lambda in_dim, hidden_dims, out_dim, result:
    result.input_dim == in_dim and
    result.output_dim == out_dim)
def build_mlp(
    in_dim: int, hidden_dims: list[int], out_dim: int
) -> nn.Sequential:
    layers = []
    prev_dim = in_dim

    for i, h_dim in enumerate(hidden_dims):
        with invariant():
            have("prev_dim > 0")
            have("len(layers) == 2 * i")
        layers.append(nn.Linear(prev_dim, h_dim))
        layers.append(nn.ReLU())
        prev_dim = h_dim
        have("prev_dim == hidden_dims[i]")
        have("prev_dim > 0")
```

```
    layers.append(nn.Linear(prev_dim, out_dim))
    have("the final linear layer maps from "
         "hidden_dims[-1] to out_dim")

    model = nn.Sequential(*layers)
    return model
```

The system verifies that each `nn.Linear` layer's input dimension matches the previous layer's output dimension. The loop invariant tracks `prev_dim` through the layer construction. The final layer's input dimension equals the last hidden dimension, which is positive by the precondition.

9.1.2 Attention mechanism shape proof

```
from deppy import requires, ensures, have

@requires(lambda query, key, value:
    query.dim() == 3 and key.dim() == 3 and
    value.dim() == 3)
@requires(lambda query, key, value:
    query.shape[0] == key.shape[0] == value.shape[0])
@requires(lambda query, key, value:
    query.shape[2] == key.shape[2])
@requires(lambda query, key, value:
    key.shape[1] == value.shape[1])
@ensures(lambda query, key, value, result:
    result.shape == (query.shape[0], query.shape[1],
                     value.shape[2]))
def scaled_dot_product_attention(
    query: torch.Tensor,
    key: torch.Tensor,
    value: torch.Tensor,
) -> torch.Tensor:
    d_k = query.shape[2]
    have("d_k > 0") # from query.dim() == 3

    scores = torch.matmul(query, key.transpose(-2, -1))
    have("scores.shape == (query.shape[0], query.shape[1],"
         " key.shape[1])")

    scores = scores / (d_k ** 0.5)
    have("scores.shape unchanged, d_k > 0 prevents div-by-zero")

    weights = torch.softmax(scores, dim=-1)
    have("weights.shape == scores.shape")
    have("forall weights values: 0 <= w <= 1")
    have("weights sum to 1 along last dimension")

    output = torch.matmul(weights, value)
    have("output.shape == (query.shape[0], query.shape[1],"
         " value.shape[2])")

    return output
```

The proof traces shapes through each operation:

- `key.transpose(-2, -1)` swaps the last two dimensions: shape becomes (B, D_k, S_k).
- `torch.matmul(query, key.T)`: matrix multiply on last two dims: $(B, S_q, D_k) \times (B, D_k, S_k) = (B, S_q, S_k)$.
- Division by $\sqrt{d_k}$: scalar operation, shape preserved.
- `torch.softmax`: element-wise, shape preserved.
- `torch.matmul(weights, value)`: $(B, S_q, S_k) \times (B, S_k, D_v) = (B, S_q, D_v)$.

Each step is within the decidable shape theory (linear arithmetic on dimension sizes).

9.2 Gradient correctness

9.2.1 Proving that gradients are well-defined

The system can verify that gradient computation does not encounter singularities:

```
from deppy import requires, ensures, have

@requires(lambda x: x.requires_grad)
@requires(lambda x: x.dim() == 2)
@ensures(lambda x, result:
    result.shape == x.shape)
@ensures(lambda x, result:
    not torch.isnan(result).any())
def safe_softmax_cross_entropy(
    x: torch.Tensor,
    targets: torch.Tensor,
) -> torch.Tensor:
    # log-sum-exp trick for numerical stability
    max_x = x.max(dim=1, keepdim=True).values
    have("max_x.shape == (x.shape[0], 1)")
    have("no NaN: max of finite values is finite")

    shifted = x - max_x
    have("shifted.max(dim=1) == 0 for each row")
    have("shifted values are <= 0")

    exp_shifted = torch.exp(shifted)
    have("exp values are in (0, 1] since shifted <= 0")
    have("no overflow: exp(0) = 1 is the maximum")

    sum_exp = exp_shifted.sum(dim=1, keepdim=True)
    have("sum_exp > 0: sum of positive values")
    have("sum_exp >= 1: at least one exp(0) = 1 term")

    log_softmax = shifted - torch.log(sum_exp)
    have("log(sum_exp) is finite: sum_exp >= 1 > 0")
    have("no NaN in log_softmax")

    loss = -log_softmax.gather(1, targets.unsqueeze(1))
    have("loss is finite: log_softmax is finite")

    return loss.mean()
```

The proof traces numeric safety through the log-sum-exp computation:

1. Subtracting the max ensures all shifted values are ≤ 0.
2. $\exp(\text{shifted}) \in (0, 1]$: no overflow.
3. The sum is ≥ 1 (at least one term equals $\exp(0) = 1$): no log-of-zero.
4. The result is finite throughout: no NaN propagation.

This is a real-world pattern (the log-sum-exp trick) that is numerically stable but whose stability is non-obvious. The proof mode makes the reasoning explicit and machine-checked.

9.3 Training loop correctness

9.3.1 Proving that a training loop maintains model invariants

```
from deppy import requires, ensures, have, invariant, old
import torch
import torch.nn as nn

@requires(lambda model, train_loader, epochs:
    epochs > 0 and len(train_loader) > 0)
@ensures(lambda model, train_loader, epochs, result:
    len(result) == epochs)
@ensures(lambda model, train_loader, epochs, result:
    forall(result, lambda loss: isinstance(loss, float)))
def train(
    model: nn.Module,
    train_loader,
    epochs: int,
    lr: float = 0.01,
) -> list[float]:
    optimizer = torch.optim.SGD(model.parameters(), lr=lr)
    criterion = nn.MSELoss()
    epoch_losses = []

    for epoch in range(epochs):
        with invariant():
            have("len(epoch_losses) == epoch")
            have("forall(epoch_losses, "
                 "lambda l: isinstance(l, float))")

        batch_losses = []
        for batch_x, batch_y in train_loader:
            pred = model(batch_x)
            have("pred.shape[0] == batch_x.shape[0]")

            loss = criterion(pred, batch_y)
            have("loss is a scalar tensor")
            have("loss.requires_grad")

            optimizer.zero_grad()
            loss.backward()
            have("all model.parameters() have .grad set")

            optimizer.step()
            batch_losses.append(loss.item())
```

```
        epoch_loss = sum(batch_losses) / len(batch_losses)
        have("len(batch_losses) > 0") # train_loader nonempty
        have("isinstance(epoch_loss, float)")
        epoch_losses.append(epoch_loss)

    have("len(epoch_losses) == epochs")
    return epoch_losses
```

The proof verifies:

- The epoch losses list grows by exactly one per epoch.
- Each epoch loss is a float (not a tensor, not None).
- The division by `len(batch_losses)` is safe (the loader is nonempty).
- The model's output shape is compatible with the loss function.

9.4 Model architecture validation

9.4.1 Proving that a ResNet skip connection is shape-compatible

```
from deppy import requires, ensures, invariant_of, have

@invariant_of(lambda self:
    self.conv1.in_channels == self.in_channels and
    self.conv2.out_channels == self.out_channels and
    (self.shortcut is None and
     self.in_channels == self.out_channels) or
    (self.shortcut is not None and
     self.shortcut.in_channels == self.in_channels and
     self.shortcut.out_channels == self.out_channels))
class ResidualBlock(nn.Module):
    def __init__(self, in_channels: int, out_channels: int):
        super().__init__()
        self.in_channels = in_channels
        self.out_channels = out_channels
        self.conv1 = nn.Conv2d(in_channels, out_channels, 3,
                               padding=1)
        self.conv2 = nn.Conv2d(out_channels, out_channels, 3,
                               padding=1)
        self.shortcut = None
        if in_channels != out_channels:
            self.shortcut = nn.Conv2d(in_channels, out_channels,
                                      1)

    @requires(lambda self, x:
        x.dim() == 4 and x.shape[1] == self.in_channels)
    @ensures(lambda self, x, result:
        result.shape == (x.shape[0], self.out_channels,
                         x.shape[2], x.shape[3]))
    def forward(self, x: torch.Tensor) -> torch.Tensor:
        identity = x

        out = self.conv1(x)
```

```
        have("out.shape[1] == self.out_channels")
        out = torch.relu(out)
        out = self.conv2(out)
        have("out.shape[1] == self.out_channels")

        if self.shortcut is not None:
            identity = self.shortcut(x)
            have("identity.shape[1] == self.out_channels")
        else:
            have("self.in_channels == self.out_channels")
            have("identity.shape[1] == self.out_channels")

        # The addition is shape-safe
        have("out.shape == identity.shape")
        out = out + identity
        return torch.relu(out)
```

The class invariant ensures that the shortcut connection exists exactly when the input and output channel counts differ. The forward method proof shows that the skip connection's output shape always matches the main path's output shape, so the addition is safe.

Chapter 10

Proof Mode for Real-World Python: Effects, I/O, and the Trusted Computing Base

Real Python programs are not pure functions. They read files, make network requests, mutate global state, and interact with databases. This chapter describes how proof mode handles effectful computation while maintaining soundness.

10.1 The effect discipline

The system tracks effects using a lightweight effect system embedded in the section structure. Each site carries an effect annotation that classifies the operations at that site:

- **Pure**: no side effects; the result depends only on the inputs.
- **Read**: reads from external state (files, databases, network) but does not modify it.
- **Write**: modifies external state.
- **Alloc**: allocates a new resource (file handle, socket, database connection).
- **Free**: releases a resource.
- **Raise**: may raise an exception.
- **Diverge**: may not terminate.

Effects compose: a function that calls both a Read function and a Write function has both Read and Write effects. The system infers effects automatically; the programmer does not need to annotate them. However, the programmer can constrain effects:

```
from deppy import pure

@pure
def compute_hash(data: bytes) -> int:
    """Must be a pure function (no side effects)."""
    h = 0
    for b in data:
        h = (h * 31 + b) % (2**64)
    return h
```

The `@pure` decorator asserts that the function has no effects. The system verifies this by checking that no operation in the function's body has a Write, Alloc, Free, or Raise effect.

10.2 Resource tracking and linear types

Resources (file handles, database connections, locks) must be acquired and released exactly once. The proof mode supports linear-type-style tracking:

```
from deppy import requires, ensures, owned, consumed

@ensures(lambda path, result: owned(result))
def open_file(path: str) -> "FileHandle":
    return open(path, 'r')

@requires(lambda handle: owned(handle))
@ensures(lambda handle, result: consumed(handle))
def close_file(handle: "FileHandle") -> None:
    handle.close()

@requires(lambda handle: owned(handle))
@ensures(lambda handle, result: owned(handle))
def read_line(handle: "FileHandle") -> str:
    return handle.readline()
```

The `owned(handle)` predicate asserts that the resource is currently owned (open, valid, not yet released). The `consumed(handle)` predicate asserts that the resource has been released. The system tracks ownership through the section structure:

- Opening a resource establishes ownership.
- Using a resource requires ownership.
- Closing a resource consumes ownership.
- Using a consumed resource creates an error site (use-after-close).
- Failing to close a resource before the owning scope exits creates a resource-leak warning.

Context managers as ownership boundaries. Python's `with` statement provides automatic resource management:

```
with open("data.txt") as f:
    # f is owned here
    data = f.read()
# f is consumed here (automatically closed)
```

The system recognizes `with` as an ownership boundary: the resource is owned inside the block and consumed at the block exit. This means that well-structured code using context managers gets resource safety for free.

10.3 State-dependent types for mutable objects

Mutable objects have state that changes over time. The proof mode tracks object state through *state-dependent types*: the type of an object includes its current state.

```
from deppy import state, transition, requires, ensures

class StateMachine:
    @state("idle")
    def __init__(self):
        self.data = None
```

```
    @requires(lambda self: self.in_state("idle"))
    @transition("idle", "running")
    def start(self, data):
        self.data = data

    @requires(lambda self: self.in_state("running"))
    @transition("running", "done")
    def stop(self) -> object:
        result = self.data
        self.data = None
        return result

    @requires(lambda self: self.in_state("done"))
    @transition("done", "idle")
    def reset(self):
        pass
```

The `@state` and `@transition` decorators define a state machine. The system verifies that:

- Every method is called only in its required state.
- State transitions are valid (only declared transitions are allowed).
- The state machine is used correctly at every call site.

This pattern is useful for protocol implementations, connection managers, and any object whose methods must be called in a specific order.

10.4 Proving properties across module boundaries

In large codebases, functions are spread across modules. The proof mode supports cross-module verification through *module contracts*:

```
# module: data_loader.py
from deppy import module_ensures

@module_ensures("""
    All functions in this module that return DataFrames
    guarantee that the DataFrame has the columns specified
    in the function's @ensures annotation.
    All functions handle missing files by raising FileNotFoundError.
""")

@ensures(lambda path, result:
    set(result.columns) >= {"id", "name", "value"})
@ensures(lambda path, result: len(result) > 0)
def load_data(path: str) -> "pd.DataFrame":
    ...
```

The `@module_ensures` decorator is a documentation-level annotation that describes the module's guarantees. Individual function contracts are checked mechanically; the module contract provides context for human readers.

When function `g` in module B calls function `f` in module A, the system uses `f`'s contract as an interface. If `f` is fully verified, the contract is trusted. If `f` is only inferred (Level 0), the contract is

the inferred section, which may be weaker. The system reports the trust level at each cross-module boundary.

10.5 The trusted computing base

The *trusted computing base* (TCB) is the set of components whose correctness the system assumes without proof:

1. **The Python interpreter.** The system assumes that Python executes code according to the language specification.
2. **The SMT solver (Z3).** The system assumes that Z3 returns correct results.
3. **Library axioms.** The system assumes that library theory packs correctly model their libraries (e.g., that `len(xs)` returns a non-negative integer, that `torch.matmul` follows matrix multiplication shape rules).
4. **The proof checker.** The system assumes that the proof checker correctly verifies proof terms.
5. **Assumed axioms.** Properties marked with `@assume` are trusted without proof.

The TCB is explicitly documented and minimized:

- Library axioms are generated from specifications, not from implementation details, and can be tested against the actual library.
- The proof checker is small (a few hundred lines) and can be independently audited.
- `@assume` usage is tracked and reported, so the programmer knows exactly what is trusted.

10.6 Soundness and the escape hatches

The system provides several escape hatches that compromise soundness in exchange for practicality. Each escape hatch is tracked:

`@assume(P)`. Asserts that P is true without proof. The system treats P as an axiom. If P is false, the system may derive incorrect results.

`sorry()`. Marks a proof obligation as deferred. The system proceeds as if the obligation were proved. All `sorry()` uses are reported in the final verification summary.

`@unchecked`. Marks a function as unchecked. The system does not analyze the function's body. The function's contract (if any) is trusted without verification.

`cast(T, x)`. Asserts that `x` has type `T` without verification. This is Python's standard `typing.cast`, but the system tracks its use.

The verification summary reports all escape hatch uses:

```
VERIFICATION SUMMARY:
  Functions analyzed: 47
  Obligations discharged: 312 / 318
  Obligations deferred (sorry): 4
  Assumed axioms: 2
  Unchecked functions: 1
  Type casts: 3
```

```
Trust level: 96.2% verified
```

This gives the programmer a clear picture of how much of their code is fully verified and where the gaps are.

Chapter 11

Advanced Proof Techniques: Abstraction, Refinement, and Parametricity

This chapter covers advanced proof techniques that enable reasoning about complex software systems while keeping the annotation burden manageable.

11.1 Data abstraction and representation invariants

A key technique for managing proof complexity is *data abstraction*: proving properties about an abstract interface and then showing that the concrete implementation satisfies the interface.

```
from deppy import abstract, refines, rep_invariant

@abstract
class AbstractSet:
    """Abstract specification of a set."""
    @ensures(lambda self, result: result >= 0)
    def size(self) -> int: ...

    @ensures(lambda self, x, result:
        isinstance(result, bool))
    def contains(self, x: int) -> bool: ...

    @ensures(lambda self, x:
        self.contains(x) and
        self.size() == old(self.size()) +
        (0 if old(self.contains(x)) else 1))
    def add(self, x: int) -> None: ...

    @requires(lambda self, x: self.contains(x))
    @ensures(lambda self, x:
        not self.contains(x) and
        self.size() == old(self.size()) - 1)
    def remove(self, x: int) -> None: ...

@refines(AbstractSet)
```

```python
@rep_invariant(lambda self:
    len(self.items) == len(set(self.items)) and
    all(isinstance(x, int) for x in self.items))
class SortedListSet:
    """Concrete implementation using a sorted list."""
    def __init__(self):
        self.items: list[int] = []

    def size(self) -> int:
        return len(self.items)

    def contains(self, x: int) -> bool:
        # binary search
        lo, hi = 0, len(self.items)
        while lo < hi:
            mid = (lo + hi) // 2
            if self.items[mid] == x:
                return True
            elif self.items[mid] < x:
                lo = mid + 1
            else:
                hi = mid
        return False

    def add(self, x: int) -> None:
        if not self.contains(x):
            # insert in sorted position
            pos = 0
            while pos < len(self.items) and self.items[pos] < x:
                pos += 1
            self.items.insert(pos, x)

    def remove(self, x: int) -> None:
        # binary search for position
        lo, hi = 0, len(self.items)
        while lo < hi:
            mid = (lo + hi) // 2
            if self.items[mid] == x:
                self.items.pop(mid)
                return
            elif self.items[mid] < x:
                lo = mid + 1
            else:
                hi = mid
```

The `@refines(AbstractSet)` decorator instructs the system to verify that `SortedListSet` satisfies every contract in `AbstractSet`. The `@rep_invariant` specifies the concrete representation invariant (no duplicates, all elements are integers). The system verifies:

1. The representation invariant is established by `__init__`.
2. Each method preserves the representation invariant.
3. Each method satisfies the corresponding abstract contract.
4. The abstraction function (mapping concrete state to abstract state) is well-defined.

11.2 Simulation and refinement proofs

When replacing one implementation with another, the programmer can prove that the new implementation refines (is at least as correct as) the old one:

```
from deppy import simulates

@simulates(SortedListSet, simulation_relation=lambda old, new:
    set(old.items) == new.table.keys())
class HashSetImpl:
    def __init__(self):
        self.table: dict[int, bool] = {}

    def size(self) -> int:
        return len(self.table)

    def contains(self, x: int) -> bool:
        return x in self.table

    def add(self, x: int) -> None:
        self.table[x] = True

    def remove(self, x: int) -> None:
        del self.table[x]
```

The `@simulates` decorator asserts that `HashSetImpl` simulates `SortedListSet` under the given simulation relation. The system verifies that for every method, if the simulation relation holds before the call, it holds after the call, and the observable behavior (return values, exceptions) is the same.

11.3 Parametricity and generic proofs

Generic functions parameterized by type variables enjoy *parametricity*: their behavior is uniform across all type instantiations. This gives free theorems that the system can exploit:

```
from deppy import ensures, forall, TypeVar

T = TypeVar("T")

@ensures(lambda xs, result:
    len(result) == len(xs))
@ensures(lambda xs, result:
    forall(range(len(xs)),
           lambda i: result[i] == xs[len(xs) - 1 - i]))
def reverse(xs: list[T]) -> list[T]:
    return xs[::-1]
```

Because `reverse` is parametric in `T`, the system can derive:

- `reverse(reverse(xs)) == xs` (involution).
- `len(reverse(xs)) == len(xs)` (length preservation).
- For any function `f`, `map(f, reverse(xs)) == reverse(map(f, xs))` (naturality).

These "free theorems" follow from parametricity and do not require separate proofs. The system derives them automatically for parametric functions.

11.4 Proof by refinement: stepwise development

The proof mode supports stepwise development, where the programmer starts with a high-level specification and refines it into an implementation through a series of verified steps:

Step 1: Specification.

```
@abstract
@ensures(lambda xs, result: is_sorted(result))
@ensures(lambda xs, result: is_permutation(result, xs))
def sort_spec(xs: list[int]) -> list[int]: ...
```

Step 2: Algorithm choice.

```
@refines(sort_spec)
def merge_sort(xs: list[int]) -> list[int]:
    if len(xs) <= 1:
        return xs[:]
    mid = len(xs) // 2
    return merge(merge_sort(xs[:mid]), merge_sort(xs[mid:]))
```

Step 3: Implementation details.

```
@refines(merge_sort)
def merge_sort_inplace(xs: list[int]) -> list[int]:
    """In-place merge sort with auxiliary buffer."""
    ...
```

Each refinement step is verified: the refined version satisfies the same contract as the original, plus any additional properties the refined version establishes.

11.5 Proof composition and modular verification

The system supports modular verification: each module is verified independently, and cross-module verification uses contracts as interfaces. This means that:

- Changing a function's implementation (without changing its contract) does not require re-verifying callers.
- Adding a new function does not affect existing verification results.
- Removing a function invalidates only code that calls it.

11.5.1 Assume-guarantee reasoning

For mutually-recursive modules, the system supports assume-guarantee reasoning:

```
# module A assumes module B's contract
from deppy import assume_module

@assume_module("B", contract={
    "process": {
        "requires": lambda x: x > 0,
        "ensures": lambda x, r: r >= 0,
    }
})
def compute(x: int) -> int:
```

```
    from B import process
    return process(x) + 1
```

The `@assume_module` decorator declares what module A assumes about module B. The system verifies A under this assumption. Separately, the system verifies that B actually satisfies the assumed contract. If both verifications succeed, the combined system is correct.

11.6 Proof of absence of common vulnerability patterns

The proof mode can verify the absence of common security vulnerabilities:

SQL injection.

```
from deppy import requires, Refine, sanitized

@requires(lambda query: sanitized(query, "sql"))
def execute_query(query: str) -> list:
    cursor.execute(query)
    return cursor.fetchall()
```

The `sanitized(query, "sql")` predicate asserts that the query has been sanitized against SQL injection. The system tracks sanitization status through the program and creates error sites where unsanitized input reaches `execute_query`.

Path traversal.

```
from deppy import requires

@requires(lambda path: ".." not in path.parts)
@requires(lambda path: path.is_relative_to(BASE_DIR))
def read_user_file(path: "Path") -> str:
    return path.read_text()
```

The precondition ensures that the path does not escape the base directory. The system propagates this requirement to all callers.

Integer overflow in serialization.

```
from deppy import requires, Refine

@requires(lambda size: 0 <= size <= MAX_BUFFER_SIZE)
def allocate_buffer(
    size: Refine[int, lambda s: 0 <= s <= MAX_BUFFER_SIZE]
) -> bytes:
    return bytes(size)
```

The bounded-size requirement prevents denial-of-service via oversized allocations.

Chapter 12

The Annotation Economy: Maximizing Proof Value with Minimal Effort

This chapter addresses the central question of proof mode: how should a programmer invest annotation effort to maximize the verification benefit?

12.1 The information-theoretic view

From an information-theoretic perspective, an annotation provides information that resolves uncertainty in the section structure. The "value" of an annotation is the number of obligations it helps discharge, weighted by the importance of those obligations.

High-value annotations.

- **Loop invariants**: A single loop invariant can discharge dozens of obligations (one per iteration, one for the postcondition, several for callers).
- **Library contracts**: A contract on a widely-used utility function propagates to every caller.
- **Class invariants**: A class invariant enriches every method's section.
- **Key lemmas**: A lemma about a core data structure enables verification of all code that uses the structure.

Low-value annotations.

- **Annotations the inference engine already knows**: If the system already inferred that `len(xs) > 0`, annotating it provides zero additional information.
- **Annotations in dead code**: Verifying unreachable paths provides no runtime safety benefit.
- **Redundant postconditions**: If the postcondition follows trivially from the body, the annotation is documentation rather than verification.

12.2 The annotation advisor

The system includes an *annotation advisor* that suggests where annotation effort should be invested:

```
ANNOTATION ADVISOR REPORT:
  High-impact opportunities:
    1. Loop invariant at sort.py:45 (would discharge 8 obligations)
    2. Contract for utils.validate() (called 23 times)
```

```
  3. Class invariant for Database (5 methods, 12 obligations)

Already well-covered:
  - math_utils.py: 94% of obligations auto-discharged
  - config.py: pure functions, fully inferred

Residual clusters:
  - 6 residuals in parser.py relate to grammar validity
    (consider a grammar-correctness lemma)
  - 4 residuals in network.py relate to connection state
    (consider a state machine annotation)
```

The advisor analyzes:

1. **Obligation density**: functions with many undischarged obligations are candidates for annotation.
2. **Call frequency**: functions called from many sites amplify the benefit of a contract.
3. **Residual clustering**: when multiple residuals relate to the same underlying property, a single lemma can discharge all of them.
4. **Inference gaps**: sites where the inferred section is significantly weaker than what the code actually guarantees.

12.3 Progressive verification: a worked example

Consider a small project with 500 lines of Python across 5 modules. We walk through the progressive verification workflow:

Phase 1: Zero-annotation baseline. The system analyzes all 500 lines and produces:

- 180 obligations auto-discharged (arithmetic, bounds, types).
- 42 residuals reported.
- Section quality: 81% of sites have useful inferred sections.

Phase 2: Add contracts to the core utility module (8 functions).

- 8 `@requires`/`@ensures` annotations added (24 lines).
- 15 additional obligations discharged (callers of the utilities).
- Residuals reduced to 27.

Phase 3: Add loop invariants to the main algorithm (2 loops).

- 2 loop invariants added (6 lines).
- 9 additional obligations discharged.
- Residuals reduced to 18.

Phase 4: Add a class invariant to the central data structure.

- 1 class invariant added (3 lines).
- 7 additional obligations discharged.
- Residuals reduced to 11.

Phase 5: Add 2 key lemmas.

- 2 lemmas stated and proved (12 lines).
- 6 additional obligations discharged.
- Residuals reduced to 5.

Phase 6: Accept remaining residuals. The 5 remaining residuals involve properties about external data (user input, file contents) that cannot be verified statically. The programmer marks them as runtime-checked.

Summary.

Phase	Annotations	Discharged	Residuals	Trust
1: Baseline	0	180	42	81%
2: Utility contracts	24	195	27	88%
3: Loop invariants	30	204	18	92%
4: Class invariant	33	211	11	95%
5: Key lemmas	45	217	5	98%
6: Runtime checks	45	222	0	100%

Total annotation effort: 45 lines for 500 lines of code (9% overhead). The system went from 81% auto-verified to 100% (verified + runtime-checked) with targeted, incremental investment.

12.4 When not to prove

Not every property is worth proving. The proof mode should be used judiciously:

- **Don't prove UI layout properties.** UI correctness is visual and subjective; formal verification adds no value.
- **Don't prove performance properties.** While the system can verify termination, it cannot verify time complexity. Use benchmarks instead.
- **Don't prove external behavior.** The system cannot verify that a web API returns correct data or that a database query is semantically correct. Use integration tests.
- **Don't over-specify.** Over-specified contracts are brittle: they break when the implementation changes, even if the change is correct. Specify the essential properties and leave the rest to inference.

The golden rule: *prove what matters, infer what doesn't, and test what can't be proved.*

Part II

Advanced Sheaf Theory, Optimization, and the Relevance Problem

Chapter 13

The Full Sheaf-Theoretic Framework for Type Inference

Parts I–IV developed the sheaf-descent idea informally: types are sections, local type information glues into global types, and the optimization objective maximizes information. This chapter makes the mathematics fully precise, introducing the cohomological machinery that powers automatic deduction and the obstruction theory that identifies when and why type inference fails.

13.1 Sites, covers, and the Grothendieck topology

Definition 13.1.1 (Program site category)**.** Let $\mathbf{S}_P$ be the category whose objects are *program sites* (syntactic locations in a program P where a value is produced, consumed, or tested) and whose morphisms are *data-flow edges* $s \to t$ indicating that information at site s is available at site t. Composition of morphisms corresponds to transitive data flow.

Definition 13.1.2 (Cover)**.** A *cover* of a program site s is a family $\{u_i \to s\}_{i \in I}$ of data-flow edges such that the type of the value at s is completely determined by the types at the u_i. Formally, the cover satisfies the *local determination property*: any two sections that agree on all u_i must agree on s.

Definition 13.1.3 (Grothendieck topology on $\mathbf{S}_P$)**.** The collection of covers defines a Grothendieck topology J on $\mathbf{S}_P$. The triple $(\mathbf{S}_P, J)$ is a *site* in the sense of topos theory. The Grothendieck topology must satisfy:

1. **Identity cover.** $\{\mathrm{id}_s : s \to s\}$ is a cover of s.
2. **Stability.** If $\{u_i \to s\}$ is a cover and $t \to s$ is a morphism, then $\{u_i \times_s t \to t\}$ is a cover of t.
3. **Transitivity.** If $\{u_i \to s\}$ is a cover and for each i, $\{v_{ij} \to u_i\}$ is a cover, then $\{v_{ij} \to s\}$ is a cover.

These axioms ensure that type information composes correctly: if we can determine a type from its parts, and each part from its sub-parts, then we can determine the type from the sub-parts.

13.2 The type presheaf and the sheaf condition

Definition 13.2.1 (Type presheaf)**.** A *type presheaf* on $(\mathbf{S}_P, J)$ is a contravariant functor

$$\mathcal{T} : \mathbf{S}_P^{\mathrm{op}} \to \mathbf{Lat}$$

where **Lat** is the category of complete lattices and monotone maps. For each site s, $\mathcal{T}(s)$ is the lattice of possible type assignments at s. For each morphism $f : s \to t$, the *restriction map* $\mathcal{T}(f) : \mathcal{T}(t) \to \mathcal{T}(s)$ restricts the type information at t to what is observable at s.

Definition 13.2.2 (Sheaf condition)**.** The presheaf $\mathcal{T}$ is a *sheaf* if for every cover $\{u_i \to s\}$ and every family of sections $\sigma_i \in \mathcal{T}(u_i)$ that agree on overlaps (i.e., $\mathcal{T}(p_1)(\sigma_i) = \mathcal{T}(p_2)(\sigma_j)$ for all overlap morphisms), there exists a unique section $\sigma \in \mathcal{T}(s)$ that restricts to each σ_i.

Interpretation. The sheaf condition says: if local type assignments are consistent on overlaps, they glue into a unique global type. This is exactly what type inference should do—combine local evidence into a global type assignment.

When the sheaf condition fails—when local sections do not glue—this indicates a type error. The *obstruction* to gluing identifies the nature of the error.

13.3 Cohomology of the type sheaf

The sheaf cohomology groups $H^n(\mathbf{S}_P, \mathcal{T})$ measure the obstructions to extending local type information to global type assignments.

Definition 13.3.1 (Čech cohomology)**.** Given a cover $\mathcal{U} = \{u_i \to s\}$, define the Čech complex:

$$\begin{aligned}
C^0(\mathcal{U}, \mathcal{T}) &= \prod_i \mathcal{T}(u_i) \\
C^1(\mathcal{U}, \mathcal{T}) &= \prod_{i<j} \mathcal{T}(u_i \times_s u_j) \\
C^2(\mathcal{U}, \mathcal{T}) &= \prod_{i<j<k} \mathcal{T}(u_i \times_s u_j \times_s u_k) \\
&\vdots
\end{aligned}$$

with coboundary maps $\delta^n : C^n \to C^{n+1}$ defined by the alternating sum of restriction maps.

H^0: Global sections = valid type assignments. $H^0(\mathcal{U}, \mathcal{T}) = \ker(\delta^0)$ is the group of global sections—families of local types that agree on all overlaps. This is precisely the set of valid type assignments for the program.

H^1: Obstructions = type errors. $H^1(\mathcal{U}, \mathcal{T}) = \ker(\delta^1)/\mathrm{im}(\delta^0)$ measures the failure of the sheaf condition. A nonzero class in H^1 represents a *type error*: local sections that are pairwise consistent but cannot be glued into a global section. This happens when there is a "twist"—the local sections form a non-trivial cocycle.

H^2: Higher obstructions = structural type incompatibilities. H^2 measures obstructions to extending partial gluings. In practice, H^2 detects situations where three or more type constraints interact in a way that no pairwise analysis would catch—a higher-order type conflict.

Theorem 13.3.2 (Type inference as cohomology vanishing)**.** *A program P is well-typed if and only if $H^1(\mathbf{S}_P, \mathcal{T}) = 0$. That is, every collection of pairwise-consistent local type assignments glues into a global type assignment.*

Proof sketch. The forward direction: if P is well-typed, then a global type assignment exists, which restricts to consistent local assignments, so every cocycle is a coboundary. The reverse direction: if $H^1 = 0$, then every choice of pairwise-consistent local types glues, so in particular the types inferred by the local analysis glue into a global assignment. □

13.4 Descent data and effective descent

The gluing process—constructing a global section from local sections—is formalized by *descent data*.

Definition 13.4.1 (Descent datum)**.** A *descent datum* for a cover $\{u_i \to s\}$ is a pair $(\{\sigma_i\}, \{\phi_{ij}\})$ where:

- $\sigma_i \in \mathcal{T}(u_i)$ is a local section at each cover element.
- $\phi_{ij} : \mathcal{T}(p_1)(\sigma_i) \xrightarrow{\sim} \mathcal{T}(p_2)(\sigma_j)$ is an isomorphism on overlaps.
- The cocycle condition holds: $\phi_{jk} \circ \phi_{ij} = \phi_{ik}$ on triple overlaps.

Definition 13.4.2 (Effective descent)**.** Descent is *effective* if every descent datum arises from a unique global section. Equivalently, the natural map

$$\mathcal{T}(s) \to \mathrm{Desc}(\mathcal{U}, \mathcal{T})$$

is an isomorphism, where Desc is the category of descent data.

Type inference as effective descent. The type inference algorithm constructs descent data (local types with consistency witnesses on overlaps) and then "descends" to a global type. The effectiveness of descent guarantees that the inferred global type is unique—there is no ambiguity in the type inference result.

When descent is not effective (because the sheaf condition fails at some cover), the system reports the failure as a type error, with the cocycle class in H^1 identifying the precise nature of the inconsistency.

13.5 Stalks and the local-to-global principle

Definition 13.5.1 (Stalk)**.** The *stalk* of $\mathcal{T}$ at a site s is

$$\mathcal{T}_s = \varinjlim_{s \in U} \mathcal{T}(U)$$

the colimit over all covers containing s. Informally, the stalk is the "best possible type" at s, combining all available information.

Theorem 13.5.2 (Local-to-global for types)**.** *A morphism of sheaves $f : \mathcal{T} \to \mathcal{T}'$ is an isomorphism if and only if $f_s : \mathcal{T}_s \to \mathcal{T}'_s$ is an isomorphism at every stalk. That is, type equivalence can be checked locally.*

This theorem justifies the system's architecture: type checking can be decomposed into local checks at each site, and if all local checks pass, the global type assignment is valid. This is the formal foundation for the compositional verification strategy.

13.6 Sheaf morphisms and type transformations

When a function transforms data (e.g., `torch.reshape`), it induces a morphism between type sheaves. The system must track how these morphisms interact with sections.

Definition 13.6.1 (Pushforward)**.** Given a program morphism $f : P \to Q$ (e.g., a function call), the *pushforward* $f_*\mathcal{T}$ is the sheaf on $\mathbf{S}_Q$ defined by $(f_*\mathcal{T})(s) = \mathcal{T}(f^{-1}(s))$. This models how the caller's type information is available at the callee's sites.

Definition 13.6.2 (Pullback)**.** The *pullback* $f^*\mathcal{T}'$ is the sheaf on $\mathbf{S}_P$ defined by $(f^*\mathcal{T}')(s) = \mathcal{T}'(f(s))$. This models how the callee's type information flows back to the caller.

The adjunction $f^* \dashv f_*$ captures the bidirectional flow of type information across function boundaries: forward (pushforward) and backward (pullback) propagation.

Chapter 14

The Optimization Problem: Maximizing Type Information

With the sheaf-theoretic framework in place, we can formalize the central optimization problem: among all valid type assignments (global sections), find the one that maximizes information content.

14.1 The information lattice

Definition 14.1.1 (Information ordering)**.** Given two type assignments σ, τ at a site s, we say σ is *more informative* than τ (written $\sigma \sqsupseteq \tau$) if σ refines τ—every fact implied by σ is also implied by τ, and σ may imply additional facts.

For refinement types, the ordering is:

$$\{x : A \mid \phi(x) \wedge \psi(x)\} \sqsupseteq \{x : A \mid \phi(x)\}$$

A more refined type (stronger predicate) is more informative.

Definition 14.1.2 (Information content)**.** The *information content* of a type assignment σ at site s is a measure $\mathcal{I}(\sigma, s) \in [0, \infty)$ that quantifies how much the type constrains the possible values. Formally:

$$\mathcal{I}(\sigma, s) = -\log \frac{|\{v : v \models \sigma(s)\}|}{|\{v : v \models \top\}|}$$

where $\top$ is the unconstrained type. A more restrictive type has higher information content.

In practice, we use an approximation: the number of atomic predicates in the refinement, weighted by their discriminative power.

14.2 The global optimization objective

Definition 14.2.1 (Maximum-information type inference)**.** Given a program P with site category $\mathbf{S}_P$ and type sheaf $\mathcal{T}$, the *maximum-information type inference problem* is:

$$\sigma^* = \arg \max_{\sigma \in H^0(\mathbf{S}_P, \mathcal{T})} \sum_{s \in \mathbf{S}_P} w(s) \cdot \mathcal{I}(\sigma, s)$$

where $w(s) \geq 0$ is a weight that reflects the importance of site s (e.g., higher weight for sites involved in potential runtime errors).

Why maximize? A more informative type assignment detects more bugs, provides more useful IDE feedback, and enables more downstream verification. The trivial type assignment ($\top$ at every site) is always valid but useless. The system should find the most informative valid assignment.

Why weighted? Not all sites are equally important. Sites where a runtime error could occur (division, indexing, attribute access) should be weighted more heavily, so the optimizer prioritizes inferring precise types at those sites.

14.3 Reformulation as a constraint satisfaction problem

The optimization problem can be reformulated as a weighted MaxSAT or MaxSMT problem:

Hard constraints (must be satisfied).

- **Overlap consistency.** For every overlap $u_i \times_s u_j$, the restrictions of σ_i and σ_j must agree.
- **Semantic soundness.** For every operation site s with semantics $[\![s]\!]$, the output type must be consistent with applying $[\![s]\!]$ to the input types.
- **Data-flow consistency.** For every data-flow edge $s \to t$, $\sigma(t)$ must be at least as general as $\sigma(s)$ restricted to t.

Soft constraints (maximize satisfaction).

- For each site s and each candidate predicate ϕ that could refine the type at s, add a soft clause $\sigma(s) \models \phi$ with weight $w(s) \cdot \mathcal{I}(\phi)$.
- The MaxSMT solver maximizes the total weight of satisfied soft clauses.

Theorem 14.3.1 (Soundness of MaxSMT formulation)**.** *The MaxSMT solution σ^* is a valid type assignment (all hard constraints are satisfied) that maximizes the weighted information content among all valid assignments.*

14.4 The lattice structure of solutions

Theorem 14.4.1 (Lattice of valid type assignments)**.** *The set of valid type assignments $H^0(\mathbf{S}_P, \mathcal{T})$ forms a complete lattice under the information ordering. The least element is $\top$ (no information). The greatest element is the* principal type*— the most informative valid assignment.*

Proof sketch. The meet of two valid assignments restricts to the common information (intersection of refinement predicates). The join takes the stronger of the two at each site and verifies consistency; if both are valid and consistent, their join is valid. The greatest element exists by Zorn's lemma applied to the chain of increasingly-informative valid assignments. □

The principal type. The principal type is the unique most-informative valid type assignment. If it exists and is computable, it is the ideal output of the inference engine. In general, the principal type may not be computable (the predicate language is too expressive), but within decidable fragments, the MaxSMT solver converges to it.

14.5 Approximation algorithms

The exact MaxSMT problem may be intractable for large programs. The system uses several approximation strategies:

Fixpoint iteration. Start with the least informative assignment ($\top$ at every site). At each iteration, for each site s, strengthen the type at s by adding predicates that are implied by the current types at s's predecessors. Repeat until convergence (fixpoint). This is equivalent to abstract interpretation and converges monotonically to a valid type assignment.

Widening. When fixpoint iteration is slow (large loops, deep recursion), apply widening operators that sacrifice some precision for faster convergence. The widening point is at loop headers: after a fixed number of iterations, generalize the loop-header type to an over-approximation.

Stratified solving. Decompose the program into strongly-connected components (SCCs) of the data-flow graph. Solve each SCC independently (from leaves to roots). Within each SCC, use fixpoint iteration. This reduces the problem size and exploits the acyclic structure of most programs.

Demand-driven inference. Instead of inferring types at all sites, infer types only at sites that are "demanded"—sites involved in potential errors, sites with user annotations, or sites explicitly queried. This reduces work for large programs where most sites are uninteresting.

14.6 Duality: the backward optimization problem

The forward problem maximizes output information given inputs. The *backward* or *dual* problem minimizes input requirements while ensuring output properties:

$$\rho^* = \arg \min_{\rho \in \mathcal{T}(\text{input})} \mathcal{I}(\rho) \quad \text{subject to} \quad \forall s \in \text{error-sites} : \text{viable}(\sigma_s^*, \rho)$$

This finds the *weakest precondition*—the least restrictive input requirement that ensures all error sites are viable (no runtime errors).

Theorem 14.6.1 (Duality)**.** *The forward maximum-information problem and the backward weakest-precondition problem are Lagrangian duals. The optimal Lagrange multipliers correspond to the "importance weights" at each error site.*

This duality is the formal justification for the bidirectional synthesis described in Part I: the forward and backward passes are two views of the same optimization problem.

Chapter 15

The Relevance Problem: Which Propositions Matter for Bug Detection?

A dependent type system can decorate every term with an enormous number of propositions: the integer 42 satisfies $x > 0$, $x < 100$, x is even, $x \equiv 0 \pmod 6$, and infinitely many others. Most of these propositions are irrelevant to detecting bugs in the surrounding code. This chapter formalizes the *relevance problem*: determining which propositions are worth tracking.

15.1 The relevance criterion

Definition 15.1.1 (Bug-relevant proposition)**.** A proposition ϕ at site s is *bug-relevant* if there exists a reachable error site e such that knowing ϕ at s changes the viability analysis at e. Formally, ϕ is bug-relevant if:

$$\exists e \in \text{ErrorSites}(P): \quad \text{viable}(e \mid \sigma \cup \{\phi\}) \neq \text{viable}(e \mid \sigma)$$

where σ is the current section without ϕ.

Intuitively, a proposition is bug-relevant if adding it to the type information either (a) allows the system to prove that an error cannot occur (the error site becomes viable), or (b) allows the system to prove that an error must occur (the error site becomes non-viable and should be reported).

The relevance filter. The system should track only bug-relevant propositions. This dramatically reduces the number of predicates in the MaxSMT problem and focuses the analysis on what matters.

15.2 Computing relevance: the backward slice

Bug-relevance can be computed by a backward analysis from error sites:

Definition 15.2.1 (Error-site backward slice)**.** The *backward slice* from an error site e is the set of sites s such that the type at s can influence the viability predicate at e. Formally:

$$\text{Slice}(e) = \{s \in \mathbf{S}_P \mid \exists \text{data-flow path } s \rightsquigarrow e\}$$

A proposition ϕ at site s can be bug-relevant only if $s \in \text{Slice}(e)$ for some error site e. This is a necessary but not sufficient condition.

Definition 15.2.2 (Predicate relevance)**.** A predicate ϕ over the value at site s is *relevant to error site* e if:

1. $s \in \text{Slice}(e)$.
2. The viability predicate at e, when pulled back to s through the data-flow path, involves the variable constrained by ϕ.
3. The decision procedure for the viability predicate at e can use ϕ (i.e., ϕ is in a compatible theory).

15.3 The predicate abstraction framework

Rather than tracking arbitrary propositions, the system operates over a finite *predicate abstraction*: a finite set of predicates that are sufficient to capture the relevant type information.

Definition 15.3.1 (Predicate domain)**.** A *predicate domain* Π for a program P is a finite set of predicates $\{\phi_1, \ldots, \phi_n\}$ over the program's variables. The *abstract type* at site s is a Boolean assignment to the predicates in Π that are defined at s.

Definition 15.3.2 (Sufficient predicate domain)**.** Π is *sufficient for bug detection* if for every error site e in P, the viability predicate at e can be expressed as a Boolean combination of predicates in Π (possibly pulled back through the data flow).

Theorem 15.3.3 (Existence of finite sufficient predicate domains)**.** *If the viability predicates at all error sites are in a decidable theory $\mathcal{L}$ with finite quantifier elimination, then a finite sufficient predicate domain exists.*

Proof sketch. Each viability predicate involves a finite number of variables and constants. Quantifier elimination for $\mathcal{L}$ produces a finite set of atomic predicates. The union of these sets across all error sites is the sufficient domain. □

15.4 Automatic predicate discovery

The system discovers predicates automatically through several mechanisms:

Error-site-driven discovery. For each error site e with viability predicate V_e, the system:

1. Identifies the free variables of V_e.
2. Traces each variable back through the data flow to find its origins.
3. At each origin site, creates predicates corresponding to the atoms of V_e specialized to the origin's context.

For example, if the viability predicate at an indexing site is $0 \leq i < \text{len}(xs)$, the system traces i and $\text{len}(xs)$ back through the data flow and creates predicates $i \geq 0$, $i < \text{len}(xs)$, and $\text{len}(xs) > 0$ at the appropriate upstream sites.

Guard-derived predicates. Every conditional branch (`if`, `while`, `assert`) introduces a predicate. The system adds the guard's predicate (and its negation) to the predicate domain.

Annotation-derived predicates. Every user annotation (`@requires`, `@ensures`, `@invariant_of`) introduces predicates. The system adds the annotation's predicates to the domain.

Theory-derived predicates. Library theories (PyTorch shapes, Pandas columns) introduce domain-specific predicates. For example, the PyTorch shape theory adds dimension-equality and positivity predicates for all tensor-valued sites.

15.5 Predicate domain refinement

The initial predicate domain may be too coarse (missing predicates that are needed) or too fine (containing predicates that are irrelevant). The system refines the domain through a CEGAR-like (counterexample-guided abstraction refinement) loop:

Step 1: Solve with current domain. Run the MaxSMT inference with the current predicate domain Π.

Step 2: Check residuals. For each residual obligation (an obligation that could not be discharged), check whether adding new predicates to Π would help.

Step 3: Refine. If a residual's viability predicate involves a sub-expression not represented in Π, add the sub-expression as a new predicate. Re-run from Step 1.

Step 4: Coarsen. After convergence, remove predicates from Π that were never used in any discharged obligation. This keeps the domain minimal.

Theorem 15.5.1 (CEGAR termination)**.** *If the predicate theory has finite quantifier elimination, the CEGAR loop terminates after finitely many refinement steps.*

15.6 Relevance scoring

Not all bug-relevant predicates are equally important. The system assigns a *relevance score* to each predicate based on:

Definition 15.6.1 (Relevance score)**.** The relevance score of predicate ϕ at site s is:

$$R(\phi, s) = \sum_{e \in \mathrm{ErrorSites}(P)} w(e) \cdot \mathbb{1}[\phi \text{ contributes to discharging viability at } e]$$

where $w(e)$ is the error site's importance weight.

Predicates with high relevance scores should be tracked with high priority. Predicates with zero relevance are irrelevant and can be dropped.

Importance weighting of error sites. The weights $w(e)$ can be set by:

- **Severity**: runtime crashes (e.g., division by zero) are weighted higher than warnings.
- **Reachability**: error sites on hot paths are weighted higher.
- **User priority**: the programmer can explicitly flag certain error sites as high-priority.
- **Historical data**: error sites that have caused bugs before are weighted higher.

Chapter 16

User-Guided Relevance and the Interactive Query Interface

The automatic relevance analysis from the previous chapter determines which propositions matter for bug detection. But users have domain knowledge that the system lacks: they know which properties are important to their application, which invariants they suspect might be violated, and which components are most critical. This chapter describes the interactive relevance interface.

16.1 The `@track` annotation: user-declared relevance

The simplest form of user guidance is declaring that a specific proposition should be tracked, even if the automatic analysis does not identify it as bug-relevant:

```
from deppy import track

@track(lambda self: self.balance >= 0, name="non_negative_balance")
class BankAccount:
    def __init__(self, initial: float):
        self.balance = initial

    def withdraw(self, amount: float):
        self.balance -= amount

    def deposit(self, amount: float):
        self.balance += amount
```

The `@track` decorator instructs the system to:

1. Add the predicate `self.balance` ≥ 0 to the predicate domain.
2. Track this predicate through all methods of `BankAccount`.
3. Report any method that might violate the predicate.
4. Report any call sequence that might lead to a violation.

The key difference from `@invariant_of` is that `@track` does not assert that the property holds—it merely instructs the system to track whether it holds. The system reports both confirmations ("this method preserves non-negative balance when called with amount $\leq$ balance") and violations ("`withdraw` can make balance negative when amount $>$ balance").

16.2 Property queries: asking the system what it knows

The system provides a query interface where the user can ask about specific properties:

```
from deppy import query

# Does this function always return a sorted list?
query(merge_sort, "is_sorted(result)")
# -> VERIFIED: result is always sorted (given sorted inputs to merge)

# Can this function raise IndexError?
query(binary_search, "raises(IndexError)")
# -> SAFE: no IndexError possible (given len(xs) > 0)

# What is the shape of the output tensor?
query(attention, "result.shape", given={"query.shape": (B, S, D)})
# -> result.shape == (B, S, D_v) where D_v = value.shape[2]

# What preconditions does this function need?
query(train_classifier, "preconditions")
# -> features.dim() == 2, labels.dim() == 1,
# features.shape[0] == labels.shape[0], ...
```

The query interface works by:

1. Parsing the query into a proposition or a query type (property check, error reachability, shape inference, precondition synthesis).
2. Adding the query's predicates to the predicate domain if not already present.
3. Running the inference/verification engine with the augmented domain.
4. Reporting the result: verified, violated (with counterexample), or unknown (with explanation).

16.3 The exploration mode: discovering unknown properties

Beyond checking specific properties, the system can *discover* properties that the user might not have thought to ask about:

```
from deppy import explore

# What interesting properties does this function have?
explore(matrix_multiply)
# -> Discovered properties:
# 1. result.shape == (A.shape[0], B.shape[1]) [shape]
# 2. result[i][j] == sum(A[i][k]*B[k][j] for k) [value]
# 3. A.shape[1] == B.shape[0] is required [precond]
# 4. No division by zero possible [safety]
# 5. result is None-free if inputs are None-free [null]

# What could go wrong in this function?
explore(parse_config, mode="risks")
# -> Potential risks:
# 1. KeyError at line 12 if config missing "host"
# 2. ValueError at line 15 if port is not numeric
# 3. FileNotFoundError at line 8 if path invalid
# 4. TypeError at line 20 if timeout is not int/float
```

The exploration mode works by:

1. Generating candidate predicates from the function's operations (shapes, bounds, nullability, error conditions).
2. Running the inference engine to determine which predicates hold.
3. Ranking the discovered properties by relevance score.
4. Presenting the top-k properties to the user.

16.4 Relevance propagation across module boundaries

When a function f in module A calls a function g in module B, the relevance of predicates in g depends on what f needs:

Definition 16.4.1 (Cross-module relevance)**.** A predicate ϕ in module B is *relevant to module A* if:

1. There exists a call from A to a function g in B.
2. ϕ appears in g's postcondition or return-type section.
3. ϕ is bug-relevant to some error site in A (when pulled back through the call).

Demand-driven cross-module analysis. The system propagates relevance demands across module boundaries:

1. Module A determines that it needs a specific property of g's return value (e.g., the output tensor's shape).
2. This demand is communicated to module B's analysis.
3. Module B checks whether g satisfies the demanded property.
4. If yes, the property is exported as part of g's section.
5. If no, the failure is reported at the call site in A.

This demand-driven approach avoids analyzing all possible properties of every function in every module. Instead, only properties that are actually needed by callers are analyzed.

16.5 The relevance graph

The system maintains a *relevance graph* that tracks the flow of relevance through the program:

Definition 16.5.1 (Relevance graph)**.** The relevance graph $G_R = (V, E)$ has:

- Vertices V: predicate instances (ϕ, s) where ϕ is a predicate and s is a site.
- Edges E: a directed edge $(\phi, s) \to (\psi, t)$ indicates that knowing ϕ at s contributes to determining ψ at t.

The relevance graph supports several analyses:

- **Root cause analysis.** When a verification fails at site t, trace backward through the relevance graph to find which upstream sites contribute to the failure.
- **Impact analysis.** When a user adds an annotation at site s, trace forward through the relevance graph to see which downstream obligations are affected.
- **Annotation suggestion.** Find sites with high out-degree in the relevance graph (sites whose properties affect many downstream obligations). These are high-leverage annotation targets.

16.6 Relevance under uncertainty: probabilistic relevance

When the system cannot determine whether a predicate holds (the solver returns UNKNOWN), it can assign a *probabilistic relevance* based on heuristics:

Definition 16.6.1 (Probabilistic relevance)**.** The probabilistic relevance of predicate ϕ at site s is:

$$R_{\text{prob}}(\phi, s) = P(\phi \text{ is true at } s) \cdot R(\phi, s)$$

where $P(\phi \text{ is true})$ is estimated from:

- Static analysis heuristics (e.g., most list accesses are in-bounds).
- Dynamic profiling (if available).
- Prior distributions based on common patterns.

This allows the system to prioritize predicates that are both important (high relevance) and likely to hold (high probability), focusing verification effort where it is most likely to succeed.

Chapter 17

Formal Verification Integration: Connecting to External Provers

While the Z3-based solver handles decidable fragments, some properties require the full power of an interactive proof assistant. This chapter describes how the system integrates with external formal verification tools.

17.1 The verification backend interface

The system defines an abstract *verification backend* interface that can be implemented by different provers:

```
from abc import ABC, abstractmethod

class VerificationBackend(ABC):
    @abstractmethod
    def check(self, obligation: Obligation,
              context: Context) -> VerificationResult:
        """Check whether the obligation holds in the context."""
        ...

    @abstractmethod
    def supported_theories(self) -> set[str]:
        """Return the set of supported theories."""
        ...

class Z3Backend(VerificationBackend):
    """Z3 SMT solver for decidable fragments."""
    ...

class Lean4Backend(VerificationBackend):
    """Lean 4 proof assistant for interactive proofs."""
    ...

class CoqBackend(VerificationBackend):
    """Coq proof assistant for type-theoretic proofs."""
    ...

class DafnyBackend(VerificationBackend):
```

```
    """Dafny program verifier for Hoare-logic proofs."""
    ...
```

17.2 Obligation routing

The system routes each obligation to the most appropriate backend:

1. **Z3 first.** Every obligation is first sent to Z3. If Z3 discharges it (UNSAT), done.
2. **Theory matching.** If Z3 returns UNKNOWN, the system examines the obligation's theory. If it involves induction, it is routed to Lean4 or Coq. If it involves program logic, it is routed to Dafny.
3. **User preference.** The user can specify a preferred backend for specific obligations or obligation types.
4. **Fallback.** If all automated backends fail, the obligation becomes a residual for human review.

17.3 Translation to Lean 4

For obligations that require interactive proof, the system generates Lean 4 proof obligations:

```
# Python source:
@ensures(lambda xs, result:
    forall(range(len(xs)),
           lambda i: result[i] == xs[len(xs) - 1 - i]))
def reverse(xs: list[int]) -> list[int]:
    return xs[::-1]
```

The system generates:

```
-- Lean 4 translation:
theorem reverse_correct (xs : List Int) :
    forall i, i < xs.length ->
    (xs.reverse.get i) = (xs.get (xs.length - 1 - i)) := by
  intro i hi
  simp [List.get_reverse]
```

The translation preserves the logical structure while adapting to Lean's syntax and type system. The system can:

- Generate Lean 4 theorem statements from Python postconditions.
- Translate Python data structures to Lean equivalents (lists, dicts, optional types).
- Import proved Lean theorems back as trusted axioms in the Python analysis.

17.4 Proof portability and the semantic bridge

The semantic bridge between Python and the proof assistant must be trustworthy:

Definition 17.4.1 (Semantic bridge)**.** A *semantic bridge* from Python to a proof assistant L is a pair of translation functions:

$$\text{encode} : \text{Python obligation} \to L\text{-theorem statement}$$
$$\text{decode} : L\text{-proof certificate} \to \text{Python assumption}$$

such that if $L \vdash P$ (the proof assistant proves P), then the corresponding Python proposition holds for all values satisfying the encoding assumptions.

The bridge is trusted: if the translation is incorrect, the system may accept invalid proofs. To mitigate this risk:

- The translation is kept simple and auditable.
- Each translation rule is documented with a justification.
- The system can generate test cases that exercise both the Python code and the Lean theorem, checking consistency dynamically.

17.5 Hybrid verification workflows

The integration enables hybrid workflows where some properties are verified by Z3, some by Lean, and some by runtime checks:

```
from deppy import ensures, lean_proof, runtime_check

@ensures(lambda xs, result: is_sorted(result),
        verified_by="z3") # shape/bounds: Z3
@ensures(lambda xs, result: is_permutation(result, xs),
        verified_by="lean4") # permutation: Lean4
@ensures(lambda xs, result: is_stable(result, xs),
        verified_by="runtime") # stability: runtime check
def sort(xs: list[int]) -> list[int]:
    ...
```

The `verified_by` parameter specifies which backend should handle each postcondition. This allows the programmer to match the verification tool to the difficulty of the property.

Chapter 18

Information-Theoretic Optimization of Type Inference

The previous chapters formalized type inference as an optimization problem over sections of a sheaf. This chapter develops the information-theoretic foundations in full detail, showing how entropy minimization, mutual information, and rate-distortion theory provide principled answers to the questions: How much type information should the system infer? Which predicates should it track? What is the optimal trade-off between precision and computational cost?

18.1 Types as probability distributions

Rather than treating types as deterministic sets of values, we adopt a probabilistic viewpoint: a type specifies a *distribution* over possible values.

Definition 18.1.1 (Type-as-distribution)**.** A type assignment σ at site s induces a distribution P_σ over the possible runtime values at s. The distribution is:

$$P_\sigma(v) = \begin{cases} 1/|\{v : v \models \sigma(s)\}| & \text{if } v \models \sigma(s) \\ 0 & \text{otherwise} \end{cases}$$

under the maximum-entropy assumption (uniform distribution over the admitted values).

Definition 18.1.2 (Type entropy)**.** The *entropy* of a type assignment σ at site s is:

$$H(\sigma, s) = -\sum_v P_\sigma(v) \log P_\sigma(v) = \log |\{v : v \models \sigma(s)\}|$$

A more informative type has lower entropy (fewer admitted values).

Type inference as entropy minimization. The maximum-information objective from the previous chapter is equivalent to entropy minimization:

$$\sigma^* = \arg \min_{\sigma \in H^0(\mathbf{S}_P, \mathcal{T})} \sum_{s \in \mathbf{S}_P} w(s) \cdot H(\sigma, s)$$

This reframing connects type inference to the extensive literature on information theory, enabling the use of information-theoretic tools.

18.2 Mutual information between sites

The type at one site constrains the types at other sites through data flow. The *mutual information* between sites quantifies this coupling:

Definition 18.2.1 (Inter-site mutual information)**.** The mutual information between sites s and t is:

$$I(s;t) = H(\sigma, s) + H(\sigma, t) - H(\sigma, s \cup t)$$

where $H(\sigma, s \cup t)$ is the joint entropy of the types at s and t.

High mutual information means that knowing the type at s strongly constrains the type at t (or vice versa). The system uses mutual information to:

- Identify which sites are most tightly coupled (and should be analyzed together).
- Detect redundant predicates: if $I(\phi_1; \phi_2) \approx H(\phi_1)$, then ϕ_2 is nearly determined by ϕ_1 and may not need separate tracking.
- Guide the stratified solving order: analyze tightly-coupled SCCs together and loosely-coupled components independently.

18.3 Rate-distortion theory for type precision

Rate-distortion theory provides the optimal trade-off between the "rate" (amount of type information tracked) and the "distortion" (loss of precision).

Definition 18.3.1 (Type distortion)**.** The *distortion* of a type assignment σ relative to the principal type σ^* is:

$$D(\sigma) = \sum_{s \in \mathbf{S}_P} w(s) \cdot d(\sigma(s), \sigma^*(s))$$

where $d(\cdot, \cdot)$ is a distance metric on types (e.g., the number of predicates in σ^* that are not in σ).

Definition 18.3.2 (Type rate)**.** The *rate* of a type assignment σ is the total amount of information stored:

$$R(\sigma) = \sum_{s \in \mathbf{S}_P} |\{\phi \in \Pi : \sigma(s) \models \phi\}|$$

(the total number of tracked predicate instances across all sites).

Definition 18.3.3 (Rate-distortion function)**.** The *rate-distortion function* $R(D)$ is:

$$R(D) = \min_{\sigma : D(\sigma) \leq D} R(\sigma)$$

the minimum rate (number of tracked predicates) needed to achieve distortion at most D.

Theorem 18.3.4 (Optimal type precision allocation)**.** *The rate-distortion-optimal type assignment allocates more predicates to sites with higher importance weight $w(s)$ and more complex viability predicates. Sites with simple viability predicates (e.g., $x > 0$) need fewer tracked predicates; sites with complex viability predicates (e.g., involving multiple variables and array indices) need more.*

This theorem justifies the system's demand-driven approach: allocate analysis effort proportionally to the complexity and importance of each site.

18.4 The predicate selection problem

Given a budget of k predicates (to keep the analysis tractable), which predicates should the system track?

Definition 18.4.1 (Predicate selection problem)**.** Given a set of candidate predicates $\Phi = \{\phi_1, \ldots, \phi_m\}$, a budget $k \leq m$, and a set of error sites with viability predicates, find the subset $\Pi \subseteq \Phi$ with $|\Pi| \leq k$ that maximizes the number of dischargeable obligations.

Theorem 18.4.2 (NP-hardness of predicate selection)**.** *The predicate selection problem is NP-hard by reduction from maximum coverage.*

Proof sketch. Reduce from the maximum coverage problem: given a universe U, a collection of sets $S_1, \ldots, S_m$, and a budget k, find k sets whose union is maximized. Map each set S_i to a predicate ϕ_i, each element $u \in U$ to an obligation, and say that ϕ_i "helps discharge" obligation u if $u \in S_i$. □

Greedy approximation. Despite NP-hardness, the greedy algorithm achieves a $(1 - 1/e)$ approximation: iteratively select the predicate that discharges the most additional obligations. This is the standard result for submodular maximization.

Theorem 18.4.3 (Submodularity of predicate selection)**.** *The number of dischargeable obligations is a monotone submodular function of the predicate set. Therefore, the greedy algorithm achieves a* $(1 - 1/e) \approx 0.632$ *approximation ratio.*

Proof sketch. Monotonicity: adding a predicate can only help discharge more obligations (it never removes useful information). Submodularity: the marginal benefit of adding a predicate decreases as more predicates are already tracked (diminishing returns). This is because obligations that can be discharged by multiple predicates contribute less marginal benefit for each additional predicate. □

18.5 Pareto-optimal type assignments

When multiple objectives compete (precision at different sites, or precision vs. computation time), the system computes Pareto-optimal type assignments:

Definition 18.5.1 (Pareto optimality)**.** A type assignment σ is *Pareto-optimal* if there is no other valid type assignment σ' that is strictly more informative at some site and no less informative at any site. The set of Pareto-optimal assignments is the *Pareto frontier*.

In practice, the Pareto frontier arises when:

- Increasing precision at one site requires decreasing precision at another (due to solver timeout budgets).
- Different error sites require different predicates, and the total predicate budget is limited.
- The user cares about different properties at different confidence levels.

The system presents the Pareto frontier to the user, who can choose the desired trade-off:

```
PARETO FRONTIER:
  Option A: 95% of error sites verified, 12s analysis time
  Option B: 98% of error sites verified, 45s analysis time
  Option C: 100% of error sites verified, 180s analysis time
  Current: Option B (default)
```

18.6 Online optimization: incremental type inference

As the programmer edits code, the type assignments must be updated. Full re-analysis is expensive; incremental analysis updates only the affected sites.

Definition 18.6.1 (Change set)**.** A code edit produces a *change set* $\Delta \subseteq \mathbf{S}_P$: the set of sites whose code has changed. The *impact set* $\mathrm{Impact}(\Delta)$ is the transitive closure of Δ under data-flow edges.

Theorem 18.6.2 (Incremental soundness)**.** *If the type assignment σ was valid before the edit, and the edit affects only sites in Δ, then re-analyzing only* $\mathrm{Impact}(\Delta)$ *produces a valid updated assignment. Sites outside* $\mathrm{Impact}(\Delta)$ *retain their previous type assignments.*

The incremental analysis:

1. Identifies the change set Δ.
2. Computes the impact set $\mathrm{Impact}(\Delta)$.
3. Invalidates type assignments in $\mathrm{Impact}(\Delta)$.
4. Re-runs fixpoint iteration restricted to $\mathrm{Impact}(\Delta)$.
5. Re-checks obligations in $\mathrm{Impact}(\Delta)$.

For typical edits (modifying a single function), the impact set is small, and incremental analysis completes in milliseconds.

18.7 The Lagrangian relaxation of the combined problem

The combined problem—maximize type information subject to sheaf consistency, budget constraints, and solver-decidability constraints—can be approached via Lagrangian relaxation:

$$\begin{aligned}\mathcal{L}(\sigma, \lambda, \mu) = \sum_s w(s) \cdot \mathcal{I}(\sigma, s) - \sum_{(i,j)} \lambda_{ij} \cdot \mathrm{inconsistency}(\sigma_i, \sigma_j) \\ - \mu \cdot (|\Pi(\sigma)| - k)\end{aligned}$$

where λ_{ij} are Lagrange multipliers penalizing overlap inconsistency and μ is the multiplier for the predicate budget.

The dual problem $\min_{\lambda,\mu \geq 0} \max_\sigma \mathcal{L}$ can be solved by subgradient methods, alternating between:

1. Fixing the multipliers and solving for the best type assignment (the inner maximization).
2. Updating the multipliers based on constraint violations (the outer minimization).

This provides a principled approach to balancing precision, consistency, and computational cost.

Chapter 19

Obstruction Theory for Type Errors

When type inference fails, the sheaf-theoretic framework provides precise diagnostic information through *obstruction theory*: the mathematical theory of why a global section cannot be constructed.

19.1 Obstructions as cohomology classes

Recall from the cohomology chapter that $H^1(\mathbf{S}_P, \mathcal{T})$ measures the obstructions to gluing local sections. Each nonzero class in H^1 corresponds to a type error. This section makes the correspondence precise.

Definition 19.1.1 (Type-error cocycle)**.** A *type-error cocycle* is an element $\alpha \in Z^1(\mathcal{U}, \mathcal{T})$ (a 1-cocycle that is not a 1-coboundary). Concretely, α assigns to each overlap $u_i \cap u_j$ a "type mismatch"—a discrepancy between the local types at u_i and u_j that cannot be resolved by adjusting the local types.

Interpretation. The cocycle α_{ij} at the overlap $u_i \cap u_j$ is the "gap" between what site u_i expects and what site u_j provides. If α is a coboundary ($\alpha = \delta\beta$ for some 0-cochain β), then the gap can be closed by adjusting the local types by β. If α is not a coboundary, the gap is structural and cannot be closed.

Example 19.1.2 (Shape mismatch as a cocycle)**.** Consider two functions: f outputs a tensor of shape $(n, 3)$, and g expects a tensor of shape $(n, 4)$. The overlap is the call site $g(f(x))$. The cocycle assigns the "shape gap" $(n, 3) \neq (n, 4)$ to this overlap. This gap cannot be resolved by changing the types at other sites (no adjustment to f's output or g's input can make $3 = 4$), so it is a non-trivial cohomology class.

Example 19.1.3 (Resolvable inconsistency as a coboundary)**.** Consider f outputs an `int | None` and g expects an `int`. The overlap has a "None gap." But if there is a guard (`if x is not None`) between f and g, the gap is resolvable: the guard narrows the type, so the cocycle is a coboundary (the 0-cochain β is the guard's narrowing effect).

19.2 Minimal obstruction representatives

The cohomology class $[\alpha] \in H^1$ may have many representatives. The system should find the *minimal* representative: the one that identifies the most localized cause of the error.

Definition 19.2.1 (Minimal representative)**.** A representative α of a cohomology class $[\alpha]$ is *minimal* if the support of α (the set of overlaps where $\alpha_{ij} \neq 0$) has minimum cardinality among all representatives.

Theorem 19.2.2 (Minimal error localization)**.** *The minimal representative of a type-error cohomology class identifies the smallest set of code locations that jointly cause the type error.*

This is the formal version of "error localization": the system should blame the smallest set of locations, not the entire program. The minimal representative provides this.

19.3 Error classification by obstruction type

Different types of type errors correspond to different cohomology structures:

H^1 errors: pairwise inconsistencies. These are the most common errors: two sites disagree on the type of a shared value. Examples: shape mismatch, type mismatch, arity mismatch.

H^2 errors: three-way inconsistencies. These are rarer: three sites are pairwise consistent but jointly inconsistent. Example: function f returns a value that satisfies both predicates ϕ and ψ, but the conjunction $\phi \wedge \psi$ is unsatisfiable (the predicates contradict when combined).

Torsion errors: periodic inconsistencies. In loop-carried data flow, the type assignment at the loop header must be consistent with itself after one iteration. If the loop body changes the type in a way that cannot be undone (e.g., narrowing a type that the loop condition requires to be wide), the inconsistency is "periodic" and shows up as a torsion element in the cohomology.

19.4 Obstruction-guided repair

When a type error is detected, the obstruction class guides the repair:

Definition 19.4.1 (Repair as coboundary selection)**.** A *repair* for a type error $[\alpha] \in H^1$ is a 0-cochain $\beta \in C^0$ such that $\alpha' = \alpha - \delta\beta$ has smaller support than α. Ideally, $\alpha' = 0$ (the error is fully repaired). The 0-cochain β specifies how to adjust the local types to resolve the inconsistency.

The system computes repair suggestions by solving:

$$\beta^* = \arg\min_{\beta} \|\alpha - \delta\beta\|$$

where $\|\cdot\|$ measures the "size" of the remaining inconsistency (e.g., the number of overlaps with nonzero type gap).

Repair ranking. Multiple repairs may exist. The system ranks them by:

- **Locality**: repairs that change fewer sites are preferred.
- **Minimality**: repairs that make smaller type changes are preferred.
- **Semantic plausibility**: repairs that correspond to common coding patterns (adding a guard, adding a cast, changing a function signature) are preferred.

```
TYPE ERROR at line 42: shape mismatch
  f() returns shape (n, 3) but g() expects shape (n, 4)

  Repair suggestions (ranked):
    1. Add a projection: result = f(x)[:, :4]   [if n >= 4]
    2. Change f() to return shape (n, 4)
    3. Change g() to accept shape (n, 3)
    4. Add a reshape: result = f(x).reshape(n, 4)
       [requires n*3 divisible by 4]
```

19.5 Spectral sequences for complex type errors

For complex programs with deeply nested function calls and multiple interacting type constraints, the cohomology may need to be computed using *spectral sequences*.

Definition 19.5.1 (Spectral sequence for type inference)**.** The *Leray spectral sequence* for a composition of covers $\mathcal{U} \to \mathcal{V} \to \mathbf{S}_P$ has E_2 page:

$$E_2^{p,q} = H^p(\mathcal{V}, H^q(\mathcal{U}/\mathcal{V}, \mathcal{T}))$$

converging to $H^{p+q}(\mathbf{S}_P, \mathcal{T})$.

The practical interpretation: first compute local cohomology within each function (H^q over the function's internal cover), then compute global cohomology across functions (H^p over the inter-function cover). This decomposes the global type-error analysis into manageable local analyses, analogous to the stratified solving strategy.

Differential diagnosis. The differentials $d_r : E_r^{p,q} \to E_r^{p+r,q-r+1}$ in the spectral sequence correspond to type errors that "cross abstraction boundaries": errors that cannot be detected within a single function but arise from the interaction between functions. These are the hardest errors to diagnose, and the spectral sequence provides a systematic way to enumerate them.

Chapter 20

Topos-Theoretic Semantics: The Internal Logic of the Type Topos

The sheaf-theoretic framework developed so far uses sheaves as a technical device for gluing local type information. This chapter goes deeper: the category of sheaves on the program site forms a *topos*, and the internal logic of this topos provides a powerful language for reasoning about types, propositions, and their relevance to bug detection.

20.1 The type topos

Definition 20.1.1 (Type topos)**.** The *type topos* $\mathbf{Sh}(\mathbf{S}_P, J)$ is the category of sheaves of types on the site $(\mathbf{S}_P, J)$. Objects are type sheaves; morphisms are natural transformations (type-preserving maps between type sheaves).

As a topos, $\mathbf{Sh}(\mathbf{S}_P, J)$ has:

- A terminal object $\mathbf{1}$: the trivial type sheaf (a single type at every site).
- Products $\mathcal{T} \times \mathcal{T}'$: pair types.
- Exponentials $\mathcal{T}'^{\mathcal{T}}$: function types.
- A subobject classifier Ω: the "truth value" sheaf.
- An internal logic: a higher-order logic that can express propositions about types.

20.2 The subobject classifier and propositions

Definition 20.2.1 (Subobject classifier)**.** The subobject classifier Ω in the type topos is the sheaf defined by:

$$\Omega(s) = \{\text{sieves on } s\}$$

where a *sieve* on s is a downward-closed set of morphisms into s. A global section of Ω is a "truth value" that may vary across the program.

Interpretation. In a standard set-theoretic context, $\Omega = \{\text{true}, \text{false}\}$. In the type topos, truth values are more nuanced: a proposition may be true at some sites and false (or unknown) at others. The subobject classifier captures this *local truth*.

Example 20.2.2 (Local truth for bounds checking)**.** Consider the proposition "$i < \text{len}(xs)$" in a program with a guard `if i < len(xs)`. This proposition is:

- True at sites inside the true branch.
- False at sites inside the false branch.
- Unknown at sites before the guard.

The truth value is a section of Ω, not a single Boolean.

20.3 Propositions as subobjects of the type sheaf

In the topos, a proposition about a type is a *subobject*: a monomorphism $\mathcal{P} \hookrightarrow \mathcal{T}$ that picks out the values satisfying the proposition.

Definition 20.3.1 (Refinement as subobject)**.** The refinement type $\{x : A \mid \phi(x)\}$ corresponds to the subobject $\phi^{-1}(\text{true}) \hookrightarrow A$ in the type topos. The "characteristic map" $\chi_\phi : A \to \Omega$ sends each value to its local truth value under ϕ.

This formalization captures the key insight: a refinement type is not just a set constraint; it is a *site-dependent* constraint whose truth may vary across the program.

20.4 The internal logic for type reasoning

The internal logic of the type topos is an intuitionistic higher-order logic with the following connectives:

- **Conjunction** $\phi \wedge \psi$: a value satisfies both ϕ and ψ at a site iff it satisfies both propositions at that site.
- **Disjunction** $\phi \vee \psi$: a value satisfies ϕ or ψ at a site. Note: this is constructive—disjunction does not imply that we know which disjunct holds.
- **Implication** $\phi \Rightarrow \psi$: at every site where ϕ holds, ψ also holds. This is the *local implication*.
- **Universal quantification** $\forall x.\phi(x)$: for all values x at the site, $\phi(x)$ holds.
- **Existential quantification** $\exists x.\phi(x)$: there exists a value x at the site satisfying $\phi(x)$.
- **Negation** $\neg\phi = (\phi \Rightarrow \bot)$: ϕ implies absurdity. Note: $\neg\neg\phi$ does not imply ϕ (the logic is intuitionistic).

Why intuitionistic? The excluded middle ($\phi \vee \neg\phi$) does not hold in general in the type topos, because a proposition may be neither provable nor refutable at a given site. This matches the reality of type inference: the system may not be able to determine whether a property holds or not (the solver may time out). Intuitionistic logic is the natural logic for this setting.

Recovering classical reasoning. For decidable predicates (those in the solver's decidable fragment), the excluded middle does hold: the solver either proves ϕ or proves $\neg\phi$. The system tracks which predicates are decidable and allows classical reasoning for them while maintaining intuitionistic reasoning for undecidable predicates.

20.5 The forcing semantics: what is "true at a site"?

Definition 20.5.1 (Forcing)**.** A site s *forces* a proposition ϕ (written $s \Vdash \phi$) if ϕ holds in the section assigned to s. The forcing relation satisfies:

$$\begin{aligned}
s \Vdash \phi \wedge \psi &\iff s \Vdash \phi \text{ and } s \Vdash \psi \\
s \Vdash \phi \vee \psi &\iff \text{there is a cover } \{u_i \to s\} \text{ such that each } u_i \Vdash \phi \text{ or } u_i \Vdash \psi \\
s \Vdash \phi \Rightarrow \psi &\iff \forall (t \to s) : t \Vdash \phi \implies t \Vdash \psi \\
s \Vdash \forall x.\phi(x) &\iff \forall (t \to s), \forall a \in \mathcal{T}(t) : t \Vdash \phi(a) \\
s \Vdash \exists x.\phi(x) &\iff \text{there is a cover } \{u_i \to s\} \text{ and } a_i \in \mathcal{T}(u_i) \text{ with } u_i \Vdash \phi(a_i)
\end{aligned}$$

Interpretation for type inference.

- $s \Vdash (x > 0)$ means: at program point s, the system has established that $x > 0$.
- $s \Vdash (\text{len}(xs) = n)$ means: at program point s, the system knows the list's length.
- The forcing relation propagates through data flow: if $s \Vdash \phi$ and $s \to t$ is a data-flow edge that preserves ϕ, then $t \Vdash \phi$.

20.6 Relevance in the internal logic

The internal logic provides a precise language for expressing relevance:

Definition 20.6.1 (Bug-relevance in the internal logic)**.** A proposition ϕ at site s is *bug-relevant* if:

$$s \Vdash \left(\phi \Rightarrow \bigwedge_{e \in E(s)} \text{viable}(e) \right) \wedge \left(\neg\phi \Rightarrow \bigvee_{e \in E(s)} \neg\text{viable}(e) \right)$$

where $E(s)$ is the set of error sites reachable from s. That is: ϕ being true implies all reachable errors are viable (safe), and ϕ being false implies some error is non-viable (a bug).

This definition captures the intuition precisely: a bug-relevant proposition is one that *separates* safe behavior from buggy behavior.

20.7 The Lawvere-Tierney topology for decidable propositions

Definition 20.7.1 (Lawvere-Tierney topology)**.** A *Lawvere-Tierney topology* on the type topos is a morphism $j : \Omega \to \Omega$ satisfying:

1. $j \circ \text{true} = \text{true}$ (truth is preserved).
2. $j \circ j = j$ (idempotent).
3. $j \circ \wedge = \wedge \circ (j \times j)$ (commutes with conjunction).

The decidability boundary defines a Lawvere-Tierney topology: $j(\phi) = \text{true}$ if ϕ is decidable (the solver can determine its truth), and $j(\phi) = \phi$ otherwise. The *j-sheaves* are the types whose propositions are all decidable—the "decidable fragment" of the type system.

The decidable subtoposes. The subtoposes $\mathbf{Sh}_j(\mathbf{S}_P) \subseteq \mathbf{Sh}(\mathbf{S}_P)$ correspond to different decidable fragments:

- j_{LIA}: linear integer arithmetic.
- j_{LRA}: linear real arithmetic.
- j_{BV}: bitvector arithmetic.
- j_{ARR}: array theory.

The system operates primarily within $\mathbf{Sh}_{j_{\mathrm{LIA}}}$ (where most practical type reasoning lives) and escapes to the full topos when necessary.

20.8 Modalities for annotation levels

The annotation levels (0–5 from Part IV) correspond to modalities in the topos:

- **Level 0** ($\Box_{\mathrm{infer}}$): only propositions that the inference engine can derive.
- **Level 1** ($\Box_{\mathrm{contract}}$): propositions stated by contracts and verified by the solver.
- **Level 2** ($\Box_{\mathrm{invariant}}$): propositions maintained by invariants and verified inductively.
- **Level 3–5** ($\Box_{\mathrm{proof}}$): propositions with human-provided proofs.

Each modality defines a subtopos of "known truths" at that level. Higher levels include all lower levels. The full type topos includes all levels, plus propositions that are true but unknown at any level.

This topos-theoretic perspective reveals the mathematical structure of the annotation economy: the annotation levels form a chain of subtoposes, and the "value" of an annotation is its contribution to extending the subtopos from one level to the next.

Chapter 21

Automated Deduction Strategies for Richer Dependent Types

This chapter presents advanced deduction strategies that go beyond standard SMT solving to infer richer dependent types automatically, without user annotations.

21.1 Abductive inference: hypothesizing missing predicates

Standard forward inference derives conclusions from known facts. *Abductive inference* works in reverse: given a desired conclusion (a viability predicate that must hold), hypothesize the missing premises.

Definition 21.1.1 (Abductive inference for types)**.** Given a viability predicate V at an error site e and the current section σ at e (which does not imply V), the *abductive inference problem* is:

$$\text{Find the weakest } \phi \text{ such that } \sigma \cup \{\phi\} \models V$$

Algorithm.

1. Negate V and conjoin with σ: form $\sigma \wedge \neg V$.
2. Use Z3's `unsat_core()` to find the minimal set of facts in σ that participate in the (un)satisfiability check.
3. If $\sigma \wedge \neg V$ is satisfiable, the satisfying model identifies the "gap": the conditions under which V fails.
4. Negate the gap conditions to get ϕ: the missing predicate.
5. Trace ϕ backward through data flow to find where ϕ should be established.

Example 21.1.2. Error site: `xs[i]` with viability $0 \leq i < \text{len}(xs)$. Current section: $i = n - 1$ (from a loop), $\text{len}(xs) = m$. The gap: $n - 1 < m$ is not implied. The abduced predicate: $n \leq m$. Trace backward: n comes from a loop bound, m comes from the input. The system synthesizes the precondition $n \leq \text{len}(xs)$.

21.2 Craig interpolation for interface synthesis

When two modules interact, the system needs to find the "right" interface: a set of predicates that is strong enough for the caller's needs and weak enough for the callee to satisfy.

Definition 21.2.1 (Craig interpolant)**.** Given formulas A and B such that $A \models B$, a *Craig interpolant* is a formula I such that:

1. $A \models I$ (the implementation implies the interface).
2. $I \models B$ (the interface implies the caller's needs).
3. The variables of I are shared between A and B (the interface uses only shared vocabulary).

Application to type inference. Let A be the callee's postcondition (what the function actually guarantees) and B be the caller's requirement (what the caller needs to know about the return value). The interpolant I is the optimal function signature: it exports exactly the information the caller needs, no more and no less.

Z3 supports Craig interpolation for linear arithmetic and some other theories. The system uses this to automatically synthesize function signatures that are both sufficient for callers and provable from the implementation.

21.3 Quantifier instantiation for bounded domains

Many dependent type predicates involve bounded quantification: $\forall i \in [0, n).\phi(i)$. The solver handles these by instantiation:

Finite instantiation. When n is a known constant (or small symbolic bound), the quantifier is expanded into a finite conjunction: $\phi(0) \wedge \phi(1) \wedge \cdots \wedge \phi(n-1)$.

Inductive instantiation. When n is symbolic but the predicate has an inductive structure, the system uses inductive reasoning:

- Base case: $\phi(0)$.
- Inductive step: $\phi(k) \Rightarrow \phi(k+1)$ for $0 \leq k < n$.
- Conclusion: $\forall i \in [0, n).\phi(i)$.

The system attempts inductive instantiation automatically for quantified postconditions of loops and recursive functions.

E-matching. For quantifiers over complex expressions, the system uses E-matching: it identifies ground instances of the quantified variable that appear in the current context and instantiates the quantifier at those instances. This is Z3's standard approach for quantifier handling.

21.4 Abstract interpretation with predicate abstraction

The system uses abstract interpretation to infer loop invariants and function summaries automatically:

Definition 21.4.1 (Predicate abstract domain)**.** Given a predicate domain $\Pi = \{\phi_1, \ldots, \phi_k\}$, the *predicate abstract domain* is the Boolean lattice 2^Π (subsets of Π). An abstract value $S \subseteq \Pi$ represents the conjunction $\bigwedge_{\phi \in S} \phi$.

The abstract interpretation computes, for each program point, the set of predicates that hold:

1. Initialize all program points with Π (all predicates hold).
2. For each program point s, compute the abstract transfer function: which predicates are preserved by the operations at s?

3. Iterate until fixpoint.
4. The fixpoint gives the strongest invariant expressible in Π.

This is the standard approach from model checking (SLAM, BLAST), adapted to the sheaf-descent framework. The key difference is that the predicate domain Π is not fixed a priori but is dynamically refined by the CEGAR loop described earlier.

21.5 Symbolic execution for path-sensitive inference

Path-sensitive analysis tracks different type information along different execution paths:

```
def process(x: int | str) -> int:
    if isinstance(x, int):
        return x + 1 # path 1: x is int
    else:
        return len(x) # path 2: x is str
```

The path-insensitive analysis would merge the two paths and lose the type narrowing. The path-sensitive analysis maintains separate sections:

- Path 1: $x : \texttt{int}$, result $= x + 1 : \texttt{int}$.
- Path 2: $x : \texttt{str}$, result $= \text{len}(x) : \texttt{int}$.

The system merges paths at join points but preserves path conditions that affect type-critical properties.

Bounded symbolic execution. For loops, the system unrolls a bounded number of iterations and analyzes each unrolled path separately. After the bound, it widens to an over-approximation. The bound is configurable (default: 3 iterations for inference, unlimited for proof checking).

21.6 Liquid types: type inference as predicate abduction

The system incorporates ideas from *Liquid Types* (Rondon, Kawaguchi, Jhala): automatically inferring refinement types using predicate abduction and Horn clause solving.

Definition 21.6.1 (Liquid type inference)**.** Given a program with types annotated as "holes" κ_i (unknown refinements), the system:

1. Generates subtyping constraints between types with holes.
2. Converts the constraints to Horn clauses over the predicate domain.
3. Solves the Horn clauses using a fixpoint solver.
4. The solution assigns predicates to each hole, giving the strongest valid refinement.

The integration with the sheaf framework adds a key capability: the predicate domain is not fixed but dynamically refined based on the relevance analysis. This means the system discovers predicates that are both expressible and useful, rather than requiring a predefined set.

21.7 Machine-learning-guided predicate suggestion

For predicates that fall outside the decidable fragment, the system can use machine learning to suggest likely predicates:

- **Pattern matching**: recognize common code patterns (e.g., "loop over range(len(xs))") and suggest standard predicates (e.g., "loop variable is a valid index").

- **Historical data**: learn from previously verified programs which predicates were useful in similar contexts.
- **Natural language**: use the function's docstring or comments to suggest predicates (e.g., "returns a sorted list" suggests $\forall i < \text{len}(r) - 1.r[i] \leq r[i+1]$).

Machine-learning suggestions are never trusted; they are treated as candidates that must be verified by the solver. The value is in reducing the search space for predicate discovery.

Chapter 22

Enriched Category Theory for Type Propagation and Kan Extensions

The interprocedural transport of type information—how types flow across function calls, module boundaries, and library interfaces—can be formalized using enriched category theory and Kan extensions. This chapter develops the mathematics and shows how it enables optimal type propagation.

22.1 Enriched categories of types

Standard categories have sets of morphisms. *Enriched categories* have morphism objects in a monoidal category $\mathcal{V}$, allowing richer structure.

Definition 22.1.1 ($\mathcal{V}$-enriched type category)**.** Let $\mathcal{V} = (\mathbf{Lat}, \wedge, \top)$ be the monoidal category of complete lattices with meet as the tensor product. A $\mathcal{V}$-enriched category $\mathbf{T}$ has:

- Objects: program sites $s, t, \ldots$
- Hom-objects: $\mathbf{T}(s, t) \in \mathbf{Lat}$, the lattice of *type transfer functions* from s to t.
- Composition: a morphism $\circ : \mathbf{T}(t, u) \otimes \mathbf{T}(s, t) \to \mathbf{T}(s, u)$ in $\mathcal{V}$ (composition of transfer functions).

Interpretation. A type transfer function $f \in \mathbf{T}(s, t)$ is a lattice morphism that maps a type at s to a type at t. For example, the operation `len(xs)` induces a transfer function that maps the type of `xs` (a list with refinements) to the type of the result (an integer with the refinement ≥ 0).

The enrichment in **Lat** means that the set of transfer functions is itself a lattice: there is a most informative transfer function (the one that preserves the most type information) and a least informative one (the one that loses all information).

22.2 Weighted colimits for type synthesis

Type inference at a site s with multiple inputs $u_1, \ldots, u_n$ is a *weighted colimit* in the enriched category:

Definition 22.2.1 (Type synthesis as weighted colimit)**.** The inferred type at site s with inputs from sites $u_1, \ldots, u_n$ is:

$$\sigma(s) = \mathrm{colim}_W F$$

where:

- $F : \{u_1, \ldots, u_n\} \to \mathbf{T}$ maps each input site to its type.
- $W : \{u_1, \ldots, u_n\}^{\mathrm{op}} \to \mathcal{V}$ is the *weight* functor that assigns importance to each input.
- The colimit synthesizes the optimal type at s by combining the input types according to the weights and transfer functions.

Concrete construction. For a binary operation $s = f(u_1, u_2)$, the weighted colimit is:

$$\sigma(s) = \bigsqcup \{f_*(\sigma(u_1), \sigma(u_2)) \mid f_* \text{ is the transfer function of } f\}$$

the join (strongest common type) of all types that can be derived from the inputs through f's semantics.

22.3 Kan extensions for interprocedural transport

The key challenge of interprocedural analysis is: given a function g that calls function f, how should f's type information be transported into g's context?

Definition 22.3.1 (Left Kan extension for forward transport)**.** Given the inclusion functor $i : \mathbf{S}_f \hookrightarrow \mathbf{S}_g$ (embedding f's sites into g's context at the call site) and the type sheaf $\mathcal{T}_f$ on $\mathbf{S}_f$, the *left Kan extension*

$$\mathrm{Lan}_i \mathcal{T}_f : \mathbf{S}_g \to \mathbf{Lat}$$

is the optimal way to extend f's type information to all of g's sites. At each site $s \in \mathbf{S}_g$:

$$(\mathrm{Lan}_i \mathcal{T}_f)(s) = \mathrm{colim}_{(i \downarrow s)} \mathcal{T}_f$$

the colimit over all sites in f that can reach s.

Definition 22.3.2 (Right Kan extension for backward transport)**.** Dually, the *right Kan extension* $\mathrm{Ran}_i \mathcal{T}_g$ transports g's type requirements backward into f's context:

$$(\mathrm{Ran}_i \mathcal{T}_g)(s) = \lim_{(s \downarrow i)} \mathcal{T}_g$$

the limit over all sites in g that can be reached from s.

Interpretation.

- The left Kan extension is *forward propagation*: it pushes f's output type into g's sites as strongly as possible.
- The right Kan extension is *backward propagation*: it pulls g's requirements back into f's sites as weakly as possible.

Theorem 22.3.3 (Optimality of Kan extensions)**.** *The left Kan extension gives the strongest possible forward propagation, and the right Kan extension gives the weakest possible backward propagation, among all extensions that respect the data flow and the type transfer functions.*

Proof sketch. By the universal property of Kan extensions: Lan_i is left adjoint to restriction i^*, so it is the *best* (strongest) extension. Dually, Ran_i is right adjoint to i^*, so it is the best (weakest) backward extension. □

22.4 The pointwise formula and computational realization

In practice, Kan extensions are computed pointwise:

Theorem 22.4.1 (Pointwise Kan extension)**.** *If $\mathcal{V}$ is complete and cocomplete (which* **Lat** *is), then the Kan extensions exist and are computed pointwise:*

$$(\mathrm{Lan}_i \mathcal{T}_f)(s) = \int^{t \in \mathbf{S}_f} \mathbf{S}_g(i(t), s) \otimes \mathcal{T}_f(t)$$

$$(\mathrm{Ran}_i \mathcal{T}_g)(s) = \int_{t \in \mathbf{S}_f} [\mathbf{S}_g(s, i(t)), \mathcal{T}_g(i(t))]$$

where $\int^t$ denotes the coend and $\int_t$ the end.

Algorithmic realization. The pointwise formula translates directly to an algorithm:

1. For each site s in the caller, find all sites t in the callee that can reach s (the "comma category" $i \downarrow s$).
2. For each such t, apply the transfer function along the path $i(t) \to s$ to transport $\mathcal{T}_f(t)$.
3. Take the join (for forward propagation) or meet (for backward propagation) of all transported types.

This is exactly what the interprocedural transport already does in the implementation, but the Kan extension framework proves it is *optimal*.

22.5 Profunctors for type interfaces

The interface between two modules can be modeled as a *profunctor*:

Definition 22.5.1 (Type profunctor)**.** A *type profunctor* $H : \mathbf{S}_A \nrightarrow \mathbf{S}_B$ is a $\mathcal{V}$-enriched functor $H : \mathbf{S}_A^{\mathrm{op}} \times \mathbf{S}_B \to \mathcal{V}$. For each pair (a, b) of sites in modules A and B, $H(a, b)$ is the lattice of "type relationships" between a and b.

Composition of interfaces. If module A calls module B which calls module C, the composed interface $A \to C$ is the profunctor composition:

$$(H_2 \circ H_1)(a, c) = \int^{b \in \mathbf{S}_B} H_1(a, b) \otimes H_2(b, c)$$

This captures transitive type propagation across chains of function calls.

Identity profunctor. The identity profunctor $\mathrm{Id}(a, b) = \mathbf{S}(a, b)$ represents "no interface overhead": direct data flow with no loss of type information.

22.6 The fibered category of type families

As programs evolve, the type sheaf changes. The collection of all type sheaves (one per program version) forms a *fibered category*:

Definition 22.6.1 (Type fibration)**.** The *type fibration* $\pi : \mathbf{T} \to \mathbf{Prog}$ has:

- Base category **Prog**: programs and program transformations (edits, refactorings).
- Fiber $\mathbf{T}_P$ over each program P: the type topos $\mathbf{Sh}(\mathbf{S}_P, J_P)$.
- Transition functors: for each program transformation $f : P \to Q$, a functor $f^* : \mathbf{T}_Q \to \mathbf{T}_P$ that "translates" types between program versions.

Incremental analysis as base change. When the programmer edits the program ($f : P \to Q$), the transition functor f^* identifies which types can be preserved (unchanged sites) and which must be recomputed (changed or affected sites). This is the formal foundation for incremental type inference.

Chapter 23

The Relevance Optimization Problem: Formal Treatment

This chapter formalizes the relevance optimization problem with full mathematical precision and develops algorithms that determine, for each proposition the system could attach to a term, whether that proposition is worth tracking for the purpose of bug detection.

23.1 The space of decorations

Definition 23.1.1 (Decoration)**.** A *decoration* of a term t at site s is a proposition ϕ such that $s \Vdash \phi(t)$ (the system can establish that t satisfies ϕ at site s). The *decoration space* $\mathcal{D}(t, s)$ is the set of all propositions that the system can prove about t at s.

The decoration space is typically enormous: for an integer variable, it includes $x > 0$, $x \leq 100$, x is even, $x \equiv 3 \pmod 7$, and infinitely many others. The relevance problem is to select the *useful* subset.

Definition 23.1.2 (Useful decoration)**.** A decoration $\phi \in \mathcal{D}(t, s)$ is *useful* if removing it from the type at s would cause at least one obligation to become undischargeable. Formally:

$$\phi \in \mathcal{D}_{\text{useful}}(t, s) \iff \exists e \in \text{Obligations}(P) : \sigma \setminus \{\phi\} \not\models V_e \wedge \sigma \models V_e$$

23.2 Minimal useful decoration sets

Definition 23.2.1 (Minimal useful set)**.** A set $\Phi \subseteq \mathcal{D}(t, s)$ is a *minimal useful decoration set* if:

1. Every useful decoration is in Φ or is implied by Φ.
2. No proper subset of Φ has property 1.

Theorem 23.2.2 (Existence and computation)**.** *The minimal useful decoration set exists and can be computed as the set of unsatisfiable core predicates over all obligations:*

$$\Phi^* = \bigcup_{e \in \text{Obligations}} \text{unsat_core}(e, \sigma)$$

where $\text{unsat_core}(e, \sigma)$ *is the minimal subset of* σ *that implies the viability predicate* V_e*.*

Proof sketch. Each obligation e depends on a subset of the section's predicates. The unsat core identifies this subset. The union across all obligations gives the predicates that are needed by at least one obligation. Any predicate not in this union is not needed by any obligation and can be dropped. □

23.3 The relevance lattice

Definition 23.3.1 (Relevance lattice)**.** The *relevance lattice* $\mathcal{R}$ at a site s is the lattice of subsets of the decoration space, ordered by inclusion, with the additional structure:

- Bottom: $\emptyset$ (no decorations tracked).
- Top: $\mathcal{D}(t, s)$ (all decorations tracked).
- Meet: intersection (common decorations).
- Join: union (combined decorations).

The minimal useful set Φ^* is a specific element of this lattice. The system should compute Φ^* or a close approximation.

23.4 Relevance propagation as a Galois connection

The relationship between decorations at different sites is captured by a Galois connection:

Definition 23.4.1 (Relevance Galois connection)**.** For sites s and t connected by a data-flow edge $f : s \to t$, define:

$$\alpha : \mathcal{R}(s) \to \mathcal{R}(t), \quad \alpha(\Phi) = \{f_*(\phi) \mid \phi \in \Phi\}$$
$$\gamma : \mathcal{R}(t) \to \mathcal{R}(s), \quad \gamma(\Psi) = \{f^*(\psi) \mid \psi \in \Psi\}$$

where f_* pushes decorations forward and f^* pulls decorations backward. The pair (α, γ) forms a Galois connection: $\alpha(\Phi) \subseteq \Psi \iff \Phi \subseteq \gamma(\Psi)$.

Forward relevance propagation. If a decoration ϕ at s is relevant, then the pushed-forward decoration $f_*(\phi)$ at t may also be relevant (it contributes to the same downstream obligations through t).

Backward relevance propagation. If a decoration ψ at t is relevant (needed for some obligation at or beyond t), then the pulled-back decoration $f^*(\psi)$ at s is also relevant (it is needed to establish ψ at t).

23.5 The relevance fixpoint

Combining forward and backward propagation gives a fixpoint computation:

Algorithm 7 Relevance fixpoint computation

```
Initialize: R(s) ← ∅ for all sites s
Seed: For each error site e, add V_e's atoms to R(e)
repeat
    for all data-flow edges f : s → t do
        R(s) ← R(s) ∪ f*(R(t))        ▷ backward
        R(t) ← R(t) ∪ f_*(R(s))       ▷ forward
    end for
until convergence
return R
```

Theorem 23.5.1 (Convergence of relevance fixpoint)**.** *If the decoration space at each site is finite (which it is when the predicate domain is finite), the relevance fixpoint converges in at most* $|\Pi| \cdot |\mathbf{S}_P|$ *iterations.*

23.6 User interaction with the relevance system

The user can interact with the relevance system at several levels:

Querying relevance.

```
from deppy import why_relevant, why_irrelevant

# Why is this predicate tracked?
why_relevant(x, "x > 0", at_line=42)
# -> "x > 0" is relevant because:
# - It is needed to prove xs[x-1] is in bounds (line 45)
# - It is needed to prove sqrt(x) is real-valued (line 48)

# Why is this predicate NOT tracked?
why_irrelevant(x, "x % 2 == 0", at_line=42)
# -> "x % 2 == 0" is irrelevant because:
# - No error site reachable from line 42 depends on
# the parity of x
# - Dropping this predicate does not affect any obligation
```

Forcing relevance.

```
from deppy import force_relevant

# Track this predicate even if automatic analysis
# considers it irrelevant:
force_relevant(matrix, "matrix.is_symmetric()",
              reason="symmetric matrices have real eigenvalues")
```

Suppressing relevance.

```
from deppy import suppress

# Don't bother tracking this (known imprecision):
suppress(x, "x < 2**63", reason="Python ints don't overflow")
```

23.7 Relevance-weighted MaxSMT

The relevance scores feed directly into the MaxSMT objective function:

$$\sigma^* = \arg \max_{\sigma \in H^0(\mathbf{S}_P, \mathcal{T})} \sum_{(\phi, s) \in \mathcal{R}} R(\phi, s) \cdot \mathbb{1}[\sigma(s) \models \phi]$$

This weighted objective ensures that the solver spends its effort on predicates that are both provable and useful for bug detection. Irrelevant predicates have $R = 0$ and are ignored by the solver.

Theorem 23.7.1 (Optimality of relevance-weighted inference)**.** *The relevance-weighted MaxSMT solution discharges the maximum possible number of obligations (weighted by error-site importance) among all type assignments that are consistent and use at most k predicates per site.*

Chapter 24

Connections to Homotopy Type Theory and Higher-Dimensional Type Structure

This chapter explores deeper connections between the sheaf-descent framework and homotopy type theory (HoTT). While the practical system operates in a decidable fragment, the HoTT perspective reveals structural insights that improve automatic deduction and clarify the space of type equivalences.

24.1 Types as spaces, programs as paths

In HoTT, types are interpreted as spaces and terms as points in those spaces. Equalities between terms are paths; equalities between equalities are homotopies. This provides a geometric intuition for type equivalence.

Program paths. Two programs f and g of the same type $A \to B$ are *path-connected* if there exists a continuous transformation from f to g that preserves typing. In our framework, this means that the type sheaves of f and g are equivalent (their sections carry the same information).

Practical consequence. Two implementations of the same specification are path-connected. The system can use this to transfer proofs between implementations: if we prove a property of implementation f and show that g is path-connected to f (via the refinement proofs from Part IV), the property transfers to g automatically.

24.2 The univalence principle for type equivalence

Definition 24.2.1 (Univalence for dependent types)**.** Two types A and B are *equivalent* ($A \simeq B$) if there exist functions $f : A \to B$ and $g : B \to A$ with $g \circ f \sim \mathrm{id}_A$ and $f \circ g \sim \mathrm{id}_B$. The univalence axiom states:

$$(A \simeq B) \simeq (A =_{\mathcal{U}} B)$$

Equivalence of types *is* equality of types.

Application to type inference. Univalence means that the system should not distinguish between equivalent types. If $A = \{x : \texttt{int} \mid x > 0\}$ and $B = \{x : \texttt{int} \mid x \geq 1\}$, then $A \simeq B$ (they have the same elements), so the system should treat them as the same type. This is captured by the normalization step in the solver: equivalent refinement predicates are simplified to a canonical form before comparison.

Automatic equivalence detection. The system should detect type equivalences automatically:

- $\{x : A \mid \phi(x) \wedge \psi(x)\} \simeq \{x : A \mid \psi(x) \wedge \phi(x)\}$ (conjunction is commutative).
- $\{x : \texttt{int} \mid x > 0 \wedge x > 0\} \simeq \{x : \texttt{int} \mid x > 0\}$ (idempotence).
- $\{x : \texttt{int} \mid x > 0 \wedge x < 10\} \simeq \{x : \texttt{int} \mid 0 < x < 10\}$ (rewriting).
- $\texttt{list}[A]$ where $\text{len} = 0 \simeq$ the unit type (an empty list carries no element information).

Normalizing types to canonical forms reduces the number of distinct types the system must track, improving both efficiency and readability of the inferred types.

24.3 Higher inductive types for recursive data structures

Higher inductive types (HITs) generalize ordinary inductive types by allowing constructors that produce paths (equalities) in addition to points (elements).

Application: quotient types. A common programming pattern is working with values modulo an equivalence relation:

```
# Rational numbers: (p, q) with q != 0, modulo (p,q) ~ (p*k, q*k)
class Rational:
    def __init__(self, p: int, q: Refine[int, lambda q: q != 0]):
        g = gcd(abs(p), abs(q))
        self.p = p // g
        self.q = abs(q) // g
```

The type of `Rational` is a *quotient type*: two pairs (p_1, q_1) and (p_2, q_2) represent the same rational if $p_1 q_2 = p_2 q_1$. As a HIT, this is:

$$\begin{aligned}\texttt{Rational} = \text{HIT}\quad & \\ & \text{mk} : (p : \mathbb{Z}) \times (q : \mathbb{Z}_{>0}) \to \texttt{Rational} \\ & \text{path} : \forall p_1, q_1, p_2, q_2.\ p_1 q_2 = p_2 q_1 \to \text{mk}(p_1, q_1) = \text{mk}(p_2, q_2)\end{aligned}$$

The system should recognize quotient types and track the canonical representative (after normalization by GCD), ensuring that the type refinements are invariant under the equivalence relation.

Application: sets as quotients of lists. Python's `set` type is a quotient of `list` by permutation and deduplication. The system should track set properties (membership, cardinality) without being sensitive to the internal representation order.

24.4 Transport along type paths

When a type changes along a data-flow path (e.g., a list is reversed, a tensor is transposed), the predicates must be *transported* along the type path:

Definition 24.4.1 (Transport)**.** Given a path $p : A =_{\mathcal{U}} B$ and a predicate $\phi : A \to \text{Prop}$, the *transport* of ϕ along p is:

$$p_*(\phi) : B \to \text{Prop}, \quad p_*(\phi)(b) = \phi(p^{-1}(b))$$

Example 24.4.2 (Transport through list reversal)**.** If $\phi = (xs[0] = 42)$ and p is the reversal operation `reverse` : `list` $\to$ `list`, then:

$$p_*(\phi) = (\text{reverse}(xs)[0] = 42) = (xs[-1] = 42)$$

The predicate "first element is 42" becomes "last element is 42" after reversal.

Transport is what the system does when propagating refinement predicates through operations: the predicate's variables are rewritten according to the operation's semantics. The HoTT perspective formalizes this as path transport, ensuring that the rewriting is coherent (composing two transports gives the transport along the composed path).

24.5 Identity types and equality reasoning

In HoTT, the identity type $a =_A b$ is the type of proofs that a and b are equal. The system uses identity types for equality reasoning:

- **Reflexivity**: $\text{refl} : a =_A a$. Every value is equal to itself.
- **Symmetry**: if $p : a =_A b$, then $p^{-1} : b =_A a$.
- **Transitivity**: if $p : a =_A b$ and $q : b =_A c$, then $p \cdot q : a =_A c$.
- **Congruence**: if $p : a =_A b$ and $f : A \to B$, then $\text{ap}_f(p) : f(a) =_B f(b)$.

These are the proof terms from Part IV, now grounded in the HoTT framework. The advantage of the HoTT perspective is that it provides a principled account of when equality reasoning is valid across type transformations.

24.6 The fundamental groupoid of a type

Definition 24.6.1 (Fundamental groupoid)**.** The *fundamental groupoid* $\Pi_1(A)$ of a type A has:

- Objects: elements of A.
- Morphisms: paths (equalities) between elements.
- Composition: path concatenation.
- Inverses: path reversal.

For practical types, the fundamental groupoid captures the "transformation group" of the type:

- For integers: the trivial groupoid (each integer is distinct).
- For tensors: the groupoid includes shape-preserving transformations (transpositions, reshapes that preserve element count).
- For sets: the symmetric group (any permutation is a valid path).
- For quotient types: the equivalence relation's groupoid.

The system uses the groupoid structure to determine which predicates are invariant under type transformations and which need to be transported.

24.7 Truncation levels and proof irrelevance

HoTT introduces *truncation levels* that classify types by the complexity of their equality structure:

- (-2)**-truncated**: contractible types (a single value).
- (-1)**-truncated**: propositions (at most one proof). The truth value is what matters, not how it was proved.
- 0**-truncated**: sets (equality is a proposition). This is the level of ordinary types like `int` and `str`.
- 1**-truncated**: groupoids.
- n**-truncated**: higher groupoids.

Proof irrelevance. Propositions ((-1)-truncated types) have the property that all proofs are equal. This is critical for the system's efficiency: when a refinement predicate is proved, the system does not need to track *how* it was proved—only that it was proved. The proof can be erased at runtime and discarded during analysis after the obligation is discharged.

Sets and decidable equality. Most Python types are 0-truncated (sets): equality between values is a proposition (either they are equal or they are not). This justifies the system's use of SMT equality reasoning for most type comparisons.

Chapter 25

Synthesis of Relevant Propositions: Algorithms and Complexity

This chapter presents concrete algorithms for synthesizing the propositions that should be tracked at each program site, combining the theoretical framework from the preceding chapters with practical computational strategies.

25.1 The proposition synthesis pipeline

The full pipeline for determining which propositions to track at each site:

1. **Error site enumeration.** Identify all sites where a runtime error could occur and extract the viability predicate at each.
2. **Backward slicing.** For each error site e, compute the backward slice: all sites that could influence e's viability.
3. **Predicate extraction.** For each site s in a backward slice, extract candidate predicates from the viability predicate V_e pulled back to s.
4. **Relevance filtering.** Remove predicates that cannot affect any viability predicate (zero relevance score).
5. **Predicate selection.** If the remaining predicates exceed the budget, select the top-k by greedy submodular maximization.
6. **Fixpoint inference.** Run the MaxSMT inference with the selected predicates to compute the optimal type assignment.
7. **CEGAR refinement.** If residuals remain, add new predicates and repeat from step 3.

25.2 Complexity analysis

Theorem 25.2.1 (Complexity of the proposition synthesis pipeline)**.** *Let n be the number of program sites, m the number of error sites, and k the predicate budget. The pipeline has the following complexity:*

1. *Error site enumeration: $O(n)$ (one pass over the AST).*
2. *Backward slicing: $O(n \cdot m)$ (one backward traversal per error site, amortized via shared computation).*
3. *Predicate extraction: $O(m \cdot p)$ where p is the average number of atoms per viability predicate.*
4. *Relevance filtering: $O(m \cdot p \cdot n)$ (check each predicate against each error site's backward slice).*

5. *Predicate selection:* $O(k \cdot m \cdot p)$ *(greedy selection with* k *iterations, each evaluating* $O(m \cdot p)$ *marginal gains).*
6. *Fixpoint inference:* $O(n \cdot k^2 \cdot T_{\text{SMT}})$ *where* T_{SMT} *is the average SMT query time.*
7. *CEGAR refinement:* $O(r)$ *refinement rounds, each repeating steps 3–6 with an expanded predicate set.*

Total: $O(r \cdot n \cdot k^2 \cdot T_{\text{SMT}})$*, dominated by the fixpoint inference step.*

25.3 Optimizations for large programs

Incremental slicing. After a code edit, only the changed function's backward slices need to be recomputed. Other slices are cached.

Parallelization. Independent SCCs (strongly connected components of the call graph) can be analyzed in parallel. Within each SCC, the fixpoint iterations are sequential, but the SMT queries within each iteration can be parallelized across sites.

Tiered analysis.

1. **Tier 1 (fast, imprecise)**: flow-insensitive analysis with a small predicate domain. Runs in seconds for 10K-line programs. Catches simple bugs (type mismatches, obvious null dereferences).
2. **Tier 2 (moderate, precise)**: flow-sensitive analysis with the full predicate domain. Runs in minutes for 10K-line programs. Catches shape errors, bounds violations, numeric safety issues.
3. **Tier 3 (slow, complete)**: path-sensitive analysis with CEGAR refinement. Runs in minutes to hours for 10K-line programs. Handles complex interactions and proves functional properties.

The system defaults to Tier 2 and escalates to Tier 3 only for functions with proof annotations or unresolved residuals.

25.4 The relevance oracle: user-system interaction protocol

When the automatic analysis cannot determine relevance, it consults the user through a structured protocol:

```
RELEVANCE QUERY at line 42:
  The predicate "x > 0" at line 42 MIGHT be relevant to
  the IndexError at line 58. The system cannot determine
  whether x's positivity affects the array access at line 58.

  Options:
    [1] Mark as relevant (track it, may find additional bugs)
    [2] Mark as irrelevant (drop it, faster analysis)
    [3] Show me the data flow from line 42 to line 58
    [4] Let me provide additional context
```

```
CONTEXT NEEDED at line 30:
  The function "load_data" returns a DataFrame, but the system
  cannot determine whether the DataFrame has the required
  columns {"name", "age", "score"} needed at line 45.

  Options:
    [1] The DataFrame always has these columns
        (add as @ensures)
    [2] The DataFrame might not have these columns
        (add runtime check)
    [3] Show me the implementation of load_data
    [4] I'll add a contract manually
```

The interaction protocol is designed to minimize user effort: the system presents binary or multiple-choice questions rather than open-ended queries, and each response immediately refines the analysis.

25.5 Relevance for proof obligations

When the system is in proof mode (Part IV), relevance extends to proof obligations:

Definition 25.5.1 (Proof-relevant proposition)**.** A proposition ϕ at site s is *proof-relevant* if it is used in the proof of at least one obligation. Formally:

$$\phi \in \mathcal{D}_{\text{proof-relevant}}(t, s) \iff \exists e \in \text{Obligations}_{\text{proved}}(P) : \phi \in \text{proof_dependencies}(e)$$

Proof-relevant propositions are a superset of bug-relevant propositions: a proposition might not directly affect a viability predicate but might be needed as a lemma in a proof chain that eventually establishes viability.

Example 25.5.2. The proposition "the list is sorted" is not directly needed for bounds checking (bug relevance), but it is needed to prove that binary search returns the correct result (proof relevance). If the user has asked for a correctness proof, sortedness becomes relevant; if they have only asked for safety checking, it is irrelevant.

The system distinguishes between bug-relevance (always computed) and proof-relevance (computed only when proof mode is active) to avoid tracking unnecessary propositions in the common case.